

benchmark_results_summary

June 12, 2026

1 AutoQSAR Benchmark Summary

This notebook reads the output from `run_autoqsar_ga_benchmarks.py` and summarizes model performance across datasets.

By default it loads the most recent run under `./benchmark_results/`, then builds: - a model-performance summary across datasets - a leaderboard-relative summary using cached top-10 reference artifacts from the run directory - diagnostic visuals for model spread, predictions, and (if present) GA convergence - feature-family selection representation analysis from selector outputs (coverage, enrichment, and drop/watchlist suggestions)

```
[53]: # @title 0. Choose a benchmark run { display-mode: "form" }
benchmark_run_dir = "AUTO" # @param {type:"string"}
show_top_n_models = 10 # @param {type:"slider", min:3, max:20, step:1}
rmse_metric_column_preference = "AUTO" # @param {type:"string"}
prediction_sample_cap = 5000 # @param {type:"integer"}
```

```
[54]: from pathlib import Path
import json
import math

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from IPython.display import Markdown, display

pd.set_option("display.max_columns", 200)
pd.set_option("display.width", 200)

def display_note(text):
    display(Markdown(text))

def resolve_benchmark_dir(path_text="AUTO"):
    path_text = str(path_text).strip()
    if path_text and path_text.upper() != "AUTO":
```

```

        candidate = Path(path_text).expanduser().resolve()
        if not candidate.exists():
            raise FileNotFoundError(f"Benchmark directory not found: {candidate}")
        return candidate

    root = Path("./benchmark_results").resolve()
    if not root.exists():
        for parent in Path(".").resolve().parents:
            candidate = parent / "benchmark_results"
            if candidate.exists():
                root = candidate
                break
        else:
            raise FileNotFoundError("No ./benchmark_results directory exists yet.")
    candidates = sorted([p for p in root.iterdir() if p.is_dir()], key=lambda p:
        p.stat().st_mtime, reverse=True)
    if not candidates:
        raise FileNotFoundError("No benchmark run directories were found under ./benchmark_results.")
    return candidates[0]

def read_optional_csv(path):
    path = Path(path)
    if path.exists():
        return pd.read_csv(path)
    return None

def read_optional_json(path):
    path = Path(path)
    if path.exists():
        return json.loads(path.read_text(encoding="utf-8"))
    return None

def pick_column(columns, candidates):
    cols = list(columns)
    for candidate in candidates:
        if candidate in cols:
            return candidate
    lowered = {str(col).strip().lower(): col for col in cols}
    for candidate in candidates:
        match = lowered.get(str(candidate).strip().lower())
        if match is not None:

```

```

        return match
    return None

def metric_column(metric_df, preferred="AUTO", metric_hint="rmse"):
    if metric_df is None or metric_df.empty:
        return None
    if str(preferred).strip() and str(preferred).strip().upper() != "AUTO":
        if preferred not in metric_df.columns:
            raise ValueError(f"Requested metric column not found: {preferred}")
        return preferred
    candidates = []
    hint = str(metric_hint).strip().lower()
    if hint == "rmse":
        candidates = [
            "test_rmse", "Test RMSE", "cv_rmse", "CV RMSE", "mean_test_rmse",
            "rmse", "RMSE"
        ]
    elif hint == "r2":
        candidates = ["test_r2", "Test R2", "cv_r2", "CV R2", "r2", "R2"]
    return pick_column(metric_df.columns, candidates)

CLASSIFICATION_METRIC_CANDIDATES = {
    "roc_auc": ["test_roc_auc", "Test ROC AUC", "roc_auc", "ROC AUC", "AUROC",
    ↪ "test_auroc", "cv_roc_auc", "CV ROC AUC"],
    "auprc": ["test_auprc", "Test AUPRC", "AUPRC", "auprc", "cv_auprc", "CV
    ↪ AUPRC", "average_precision"],
    "balanced_accuracy": ["test_balanced_accuracy", "Test Balanced Accuracy",
    ↪ "balanced_accuracy", "Balanced Accuracy", "cv_balanced_accuracy", "CV
    ↪ Balanced Accuracy"],
    "mcc": ["test_mcc", "MCC", "mcc", "cv_mcc"],
}

# TDC CYP benchmark tasks whose leaderboard-primary metric is AUPRC. Keep these
# row-specific so other classification datasets can continue to use AUROC or the
# metric declared in their benchmark metadata.
TDC_AUPRC_PRIMARY_DATASETS = {
    "tdc_cyp2c9_veith",
    "tdc_cyp2d6_veith",
    "tdc_cyp3a4_veith",
    "tdc_cyp2c9_substrate_carbonmangels",
    "tdc_cyp2d6_substrate_carbonmangels",
    "tdc_cyp3a4_substrate_carbonmangels",
}

```

```

def classification_metric_column(metric_df, preferred="AUTO", metric_kind=None):
    if metric_df is None or metric_df.empty:
        return None
    if str(preferred).strip() and str(preferred).strip().upper() != "AUTO":
        if preferred not in metric_df.columns:
            raise ValueError(f"Requested classification metric column not found:
↪ {preferred}")
        return preferred

    if metric_kind is not None:
        candidates = CLASSIFICATION_METRIC_CANDIDATES.get(str(metric_kind).
↪ strip().lower())
        if candidates:
            return pick_column(metric_df.columns, candidates)

    ordered_candidates = []
    for kind in ["roc_auc", "auprc", "balanced_accuracy", "mcc"]:
        ordered_candidates.extend(CLASSIFICATION_METRIC_CANDIDATES[kind])
    return pick_column(metric_df.columns, ordered_candidates)

def infer_classification_metric_kind(dataset_name="", primary_metric=""):
    dataset_key = str(dataset_name or "").strip().lower()
    metric_text = str(primary_metric or "").strip().lower()

    if dataset_key in TDC_AUPRC_PRIMARY_DATASETS:
        return "auprc"
    if "auprc" in metric_text or "pr_auc" in metric_text or "average_precision"
↪ in metric_text:
        return "auprc"
    if "balanced" in metric_text:
        return "balanced_accuracy"
    if "mcc" in metric_text or "matthews" in metric_text:
        return "mcc"
    if "roc" in metric_text or "auROC" in metric_text or metric_text == "auc":
        return "roc_auc"
    return "roc_auc"

def classification_metric_column_for_row(row, columns, fallback_col=None):
    metric_kind = infer_classification_metric_kind(row.get("dataset", ""), row.
↪ get("primary_metric", ""))
    selected = pick_column(columns, CLASSIFICATION_METRIC_CANDIDATES.
↪ get(metric_kind, []))
    if selected is not None:
        return selected, metric_kind
    if fallback_col in columns:

```

```

        return fallback_col, infer_classification_metric_kind("", fallback_col)
    fallback = classification_metric_column(pd.DataFrame(columns=columns))
    return fallback, infer_classification_metric_kind("", fallback)

def metric_is_lower_better(metric_name):
    text = str(metric_name or "").strip().lower()
    if not text:
        return True
    if any(key in text for key in ["rmse", "mae", "mse", "loss", "error"]):
        return True
    if any(key in text for key in ["balanced_accuracy", "accuracy", "roc_auc", "au
    ↪ "auroc", "auc", "auprc", "mcc", "f1", "r2", "spearman", "pearson"]):
        return False
    return True

def add_analysis_metric_columns(frame, rmse_col=None, classification_col=None):
    result = frame.copy()
    if result.empty:
        result["task_kind"] = []
        result["analysis_metric"] = []
        result["analysis_metric_kind"] = []
        result["analysis_metric_value"] = []
        result["analysis_metric_direction"] = []
        result["analysis_delta_from_best"] = []
        return result

    rmse_col = rmse_col if rmse_col in result.columns else ↵
    ↪ metric_column(result, "AUTO", metric_hint="rmse")
    classification_col = classification_col if classification_col in result.
    ↪ columns else classification_metric_column(result)

    rmse_values = pd.to_numeric(result[rmse_col], errors="coerce") if rmse_col ↵
    ↪ is not None else pd.Series(np.nan, index=result.index)
    class_probe_values = pd.Series(False, index=result.index)
    for candidates in CLASSIFICATION_METRIC_CANDIDATES.values():
        probe_col = pick_column(result.columns, candidates)
        if probe_col is not None:
            class_probe_values = class_probe_values | pd.
            ↪ to_numeric(result[probe_col], errors="coerce").notna()

    primary_text = result.get("primary_metric", pd.Series("", index=result.
    ↪ index)).fillna("").astype(str).str.lower()
    looks_classification = (

```

```

        primary_text.str.
        ↳contains("auc|accuracy|mcc|f1|auprc|average_precision", regex=True)
            | class_probe_values
    )

    result["task_kind"] = np.where(rmse_values.notna() & ~looks_classification,
    ↳"regression", np.where(looks_classification, "classification", "unknown"))
    result["analysis_metric"] = rmse_col or "rmse"
    result["analysis_metric_kind"] = "rmse"
    result["analysis_metric_value"] = rmse_values
    result["analysis_metric_direction"] = np.where(result["task_kind"].
    ↳eq("classification"), "higher", "lower")

    class_mask = result["task_kind"].eq("classification")
    for row_index, row in result.loc[class_mask].iterrows():
        metric_col, metric_kind = classification_metric_column_for_row(row,
    ↳result.columns, fallback_col=classification_col)
        result.loc[row_index, "analysis_metric"] = metric_col or
    ↳"classification_metric"
        result.loc[row_index, "analysis_metric_kind"] = metric_kind
        if metric_col in result.columns:
            result.loc[row_index, "analysis_metric_value"] = pd.
    ↳to_numeric(result.loc[row_index, metric_col], errors="coerce")
        else:
            result.loc[row_index, "analysis_metric_value"] = np.nan

    result["analysis_metric_value"] = pd.
    ↳to_numeric(result["analysis_metric_value"], errors="coerce")
    result["analysis_delta_from_best"] = np.nan

    for dataset_name, idx in result.groupby("dataset", dropna=False).groups.
    ↳items():
        local = result.loc[idx]
        values = pd.to_numeric(local["analysis_metric_value"], errors="coerce")
        valid = values.notna()
        if not valid.any():
            continue
        direction = str(local.loc[valid, "analysis_metric_direction"].mode().
    ↳iloc[0])
        best_value = values[valid].min() if direction == "lower" else
    ↳values[valid].max()
        if direction == "lower":
            result.loc[idx, "analysis_delta_from_best"] = values - best_value
        else:
            result.loc[idx, "analysis_delta_from_best"] = best_value - values

```

```

return result

def best_rows_by_dataset(frame):
    if frame is None or frame.empty:
        return pd.DataFrame()
    work = frame.copy()
    if "analysis_metric_value" not in work.columns:
        work = add_analysis_metric_columns(work)
    work = work.loc[pd.to_numeric(work["analysis_metric_value"],
↪errors="coerce").notna()].copy()
    if work.empty:
        return work
    work["_analysis_sort_value"] = np.where(
        work["analysis_metric_direction"].astype(str).eq("higher"),
        -pd.to_numeric(work["analysis_metric_value"], errors="coerce"),
        pd.to_numeric(work["analysis_metric_value"], errors="coerce"),
    )
    return (
        work.sort_values(["dataset", "_analysis_sort_value", "model"],
↪ascending=[True, True, True])
        .groupby("dataset", as_index=False, dropna=False)
        .first()
        .drop(columns=["_analysis_sort_value"], errors="ignore")
    )

def standardize_metric_frame(metric_df):
    if metric_df is None:
        return None
    frame = metric_df.copy()
    rename_map = {}
    dataset_col = pick_column(frame.columns, ["dataset", "dataset_id",
↪"dataset_name"])
    model_col = pick_column(frame.columns, ["model", "Model"])
    workflow_col = pick_column(frame.columns, ["workflow", "Workflow"])
    if dataset_col is not None:
        rename_map[dataset_col] = "dataset"
    if model_col is not None:
        rename_map[model_col] = "model"
    if workflow_col is not None:
        rename_map[workflow_col] = "workflow"
    frame = frame.rename(columns=rename_map)
    return frame

def resolve_repo_file(relative_path):

```

```

"""Find a repo-relative file whether the notebook is run from repo root or
↳ bundle dir."""
relative_path = Path(relative_path)
for base in [Path(".").resolve(), *Path(".").resolve().parents]:
    candidate = base / relative_path
    if candidate.exists():
        return candidate
return relative_path

def normalize_reference_dataset_name(dataset_name, benchmark_suite="tdc"):
    text = str(dataset_name or "").strip()
    if not text:
        return ""
    lowered = text.lower()
    if lowered.startswith(("tdc_", "polaris_", "poduam_", "chemml_")):
        return lowered
    if lowered in {"esol_delaney", "freesolv_sampl", "lipophilicity"}:
        return lowered
    suite = str(benchmark_suite or "").strip().lower()
    if suite in {"tdc", "adme", "tox", "absorption", "distribution",
↳ "metabolism", "excretion", "toxicity"}:
        return f"tdc_{lowered}"
    return lowered

def clean_reference_text(value):
    if value is None or (isinstance(value, float) and math.isnan(value)):
        return ""
    text = str(value).strip()
    return text.replace("\\u00c2\\u00b1", "+/-").replace("\\u00b1", "+/-")

def format_reference_metric_value(mean_value, std_value=""):
    mean_text = clean_reference_text(mean_value)
    std_text = clean_reference_text(std_value)
    if std_text:
        return f"{mean_text} +/- {std_text}"
    return mean_text

def load_manual_benchmark_reference(path="data/benchmark_leaderboards/
↳ TDC_ADMET_Benchmark_Performance_by_Model.csv"):
    path = resolve_repo_file(path)
    frame = read_optional_csv(path)
    if frame is None or frame.empty:
        return pd.DataFrame()

```



```

required = {"Dataset", "Metric", "Direction", "Model", "Score_Mean"}
missing = sorted(required - set(frame.columns))
if missing:
    raise ValueError(f"Manual benchmark reference is missing required_
↳columns: {missing}")

work = frame.copy()
work["Score_Mean"] = pd.to_numeric(work["Score_Mean"], errors="coerce")
if "Score_StdDev" in work.columns:
    work["Score_StdDev"] = pd.to_numeric(work["Score_StdDev"],
↳errors="coerce")
else:
    work["Score_StdDev"] = np.nan

rows = []
captured_at = pd.Timestamp(path.stat().st_mtime, unit="s").
↳strftime("%Y-%m-%d") if path.exists() else ""
for _, item in work.iterrows():
    benchmark_id = clean_reference_text(item.get("Dataset"))
    task_category = clean_reference_text(item.get("Task_Category"))
    dataset_name = normalize_reference_dataset_name(benchmark_id, "tdc")
    score_mean = item.get("Score_Mean", np.nan)
    score_std = item.get("Score_StdDev", np.nan)
    if pd.isna(score_mean):
        continue
    rank_text = clean_reference_text(item.get("TDC_Rank"))
    rows.append(
        {
            "dataset": dataset_name,
            "dataset_source": f"Manual benchmark reference: {benchmark_id}",
            "benchmark_suite": "tdc",
            "benchmark_id": benchmark_id,
            "task_category": task_category,
            "leaderboard_url": clean_reference_text(item.get("DOI_or_URL")),
            "leaderboard_dataset_split": "reported in source literature/TDC_
↳reference table",
            "leaderboard_metric_name": clean_reference_text(item.
↳get("Metric")),
            "leaderboard_metric_direction": clean_reference_text(item.
↳get("Direction")),
            "rank": rank_text,
            "rank_numeric": pd.to_numeric(rank_text, errors="coerce"),
            "model": clean_reference_text(item.get("Model")),
            "metric_value": format_reference_metric_value(score_mean,
↳score_std if pd.notna(score_std) else ""),

```

```

        "metric_value_numeric": float(score_mean),
        "is_top10_entry": True,
        "reference_source": "manual_tdc_admet_reference",
        "captured_at": captured_at,
        "paper_short": clean_reference_text(item.get("Paper_Short")),
        "year": clean_reference_text(item.get("Year")),
        "reference": clean_reference_text(item.get("Reference")),
        "doi_or_url": clean_reference_text(item.get("DOI_or_URL")),
        "manual_reference_source_file": str(path).replace("\\", "/"),
    }
)

result = pd.DataFrame(rows)
if result.empty:
    return result

result["rank_numeric"] = pd.to_numeric(result["rank_numeric"],
errors="coerce")
for (_dataset, _metric), idx in result.groupby(["dataset",
leaderboard_metric_name"], dropna=False).groups.items():
    local = result.loc[idx].copy()
    direction = str(local["leaderboard_metric_direction"].dropna().
astype(str).iloc[0]).strip().lower()
    ascending = not direction.startswith("higher")
    local = local.sort_values(["metric_value_numeric", "model"],
ascending=[ascending, True])
    missing_rank = result.loc[local.index, "rank_numeric"].isna()
    if bool(missing_rank.any()):
        result.loc[local.index[missing_rank], "rank_numeric"] = np.
arange(1, int(missing_rank.sum()) + 1)
        result.loc[local.index[missing_rank], "rank"] = result.loc[local.
index[missing_rank], "rank_numeric"].astype(int).astype(str)
    return result

def combine_leaderboard_reference_frames(*frames):
    parts = []
    for frame in frames:
        if frame is None or frame.empty:
            continue
        work = frame.copy()
        if "dataset" in work.columns:
            work["dataset"] = work["dataset"].astype(str).str.strip()
        if "benchmark_id" in work.columns:
            work["benchmark_id"] = work["benchmark_id"].astype(str).str.strip()
        if "metric_value_numeric" in work.columns:

```

```

        work["metric_value_numeric"] = pd.
↳to_numeric(work["metric_value_numeric"], errors="coerce")
        if "rank_numeric" in work.columns:
            work["rank_numeric"] = pd.to_numeric(work["rank_numeric"],
↳errors="coerce")
        if "is_top10_entry" not in work.columns:
            work["is_top10_entry"] = True
        parts.append(work)
    if not parts:
        return pd.DataFrame()
    combined = pd.concat(parts, ignore_index=True, sort=False)
    subset_cols = [c for c in ["dataset", "benchmark_id",
↳"leaderboard_metric_name", "model", "metric_value_numeric",
↳"reference_source", "paper_short", "doi_or_url"] if c in combined.columns]
    if subset_cols:
        combined = combined.drop_duplicates(subset=subset_cols, keep="first")
    return combined.reset_index(drop=True)

```

```

[55]: RUN_DIR = resolve_benchmark_dir(benchmark_run_dir)
SUMMARY_METRICS_FILE = standardize_metric_frame(read_optional_csv(RUN_DIR /
↳"summary_metrics.csv"))
TEST_RMSE_PIVOT = read_optional_csv(RUN_DIR / "test_rmse_pivot.csv")
PREDICTIONS = read_optional_csv(RUN_DIR / "predictions.csv")
GA_HISTORY = read_optional_csv(RUN_DIR / "ga_history.csv")
RUN_CONFIG = read_optional_json(RUN_DIR / "run_config.json")
LEADERBOARD_TOP10 = read_optional_csv(RUN_DIR / "leaderboard_top10_reference.
↳csv")
LEADERBOARD_COMPARISON = read_optional_csv(RUN_DIR /
↳"leaderboard_comparison_by_dataset.csv")
MANUAL_LEADERBOARD_REFERENCE = load_manual_benchmark_reference()
REFERENCE_LEADERBOARD_ROWS =
↳combine_leaderboard_reference_frames(LEADERBOARD_TOP10,
↳MANUAL_LEADERBOARD_REFERENCE)

run_status_rows = []
for status_path in sorted(RUN_DIR.glob("*/run_status.json")):
    payload = read_optional_json(status_path)
    if not isinstance(payload, dict):
        continue
    run_status_rows.append(
        {
            "dataset": status_path.parent.name,
            "status": payload.get("status", "unknown"),
            "checkpoint_stage": payload.get("checkpoint_stage", ""),
            "n_metrics_rows": payload.get("n_metrics_rows", np.nan),
            "started_at": payload.get("started_at", ""),

```

```

        "completed_at": payload.get("completed_at", ""),
        "elapsed_seconds": payload.get("elapsed_seconds", np.nan),
    }
)
RUN_STATUS = pd.DataFrame(run_status_rows)

def _row_has_error_text_series(series):
    text = series.fillna("").astype(str).str.strip().str.lower()
    return text.ne("") & ~text.isin({"nan", "none"})

dataset_metric_tables = []
for metrics_path in sorted(RUN_DIR.glob("*/metrics.csv")):
    dataset_name = metrics_path.parent.name
    try:
        frame = pd.read_csv(metrics_path)
    except Exception:
        continue
    frame = standardize_metric_frame(frame)
    if frame is None or frame.empty:
        continue
    if "dataset" not in frame.columns:
        frame["dataset"] = dataset_name
    dataset_metric_tables.append(frame)

combined_metric_frames = []
if SUMMARY_METRICS_FILE is not None and not SUMMARY_METRICS_FILE.empty:
    summary_frame = SUMMARY_METRICS_FILE.copy()
    if "dataset" not in summary_frame.columns:
        raise ValueError("summary_metrics.csv is missing a dataset column.")
    summary_frame["_metrics_source"] = "summary_metrics_csv"
    combined_metric_frames.append(summary_frame)

if dataset_metric_tables:
    dataset_frame = pd.concat(dataset_metric_tables, ignore_index=True)
    dataset_frame["_metrics_source"] = "dataset_metrics"
    combined_metric_frames.append(dataset_frame)

if not combined_metric_frames:
    raise ValueError(f"No metrics tables were found under {RUN_DIR}")

combined_metrics = pd.concat(combined_metric_frames, ignore_index=True)
combined_metrics = standardize_metric_frame(combined_metrics)

if combined_metrics is None or combined_metrics.empty:
    raise ValueError(f"No metrics tables were found under {RUN_DIR}")
if "dataset" not in combined_metrics.columns:

```

```

        raise ValueError("Could not identify a dataset column in the metrics table.
        ↪")
    if "model" not in combined_metrics.columns:
        raise ValueError("Could not identify a model column in the metrics table.")

    if "workflow" not in combined_metrics.columns:
        combined_metrics["workflow"] = ""

    combined_metrics["_source_rank"] = combined_metrics.get("_metrics_source", "").
    ↪map(
        {"dataset_metrics": 0, "summary_metrics_csv": 1}
    ).fillna(9)
    combined_metrics["_error_rank"] = _row_has_error_text_series(combined_metrics.
    ↪get("error", pd.Series(index=combined_metrics.index, dtype=object))).
    ↪astype(int)
    status_series = combined_metrics.get("status", pd.Series(index=combined_metrics.
    ↪index, dtype=object)).fillna("").astype(str).str.strip().str.lower()
    combined_metrics["_status_rank"] = np.where(status_series.str.
    ↪startswith("skipped") | status_series.str.contains("error", regex=False), 1,
    ↪0)

    _rmse_probe_col = metric_column(combined_metrics,
    ↪rmse_metric_column_preference, metric_hint="rmse")
    if _rmse_probe_col is not None:
        combined_metrics["_rmse_rank"] = pd.
        ↪to_numeric(combined_metrics[_rmse_probe_col], errors="coerce")
    else:
        combined_metrics["_rmse_rank"] = np.nan
    combined_metrics["_rmse_rank"] = combined_metrics["_rmse_rank"].fillna(np.inf)

    key_cols = ["dataset", "model", "workflow"]
    combined_metrics = combined_metrics.sort_values(
        by=key_cols + ["_error_rank", "_status_rank", "_rmse_rank", "_source_rank"],
        ascending=[True, True, True, True, True, True, True],
    )
    SUMMARY_METRICS = combined_metrics.groupby(key_cols, as_index=False,
    ↪dropna=False).first()
    SUMMARY_METRICS = SUMMARY_METRICS.drop(columns=["_source_rank", "_error_rank",
    ↪"_status_rank", "_rmse_rank", "_metrics_source"], errors="ignore")

    if SUMMARY_METRICS is None or SUMMARY_METRICS.empty:
        raise ValueError(f"No metrics tables were found under {RUN_DIR}")

    if "dataset" not in SUMMARY_METRICS.columns:
        raise ValueError("Could not identify a dataset column in the metrics table.
        ↪")

```

```

if "model" not in SUMMARY_METRICS.columns:
    raise ValueError("Could not identify a model column in the metrics table.")

if {"model", "workflow"}.issubset(set(SUMMARY_METRICS.columns)):
    tabpfn_mask = SUMMARY_METRICS["model"].astype(str).str.contains("tabpfn",
    ↪case=False, na=False)
    SUMMARY_METRICS.loc[tabpfn_mask, "workflow"] = "TabPFN deep learning"
    cfa_mask = SUMMARY_METRICS["model"].astype(str).str.startswith("CFA (",
    ↪na=False)
    SUMMARY_METRICS.loc[cfa_mask, "workflow"] = "CFA fusion"

RMSE_COL = metric_column(SUMMARY_METRICS, rmse_metric_column_preference,
    ↪metric_hint="rmse")
CLASSIFICATION_FALLBACK_COL = classification_metric_column(SUMMARY_METRICS)
# Backward-compatible name used by older display cells; row-specific primary
# classification metrics are stored in analysis_metric/analysis_metric_kind.
BALANCED_ACCURACY_COL = CLASSIFICATION_FALLBACK_COL
R2_COL = metric_column(SUMMARY_METRICS, "AUTO", metric_hint="r2")
SUMMARY_ANALYSIS_METRICS = add_analysis_metric_columns(SUMMARY_METRICS,
    ↪rmse_col=RMSE_COL, classification_col=CLASSIFICATION_FALLBACK_COL)
SUMMARY_RMSE_PIVOT = (
    SUMMARY_METRICS.pivot_table(index="dataset", columns="model",
    ↪values=RMSE_COL, aggfunc="mean").reset_index()
    if RMSE_COL is not None
    else pd.DataFrame()
)

print(f"Benchmark run directory: {RUN_DIR}")
print(f"Datasets: {SUMMARY_METRICS['dataset'].unique()}")
print(f"Models: {SUMMARY_METRICS['model'].unique()}")
print(f"Rows in merged summary table: {len(SUMMARY_METRICS):,}")
if SUMMARY_METRICS_FILE is not None:
    print(f"Rows in summary_metrics.csv: {len(SUMMARY_METRICS_FILE):,}")
print(f"Rows from dataset-level metrics after dedupe: {len(SUMMARY_METRICS):,}")
print(f"Regression metric column used: {RMSE_COL}")
print(f"Classification fallback metric column used:
    ↪{CLASSIFICATION_FALLBACK_COL}")
if not SUMMARY_ANALYSIS_METRICS.empty and "analysis_metric_kind" in
    ↪SUMMARY_ANALYSIS_METRICS.columns:
    classification_primary_summary = (
        SUMMARY_ANALYSIS_METRICS.loc[SUMMARY_ANALYSIS_METRICS["task_kind"]
    ↪eq("classification")]
        .groupby(["analysis_metric_kind", "analysis_metric"],
    ↪dropna=False)["dataset"]
        .nunique()
        .reset_index(name="datasets")

```

```

        .sort_values(["analysis_metric_kind", "analysis_metric"])
    )
    if not classification_primary_summary.empty:
        print("Classification primary metrics by dataset count:")
        display(classification_primary_summary.reset_index(drop=True))
if R2_COL is not None:
    print(f"R2 column used: {R2_COL}")
if not SUMMARY_ANALYSIS_METRICS.empty:
    task_count_source = SUMMARY_ANALYSIS_METRICS.copy()
    task_error_col = pick_column(task_count_source.columns, ["error", "Error"])
    if task_error_col is not None:
        task_count_source = task_count_source.loc[
            task_count_source[task_error_col].isna()
            | (task_count_source[task_error_col].astype(str).str.strip() == "")
        ].copy()
    task_count_source = task_count_source.
    ↪loc[task_count_source["analysis_metric_value"].notna()].copy()
    task_counts = task_count_source.groupby("task_kind",
    ↪dropna=False)["dataset"].nunique().to_dict()
    print(f"Datasets by inferred task kind with successful metrics:
    ↪{task_counts}")

if "model" in SUMMARY_METRICS.columns:
    tabpfn_rows = SUMMARY_METRICS[SUMMARY_METRICS["model"].astype(str).str.
    ↪contains("TabPFNRegressor", na=False)]
    if not tabpfn_rows.empty:
        print(
            "TabPFNRegressor rows retained for analysis: "
            f"{len(tabpfn_rows)} rows across {tabpfn_rows['dataset'].nunique()}
    ↪datasets."
        )

if RUN_CONFIG is not None:
    display_note("Loaded `run_config.json` for this benchmark run.")

if not RUN_STATUS.empty:
    completed = int((RUN_STATUS["status"].astype(str).str.lower() ==
    ↪"completed").sum())
    print(f"Datasets with run_status.json: {len(RUN_STATUS)} (completed:
    ↪{completed})")

if LEADERBOARD_TOP10 is not None:
    print(f"Leaderboard top-10 reference rows from run artifact:
    ↪{len(LEADERBOARD_TOP10):,}")
if MANUAL_LEADERBOARD_REFERENCE is not None and not
    ↪MANUAL_LEADERBOARD_REFERENCE.empty:

```

```

print(
    "Manual benchmark reference rows loaded: "
    f"{len(MANUAL_LEADERBOARD_REFERENCE):,} across_
↳{MANUAL_LEADERBOARD_REFERENCE['dataset'].nunique()} datasets"
)
if REFERENCE_LEADERBOARD_ROWS is not None and not REFERENCE_LEADERBOARD_ROWS.
↳empty:
    print(
        "Combined leaderboard/reference rows available for comparison: "
        f"{len(REFERENCE_LEADERBOARD_ROWS):,} across_
↳{REFERENCE_LEADERBOARD_ROWS['dataset'].nunique()} datasets"
    )
if LEADERBOARD_COMPARISON is not None:
    print(f"Precomputed leaderboard comparison rows from run artifact:_
↳{len(LEADERBOARD_COMPARISON):,}")

```

C:\Users\scott\AppData\Local\Temp\ipykernel_7244\2053725435.py:58:

PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance. Consider joining all columns at once using `pd.concat(axis=1)` instead. To get a de-fragmented frame, use `newframe = frame.copy()`

```
dataset_frame["_metrics_source"] = "dataset_metrics"
```

C:\Users\scott\AppData\Local\Temp\ipykernel_7244\2053725435.py:96:

PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance. Consider joining all columns at once using `pd.concat(axis=1)` instead. To get a de-fragmented frame, use `newframe = frame.copy()`

```
SUMMARY_METRICS = combined_metrics.groupby(key_cols, as_index=False,
dropna=False).first()
```

Benchmark run directory:

C:\Users\scott\AutoQSAR\benchmark_results\benchmark_name_date

Datasets: 34

Models: 26

Rows in merged summary table: 706

Rows from dataset-level metrics after dedupe: 706

Regression metric column used: test_rmse

Classification fallback metric column used: test_roc_auc

Classification primary metrics by dataset count:

	analysis_metric_kind	analysis_metric	datasets	
0		auprc	test_auprc	6
1		roc_auc	test_roc_auc	16

R2 column used: test_r2

Datasets by inferred task kind with successful metrics: {'classification': 14, 'regression': 20}

TabPFNRegressor rows retained for analysis: 24 rows across 24 datasets.

Loaded run_config.json for this benchmark run.

Datasets with run_status.json: 45 (completed: 44)

Leaderboard top-10 reference rows from run artifact: 148

Manual benchmark reference rows loaded: 190 across 28 datasets

Combined leaderboard/reference rows available for comparison: 377 across 37 datasets

```
[56]: config_summary = pd.DataFrame([RUN_CONFIG]) if isinstance(RUN_CONFIG, dict)
      ↪ else pd.DataFrame()
      if not config_summary.empty:
          display(config_summary.T.rename(columns={0: "value"}))

      analysis_metrics = SUMMARY_ANALYSIS_METRICS.copy()
      metric_error_col = pick_column(analysis_metrics.columns, ["error", "Error"])
      if metric_error_col is not None:
          analysis_metrics = analysis_metrics.loc[
              analysis_metrics[metric_error_col].isna()
              | (analysis_metrics[metric_error_col].astype(str).str.strip() == "")
          ].copy()
      analysis_metrics = add_analysis_metric_columns(analysis_metrics,
      ↪ rmse_col=RMSE_COL, classification_col=BALANCED_ACCURACY_COL)
      analysis_metrics = analysis_metrics.
      ↪ loc[analysis_metrics["analysis_metric_value"].notna()].copy()

      best_by_dataset = best_rows_by_dataset(analysis_metrics)
      best_display_cols = [
          col
          for col in [
              "dataset",
              "task_kind",
              "workflow",
              "model",
              "analysis_metric",
              "analysis_metric_kind",
              "analysis_metric_value",
              "analysis_delta_from_best",
              RMSE_COL,
              "test_roc_auc",
              "test_auprc",
              "test_balanced_accuracy",
              "test_mcc",
              R2_COL,
          ]
          if col is not None and col in best_by_dataset.columns
      ]
      display_note("### Best model per dataset")
```

```

display(best_by_dataset[best_display_cols].sort_values(["task_kind",
↳ "dataset"]).reset_index(drop=True))

summary_rows = []
group_cols = [col for col in ["workflow", "model"] if col in analysis_metrics.
↳ columns]
for keys, group in analysis_metrics.groupby(group_cols, dropna=False):
    if not isinstance(keys, tuple):
        keys = (keys,)
    row = dict(zip(group_cols, keys))
    row.update(
        {
            "datasets": int(group["dataset"].nunique()),
            "regression_datasets": int(group.loc[group["task_kind"]
↳ eq("regression"), "dataset"].nunique()),
            "classification_datasets": int(group.loc[group["task_kind"]
↳ eq("classification"), "dataset"].nunique()),
            "mean_delta_from_dataset_best":
↳ float(group["analysis_delta_from_best"].mean(skipna=True)),
            "median_delta_from_dataset_best":
↳ float(group["analysis_delta_from_best"].median(skipna=True)),
            "mean_rmse": float(pd.to_numeric(group[RMSE_COL], errors="coerce").
↳ mean(skipna=True)) if RMSE_COL in group.columns else np.nan,
            "mean_classification_primary_metric": float(pd.to_numeric(group.
↳ loc[group["task_kind"].eq("classification"), "analysis_metric_value"],
↳ errors="coerce").mean(skipna=True)),
            "mean_test_roc_auc": float(pd.to_numeric(group["test_roc_auc"],
↳ errors="coerce").mean(skipna=True)) if "test_roc_auc" in group.columns else
↳ np.nan,
            "mean_test_auprc": float(pd.to_numeric(group["test_auprc"],
↳ errors="coerce").mean(skipna=True)) if "test_auprc" in group.columns else
↳ np.nan,
            "mean_balanced_accuracy": float(pd.
↳ to_numeric(group["test_balanced_accuracy"], errors="coerce").
↳ mean(skipna=True)) if "test_balanced_accuracy" in group.columns else np.nan,
            "mean_mcc": float(pd.to_numeric(group["test_mcc"], errors="coerce").
↳ mean(skipna=True)) if "test_mcc" in group.columns else np.nan,
        }
    )
    summary_rows.append(row)

model_summary = pd.DataFrame(summary_rows)
display_note("### Cross-dataset model summary")
if model_summary.empty:
    display_note("No successful model rows with usable regression/
↳ classification metrics were available.")

```

```

else:
    display(model_summary.sort_values("mean_delta_from_dataset_best",
    ↪ascending=True).head(int(show_top_n_models)).reset_index(drop=True))

regression_best = best_by_dataset.loc[best_by_dataset["task_kind"].
    ↪eq("regression")].copy()
if not regression_best.empty:
    display_note("### Regression winner diagnostics")
    display(
        regression_best.groupby(["model", "workflow"], dropna=False)
        .agg(
            best_dataset_count=("dataset", "nunique"),
            median_best_rmse=(RMSE_COL, "median") if RMSE_COL in
    ↪regression_best.columns else ("analysis_metric_value", "median"),
            mean_best_rmse=(RMSE_COL, "mean") if RMSE_COL in regression_best.
    ↪columns else ("analysis_metric_value", "mean"),
        )
        .reset_index()
        .sort_values(["best_dataset_count", "median_best_rmse", "model"],
    ↪ascending=[False, True, True])
        .reset_index(drop=True)
    )

    regression_cols = [
        col
        for col in [
            "dataset",
            "model",
            "workflow",
            RMSE_COL,
            R2_COL,
            "analysis_delta_from_best",
            "elapsed_seconds",
        ]
        if col is not None and col in analysis_metrics.columns
    ]
    display_note("Regression dataset-level winners")
    display(regression_best[regression_cols].sort_values(["dataset", "model"]).
    ↪reset_index(drop=True))

    regression_all = analysis_metrics.loc[analysis_metrics["task_kind"].
    ↪eq("regression")].copy()
    if not regression_all.empty:

```

```

        regression_all["rmse_rank_within_dataset"] = regression_all.
↳groupby("dataset")[RMSE_COL].rank(method="min", ascending=True) if RMSE_COL_
↳in regression_all.columns else regression_all.
↳groupby("dataset")["analysis_metric_value"].rank(method="min",_
↳ascending=True)
        regression_detail_cols = [
            col
            for col in [
                "dataset",
                "model",
                "workflow",
                RMSE_COL,
                R2_COL,
                "rmse_rank_within_dataset",
                "analysis_delta_from_best",
                "elapsed_seconds",
            ]
            if col is not None and col in regression_all.columns
        ]
        display_note("Regression model rankings by dataset")
        display(regression_all[regression_detail_cols].sort_values(["dataset",_
↳"rmse_rank_within_dataset", "model"]).reset_index(drop=True))

classification_best = best_by_dataset.loc[best_by_dataset["task_kind"] .
↳eq("classification")].copy()
if not classification_best.empty:
    display_note("### Classification winner diagnostics")
    display(
        classification_best.groupby(["model", "workflow"], dropna=False)
        .agg(best_dataset_count=("dataset", "nunique"))
        .reset_index()
        .sort_values(["best_dataset_count", "model"], ascending=[False, True])
        .reset_index(drop=True)
    )

    adaboost_best_datasets = classification_best.
↳loc[classification_best["model"].astype(str).str.lower().eq("adaboost"),_
↳"dataset"].tolist()
    if adaboost_best_datasets:
        ada_cols = [
            col
            for col in [
                "dataset",
                "model",
                "workflow",
                "analysis_metric",
                "analysis_metric_kind",

```

```

        "analysis_metric_value",
        "test_roc_auc",
        "cv_roc_auc",
        "test_auprc",
        "test_balanced_accuracy",
        "cv_balanced_accuracy",
        "test_mcc",
        "analysis_delta_from_best",
        "elapsed_seconds",
    ]
    if col in analysis_metrics.columns
]
display_note(
    "AdaBoost is only counted as best where it has the highest held-out
    ↳primary classification metric for that dataset. "
    "CYP leaderboard tasks use AUPRC where designated; other
    ↳classification tasks use their declared primary metric or AUROC fallback."
)
display(
    analysis_metrics.loc[
        analysis_metrics["dataset"].isin(adaboost_best_datasets)
        & analysis_metrics["task_kind"].eq("classification"),
        ada_cols,
    ]
    .sort_values(["dataset", "analysis_delta_from_best", "model"])
    .reset_index(drop=True)
)
else:
    display_note(
        "AdaBoost is not the best model for any successful classification
        ↳dataset after ranking by the dataset-specific primary classification metric;
        ↳"
        "RMSE-only ranking would have been an artifact because
        ↳classification RMSE values are missing."
    )

```

	value
dataset	None
refresh_leaderboards_only	False
dataset_name	None
include_local_csv	None
output_dir	benchmark_results\all_benchmarks_no_unimol_202...
...	...
ga_models_resolved	
ga_resolution	{'mode': 'disabled', 'reason': 'empty_ga_models'}
default_feature_families	[morgan, ecfp6, fcfp6, layered, atom_pair, top...
config_signature	{"benchmark_profile": "cost_optimized", "cfa_i...

```
datasets          [{'name': 'tdc_carcinogens_lagunin', 'source':...
```

```
[121 rows x 1 columns]
```

```
C:\Users\scott\AppData\Local\Temp\ipykernel_7244\1832174106.py:245:
```

```
PerformanceWarning: DataFrame is highly fragmented.  This is usually the result  
of calling `frame.insert` many times, which has poor performance.  Consider  
joining all columns at once using `pd.concat(axis=1)` instead.  To get a de-  
fragmented frame, use `newframe = frame.copy()`  
    .first()
```

1.0.1 Best model per dataset

	dataset	task_kind	workflow	
		model analysis_metric		
	analysis_metric_value	analysis_delta_from_best		
0	tdc_ames	classification	ensemble	
	Ensemble (OOF Stacking (RidgeCV, 5-fold))	test_roc_auc		0.
	876017	0.0		
1	tdc_bbb_martins	classification	conventional	
	Random forest	test_roc_auc		0.
	932419	0.0		
2	tdc_bioavailability_ma	classification	conventional	
	CatBoost	test_roc_auc		0.
	776688	0.0		
3	tdc_carcinogens_lagunin	classification	conventional	
	AdaBoost	test_roc_auc		0.
	862626	0.0		
4	tdc_clintox	classification	conventional	
	CatBoost	test_roc_auc		0.
	948884	0.0		
5	tdc_cyp1a2_veith	classification	ensemble	
	Ensemble (OOF Stacking (RidgeCV, 5-fold))	test_roc_auc		0.
	951752	0.0		
6	tdc_cyp2c19_veith	classification	ensemble	
	Ensemble (OOF Stacking (RidgeCV, 5-fold))	test_roc_auc		0.
	904288	0.0		
7	tdc_cyp2c9_substrate_carbonmangels	classification	Chemprop v2	
	Chemprop v2 (AttentiveFP, ensemble=1)	test_roc_auc		0.
	653283	0.0		
8	tdc_cyp2c9_veith	classification	ensemble	
	Ensemble (OOF Stacking (RidgeCV, 5-fold))	test_roc_auc		0.
	901544	0.0		
9	tdc_cyp2d6_substrate_carbonmangels	classification	Chemprop v2	
	Chemprop v2 (AttentiveFP, ensemble=1)	test_roc_auc		0.
	800430	0.0		

10	tdc_cyp2d6_veith	classification	ensemble	0.894273	0.0	
	Ensemble (Weighted average (inverse train RMSE))		test_roc_auc			
11	tdc_cyp3a4_substrate_carbonmangels	classification	conventional	0.658793	0.0	
		LogisticRegression	test_roc_auc			
12	tdc_cyp3a4_veith	classification	ensemble	0.909641	0.0	
	Ensemble (OOF Stacking (RidgeCV, 5-fold))		test_roc_auc			
13	tdc_dili	classification	ensemble	0.913043	0.0	
	Ensemble (OOF Stacking (RidgeCV, 5-fold))		test_roc_auc			
14	tdc_herg	classification	conventional	0.860825	0.0	
		AdaBoost	test_roc_auc			
15	tdc_herg_karim	classification	ensemble	0.944478	0.0	
	Ensemble (Weighted average (inverse train RMSE))		test_roc_auc			
16	tdc_hia_hou	classification	CFA fusion	0.989918	0.0	
	CFA (Combinatorial Fusion)		test_roc_auc			
17	tdc_pampa_ncats	classification	ensemble	0.811231	0.0	
	Ensemble (Weighted average (inverse train RMSE))		test_roc_auc			
18	tdc_pgp_broccatelli	classification	conventional	0.924020	0.0	
		HistGradientBoosting	test_roc_auc			
19	tdc_skin_reaction	classification	ensemble	0.774839	0.0	
	Ensemble (Weighted average (inverse train RMSE))		test_roc_auc			
20	tdc_tox21	classification	conventional	0.808239	0.0	
		Tabular MLP	test_roc_auc			
21	tdc_toxcast	classification	conventional	0.791353	0.0	
		CatBoost	test_roc_auc			
22	chemml_cep_homo	regression	TabPFN deep learning	0.079691	0.0	
		TabPFNRegressor	test_rmse			
23	chemml_organic_density	regression	TabPFN deep learning	0.005035	0.0	
		TabPFNRegressor	test_rmse			
24	esol_delaney	regression	TabPFN deep learning	0.621439	0.0	
		TabPFNRegressor	test_rmse			

25	freesolv_sampl	regression	TabPFN deep learning	▮
↳	TabPFNRegressor	test_rmse	0.	
↳982177	0.0			
26	lipophilicity	regression	ensemble	▮
↳	Ensemble (Weighted average (inverse train RMSE))	test_rmse		▮
↳0.594788	0.0			
27	poduam_pod_nc_std	regression	CFA fusion	▮
↳	CFA (Combinatorial Fusion)	test_rmse	0.	
↳699850	0.0			
28	poduam_pod_rd_std	regression	conventional	▮
↳	XGBoost	test_rmse	0.	
↳554603	0.0			
29	polaris_adme_fang_hppb_1	regression	MapLight + GNN	▮
↳	MapLight + GNN (CatBoost, Strict Parity)	test_rmse	0.	
↳446652	0.0			
30	polaris_adme_fang_perm_1	regression	conventional	▮
↳	XGBoost	test_rmse	0.	
↳430300	0.0			
31	polaris_adme_fang_rclint_1	regression	conventional	▮
↳	CatBoost	test_rmse	0.	
↳534193	0.0			
32	polaris_adme_fang_rppb_1	regression	MapLight + GNN	▮
↳	MapLight + GNN (CatBoost, Strict Parity)	test_rmse	0.	
↳493558	0.0			
33	polaris_adme_fang_solu_1	regression	conventional	▮
↳	XGBoost	test_rmse	0.	
↳585986	0.0			
34	tdc_caco2_wang	regression	conventional	▮
↳	MapLight CatBoost (Strict Parity)	test_rmse	0.	
↳349680	0.0			
35	tdc_clearance_hepatocyte_az	regression	MapLight + GNN	▮
↳	MapLight + GNN (CatBoost, Strict Parity)	test_rmse	43.	
↳782188	0.0			
36	tdc_clearance_microsome_az	regression	MapLight + GNN	▮
↳	MapLight + GNN (CatBoost, Strict Parity)	test_rmse	36.	
↳331993	0.0			
37	tdc_half_life_obach	regression	TabPFN deep learning	▮
↳	TabPFNRegressor	test_rmse	17.	
↳210966	0.0			
38	tdc_hydrationfreeenergy_freesolv	regression	CFA fusion	▮
↳	CFA (Combinatorial Fusion)	test_rmse	0.	
↳644869	0.0			
39	tdc_ld50_zhu	regression	CFA fusion	▮
↳	CFA (Combinatorial Fusion)	test_rmse	0.	
↳829832	0.0			

40	tdc_lipophilicity_astazeneca	regression	conventional	␣
↪		XGBoost	test_rmse	0.
↪	644116	0.0		
41	tdc_ppbr_az	regression	MapLight + GNN	␣
↪	MapLight + GNN (CatBoost, Strict Parity)		test_rmse	11.
↪	378376	0.0		
42	tdc_solubility_aqsolddb	regression	CFA fusion	␣
↪	CFA (Combinatorial Fusion)		test_rmse	1.
↪	001087	0.0		
43	tdc_vdss_lombardo	regression	MapLight + GNN	␣
↪	MapLight + GNN (CatBoost, Strict Parity)		test_rmse	4.
↪	684131	0.0		

	test_rmse	test_roc_auc	test_r2
0	NaN	0.876017	NaN
1	NaN	0.932419	NaN
2	NaN	0.776688	NaN
3	NaN	0.862626	NaN
4	NaN	0.948884	NaN
5	NaN	0.951752	NaN
6	NaN	0.904288	NaN
7	NaN	0.653283	NaN
8	NaN	0.901544	NaN
9	NaN	0.800430	NaN
10	NaN	0.894273	NaN
11	NaN	0.658793	NaN
12	NaN	0.909641	NaN
13	NaN	0.913043	NaN
14	NaN	0.860825	NaN
15	NaN	0.944478	NaN
16	NaN	0.989918	NaN
17	NaN	0.811231	NaN
18	NaN	0.924020	NaN
19	NaN	0.774839	NaN
20	NaN	0.808239	NaN
21	NaN	0.791353	NaN
22	0.079691	NaN	0.987207
23	0.005035	NaN	0.974037
24	0.621439	NaN	0.815969
25	0.982177	NaN	0.950669
26	0.594788	NaN	0.774382
27	0.699850	NaN	0.638311
28	0.554603	NaN	0.489268
29	0.446652	NaN	0.643383
30	0.430300	NaN	0.634677
31	0.534193	NaN	0.508198
32	0.493558	NaN	0.563988

33	0.585986	NaN	0.383843
34	0.349680	NaN	0.740450
35	43.782188	NaN	0.168224
36	36.331993	NaN	0.288756
37	17.210966	NaN	0.370888
38	0.644869	NaN	0.975665
39	0.829832	NaN	0.385490
40	0.644116	NaN	0.705482
41	11.378376	NaN	0.436096
42	1.001087	NaN	0.809432
43	4.684131	NaN	0.253772

1.0.2 Cross-dataset model summary

	workflow		model
	↳ datasets regression_datasets classification_datasets		
	↳ mean_delta_from_dataset_best median_delta_from_dataset_best \		
0	TabPFN deep learning		TabPFNClassifier
↳10	0	10	0.043494
↳	0.036254		
1	conventional		Voting Classifier (KNN, SVM)
↳22	0	22	0.044036
↳	0.025171		
2	ensemble		Ensemble (Weighted average (inverse train RMSE))
↳22	9	13	0.044859
↳	0.008326		
3	conventional		LogisticRegression
↳22	0	22	0.053219
↳	0.039402		
4	conventional		SVC
↳22	0	22	0.053429
↳	0.029344		
5	MapLight + GNN		MapLight + GNN (CatBoost, Strict Parity)
↳44	22	22	0.199449
↳	0.046841		
6	conventional		CatBoost
↳44	22	22	0.214296
↳	0.021168		
7	conventional		MapLight CatBoost (Strict Parity)
↳22	22	0	0.276120
↳	0.043543		
8	conventional		XGBoost
↳44	22	22	0.313398
↳	0.017770		
9	conventional		Random forest
↳44	22	22	0.344516
↳	0.031821		

	mean_rmse	mean_balanced_accuracy
0	NaN	0.777952
1	NaN	0.814536
2	4.616541	0.863015
3	NaN	0.805353
4	NaN	0.805143
5	5.883022	0.756991
6	5.991561	0.835837
7	5.861825	NaN
8	6.190355	0.836425
9	6.246482	0.830316

1.0.3 Regression winner diagnostics

			model	workflow	
	best_dataset_count	median_best_rmse	mean_best_rmse		
0			Uni-Mol V1	Uni-Mol	
	5	0.575097	10.919354		
1		MapLight CatBoost (Strict Parity)		conventional	
	5	4.738737	12.345766		
2			TabPFNRegressor	TabPFN deep learning	
	2	0.045402	0.045402		
3		CFA (Combinatorial Fusion)		CFA fusion	
	2	0.628651	0.628651		
4		Chemprop v2 (AttentiveFP, ensemble=1)		Chemprop v2	
	2	0.790779	0.790779		
5			ElasticNetCV	conventional	
	1	0.008000	0.008000		
6			XGBoost	conventional	
	1	0.550777	0.550777		
7		Ensemble (Weighted average (inverse train RMSE))		ensemble	
	1	0.584573	0.584573		
8		Ensemble (OOF Stacking (RidgeCV, 5-fold))		ensemble	
	1	0.610331	0.610331		

Regression dataset-level winners

		dataset				
	model	workflow	test_rmse	test_r2	analysis_delta_from_best	
	elapsed_seconds					
0		chemml_cep_homo				
	TabPFNRegressor	TabPFN deep learning	0.085692	0.985208		
	0.0	47.273				
1		chemml_organic_density				
	TabPFNRegressor	TabPFN deep learning	0.005112	0.973233		
	0.0	99.982				

2	chemml_xyz_polarizability					
↳	ElasticNetCV	conventional	0.008000	0.940867		
↳	0.0	17.450				
3	esol_delaney	Ensemble (OOE Stacking (RidgeCV,				
↳	5-fold))	ensemble	0.610331	0.822921	0.0	
↳	8.113					
4	freesolv_sampl	Chemprop v2 (AttentiveFP,				
↳	ensemble=1)	Chemprop v2	1.079859	0.940368	0.	
↳	0	582.157				
5	lipophilicity	Ensemble (Weighted average (inverse train				
↳	RMSE))	ensemble	0.584573	0.782064	0.0	
↳	12598.116					
6	poduam_pod_nc_std	CFA (Combinatorial				
↳	Fusion)	CFA fusion	0.700876	0.637250	0.0	
↳	5679.306					
7	poduam_pod_rd_std					
↳	XGBoost	conventional	0.550777	0.496291	0.0	
↳	3995.047					
8	polaris_adme_fang_hppb_1	MapLight CatBoost (Strict				
↳	Parity)	conventional	0.456404	0.627641	0.0	
↳	1474.721					
9	polaris_adme_fang_perm_1					
↳	Uni-Mol V1	Uni-Mol	0.399061	0.685796	0.	
↳	0	3729.406				
10	polaris_adme_fang_rclint_1					
↳	Uni-Mol V1	Uni-Mol	0.512297	0.547689	0.	
↳	0	4754.733				
11	polaris_adme_fang_rppb_1	Chemprop v2 (AttentiveFP,				
↳	ensemble=1)	Chemprop v2	0.501699	0.549485	0.	
↳	0	629.352				
12	polaris_adme_fang_solu_1					
↳	Uni-Mol V1	Uni-Mol	0.575097	0.406530	0.	
↳	0	4667.428				
13	tdc_caco2_wang	MapLight CatBoost (Strict				
↳	Parity)	conventional	0.349680	0.740450	0.0	
↳	772.052					
14	tdc_clearance_hepatocyte_az	MapLight CatBoost (Strict				
↳	Parity)	conventional	44.472136	0.141802	0.0	
↳	1975.872					
15	tdc_clearance_microsome_az					
↳	Uni-Mol V1	Uni-Mol	35.888060	0.306031	0.	
↳	0	2832.591				
16	tdc_half_life_obach					
↳	Uni-Mol V1	Uni-Mol	17.222254	0.370062	0.	
↳	0	2434.371				

17	tdc_hydrationfreeenergy_freesolv	CFA (Combinatorial Fusion)	CFA fusion	0.556425	0.981883	0.0
				712.406		
18	tdc_ppbr_az	MapLight CatBoost (Strict Parity)	conventional	11.711870	0.402556	0.0
				6145.508		
19	tdc_vdss_lombardo	MapLight CatBoost (Strict Parity)	conventional	4.738737	0.236272	0.0
				1228.493		

Regression model rankings by dataset

	dataset		model	
	workflow test_rmse test_r2 rmse_rank_within_dataset			
	analysis_delta_from_best elapsed_seconds			
0	chemml_cep_homo		TabPFNRegressor	
	TabPFN deep learning	0.085692 0.985208	1.0	
		0.000000 47.273		
1	chemml_cep_homo		Ensemble (OOF Stacking (RidgeCV, 5-fold))	
	ensemble	0.085936 0.985124	2.0	
		0.000244 7.606		
2	chemml_cep_homo		Ensemble (Weighted average (inverse train RMSE))	
	ensemble	0.096283 0.981326	3.0	
		0.010592 7.788		
3	chemml_cep_homo		CFA (Combinatorial Fusion)	
	CFA fusion	0.102083 0.979008	4.0	
		0.016391 641.965		
4	chemml_cep_homo		HistGradientBoosting	
	conventional	0.103895 0.978256	5.0	
		0.018203 154.079		
..	...		...	
	...	...	...	
	...			
371	tdc_vdss_lombardo		Tabular MLP	
	conventional	8.336177 -1.363454	14.0	
		3.597440 1092.252		
372	tdc_vdss_lombardo		CFA (Combinatorial Fusion)	
	CFA fusion	9.807975 -2.271689	15.0	
		5.069238 9023.651		
373	tdc_vdss_lombardo		ChemML MLP (PyTorch)	
	ChemML deep learning	9.834613 -2.289485	16.0	
		5.095876 1872.755		
374	tdc_vdss_lombardo		Chemprop v2 (AttentiveFP + Selected descriptor...	
	Chemprop v2	10.449268 -2.713513	17.0	
		5.710531 2074.256		
375	tdc_vdss_lombardo		ElasticNetCV	
	conventional	19.991675 -12.592885	18.0	
		15.252938 715.473		

[376 rows x 8 columns]

1.0.4 Classification winner diagnostics

	model	workflow	
↪best_dataset_count			
0 Ensemble (OOF Stacking (RidgeCV, 5-fold))	ensemble		↪
↪ 6			
1 Ensemble (Weighted average (inverse train RMSE))	ensemble		↪
↪ 4			
2 CatBoost	conventional		↪
↪ 3			
3 AdaBoost	conventional		↪
↪ 2			
4 Chemprop v2 (AttentiveFP, ensemble=1)	Chemprop v2		↪
↪ 2			
5 CFA (Combinatorial Fusion)	CFA fusion		↪
↪ 1			
6 HistGradientBoosting	conventional		↪
↪ 1			
7 LogisticRegression	conventional		↪
↪ 1			
8 Random forest	conventional		↪
↪ 1			
9 Tabular MLP	conventional		↪
↪ 1			

AdaBoost is only counted as best where it has the highest held-out primary classification metric for that dataset. CYP leaderboard tasks use AUPRC where designated; other classification tasks use their declared primary metric or AUROC fallback.

	dataset	model	
↪	workflow test_balanced_accuracy cv_balanced_accuracy		↪
↪test_roc_auc cv_roc_auc test_auprc test_mcc \			
0 tdc_carcinogens_lagunin	AdaBoost		↪
↪ conventional	0.681818	0.786341	0.
↪862626 0.914248 0.703605 0.560968			
1 tdc_carcinogens_lagunin	XGBoost		↪
↪ conventional	0.693939	0.820574	0.
↪860606 0.930251 0.636400 0.440386			
2 tdc_carcinogens_lagunin Chemprop v2 (AttentiveFP + Selected descriptor...			↪
↪ Chemprop v2	0.750505	NaN	0.832323
↪ NaN 0.708567 0.568831			
3 tdc_carcinogens_lagunin	LogisticRegression		↪
↪ conventional	0.750505	0.904158	0.
↪830303 0.994993 0.643853 0.568831			

4	tdc_carcinogens_lagunin				CatBoost	␣
↪	conventional		0.716162	0.789118	0.	
↪	822222	0.948469	0.610155	0.555329		
5	tdc_carcinogens_lagunin				TabPFNClassifier	␣
↪	TabPFN deep learning		0.728283	0.901883	0.	
↪	818182	0.988253	0.622381	0.473617		
6	tdc_carcinogens_lagunin				HistGradientBoosting	␣
↪	conventional		0.648485	0.836434	0.	
↪	806061	0.924145	0.598551	0.356753		
7	tdc_carcinogens_lagunin				Chemprop v2 (AttentiveFP, ensemble=1)	␣
↪	Chemprop v2		0.727273	NaN	0.	
↪	787879	NaN	0.649915	0.633300		
8	tdc_carcinogens_lagunin				Extra trees	␣
↪	conventional		0.716162	0.793713	0.	
↪	777778	0.949620	0.522942	0.555329		
9	tdc_carcinogens_lagunin				Tabular MLP	␣
↪	conventional		0.705051	0.946380	0.	
↪	769697	0.974935	0.607702	0.492659		
10	tdc_carcinogens_lagunin				CFA (Combinatorial Fusion)	␣
↪	CFA fusion		0.500000	NaN	0.	
↪	767677	NaN	0.603564	0.000000		
11	tdc_carcinogens_lagunin				Ensemble (OOF Stacking (RidgeCV, 5-fold))	␣
↪	ensemble		0.705051	NaN	0.	
↪	763636	NaN	0.656097	0.492659		
12	tdc_carcinogens_lagunin				Random forest	␣
↪	conventional		0.716162	0.799193	0.	
↪	760606	0.967686	0.649543	0.555329		
13	tdc_carcinogens_lagunin				Voting Classifier (KNN, SVM)	␣
↪	conventional		0.761616	0.863087	0.	
↪	759596	0.990017	0.584645	0.628565		
14	tdc_carcinogens_lagunin				SVC	␣
↪	conventional		0.728283	0.957915	0.	
↪	757576	0.992044	0.574904	0.473617		
15	tdc_carcinogens_lagunin				MapLight + GNN (CatBoost, Strict Parity)	␣
↪	MapLight + GNN		0.716162	NaN	0.	
↪	716162	NaN	0.485931	0.555329		
16	tdc_carcinogens_lagunin				ChemML MLP (PyTorch)	␣
↪	ChemML deep learning		0.554545	NaN	0.	
↪	690909	NaN	0.510733	0.113165		
17	tdc_herg				AdaBoost	␣
↪	conventional		0.684536	0.765018	0.	
↪	860825	0.871060	0.948878	0.486360		
18	tdc_herg				Chemprop v2 (AttentiveFP, ensemble=1)	␣
↪	Chemprop v2		0.717084	NaN	0.	
↪	855376	NaN	0.933721	0.514245		

19	tdc_herg				Random forest	␣
↪	conventional		0.703976		0.836253	0.
↪	809573	0.926061	0.895213	0.537597		
20	tdc_herg				CatBoost	␣
↪	conventional		0.670250		0.831593	0.
↪	798969	0.929168	0.912551	0.460520		
21	tdc_herg				XGBoost	␣
↪	conventional		0.635346		0.827852	0.
↪	792489	0.915016	0.907321	0.340392		
22	tdc_herg				Extra trees	␣
↪	conventional		0.693667		0.829423	0.
↪	784242	0.912243	0.903046	0.487068		
23	tdc_herg	Chemprop v2 (AttentiveFP + Selected descriptor...				␣
↪	Chemprop v2		0.687334		NaN	0.755670
↪	NaN	0.888682	0.428795			
24	tdc_herg				CFA (Combinatorial Fusion)	␣
↪	CFA fusion		0.500000		NaN	0.
↪	751841	NaN	0.845701	0.000000		
25	tdc_herg				Voting Classifier (KNN, SVM)	␣
↪	conventional		0.632548		0.834204	0.
↪	741826	0.947369	0.872426	0.407044		
26	tdc_herg				HistGradientBoosting	␣
↪	conventional		0.687334		0.818670	0.
↪	741532	0.911301	0.880319	0.428795		
27	tdc_herg				TabPFNClassifier	␣
↪	TabPFN deep learning		0.639323		0.873705	0.
↪	740059	0.957866	0.876207	0.330038		
28	tdc_herg				Ensemble (OOF Stacking (RidgeCV, 5-fold))	␣
↪	ensemble		0.665832		NaN	0.
↪	730914	NaN	0.821961	0.469338		
29	tdc_herg				ChemML MLP (PyTorch)	␣
↪	ChemML deep learning		0.542857		NaN	0.
↪	729161	NaN	0.838285	0.253874		
30	tdc_herg				Tabular MLP	␣
↪	conventional		0.673049		0.898156	0.
↪	724006	0.966801	0.853186	0.402755		
31	tdc_herg				MapLight + GNN (CatBoost, Strict Parity)	␣
↪	MapLight + GNN		0.702798		NaN	0.
↪	712960	NaN	0.829723	0.489489		
32	tdc_herg				SVC	␣
↪	conventional		0.639323		0.881994	0.
↪	711782	0.946901	0.859416	0.330038		
33	tdc_herg				LogisticRegression	␣
↪	conventional		0.599264		0.902356	0.
↪	653756	0.961228	0.813562	0.220344		

	analysis_delta_from_best	elapsed_seconds
0	0.000000	66.059
1	0.002020	71.623
2	0.030303	410.341
3	0.032323	26.802
4	0.040404	83.694
5	0.044444	122.137
6	0.056566	53.321
7	0.074747	287.937
8	0.084848	37.740
9	0.092929	70.064
10	0.094949	802.124
11	0.098990	803.371
12	0.102020	32.287
13	0.103030	55.027
14	0.105051	27.078
15	0.146465	801.481
16	0.171717	135.062
17	0.000000	255.366
18	0.005449	478.734
19	0.051252	231.190
20	0.061856	278.739
21	0.068336	268.385
22	0.076583	236.295
23	0.105155	564.555
24	0.108984	840.695
25	0.118999	246.543
26	0.119293	244.904
27	0.120766	364.917
28	0.129911	841.583
29	0.131664	379.661
30	0.136819	266.379
31	0.147865	840.216
32	0.149043	226.720
33	0.207069	226.214

1.0.5 GA value diagnostics across runs

This section checks whether GA tuning actually improved benchmark performance in prior runs.

- It compares each GA-tuned model (for example, `CatBoost GA`) against its non-GA counterpart on the same dataset.
- Positive `delta_vs_baseline` means GA improved performance (direction-aware by metric type).
- This is computed across all run folders under `benchmark_results/`.

```
[57]: # GA_IMPACT_DIAGNOSTICS_MARKER
      # Evaluate GA uplift by dataset/model and aggregate across prior benchmark runs.
```

```

def ga_baseline_candidates(ga_model_name):
    ga_model_name = str(ga_model_name or "").strip()
    candidates = []
    if ga_model_name.endswith(" GA"):
        base = ga_model_name[: -len(" GA")].strip()
        if base:
            candidates.extend([base, f"Tuned {base}"])
    elif ga_model_name.startswith("Tuned "):
        base = ga_model_name[len("Tuned ") :].strip()
        if base:
            candidates.extend([base])
    return [c for c in candidates if c]

def metric_is_lower_better(metric_name):
    text = str(metric_name or "").strip().lower()
    if not text:
        return True
    if any(key in text for key in ["rmse", "mae", "mse", "loss", "error"]):
        return True
    if any(key in text for key in ["r2", "spearman", "pearson", "auc", ↵
↵"accuracy", "f1"]):
        return False
    return True

def load_run_metric_table(run_dir):
    summary = standardize_metric_frame(read_optional_csv(run_dir / ↵
↵"summary_metrics.csv"))
    if summary is not None and not summary.empty:
        return summary

    parts = []
    for metrics_path in sorted(run_dir.glob("*/metrics.csv")):
        dataset_name = metrics_path.parent.name
        frame = standardize_metric_frame(pd.read_csv(metrics_path))
        if frame is None or frame.empty:
            continue
        if "dataset" not in frame.columns:
            frame["dataset"] = dataset_name
        parts.append(frame)
    if not parts:
        return None
    return pd.concat(parts, ignore_index=True)

```

```

benchmark_root = RUN_DIR.parent if RUN_DIR.parent.name == "benchmark_results"
↳ else Path("./benchmark_results").resolve()
run_dirs = sorted([p for p in benchmark_root.iterdir() if p.is_dir()],
↳ key=lambda p: p.stat().st_mtime)

ga_rows = []
ga_attempt_rows = []
for run_dir in run_dirs:
    run_name = run_dir.name
    metrics = load_run_metric_table(run_dir)
    if metrics is None or metrics.empty:
        continue

    metrics = metrics.copy()
    rmse_col = metric_column(metrics, "AUTO", metric_hint="rmse")

    # Normalize key metric columns.
    for col in ["primary_metric_value", "primary_metric", rmse_col, "dataset",
↳ "model", "workflow"]:
        if col is not None and col in metrics.columns:
            if col in {"primary_metric_value", rmse_col}:
                metrics[col] = pd.to_numeric(metrics[col], errors="coerce")
            else:
                metrics[col] = metrics[col].astype(str)

    model_series = metrics["model"].astype(str) if "model" in metrics.columns
↳ else pd.Series(dtype=str)
    workflow_series = metrics["workflow"].astype(str).str.lower() if "workflow"
↳ in metrics.columns else pd.Series(dtype=str)
    ga_mask = model_series.str.endswith(" GA") | workflow_series.str.
↳ contains("ga", regex=False)
    ga_metrics = metrics.loc[ga_mask].copy()
    if ga_metrics.empty:
        continue

    for _, ga_row in ga_metrics.iterrows():
        dataset_name = str(ga_row.get("dataset", ""))
        ga_model_name = str(ga_row.get("model", "")).strip()
        ga_error_text = str(ga_row.get("error", "")).strip() if "error" in
↳ ga_row.index else ""
        ga_attempt_rows.append(
            {
                "run_name": run_name,
                "dataset": dataset_name,
                "ga_model": ga_model_name,

```

```

        "workflow": str(ga_row.get("workflow", "")),
        "error": ga_error_text,
        "has_error": bool(ga_error_text),
        "has_primary_metric": bool(pd.notna(ga_row.
↪get("primary_metric_value", np.nan))),
    }
)
candidates = ga_baseline_candidates(ga_model_name)
if not candidates:
    continue

    dataset_rows = metrics.loc[metrics["dataset"].astype(str) == ↪
↪dataset_name].copy()
    baseline_rows = dataset_rows.loc[dataset_rows["model"].astype(str).
↪isin(candidates)].copy()
    baseline_rows = baseline_rows.loc[~baseline_rows["model"].astype(str).
↪str.endswith(" GA")]
    if baseline_rows.empty:
        continue

    baseline = baseline_rows.iloc[0]

    ga_primary_name = str(ga_row.get("primary_metric", "")).strip().lower()
    baseline_primary_name = str(baseline.get("primary_metric", "")).strip().
↪lower()
    use_primary = (
        "primary_metric_value" in ga_row.index
        and "primary_metric_value" in baseline.index
        and pd.notna(ga_row.get("primary_metric_value", np.nan))
        and pd.notna(baseline.get("primary_metric_value", np.nan))
        and ga_primary_name
        and baseline_primary_name
        and ga_primary_name == baseline_primary_name
    )

    if use_primary:
        metric_name = ga_primary_name
        ga_value = float(ga_row.get("primary_metric_value", np.nan))
        base_value = float(baseline.get("primary_metric_value", np.nan))
        elif rmse_col is not None and rmse_col in ga_row.index and rmse_col in ↪
↪baseline.index:
            metric_name = "rmse"
            ga_value = float(ga_row.get(rmse_col, np.nan))
            base_value = float(baseline.get(rmse_col, np.nan))
        else:
            continue

```

```

        if not np.isfinite(ga_value) or not np.isfinite(base_value):
            continue

        lower_better = metric_is_lower_better(metric_name)
        delta = (base_value - ga_value) if lower_better else (ga_value -
↳base_value)
        denom = abs(base_value) if abs(base_value) > 1e-12 else np.nan
        rel_delta = (delta / denom) if np.isfinite(denom) else np.nan

        ga_rows.append(
            {
                "run_name": run_name,
                "dataset": dataset_name,
                "ga_model": ga_model_name,
                "baseline_model": str(baseline.get("model", "")),
                "metric": metric_name,
                "lower_is_better": bool(lower_better),
                "ga_value": float(ga_value),
                "baseline_value": float(base_value),
                "delta_vs_baseline": float(delta),
                "relative_delta": float(rel_delta) if pd.notna(rel_delta) else
↳np.nan,
                "ga_better": bool(delta > 0.0),
            }
        )

GA_IMPACT = pd.DataFrame(ga_rows)
GA_ATTEMPTS = pd.DataFrame(ga_attempt_rows)

if GA_IMPACT.empty:
    if GA_ATTEMPTS.empty:
        display_note("No GA model rows were found in the scanned run_
↳directories.")
    else:
        display_note("No comparable GA-vs-baseline pairs with valid metrics_
↳were found. GA attempt diagnostics are shown below.")
        attempt_summary = (
            GA_ATTEMPTS.groupby("run_name", dropna=False)
            .agg(
                ga_rows=("ga_model", "size"),
                ga_rows_with_error=("has_error", "sum"),
                ga_rows_with_primary_metric=("has_primary_metric", "sum"),
            )
            .reset_index()
        )
        display(attempt_summary.sort_values("run_name").reset_index(drop=True))

```

```

error_rows = GA_ATTEMPTS.loc[GA_ATTEMPTS["has_error"]].copy()
if not error_rows.empty:
    error_summary = (
        error_rows.groupby(["run_name", "ga_model", "error"],
↪dropna=False)
            .size()
            .reset_index(name="count")
            .sort_values("count", ascending=False)
        )
    display_note("Top GA error signatures")
    display(error_summary.head(20).reset_index(drop=True))
else:
    display_note("### GA uplift by dataset")
    display(
        GA_IMPACT.sort_values(["ga_better", "delta_vs_baseline"],
↪ascending=[False, False])
            .reset_index(drop=True)
        )

    ga_summary = (
        GA_IMPACT.groupby(["run_name", "ga_model"], dropna=False)
            .agg(
                datasets_compared=("dataset", "nunique"),
                wins=("ga_better", "sum"),
                mean_delta=("delta_vs_baseline", "mean"),
                median_delta=("delta_vs_baseline", "median"),
                mean_relative_delta=("relative_delta", "mean"),
            )
            .reset_index()
        )
    ga_summary["win_rate"] = ga_summary["wins"] /
↪ga_summary["datasets_compared"].replace(0, np.nan)

    display_note("### GA uplift summary by run and GA model")
    display(ga_summary.sort_values(["win_rate", "mean_delta"],
↪ascending=[False, False]).reset_index(drop=True))

    any_wins = GA_IMPACT.loc[GA_IMPACT["ga_better"]].copy()
    if any_wins.empty:
        display_note("GA did not beat its non-GA counterpart on any comparable
↪dataset in the scanned runs.")
    else:
        best_wins = any_wins.sort_values("delta_vs_baseline", ascending=False).
↪head(25)
        display_note("### Datasets where GA improved performance")

```

```

display(best_wins.reset_index(drop=True))

fig = px.bar(
    ga_summary.sort_values("win_rate", ascending=False),
    x="ga_model",
    y="win_rate",
    color="run_name",
    barmode="group",
    hover_data=[c for c in ["datasets_compared", "wins", "mean_delta", "
↪mean_relative_delta"] if c in ga_summary.columns],
    title="GA win rate vs non-GA baseline",
)
fig.update_layout(xaxis_title="GA model", yaxis_title="Win rate")
fig.show()

```

No GA model rows were found in the scanned run directories.

```

[58]: # Architecture coverage and prediction-diversity diagnostics

def classify_architecture_family(workflow_name, model_name):
    workflow_text = str(workflow_name or "").strip().lower()
    model_text = str(model_name or "").strip().lower()

    if "ensemble" in workflow_text or model_text.startswith("ensemble ("):
        return "Ensemble meta-model"
    if "cfa" in workflow_text or model_text.startswith("cfa ("):
        return "CFA combinatorial fusion"
    if "tabpfn" in workflow_text or "tabpfn" in model_text:
        return "TabPFN foundation model"
    if "chemprop" in workflow_text or "chemprop" in model_text:
        return "Graph neural networks (Chemprop)"
    if "maplight + gnn" in workflow_text or "maplight + gnn" in model_text:
        return "Graph transfer + tabular head"
    if "uni-mol" in workflow_text or "uni-mol" in model_text or "unimol" in_
↪model_text:
        return "3D foundation model"
    if "chemml deep learning" in workflow_text or "chemml" in model_text:
        return "Deep tabular neural nets"
    if "tuned conventional" in workflow_text:
        return "Tuned conventional ML"
    if "conventional" in workflow_text:
        return "Conventional ML"
    return "Other / unknown"

```

```

def classify_architecture_subfamily(model_name):
    model_text = str(model_name or "").strip().lower()
    if "cmpnn" in model_text:
        return "CMPNN"
    if "attentivefp" in model_text:
        return "AttentiveFP"
    if "d-mpnn" in model_text or "dmpnn" in model_text:
        return "D-MPNN"
    if "maplight + gnn" in model_text:
        return "MapLight + GNN"
    if model_text.startswith("cfa (") or "combinatorial fusion" in model_text:
        return "CFA fusion"
    if "tabpfn" in model_text:
        return "TabPFN"
    if "chemml mlp" in model_text:
        return "ChemML MLP"
    if "catboost" in model_text:
        return "CatBoost-family"
    if "xgboost" in model_text:
        return "XGBoost-family"
    if "random forest" in model_text:
        return "RandomForest-family"
    if "svr" in model_text:
        return "SVR"
    if "elasticnet" in model_text:
        return "ElasticNet-family"
    if model_text.startswith("ensemble ("):
        return "Stacking / averaging"
    return str(model_name or "")

coverage_source = SUMMARY_ANALYSIS_METRICS.copy()
error_col = pick_column(coverage_source.columns, ["error", "Error"])
if error_col is not None:
    coverage_source = coverage_source.loc[
        coverage_source[error_col].isna()
        | (coverage_source[error_col].astype(str).str.strip() == "")
    ].copy()

coverage_source = add_analysis_metric_columns(coverage_source,
    ↪rmse_col=RMSE_COL, classification_col=BALANCED_ACCURACY_COL)

if coverage_source.empty:
    display_note("No successful model rows were available for_
    ↪architecture-coverage diagnostics.")
else:
    coverage_source["architecture_family"] = coverage_source.apply(

```



```

        lambda row: classify_architecture_family(row.get("workflow", ""), row.
↪get("model", "")),
        axis=1,
    )
    coverage_source["architecture_subfamily"] = coverage_source["model"].
↪apply(classify_architecture_subfamily)

family_summary = (
    coverage_source.groupby("architecture_family", dropna=False)
    .agg(
        datasets=("dataset", "nunique"),
        models=("model", "nunique"),
        rows=("model", "size"),
        mean_delta_from_dataset_best=("analysis_delta_from_best", "mean"),
        mean_rmse=(RMSE_COL, "mean") if RMSE_COL in coverage_source.columns
↪else ("model", "size"),
        mean_balanced_accuracy=(BALANCED_ACCURACY_COL, "mean") if
↪BALANCED_ACCURACY_COL in coverage_source.columns else ("model", "size"),
    )
    .reset_index()
    .sort_values(["datasets", "rows"], ascending=[False, False])
)

display_note("### Architecture Coverage")
display(family_summary)

best_architecture_rows = best_rows_by_dataset(coverage_source)
if not best_architecture_rows.empty:
    best_architecture_rows["architecture_family"] = best_architecture_rows.
↪apply(
        lambda row: classify_architecture_family(row.get("workflow", ""),
↪row.get("model", "")),
        axis=1,
    )
    best_architecture_rows["architecture_subfamily"] =
↪best_architecture_rows["model"].apply(classify_architecture_subfamily)

best_model_counts = (
    best_architecture_rows.groupby(["model", "workflow", "task_kind"],
↪dropna=False)
    .agg(best_dataset_count=("dataset", "nunique"))
    .reset_index()
    .sort_values(["best_dataset_count", "model"], ascending=[False,
↪True])
)
display_note("### Overall best model counts")

```

```

display(best_model_counts.reset_index(drop=True))

best_family_counts = (
    best_architecture_rows.groupby(["architecture_family",
↪ "task_kind"], dropna=False)
    .agg(best_dataset_count=("dataset", "nunique"))
    .reset_index()
    .sort_values(["best_dataset_count", "architecture_family"],
↪ ascending=[False, True])
)
display_note("### Overall best architecture-family counts")
display(best_family_counts.reset_index(drop=True))

best_family_overall = (
    best_architecture_rows.groupby("architecture_family", dropna=False)
    .agg(
        winning_datasets=("dataset", "nunique"),
        regression_wins=("task_kind", lambda s: int((s.astype(str) ==
↪ "regression").sum())),
        classification_wins=("task_kind", lambda s: int((s.astype(str)
↪ == "classification").sum()))
    )
    .reset_index()
    .sort_values(["winning_datasets", "architecture_family"],
↪ ascending=[False, True])
)
best_family_overall["win_fraction"] =
↪ best_family_overall["winning_datasets"] /
↪ max(float(best_family_overall["winning_datasets"].sum()), 1.0)
display_note("### Publication Figure: Best-Model Win Counts by Family")
display(best_family_overall.reset_index(drop=True))

family_pie = px.pie(
    best_family_overall,
    names="architecture_family",
    values="winning_datasets",
    title="Best model family by dataset",
    hover_data=["regression_wins", "classification_wins",
↪ "win_fraction"],
)
family_pie.update_traces(textposition="inside",
↪ textinfo="percent+label")
family_pie.show()

family_rank_source = coverage_source.
↪ loc[coverage_source["analysis_delta_from_best"].notna()].copy()

```

```

        if not family_rank_source.empty:
            family_rank_source["family_rank_within_dataset"] =
↪family_rank_source.groupby("dataset")["analysis_delta_from_best"].
↪rank(method="min", ascending=True)
            family_rank_heatmap = (
                family_rank_source.groupby(["dataset", "architecture_family"],
↪dropna=False)["family_rank_within_dataset"]
                .min()
                .unstack()
                .sort_index()
            )
            display_note("### Publication Figure: Best Rank Reached by Each
↪Family and Dataset")
            display(family_rank_heatmap.reset_index())
            rank_fig = px.imshow(
                family_rank_heatmap,
                aspect="auto",
                color_continuous_scale="Viridis_r",
                labels={"x": "Architecture family", "y": "Dataset", "color":
↪"Best rank"}),
                title="Best rank reached by each model family within each
↪dataset",
                text_auto=".0f",
            )
            rank_fig.update_layout(height=max(420, 45 * len(family_rank_heatmap.
↪index)))
            rank_fig.show()

            winning_model_table = best_architecture_rows[[
                col for col in ["dataset", "task_kind", "architecture_family",
↪"architecture_subfamily", "workflow", "model", "analysis_metric",
↪"analysis_metric_value"]
                if col in best_architecture_rows.columns
            ]].sort_values(["task_kind", "dataset"]).reset_index(drop=True)
            display_note("### Publication Table: Winning Model by Dataset")
            display(winning_model_table)

            dataset_family_presence = (
                coverage_source.assign(present=1)
                .groupby(["dataset", "architecture_family"], dropna=False)["present"]
                .max()
                .unstack(fill_value=0)
                .sort_index()
            )

```

```

    coverage_fraction = dataset_family_presence.mean(axis=1).
↪rename("coverage_fraction").reset_index()
    coverage_fraction = coverage_fraction.sort_values("coverage_fraction",
↪ascending=False)
    display_note("### Per-dataset architecture-family coverage")
    display(coverage_fraction)

    coverage_fig = px.imshow(
        dataset_family_presence,
        aspect="auto",
        color_continuous_scale=[[0.0, "#f0f0f0"], [1.0, "#2ca25f"]],
        labels={"x": "Architecture family", "y": "Dataset", "color": "Present"},
        title="Architecture-family coverage matrix (dataset x family)",
        text_auto=True,
    )
    coverage_fig.update_layout(height=max(420, 45 * len(dataset_family_presence.
↪index))))
    coverage_fig.show()

    if PREDICTIONS is None or PREDICTIONS.empty:
        display_note("No predictions table was found; skipping
↪architecture-diversity diagnostics.")
    else:
        pred_df = PREDICTIONS.copy()
        dataset_col = pick_column(pred_df.columns, ["dataset", "dataset_id",
↪"dataset_name"])
        model_col = pick_column(pred_df.columns, ["model", "Model"])
        predicted_col = pick_column(pred_df.columns, ["predicted",
↪"prediction", "y_pred"])
        split_col = pick_column(pred_df.columns, ["split", "Split"])
        smiles_col = pick_column(pred_df.columns, ["smiles", "SMILES",
↪"canonical_smiles"])
        row_col = pick_column(pred_df.columns, ["row_id", "Row", "index"])

        required = [dataset_col, model_col, predicted_col]
        if any(col is None for col in required):
            display_note("Predictions table is missing dataset/model/prediction
↪columns; skipping diversity diagnostics.")
        else:
            pred_df = pred_df.rename(columns={dataset_col: "dataset", model_col:
↪"model", predicted_col: "predicted"})
            if split_col is not None:
                pred_df = pred_df.rename(columns={split_col: "split"})
                pred_df = pred_df.loc[pred_df["split"].astype(str).str.lower()
↪=="test"].copy()

```

```

if smiles_col is not None:
    pred_df = pred_df.rename(columns={smiles_col: "smiles_key"})
elif row_col is not None:
    pred_df = pred_df.rename(columns={row_col: "smiles_key"})
else:
    pred_df["smiles_key"] = pred_df.groupby(["dataset", "model"]).
↳cumcount().astype(str)

pred_df["predicted"] = pd.to_numeric(pred_df["predicted"],
↳errors="coerce")
pred_df = pred_df.loc[pred_df["predicted"].notna()].copy()

diversity_rows = []
pair_rows = []
for dataset_name, group in pred_df.groupby("dataset", sort=True):
    wide = group.pivot_table(index="smiles_key", columns="model",
↳values="predicted", aggfunc="mean")
    if wide.shape[1] < 2:
        continue

    corr = wide.corr(method="spearman")
    if corr.shape[0] < 2:
        continue

    corr_values = corr.to_numpy(dtype=float)
    tri = np.triu_indices_from(corr_values, k=1)
    pair_corr = corr_values[tri]
    pair_corr = pair_corr[np.isfinite(pair_corr)]
    if pair_corr.size == 0:
        continue

    abs_corr = np.abs(pair_corr)
    diversity_rows.append(
        {
            "dataset": dataset_name,
            "models_in_pairwise_corr": int(corr.shape[0]),
            "pair_count": int(pair_corr.size),
            "mean_abs_spearman": float(np.mean(abs_corr)),
            "median_abs_spearman": float(np.median(abs_corr)),
            "max_abs_spearman": float(np.max(abs_corr)),
            "diversity_index": float(1.0 - np.median(abs_corr)),
        }
    )

cols = list(corr.columns)
for i in range(len(cols)):
    for j in range(i + 1, len(cols)):

```

```

        value = corr.iat[i, j]
        if pd.isna(value):
            continue
        pair_rows.append(
            {
                "dataset": dataset_name,
                "model_a": str(cols[i]),
                "model_b": str(cols[j]),
                "spearman": float(value),
                "abs_spearman": float(abs(value)),
            }
        )

    if not diversity_rows:
        display_note("Prediction diversity diagnostics could not be
↳computed (insufficient aligned test predictions).")
    else:
        diversity_df = pd.DataFrame(diversity_rows).
↳sort_values("diversity_index", ascending=False).reset_index(drop=True)
        display_note("### Prediction Diversity Across Architectures")
        display(diversity_df)

        diversity_fig = px.bar(
            diversity_df.sort_values("diversity_index", ascending=True),
            x="dataset",
            y="diversity_index",
            hover_data=["models_in_pairwise_corr",
↳"median_abs_spearman", "max_abs_spearman"],
            title="Architecture diversity index by dataset (1 - median
↳|Spearman|)",
        )
        diversity_fig.update_layout(yaxis_title="Diversity index
↳(higher is better)", xaxis_title="Dataset")
        diversity_fig.show()

    if pair_rows:
        pair_df = pd.DataFrame(pair_rows)
        pair_df["pair"] = pair_df.apply(
            lambda row: " / ".join(sorted([str(row["model_a"]),
↳str(row["model_b"])])),
            axis=1,
        )
        pair_summary = (
            pair_df.groupby("pair", dropna=False)
            .agg(
                datasets=("dataset", "nunique"),

```

```

        mean_abs_spearman=("abs_spearman", "mean"),
        median_abs_spearman=("abs_spearman", "median"),
        max_abs_spearman=("abs_spearman", "max"),
    )
    .reset_index()
    .sort_values(["mean_abs_spearman", "datasets"],
↪ascending=[False, False])
    )
    display_note("### Most redundant model pairs_
↪(cross-dataset)")
    display(pair_summary.head(20))

```

1.0.6 Architecture Coverage

	architecture_family	datasets	models	rows	
↪mean_delta_from_dataset_best	mean_rmse	mean_balanced_accuracy			
1	Conventional ML	44	15	484	↪
↪0.690423	6.817332	0.817593			
4	Graph neural networks (Chemprop)	44	2	84	↪
↪0.875030	7.047637	0.821860			
3	Ensemble meta-model	44	2	66	↪
↪0.338555	5.975035	0.844050			
0	CFA combinatorial fusion	44	1	44	↪
↪0.452344	6.464336	0.832514			
2	Deep tabular neural nets	44	1	44	↪
↪0.690127	6.896099	0.788712			
5	Graph transfer + tabular head	44	1	44	↪
↪0.199449	5.883022	0.756991			
6	TabPFN foundation model	44	2	44	↪
↪0.472750	10.425320	0.777952			

C:\Users\scott\AppData\Local\Temp\ipykernel_7244\1832174106.py:245:
PerformanceWarning: DataFrame is highly fragmented. This is usually the result
of calling `frame.insert` many times, which has poor performance. Consider
joining all columns at once using `pd.concat(axis=1)` instead. To get a de-
fragmented frame, use `newframe = frame.copy()`
.first()

1.0.7 Overall best model counts

	model	workflow	
↪task_kind	best_dataset_count		
0	Ensemble (OOF Stacking (RidgeCV, 5-fold))	ensemble	↪
↪classification	6		
1	MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN	↪
↪regression	6		
2	TabPFNRegressor	TabPFN deep learning	↪
↪regression	5		

3	CFA (Combinatorial Fusion)	CFA fusion	␣
↪regression	4		
4	Ensemble (Weighted average (inverse train RMSE))	ensemble	␣
↪classification	4		
5	XGBoost	conventional	␣
↪regression	4		
6	CatBoost	conventional	␣
↪classification	3		
7	AdaBoost	conventional	␣
↪classification	2		
8	Chemprop v2 (AttentiveFP, ensemble=1)	Chemprop v2	␣
↪classification	2		
9	CFA (Combinatorial Fusion)	CFA fusion	␣
↪classification	1		
10	CatBoost	conventional	␣
↪regression	1		
11	Ensemble (Weighted average (inverse train RMSE))	ensemble	␣
↪regression	1		
12	HistGradientBoosting	conventional	␣
↪classification	1		
13	LogisticRegression	conventional	␣
↪classification	1		
14	MapLight CatBoost (Strict Parity)	conventional	␣
↪regression	1		
15	Random forest	conventional	␣
↪classification	1		
16	XGBoost	conventional	␣
↪regression	1		

1.0.8 Overall best architecture-family counts

	architecture_family	task_kind	best_dataset_count
0	Ensemble meta-model	classification	10
1	Conventional ML	classification	9
2	Conventional ML	regression	6
3	Graph transfer + tabular head	regression	6
4	TabPFN foundation model	regression	5
5	CFA combinatorial fusion	regression	4
6	Graph neural networks (Chemprop)	classification	2
7	CFA combinatorial fusion	classification	1
8	Ensemble meta-model	regression	1

1.0.9 Publication Figure: Best-Model Win Counts by Family

	dataset	coverage_fraction
0	chemml_cep_homo	1.000000
1	chemml_organic_density	1.000000

3	esol_delaney	1.000000
4	freesolv_sampl	1.000000
5	lipophilicity	1.000000
6	poduam_pod_nc_std	1.000000
8	polaris_adme_fang_hppb_1	1.000000
7	poduam_pod_rd_std	1.000000
9	polaris_adme_fang_perm_1	1.000000
10	polaris_adme_fang_rclint_1	1.000000
17	tdc_clearance_hepatocyte_az	1.000000
11	polaris_adme_fang_rppb_1	1.000000
12	polaris_adme_fang_solu_1	1.000000
15	tdc_caco2_wang	1.000000
27	tdc_hydrationfreeenergy_freesolv	1.000000
30	tdc_ppbr_az	1.000000
24	tdc_half_life_obach	1.000000
18	tdc_clearance_microsome_az	1.000000
33	tdc_vdss_lombardo	1.000000
14	tdc_bioavailability_ma	0.857143
13	tdc_bbb_martins	0.857143
2	chemml_xyz_polarizability	0.857143
25	tdc_herg	0.857143
21	tdc_cyp2d6_substrate_carbonmangels	0.857143
20	tdc_cyp2c9_substrate_carbonmangels	0.857143
19	tdc_clintox	0.857143
29	tdc_pgp_broccatelli	0.857143
23	tdc_dili	0.857143
22	tdc_cyp3a4_substrate_carbonmangels	0.857143
26	tdc_hia_hou	0.857143
31	tdc_skin_reaction	0.857143
28	tdc_pampa_ncats	0.857143
32	tdc_toxcast	0.857143
16	tdc_carcinogens_lagunin	0.714286

No predictions table was found; skipping architecture-diversity diagnostics.

[59]: *# Feature-family representation analysis from selector outputs*

```
feature_family_drop_presence_threshold = 0.20
feature_family_drop_enrichment_threshold = 0.25
feature_family_drop_selected_total_threshold = 25
feature_family_watch_enrichment_threshold = 0.35
```

```
def feature_family_from_name(feature_name):
    name = str(feature_name or "")
    if name.startswith("morgan_bit_"):
        return "morgan"
    if name.startswith("ecfp6_bit_"):
```

```

        return "ecfp6"
    if name.startswith("fcfp6_bit_"):
        return "fcfp6"
    if name.startswith("layered_bit_"):
        return "layered"
    if name.startswith("atom_pair_bit_"):
        return "atom_pair"
    if name.startswith("topological_torsion_bit_"):
        return "topological_torsion"
    if name.startswith("rdk_path_bit_"):
        return "rdk_path"
    if name.startswith("maccs_bit_"):
        return "maccs"
    if name.startswith("rdkit_"):
        return "rdkit"
    if name.startswith("avalon_count_"):
        return "avalon"
    if name.startswith("erg_"):
        return "erg"
    if name.startswith("maplight_morgan_") or name.startswith("maplight_desc_"):
        return "maplight"
    if name.startswith("gin_emb_"):
        return "gin_embedding"
    return "other"

def infer_feature_col(frame):
    return pick_column(frame.columns, ["feature", "Feature", "name"])

def infer_magnitude_col(frame):
    if "abs_coefficient" in frame.columns:
        return "abs_coefficient"
    if "importance" in frame.columns:
        return "importance"
    if "coefficient" in frame.columns:
        return "coefficient"
    return None

selector_rows = []
missing_selector_artifacts = []
for dataset_dir in sorted([p for p in RUN_DIR.iterdir() if p.is_dir()]):
    dataset_name = dataset_dir.name
    selected_path = dataset_dir / "selected_features.csv"
    coeff_path = dataset_dir / "selector_coefficients.csv"
    if not selected_path.exists() or not coeff_path.exists():

```

```

        missing_selector_artifacts.append(dataset_name)
        continue

    try:
        selected_df = pd.read_csv(selected_path)
        coeff_df = pd.read_csv(coeff_path)
    except Exception as exc:
        missing_selector_artifacts.append(f"{dataset_name} ({type(exc)}.
↳ __name__}")
        continue

    feature_col = infer_feature_col(coeff_df)
    if feature_col is None:
        missing_selector_artifacts.append(f"{dataset_name} (no feature column)")
        continue

    selected_feature_col = infer_feature_col(selected_df)
    selected_set = set()
    if selected_feature_col is not None:
        selected_set = set(selected_df[selected_feature_col].astype(str).
↳ tolist())

    coeff_work = coeff_df.copy()
    coeff_work["feature"] = coeff_work[feature_col].astype(str)
    coeff_work["feature_family"] = coeff_work["feature"].
↳ map(feature_family_from_name)
    coeff_work["is_selected"] = coeff_work["feature"].isin(selected_set)

    magnitude_col = infer_magnitude_col(coeff_work)
    if magnitude_col is None:
        coeff_work["feature_magnitude"] = np.nan
    elif magnitude_col == "coefficient":
        coeff_work["feature_magnitude"] = pd.
↳ to_numeric(coeff_work[magnitude_col], errors="coerce").abs()
    else:
        coeff_work["feature_magnitude"] = pd.
↳ to_numeric(coeff_work[magnitude_col], errors="coerce")

    grouped = coeff_work.groupby("feature_family", dropna=False)
    for family_name, family_df in grouped:
        selected_mask = family_df["is_selected"].astype(bool)
        selector_rows.append(
            {
                "dataset": dataset_name,
                "feature_family": str(family_name),
                "available_features": int(len(family_df)),
                "selected_features": int(selected_mask.sum()),
            }
        )

```

```

        "selected_magnitude_sum": float(
            family_df.loc[selected_mask, "feature_magnitude"].fillna(0.
↪0).sum()
        ),
    }
)

if not selector_rows:
    display_note("No selector feature artifacts were found in this run;␣
↪skipping feature-family representation analysis.")
else:
    selector_family_df = pd.DataFrame(selector_rows)

    family_summary = (
        selector_family_df.groupby("feature_family", dropna=False)
        .agg(
            datasets_with_family=("dataset", "nunique"),
            datasets_selected=("selected_features", lambda s: int((pd.
↪to_numeric(s, errors="coerce").fillna(0) > 0).sum())),
            available_features_total=("available_features", "sum"),
            selected_features_total=("selected_features", "sum"),
            mean_selected_per_dataset=("selected_features", "mean"),
            median_selected_per_dataset=("selected_features", "median"),
            selected_magnitude_total=("selected_magnitude_sum", "sum"),
        )
        .reset_index()
    )

    family_summary["dataset_presence_rate"] =␣
↪family_summary["datasets_selected"] / family_summary["datasets_with_family"].
↪replace(0, np.nan)
    family_summary["selection_rate"] =␣
↪family_summary["selected_features_total"] /␣
↪family_summary["available_features_total"].replace(0, np.nan)

    total_available = float(family_summary["available_features_total"].sum())
    total_selected = float(family_summary["selected_features_total"].sum())
    total_magnitude = float(family_summary["selected_magnitude_total"].sum())

    if total_available > 0 and total_selected > 0:
        family_summary["expected_selected_if_uniform"] =␣
↪family_summary["available_features_total"] / total_available * total_selected
        family_summary["enrichment_vs_uniform"] =␣
↪family_summary["selected_features_total"] /␣
↪family_summary["expected_selected_if_uniform"].replace(0, np.nan)
    else:

```

```

family_summary["expected_selected_if_uniform"] = np.nan
family_summary["enrichment_vs_uniform"] = np.nan

family_summary["selected_share"] =_
↪family_summary["selected_features_total"] / max(total_selected, 1.0)
family_summary["selected_magnitude_share"] =_
↪family_summary["selected_magnitude_total"] / max(total_magnitude, 1.0)

family_summary = family_summary.sort_values(["selected_features_total",_
↪"selection_rate"], ascending=[False, False]).reset_index(drop=True)

display_note("### Feature-Family Representation In Selected Features")
display(family_summary)

selected_heatmap = (
    selector_family_df.pivot_table(
        index="dataset",
        columns="feature_family",
        values="selected_features",
        aggfunc="sum",
        fill_value=0,
    )
    .sort_index()
)

fig = px.imshow(
    selected_heatmap,
    aspect="auto",
    color_continuous_scale="Viridis",
    labels={"x": "Feature family", "y": "Dataset", "color": "# selected"},
    title="Selected feature counts by dataset and family",
    text_auto=True,
)
fig.update_layout(height=max(420, 45 * len(selected_heatmap.index)))
fig.show()

enrich_plot_df = family_summary.sort_values("enrichment_vs_uniform",_
↪ascending=True)
fig = px.bar(
    enrich_plot_df,
    x="feature_family",
    y="enrichment_vs_uniform",
    hover_data=["selected_features_total", "available_features_total",_
↪"dataset_presence_rate", "selection_rate"],
    title="Feature-family selection enrichment vs uniform-by-dimension,_
↪baseline",

```

```

)
fig.add_hline(y=1.0, line_dash="dash", line_color="black")
fig.update_layout(xaxis_title="Feature family", yaxis_title="Enrichment (>1,
↳ means over-represented)")
fig.show()

def is_tree_based_model_name(model_name):
    text = str(model_name or "").strip().lower()
    return any(key in text for key in ["catboost", "xgboost", "random_
↳ forest", "extra trees", "histgradient", "adaboost", "lightgbm"])

importance_rows = []
tree_metric_source = SUMMARY_ANALYSIS_METRICS.copy()
tree_error_col = pick_column(tree_metric_source.columns, ["error", "Error"])
if tree_error_col is not None:
    tree_metric_source = tree_metric_source.loc[
        tree_metric_source[tree_error_col].isna()
        | (tree_metric_source[tree_error_col].astype(str).str.strip() == "")
    ].copy()
tree_metric_source = add_analysis_metric_columns(tree_metric_source,
↳ rmse_col=RMSE_COL, classification_col=BALANCED_ACCURACY_COL)
tree_metric_source = tree_metric_source.loc[tree_metric_source["model"].
↳ map(is_tree_based_model_name)].copy()
if not tree_metric_source.empty:
    best_tree_models = (
        tree_metric_source.sort_values(["dataset",
↳ "analysis_delta_from_best", "model"], ascending=[True, True, True])
        .groupby("dataset", as_index=False)
        .first()[["dataset", "model", "workflow", "analysis_metric",
↳ "analysis_metric_value"]]
    )
else:
    best_tree_models = pd.DataFrame(columns=["dataset", "model",
↳ "workflow", "analysis_metric", "analysis_metric_value"])

for dataset_dir in sorted([p for p in RUN_DIR.iterdir() if p.is_dir()]):
    importance_path = dataset_dir / "model_feature_importances.csv"
    if not importance_path.exists():
        continue
    try:
        imp_df = pd.read_csv(importance_path)
    except Exception:
        continue
    feature_col = infer_feature_col(imp_df)
    importance_col = infer_magnitude_col(imp_df)
    model_col = pick_column(imp_df.columns, ["model", "Model"])

```

```

workflow_col = pick_column(imp_df.columns, ["workflow", "Workflow"])
if feature_col is None or importance_col is None or model_col is None:
    continue
work = imp_df.copy()
work["dataset"] = dataset_dir.name
work["model"] = work[model_col].astype(str)
work["workflow"] = work[workflow_col].astype(str) if workflow_col is_
↳not None else ""
work["feature"] = work[feature_col].astype(str)
work["feature_family"] = work["feature"].map(feature_family_from_name)
work["importance_weight"] = pd.to_numeric(work[importance_col],_
↳errors="coerce").abs()
work = work.loc[work["importance_weight"].notna()].copy()
if work.empty:
    continue
local_best = best_tree_models.loc[best_tree_models["dataset"].
↳astype(str).eq(dataset_dir.name)]
if not local_best.empty:
    best_model_name = str(local_best.iloc[0]["model"])
    work = work.loc[work["model"].astype(str).eq(best_model_name)].
↳copy()
if work.empty:
    continue
grouped_importance = (
    work.groupby(["dataset", "model", "workflow", "feature_family"],_
↳dropna=False)
    .agg(importance_weight=("importance_weight", "sum"),_
↳features=("feature", "nunique"))
    .reset_index()
)
grouped_importance["importance_source"] = "model_feature_importances.
↳CSV"
importance_rows.extend(grouped_importance.to_dict(orient="records"))

if importance_rows:
    feature_importance_family_df = pd.DataFrame(importance_rows)
else:
    feature_importance_family_df = selector_family_df.copy()
    feature_importance_family_df["model"] = "selector"
    feature_importance_family_df["workflow"] = "feature selection"
    feature_importance_family_df["importance_weight"] = pd.
↳to_numeric(feature_importance_family_df["selected_magnitude_sum"],_
↳errors="coerce")
    missing_weight = ~feature_importance_family_df["importance_weight"].
↳gt(0)

```

```

        feature_importance_family_df.loc[missing_weight, "importance_weight"] =
    ↪pd.to_numeric(feature_importance_family_df.loc[missing_weight,
    ↪"selected_features"], errors="coerce")

        feature_importance_family_df["features"] =
    ↪feature_importance_family_df["selected_features"]

        feature_importance_family_df["importance_source"] =
    ↪"selector_coefficients_fallback"

    if not feature_importance_family_df.empty:
        feature_importance_family_df["importance_weight"] = pd.
    ↪to_numeric(feature_importance_family_df["importance_weight"],
    ↪errors="coerce").fillna(0.0)

        dataset_totals = feature_importance_family_df.
    ↪groupby("dataset")["importance_weight"].transform("sum").replace(0, np.nan)

        feature_importance_family_df["importance_share_within_dataset"] =
    ↪feature_importance_family_df["importance_weight"] / dataset_totals

        feature_importance_family_df["family_rank_within_dataset"] =
    ↪feature_importance_family_df.
    ↪groupby("dataset")["importance_share_within_dataset"].rank(method="min",
    ↪ascending=False)

    importance_summary = (
        feature_importance_family_df.groupby(["feature_family",
    ↪"importance_source"], dropna=False)
        .agg(
            datasets=("dataset", "nunique"),
            total_importance_weight=("importance_weight", "sum"),
            mean_importance_share=("importance_share_within_dataset",
    ↪"mean"),
            median_importance_share=("importance_share_within_dataset",
    ↪"median"),
            mean_family_rank=("family_rank_within_dataset", "mean"),
            median_family_rank=("family_rank_within_dataset", "median"),
        )
        .reset_index()
        .sort_values(["mean_importance_share", "datasets"],
    ↪ascending=[False, False])
    )

    display_note("### Feature-Family Importance Summary")
    display(importance_summary.reset_index(drop=True))

    importance_heatmap = (
        feature_importance_family_df.pivot_table(
            index="dataset",
            columns="feature_family",
            values="importance_share_within_dataset",

```



```

        aggfunc="sum",
        fill_value=0.0,
    )
    .sort_index()
)
display_note("### Publication Figure: Feature-Family Importance by_
↳Dataset")
display(importance_heatmap.reset_index())
fig = px.imshow(
    importance_heatmap,
    aspect="auto",
    color_continuous_scale="Viridis",
    labels={"x": "Feature family", "y": "Dataset", "color": "Importance_
↳share"},
    title="Feature-family importance share by dataset",
)
fig.update_layout(height=max(420, 45 * len(importance_heatmap.index)))
fig.show()
if not importance_rows:
    display_note(
        "No per-model `model_feature_importances.csv` artifacts were_
↳found, so this section uses selector coefficient/importances as a fallback. "
        "Future benchmark runs can replace this with best-tree-model_
↳importances automatically."
    )

drop_candidates = family_summary.loc[
    (family_summary["dataset_presence_rate"].fillna(0.0) <=_
↳float(feature_family_drop_presence_threshold))
    & (family_summary["enrichment_vs_uniform"].fillna(np.inf) <=_
↳float(feature_family_drop_enrichment_threshold))
    & (family_summary["selected_features_total"].fillna(0.0) <=_
↳float(feature_family_drop_selected_total_threshold))
].copy()

watchlist = family_summary.loc[
    (family_summary["enrichment_vs_uniform"].fillna(np.inf) <=_
↳float(feature_family_watch_enrichment_threshold))
    & (family_summary["dataset_presence_rate"].fillna(0.0) >= 0.5)
].copy()

if drop_candidates.empty:
    display_note(
        "No feature family met the strict drop-candidate rule in this run. "
        "That means no family looked both rare across datasets and_
↳consistently unselected."
    )

```

```

    )
else:
    display_note("### Suggested drop candidates (strict rule)")
    display(
        drop_candidates[[
            "feature_family",
            "datasets_selected",
            "datasets_with_family",
            "dataset_presence_rate",
            "selected_features_total",
            "available_features_total",
            "selection_rate",
            "enrichment_vs_uniform",
        ]].sort_values(["dataset_presence_rate",
↪ "selected_features_total"], ascending=[True, True]).reset_index(drop=True)
    )

    if not watchlist.empty:
        display_note("### Watchlist: under-enriched but still used across many_
↪ datasets")
        display(
            watchlist[[
                "feature_family",
                "datasets_selected",
                "datasets_with_family",
                "dataset_presence_rate",
                "selected_features_total",
                "available_features_total",
                "selection_rate",
                "enrichment_vs_uniform",
            ]].sort_values("enrichment_vs_uniform", ascending=True).
↪ reset_index(drop=True)
        )

    if missing_selector_artifacts:
        display_note(
            "Selector artifacts were missing for some datasets and were_
↪ excluded from this section: "
            + ", ".join(sorted(set(missing_selector_artifacts)))
        )

```

1.0.10 Feature-Family Representation In Selected Features

	feature_family	datasets_with_family	datasets_selected	
	available_features_total	selected_features_total	mean_selected_per_dataset	
	median_selected_per_dataset	selected_magnitude_total	\	
0	maplight	44	44	
	86949	4849	110.204545	
	74.0	244.877597		
1	fcfp6	44	43	
	42540	1908	43.363636	
	38.0	305.189177		
2	ecfp6	44	42	
	43435	1562	35.500000	
	30.0	259.898758		
3	atom_pair	44	43	
	39608	1448	32.909091	
	23.0	270.904451		
4	rdkit	44	41	
	8260	1380	31.363636	
	10.5	41.782560		
5	rdk_path	44	42	
	45036	1305	29.659091	
	24.5	163.960632		
6	morgan	44	41	
	40782	1140	25.909091	
	22.0	183.549403		
7	layered	44	43	
	43621	1109	25.204545	
	25.0	113.374569		
8	topological_torsion	44	43	
	23595	754	17.136364	
	14.0	83.294958		
9	maccs	44	43	
	5957	312	7.090909	
	7.0	20.576211		
	dataset_presence_rate	selection_rate	expected_selected_if_uniform	
	enrichment_vs_uniform	selected_share	selected_magnitude_share	
0	1.000000	0.055768	3609.758423	
	1.343303	0.307541	0.145121	
1	0.977273	0.044852	1766.082684	
	1.080357	0.121012	0.180863	
2	0.954545	0.035962	1803.239337	
	0.866219	0.099068	0.154022	
3	0.977273	0.036558	1644.358320	
	0.880587	0.091837	0.160545	

4		0.931818	0.167070		342.920615	␣
↪	4.024255	0.087525		0.024761		
5		0.954545	0.028977		1869.706153	␣
↪	0.697971	0.082768		0.097167		
6		0.931818	0.027954		1693.097885	␣
↪	0.673322	0.072303		0.108776		
7		0.977273	0.025424		1810.961278	␣
↪	0.612382	0.070337		0.067189		
8		0.977273	0.031956		979.565607	␣
↪	0.769729	0.047821		0.049363		
9		0.977273	0.052375		247.309698	␣
↪	1.261576	0.019788		0.012194		

C:\Users\scott\AppData\Local\Temp\ipykernel_7244\1322586611.py:202:
PerformanceWarning: DataFrame is highly fragmented. This is usually the result
of calling `frame.insert` many times, which has poor performance. Consider
joining all columns at once using `pd.concat(axis=1)` instead. To get a de-
fragmented frame, use `newframe = frame.copy()`
.first()[["dataset", "model", "workflow", "analysis_metric",
"analysis_metric_value"]]`

1.0.11 Feature-Family Importance Summary

	feature_family	importance_source	datasets	␣
↪	total_importance_weight	mean_importance_share	median_importance_share	␣
↪	mean_family_rank	median_family_rank		
0	rdkit	selector_coefficients_fallback	45	␣
↪	37.284222	0.193694	0.104153	4.888889 ␣
↪	4.0			
1	avalon	selector_coefficients_fallback	45	␣
↪	62.028021	0.114412	0.096383	4.688889 ␣
↪	4.0			
2	fcfp6	selector_coefficients_fallback	45	␣
↪	218.657874	0.100027	0.100893	4.400000 ␣
↪	4.0			
3	atom_pair	selector_coefficients_fallback	45	␣
↪	200.542007	0.089718	0.075964	5.066667 ␣
↪	5.0			
4	ecfp6	selector_coefficients_fallback	45	␣
↪	191.625618	0.080315	0.082549	5.777778 ␣
↪	5.0			
5	layered	selector_coefficients_fallback	45	␣
↪	80.574213	0.078548	0.067064	6.488889 ␣
↪	6.0			
6	rdk_path	selector_coefficients_fallback	45	␣
↪	116.974113	0.069731	0.069845	5.844444 ␣
↪	5.0			

7	morgan	selector_coefficients_fallback	45	
↪129.718587		0.066484	0.049620	7.711111
↪	7.0			
8	erg	selector_coefficients_fallback	45	
↪35.256852		0.063682	0.043227	7.355556
↪	8.0			
9	maccs	selector_coefficients_fallback	45	
↪15.738696		0.050939	0.023172	8.288889
↪	9.0			
10	topological_torsion	selector_coefficients_fallback	45	
↪54.181602		0.048301	0.051553	8.200000
↪	8.0			
11	maplight	selector_coefficients_fallback	45	
↪50.994763		0.044149	0.039183	7.844444
↪	8.0			

1.0.12 Publication Figure: Feature-Family Importance by Dataset

feature_family	dataset				atom_pair	avalon	
↪ecfp6	erg	fcfp6	layered	maccs	maplight	morgan	rdk_path
↪ rdkit	topological_torsion						
0				chemml_cep_homo	0.075964	0.234145	0.
↪061797	0.063200	0.123114	0.063654	0.010448	0.073347	0.030002	0.145832
↪0.114578		0.003919					
1				chemml_organic_density	0.040995	0.175720	0.
↪031285	0.030337	0.057066	0.033025	0.074716	0.075767	0.021841	0.061901
↪0.361714		0.035633					
2				chemml_xyz_polarizability	0.027411	0.001784	0.
↪082549	0.000000	0.026876	0.132362	0.009842	0.000000	0.464837	0.136813
↪0.095286		0.022239					
3				esol_delaney	0.140027	0.102625	0.
↪081701	0.004187	0.100320	0.071666	0.022433	0.064704	0.043344	0.064025
↪0.254287		0.050680					
4				freesolv_sampl	0.059041	0.108149	0.
↪083729	0.019706	0.080103	0.083223	0.057709	0.045718	0.063190	0.092647
↪0.288203		0.018581					
5				lipophilicity	0.069215	0.109937	0.
↪086424	0.110834	0.135726	0.075856	0.026796	0.041607	0.045751	0.101935
↪0.131627		0.064292					
6				poduam_pod_nc_std	0.019339	0.180388	0.
↪109891	0.006120	0.131372	0.109370	0.022096	0.127098	0.100202	0.060594
↪0.069736		0.063794					
7				poduam_pod_rd_std	0.080336	0.090837	0.
↪122289	0.003917	0.162849	0.086450	0.031597	0.075283	0.119013	0.103292
↪0.038942		0.085196					

8		polaris_adme_fang_hppb_1	0.102193	0.092261	0.	
	↵129590	0.035858	0.119935	0.058321	0.036820	0.044289 0.077870 0.118106 ↵
	↵0.125445		0.059311			
9		polaris_adme_fang_perm_1	0.112157	0.119621	0.	
	↵095562	0.059706	0.131828	0.073770	0.051967	0.043778 0.086173 0.084336 ↵
	↵0.078738		0.062363			
10		polaris_adme_fang_rclint_1	0.127290	0.087899	0.	
	↵079359	0.050722	0.112472	0.067064	0.022592	0.071055 0.094996 0.096852 ↵
	↵0.112754		0.076946			
11		polaris_adme_fang_rppb_1	0.130790	0.081099	0.	
	↵096956	0.040592	0.116583	0.030104	0.040143	0.032507 0.111173 0.094954 ↵
	↵0.128663		0.096435			
12		polaris_adme_fang_solu_1	0.145802	0.166379	0.	
	↵082774	0.091192	0.066110	0.075304	0.019689	0.040018 0.056490 0.052956 ↵
	↵0.138252		0.065033			
13		tdc_ames	0.010583	0.150545	0.	
	↵012066	0.067629	0.069121	0.048677	0.156728	0.029147 0.006519 0.018237 ↵
	↵0.421958		0.008790			
14		tdc_bbb_martins	0.116481	0.072106	0.	
	↵094335	0.042649	0.176273	0.089119	0.100796	0.032763 0.072117 0.069845 ↵
	↵0.025646		0.107870			
15		tdc_bioavailability_ma	0.198130	0.486963	0.	
	↵000000	0.000000	0.000000	0.000000	0.101690	0.111935 0.000000 0.000000 ↵
	↵0.034701		0.066581			
16		tdc_caco2_wang	0.095750	0.075334	0.	
	↵116717	0.098203	0.135304	0.060952	0.040389	0.039183 0.085741 0.089314 ↵
	↵0.095244		0.067869			
17		tdc_carcinogens_lagunin	0.000000	0.026681	0.	
	↵000000	0.050961	0.142969	0.166823	0.123994	0.056082 0.262426 0.024218 ↵
	↵0.034937		0.110909			
18		tdc_clearance_hepatocyte_az	0.174283	0.029350	0.	
	↵162413	0.033535	0.194433	0.086031	0.011164	0.035351 0.105869 0.126309 ↵
	↵0.007613		0.033648			
19		tdc_clearance_microsome_az	0.201989	0.050632	0.	
	↵150603	0.023595	0.190799	0.075642	0.008213	0.041618 0.102366 0.102298 ↵
	↵0.012811		0.039433			
20		tdc_clintox	0.034765	0.008368	0.	
	↵000000	0.101185	0.236979	0.000000	0.512952	0.000000 0.007404 0.054153 ↵
	↵0.000000		0.044195			
21		tdc_cyp1a2_veith	0.011079	0.151139	0.	
	↵006481	0.073468	0.014654	0.017373	0.002432	0.022190 0.001508 0.008681 ↵
	↵0.681268		0.009726			
22		tdc_cyp2c19_veith	0.058348	0.194566	0.	
	↵009572	0.127811	0.025708	0.013308	0.011533	0.029299 0.002486 0.010386 ↵
	↵0.510327		0.006657			

23	tdc_cyp2c9_substrate_carbonmangels	0.000000	0.000000	0.				
	↪0.000000	0.000000	0.186559	0.744525	0.037960	0.030956	0.000000	0.000000
	↪0.000000		0.000000					
24	tdc_cyp2c9_veith	0.011012	0.186821	0.				
	↪0.009355	0.080730	0.027185	0.095108	0.004845	0.026349	0.002078	0.015540
	↪0.535199		0.005777					
25	tdc_cyp2d6_substrate_carbonmangels	0.188564	0.009311	0.				
	↪125116	0.079473	0.109123	0.124163	0.068066	0.000000	0.050435	0.138419
	↪0.041518		0.065811					
26	tdc_cyp2d6_veith	0.015135	0.186561	0.				
	↪0.020251	0.099313	0.073812	0.046908	0.023172	0.084747	0.019615	0.022605
	↪0.398742		0.009138					
27	tdc_cyp3a4_substrate_carbonmangels	0.111831	0.076784	0.				
	↪210225	0.013719	0.244672	0.062832	0.026293	0.018270	0.045162	0.138658
	↪0.000000		0.051553					
28	tdc_cyp3a4_veith	0.016841	0.182382	0.				
	↪0.017728	0.097057	0.019643	0.016425	0.021244	0.032330	0.013812	0.027505
	↪0.550779		0.004253					
29	tdc_dili	0.173315	0.051422	0.				
	↪127447	0.000000	0.096079	0.225395	0.048372	0.094389	0.000000	0.145878
	↪0.018524		0.019179					
30	tdc_half_life_obach	0.000000	0.096383	0.				
	↪291356	0.043227	0.238976	0.000000	0.000000	0.084736	0.192675	0.000000
	↪0.000000		0.052648					
31	tdc_herg	0.191215	0.042511	0.				
	↪118400	0.061202	0.059186	0.096316	0.100429	0.030234	0.080558	0.098170
	↪0.065204		0.056575					
32	tdc_herg_karim	0.031842	0.217249	0.				
	↪0.009566	0.172421	0.015621	0.013141	0.011088	0.025853	0.005767	0.018568
	↪0.470938		0.007947					
33	tdc_hia_hou	0.040900	0.109123	0.				
	↪0.000000	0.201289	0.027277	0.004180	0.204910	0.093404	0.048748	0.000000
	↪0.031638		0.238532					
34	tdc_hydrationfreeenergy_freesolv	0.044181	0.129682	0.				
	↪0.042974	0.027875	0.070593	0.089701	0.090175	0.061826	0.062232	0.072573
	↪0.275109		0.033079					
35	tdc_ld50_zhu	0.020282	0.156525	0.				
	↪0.006738	0.019148	0.019915	0.097689	0.018846	0.018432	0.012557	0.040835
	↪0.584216		0.004817					
36	tdc_lipophilicity_astrazeneca	0.065034	0.126871	0.				
	↪0.083353	0.113232	0.137996	0.069065	0.030704	0.048564	0.054623	0.083758
	↪0.130438		0.056364					
37	tdc_pampa_ncats	0.117810	0.061603	0.				
	↪0.074728	0.504963	0.018485	0.027903	0.000000	0.019975	0.049620	0.020760
	↪0.104153		0.000000					

```

38          tdc_pgp_broccatelli    0.264383  0.077394  0.
↳175765  0.026297  0.100893  0.050132  0.000000  0.017695  0.087403  0.081902  ↳
↳0.055129          0.063007

39          tdc_ppbr_az    0.179934  0.061305  0.
↳116359  0.034106  0.162087  0.083167  0.018070  0.039387  0.063653  0.143319  ↳
↳0.033953          0.064659

40          tdc_skin_reaction    0.106998  0.094535  0.
↳150552  0.048436  0.016782  0.110533  0.000000  0.000000  0.041856  0.249556  ↳
↳0.068733          0.112018

41          tdc_solubility_aqsoldb    0.007768  0.035762  0.
↳002188  0.016975  0.002024  0.005891  0.010948  0.009012  0.000641  0.002564  ↳
↳0.905671          0.000555

42          tdc_tox21    0.009601  0.280835  0.
↳046723  0.068982  0.038593  0.016084  0.021114  0.073879  0.009763  0.011308  ↳
↳0.411286          0.011832

43          tdc_toxcast    0.159002  0.149519  0.
↳080560  0.021817  0.103132  0.023607  0.051933  0.014944  0.021652  0.076705  ↳
↳0.243791          0.053338

44          tdc_vdss_lombardo    0.249708  0.019413  0.
↳208718  0.000000  0.182007  0.013808  0.007340  0.028990  0.167603  0.031587  ↳
↳0.028470          0.062357

```

No per-model `model_feature_importances.csv` artifacts were found, so this section uses selector coefficient/importances as a fallback. Future benchmark runs can replace this with best-tree-model importances automatically.

No feature family met the strict drop-candidate rule in this run. That means no family looked both rare across datasets and consistently unselected.

Selector artifacts were missing for some datasets and were excluded from this section:

```

[60]: # Leaderboard-relative performance summary (using run artifacts + manual
↳reference table)

def leaderboard_metric_kind(text):
    t = str(text or "").strip().lower()
    t = t.replace("-", "_").replace(" ", "_")
    if not t:
        return None
    if "mean_squared_error" in t or t == "mse" or t.endswith("_mse"):
        return "mse"
    if "rmse" in t:
        return "rmse"
    if "mae" in t:
        return "mae"
    if "auroc" in t or "roc_auc" in t or "auc_roc" in t or t == "auc" or ("roc"
↳in t and "auc" in t):

```



```

        return "roc_auc"
    if "auprc" in t or "pr_auc" in t or "auc_pr" in t or "average_precision" in t:
        return "auprc"
    if "balanced_accuracy" in t or t == "ba":
        return "balanced_accuracy"
    if t in {"accuracy", "acc"} or t.endswith("_accuracy"):
        return "accuracy"
    if t == "mcc" or "matthews" in t:
        return "mcc"
    if "spearman" in t:
        return "spearman"
    if "pearson" in t:
        return "pearson"
    if t in {"r2", "r^2", "r_squared"} or "r2" in t or "r^2" in t:
        return "r2"
    return t

def leaderboard_direction(metric_kind, explicit_direction=None):
    explicit = str(explicit_direction or "").strip().lower()
    if explicit.startswith("lower"):
        return "lower"
    if explicit.startswith("higher"):
        return "higher"
    if metric_kind in {"mse", "rmse", "mae"}:
        return "lower"
    if metric_kind in {"r2", "spearman", "pearson", "roc_auc", "auprc",
        "balanced_accuracy", "accuracy", "mcc"}:
        return "higher"
    return "lower"

def model_metric_column(frame, metric_kind):
    if metric_kind in {"mse", "rmse"}:
        return metric_column(frame, "AUTO", metric_hint="rmse")
    if metric_kind == "r2":
        return metric_column(frame, "AUTO", metric_hint="r2")
    if metric_kind == "mae":
        return pick_column(frame.columns, ["test_mae", "Test MAE", "cv_mae",
        "CV MAE", "mae", "MAE"])
    if metric_kind == "spearman":
        return pick_column(frame.columns, ["test_spearman", "Test Spearman",
        "cv_spearman", "CV Spearman", "spearman", "Spearman"])
    if metric_kind == "pearson":
        return pick_column(frame.columns, ["test_pearson", "Test Pearson",
        "cv_pearson", "CV Pearson", "pearson", "Pearson"])

```

```

    if metric_kind == "roc_auc":
        return pick_column(frame.columns, ["test_roc_auc", "Test ROC AUC",
↪ "roc_auc", "ROC AUC", "AUROC", "test_auroc", "cv_roc_auc", "CV ROC AUC"])
    if metric_kind == "auprc":
        return pick_column(frame.columns, ["test_auprc", "AUPRC", "auprc",
↪ "cv_auprc", "CV AUPRC", "average_precision"])
    if metric_kind == "balanced_accuracy":
        return pick_column(frame.columns, ["test_balanced_accuracy", "Test
↪ Balanced Accuracy", "balanced_accuracy", "Balanced Accuracy",
↪ "cv_balanced_accuracy"])
    if metric_kind == "accuracy":
        return pick_column(frame.columns, ["test_accuracy", "accuracy",
↪ "Accuracy", "cv_accuracy", "test_balanced_accuracy", "balanced_accuracy"])
    if metric_kind == "mcc":
        return pick_column(frame.columns, ["test_mcc", "MCC", "mcc", "cv_mcc"])
    return None

def parse_numeric(series):
    return pd.to_numeric(series, errors="coerce")

def to_leaderboard_scale(value, metric_kind):
    value = float(value)
    if metric_kind == "mse":
        return value ** 2
    return value

def bool_like(value):
    return str(value).strip().lower() in {"true", "1", "yes", "y", "t"}

def choose_reference_subset(ref, dataset_name, benchmark_id=""):
    if ref is None or ref.empty:
        return pd.DataFrame()
    ref_subset = pd.DataFrame()
    if benchmark_id and "benchmark_id" in ref.columns:
        ref_subset = ref.loc[ref["benchmark_id"].astype(str).str.strip().str.
↪ lower() == str(benchmark_id).strip().lower()].copy()
    if ref_subset.empty and "dataset" in ref.columns:
        ref_subset = ref.loc[ref["dataset"].astype(str).str.strip().str.lower()
↪ == str(dataset_name).strip().lower()].copy()
    return ref_subset

```

```

def best_model_for_reference_metric(dataset_frame, metric_kind):
    metric_col = model_metric_column(dataset_frame, metric_kind)
    candidate = dataset_frame.copy()
    if "error" in candidate.columns:
        candidate = candidate.loc[candidate["error"].isna() |
        ↪(candidate["error"].astype(str).str.strip() == "")].copy()
        if metric_col is not None and metric_col in candidate.columns:
            candidate["_comparison_value_raw"] =
            ↪parse_numeric(candidate[metric_col])
            candidate["_comparison_metric_column"] = metric_col
        elif {"primary_metric", "primary_metric_value"}.issubset(candidate.columns):
            primary_kind = candidate["primary_metric"].astype(str).
            ↪apply(leaderboard_metric_kind)
            candidate = candidate.loc[primary_kind == metric_kind].copy()
            candidate["_comparison_value_raw"] =
            ↪parse_numeric(candidate["primary_metric_value"])
            candidate["_comparison_metric_column"] = "primary_metric_value"
        else:
            return None, pd.DataFrame()

    candidate = candidate.loc[candidate["_comparison_value_raw"].notna()].copy()
    if candidate.empty:
        return None, candidate
    candidate["_leaderboard_scale_value"] = candidate["_comparison_value_raw"].
    ↪apply(lambda v: to_leaderboard_scale(v, metric_kind))
    direction = leaderboard_direction(metric_kind)
    best_idx = candidate["_leaderboard_scale_value"].idxmax() if direction ==
    ↪"higher" else candidate["_leaderboard_scale_value"].idxmin()
    return candidate.loc[best_idx], candidate

def reference_values_for_metric(ref_subset, metric_kind):
    work = ref_subset.copy()
    if "leaderboard_metric_name" in work.columns:
        work["_metric_kind"] = work["leaderboard_metric_name"].astype(str).
        ↪apply(leaderboard_metric_kind)
        work = work.loc[work["_metric_kind"] == metric_kind].copy()
        if work.empty or "metric_value_numeric" not in work.columns:
            return work, np.array([], dtype=float)
        work["metric_value_numeric"] = pd.to_numeric(work["metric_value_numeric"],
        ↪errors="coerce")
        work = work.loc[work["metric_value_numeric"].notna()].copy()
        return work, work["metric_value_numeric"].to_numpy(dtype=float)

def comparison_rows_from_reference(ref):

```

```

rows = []
if ref is None or ref.empty:
    return pd.DataFrame()

work_ref = ref.copy()
if "is_top10_entry" in work_ref.columns:
    include_mask = work_ref["is_top10_entry"].isna() |
work_ref["is_top10_entry"].apply(bool_like)
    work_ref = work_ref.loc[include_mask].copy()

for dataset_name, dataset_frame in SUMMARY_METRICS.groupby("dataset",
sort=True):
    dataset_frame = dataset_frame.copy()
    benchmark_id = ""
    if "benchmark_id" in dataset_frame.columns:
        non_empty_ids = dataset_frame["benchmark_id"].dropna().astype(str)
        non_empty_ids = non_empty_ids[non_empty_ids.str.strip() != ""]
        if not non_empty_ids.empty:
            benchmark_id = non_empty_ids.iloc[0]

    ref_subset = choose_reference_subset(work_ref, dataset_name,
benchmark_id)
    if ref_subset.empty:
        continue

    metric_names = ref_subset.get("leaderboard_metric_name", pd.
Series(dtype=object)).dropna().astype(str)
    metric_kinds = []
    for metric_name in metric_names:
        kind = leaderboard_metric_kind(metric_name)
        if kind and kind not in metric_kinds:
            metric_kinds.append(kind)

    for metric_kind in metric_kinds:
        metric_ref, metric_values = reference_values_for_metric(ref_subset,
metric_kind)
        if len(metric_values) == 0:
            continue

        best_row, candidate =
best_model_for_reference_metric(dataset_frame, metric_kind)
        if best_row is None:
            continue

        explicit_direction = ""
        if "leaderboard_metric_direction" in metric_ref.columns:

```

```

        directions = metric_ref["leaderboard_metric_direction"].
↳dropna().astype(str)
        directions = directions[directions.str.strip() != ""]
        if not directions.empty:
            explicit_direction = directions.iloc[0]
        direction = leaderboard_direction(metric_kind, explicit_direction)
        sorted_values = np.sort(metric_values)
        if direction == "higher":
            sorted_values = sorted_values[::-1]

        top1 = float(sorted_values[0])
        cutoff_index = min(9, len(sorted_values) - 1)
        cutoff = float(sorted_values[cutoff_index])
        best_value = float(best_row["_leaderboard_scale_value"])
        best_value_raw = float(best_row["_comparison_value_raw"])

        if direction == "lower":
            estimated_rank = 1 + int(np.sum(metric_values < best_value))
            gap_top1 = best_value - top1
            gap_top10 = best_value - cutoff
        else:
            estimated_rank = 1 + int(np.sum(metric_values > best_value))
            gap_top1 = top1 - best_value
            gap_top10 = cutoff - best_value

        ref_metric_name = str(metric_ref["leaderboard_metric_name"].
↳dropna().astype(str).iloc[0]) if "leaderboard_metric_name" in metric_ref.
↳columns and not metric_ref["leaderboard_metric_name"].dropna().empty else
↳metric_kind

        sources = sorted(metric_ref.get("reference_source", pd.
↳Series(dtype=object)).dropna().astype(str).unique().tolist()) if
↳"reference_source" in metric_ref.columns else []

        models = metric_ref.get("model", pd.Series(dtype=object)).dropna().
↳astype(str).tolist() if "model" in metric_ref.columns else []

        rows.append(
            {
                "dataset": dataset_name,
                "benchmark_id": benchmark_id,
                "leaderboard_metric_name": ref_metric_name,
                "metric_kind": metric_kind,
                "direction": direction,
                "best_model": str(best_row.get("model", "")),
                "best_workflow": str(best_row.get("workflow", "")),
                "best_value": best_value,
                "best_value_raw_model_metric": best_value_raw,
            }
        )

```

```

        "best_value_metric_column": str(best_row.
↪get("_comparison_metric_column", "")),
        "leaderboard_top1": top1,
        "leaderboard_top10_cutoff": cutoff,
        "known_reference_count": int(len(metric_values)),
        "top10_reference_count": int(min(10, len(metric_values))),
        "reference_sources": "; ".join(sources),
        "reference_models": "; ".join(models[:20]),
        "estimated_rank_vs_top10": int(estimated_rank) if
↪estimated_rank <= 10 else 11,
        "in_top10_estimate": bool(estimated_rank <= 10),
        "gap_to_top1": float(gap_top1),
        "gap_to_top10_cutoff": float(gap_top10),
    }
    )
    return pd.DataFrame(rows)

reference_for_comparison = REFERENCE_LEADERBOARD_ROWS.copy() if
↪"REFERENCE_LEADERBOARD_ROWS" in globals() and REFERENCE_LEADERBOARD_ROWS is
↪not None else pd.DataFrame()
leaderboard_eval = comparison_rows_from_reference(reference_for_comparison)

if (leaderboard_eval is None or leaderboard_eval.empty) and
↪LEADERBOARD_COMPARISON is not None and not LEADERBOARD_COMPARISON.empty:
    leaderboard_eval = LEADERBOARD_COMPARISON.copy()

    rename_map = {
        "leaderboard_rank": "estimated_rank_vs_top10",
        "leaderboard_estimated_rank_vs_top10": "estimated_rank_vs_top10",
        "leaderboard_estimated_in_top10": "in_top10_estimate",
        "leaderboard_gap_to_top10_cutoff": "gap_to_top10_cutoff",
        "leaderboard_delta_primary": "gap_to_top1",
        "primary_metric_value": "best_value_raw_model_metric",
        "primary_metric": "best_value_metric_column",
    }
    applicable_renames = {k: v for k, v in rename_map.items() if k in
↪leaderboard_eval.columns and v not in leaderboard_eval.columns}
    if applicable_renames:
        leaderboard_eval = leaderboard_eval.rename(columns=applicable_renames)

if leaderboard_eval is None or leaderboard_eval.empty:
    display_note("No leaderboard comparison was available for this run.")
else:
    leaderboard_eval = leaderboard_eval.copy()
    if "estimated_rank_vs_top10" in leaderboard_eval.columns:

```

```

        leaderboard_eval["estimated_rank_vs_top10"] = pd.
↳to_numeric(leaderboard_eval["estimated_rank_vs_top10"], errors="coerce")
        leaderboard_eval["estimated_rank_label"] =
↳leaderboard_eval["estimated_rank_vs_top10"].apply(lambda x: ">10" if pd.
↳notna(x) and int(x) > 10 else (str(int(x)) if pd.notna(x) else "NA"))
        if "in_top10_estimate" not in leaderboard_eval.columns:
            leaderboard_eval["in_top10_estimate"] =
↳leaderboard_eval["estimated_rank_vs_top10"].fillna(11).astype(float) <= 10

        total = int(len(leaderboard_eval))
        dataset_total = int(leaderboard_eval["dataset"].nunique()) if "dataset" in
↳leaderboard_eval.columns else total
        rank_series = pd.to_numeric(leaderboard_eval.get("estimated_rank_vs_top10",
↳pd.Series(dtype=float)), errors="coerce")
        comparable_mask = rank_series.notna()
        if not comparable_mask.any() and "gap_to_top10_cutoff" in leaderboard_eval.
↳columns:
            comparable_mask = pd.
↳to_numeric(leaderboard_eval["gap_to_top10_cutoff"], errors="coerce").notna()
            comparable = int(comparable_mask.sum())

        in_top10_series = pd.to_numeric(leaderboard_eval.get("in_top10_estimate",
↳pd.Series(dtype=float)), errors="coerce")
        in_top10 = int(in_top10_series[comparable_mask].fillna(0).astype(int).
↳sum()) if comparable > 0 else 0
        top1_count = int((rank_series[comparable_mask] == 1).sum()) if comparable >
↳0 else 0

        display_note(
            f"### Leaderboard Performance Overview\n"
            f"Comparison rows: **{total}** across **{dataset_total}** datasets \n"
            f"Rows with leaderboard-comparable metrics: **{comparable}** \n"
            f"Estimated in top-10: **{in_top10}/{comparable if comparable > 0 else
↳1}** \n"
            f"Estimated #1: **{top1_count}**"
        )

        display_cols = [
            "dataset",
            "leaderboard_metric_name",
            "metric_kind",
            "direction",
            "best_model",
            "best_workflow",
            "best_value",
            "best_value_raw_model_metric",

```

```

        "best_value_metric_column",
        "leaderboard_top1",
        "leaderboard_top10_cutoff",
        "known_reference_count",
        "reference_sources",
        "estimated_rank_label",
        "in_top10_estimate",
        "gap_to_top1",
        "gap_to_top10_cutoff",
    ]
    display_cols = [c for c in display_cols if c in leaderboard_eval.columns]
    sort_cols = []
    sort_ascending = []
    if "in_top10_estimate" in leaderboard_eval.columns:
        sort_cols.append("in_top10_estimate")
        sort_ascending.append(False)
    if "gap_to_top10_cutoff" in leaderboard_eval.columns:
        sort_cols.append("gap_to_top10_cutoff")
        sort_ascending.append(True)

    display_df = leaderboard_eval[display_cols].copy()
    if sort_cols:
        display_df = display_df.sort_values(by=sort_cols,
↪ascending=sort_ascending)
        display(display_df.reset_index(drop=True))

    if "gap_to_top10_cutoff" in leaderboard_eval.columns:
        plot_df = leaderboard_eval.sort_values("gap_to_top10_cutoff",
↪ascending=True).copy()
        fig = px.bar(
            plot_df,
            x="dataset",
            y="gap_to_top10_cutoff",
            color="metric_kind" if "metric_kind" in plot_df.columns else None,
            hover_data=[col for col in ["best_model", "best_value",
↪"leaderboard_top10_cutoff", "estimated_rank_label", "known_reference_count",
↪"reference_sources"] if col in plot_df.columns],
            title="Gap to leaderboard/reference top-10 cutoff (negative =
↪better than cutoff)",
        )
        fig.add_hline(y=0.0, line_dash="dash", line_color="black")
        fig.update_layout(yaxis_title="Gap to top-10 cutoff",
↪xaxis_title="Dataset")
        fig.show()

    if "estimated_rank_label" in leaderboard_eval.columns:
        rank_order = [str(i) for i in range(1, 11)] + [">>10"]

```



```

rank_counts = (
    leaderboard_eval.groupby("estimated_rank_label", dropna=False)
    .size()
    .reset_index(name="comparison_rows")
)
rank_counts["estimated_rank_label"] = pd.
↳Categorical(rank_counts["estimated_rank_label"], categories=rank_order,
↳ordered=True)
rank_counts = rank_counts.sort_values("estimated_rank_label")
fig = px.bar(rank_counts, x="estimated_rank_label",
↳y="comparison_rows", title="Estimated leaderboard rank distribution")
fig.update_layout(xaxis_title="Estimated rank vs top-10",
↳yaxis_title="Comparison rows")
fig.show()

```

1.0.13 Leaderboard Performance Overview

Comparison rows: **37** across **37** datasets

Rows with leaderboard-comparable metrics: **37**

Estimated in top-10: **35/37**

Estimated #1: **7**

	dataset	leaderboard_metric_name	metric_kind	
		direction	best_model	
	↳best_workflow	best_value	best_value_raw_model_metric \	
0	esol_delaney	Test RMSE	rmse	
	↳lower	TabPFNRegressor	TabPFN deep learning	
	↳ 0.621439	0.621439		
1	tdc_hydrationfreeenergy_freesolv	RMSE	rmse	
	↳lower	CFA (Combinatorial Fusion)	CFA fusion	
	↳ 0.644869	0.644869		
2	tdc_ppbr_az	MAE	mae	
	↳lower	MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN	
	↳ 7.329016	7.329016		
3	polaris_adme_fang_rppb_1	mean_squared_error	mse	
	↳lower	MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN	
	↳ 0.243599	0.493558		
4	lipophilicity	Test RMSE	rmse	
	↳lower	Ensemble (Weighted average (inverse train RMSE))	ensemble	
	↳ 0.594788	0.594788		
5	tdc_skin_reaction	AUROC	roc_auc	
	↳higher	Ensemble (Weighted average (inverse train RMSE))	ensemble	
	↳ 0.774839	0.774839		
6	tdc_cyp2d6_veith	AUPRC	auprc	
	↳higher	Ensemble (Weighted average (inverse train RMSE))	ensemble	
	↳ 0.727009	0.727009		

7	tdc_clintox	AUROC	roc_auc
↪higher		CatBoost	conventional
↪	0.948884	0.948884	
8	polaris_adme_fang_hppb_1	mean_squared_error	mse
↪lower	MapLight + GNN (CatBoost, Strict Parity)		MapLight + GNN
↪	0.199498	0.446652	
9	tdc_cyp3a4_veith	AUPRC	auprc
↪higher	Ensemble (OOF Stacking (RidgeCV, 5-fold))		ensemble
↪	0.889038	0.889038	
10	tdc_bioavailability_ma	AUROC	roc_auc
↪higher		CatBoost	conventional
↪	0.776688	0.776688	
11	tdc_carcinogens_lagunin	AUROC	roc_auc
↪higher		AdaBoost	conventional
↪	0.862626	0.862626	
12	polaris_adme_fang_rclint_1	mean_squared_error	mse
↪lower		CatBoost	conventional
↪	0.285363	0.534193	
13	tdc_cyp2d6_substrate_carbonmangels	AUPRC	auprc
↪higher	Chemprop v2 (AttentiveFP, ensemble=1)		Chemprop v2
↪	0.683938	0.683938	
14	tdc_vdss_lombardo	Spearman	spearman
↪higher	MapLight + GNN (CatBoost, Strict Parity)		MapLight + GNN
↪	0.671327	0.671327	
15	tdc_clearance_hepatocyte_az	Spearman	spearman
↪higher	MapLight + GNN (CatBoost, Strict Parity)		MapLight + GNN
↪	0.523087	0.523087	
16	tdc_toxcast	AUROC	roc_auc
↪higher		CatBoost	conventional
↪	0.791353	0.791353	
17	poduam_pod_rd_std	RMSE	rmse
↪lower		XGBoost	conventional
↪	0.554603	0.554603	
18	polaris_adme_fang_perm_1	mean_squared_error	mse
↪lower		XGBoost	conventional
↪	0.185158	0.430300	
19	tdc_cyp2c9_substrate_carbonmangels	AUPRC	auprc
↪higher	Ensemble (OOF Stacking (RidgeCV, 5-fold))		ensemble
↪	0.427480	0.427480	
20	tdc_dili	AUROC	roc_auc
↪higher	Ensemble (OOF Stacking (RidgeCV, 5-fold))		ensemble
↪	0.913043	0.913043	
21	tdc_herg	AUROC	roc_auc
↪higher		AdaBoost	conventional
↪	0.860825	0.860825	

22	tdc_solubility_aqsolddb	MAE	mae	┐
↳ lower	CFA (Combinatorial Fusion)		CFA fusion	┐
↳ 0.727744	0.727744			
23	tdc_ames	AUROC	roc_auc	┐
↳ higher	Ensemble (OOF Stacking (RidgeCV, 5-fold))		ensemble	┐
↳ 0.876017	0.876017			
24	tdc_cyp2c9_veith	AUPRC	auprc	┐
↳ higher	Ensemble (OOF Stacking (RidgeCV, 5-fold))		ensemble	┐
↳ 0.810239	0.810239			
25	poduam_pod_nc_std	RMSE	rmse	┐
↳ lower	CFA (Combinatorial Fusion)		CFA fusion	┐
↳ 0.699850	0.699850			
26	tdc_bbb_martins	AUROC	roc_auc	┐
↳ higher	Random forest		conventional	┐
↳ 0.932419	0.932419			
27	tdc_ld50_zhu	MAE	mae	┐
↳ lower	CFA (Combinatorial Fusion)		CFA fusion	┐
↳ 0.580239	0.580239			
28	tdc_lipophilicity_astrazeneca	MAE	mae	┐
↳ lower	XGBoost		conventional	┐
↳ 0.492100	0.492100			
29	tdc_caco2_wang	MAE	mae	┐
↳ lower	MapLight CatBoost (Strict Parity)		conventional	┐
↳ 0.272436	0.272436			
30	tdc_hia_hou	AUROC	roc_auc	┐
↳ higher	CFA (Combinatorial Fusion)		CFA fusion	┐
↳ 0.989918	0.989918			
31	tdc_pgp_broccatelli	AUROC	roc_auc	┐
↳ higher	HistGradientBoosting		conventional	┐
↳ 0.924020	0.924020			
32	tdc_cyp3a4_substrate_carbonmangels	AUROC	roc_auc	┐
↳ higher	LogisticRegression		conventional	┐
↳ 0.658793	0.658793			
33	tdc_clearance_microsome_az	Spearman	spearman	┐
↳ higher	MapLight + GNN (CatBoost, Strict Parity)		MapLight + GNN	┐
↳ 0.605587	0.605587			
34	tdc_tox21	AUROC	roc_auc	┐
↳ higher	Tabular MLP		conventional	┐
↳ 0.808239	0.808239			
35	polaris_adme_fang_solu_1	mean_squared_error	mse	┐
↳ lower	XGBoost		conventional	┐
↳ 0.343380	0.585986			
36	tdc_half_life_obach	Spearman	spearman	┐
↳ higher	TabPFNRegressor	TabPFN deep learning		
↳ 0.391247	0.391247			

	best_value_metric_column	leaderboard_top1	leaderboard_top10_cutoff	
	known_reference_count	reference_sources	estimated_rank_label	
	in_top10_estimate	gap_to_top1 \		
0	test_rmse	0.8851	1.7406	┐
↪	2	moleculenet	1	┐
↪	True	-0.263661		
1	test_rmse	0.6540	1.2610	┐
↪	6	manual_tdc_admet_reference	1	┐
↪	True	-0.009131		
2	test_mae	0.6790	7.9140	┐
↪	16	manual_tdc_admet_reference; tdc	3	┐
↪	True	6.650016		
3	test_rmse	0.2300	0.6340	┐
↪	10	polaris	3	┐
↪	True	0.013599		
4	test_rmse	0.7806	0.9621	┐
↪	2	moleculenet	1	┐
↪	True	-0.185812		
5	test_roc_auc	0.7410	0.4990	┐
↪	11	manual_tdc_admet_reference	1	┐
↪	True	-0.033839		
6	test_auprc	0.8110	0.4640	┐
↪	6	manual_tdc_admet_reference	2	┐
↪	True	0.083991		
7	test_roc_auc	0.9960	0.7320	┐
↪	6	manual_tdc_admet_reference	2	┐
↪	True	0.047116		
8	test_rmse	0.1430	0.3830	┐
↪	10	polaris	3	┐
↪	True	0.056498		
9	test_auprc	0.9230	0.7500	┐
↪	6	manual_tdc_admet_reference	2	┐
↪	True	0.033962		
10	test_roc_auc	0.7480	0.6400	┐
↪	8	manual_tdc_admet_reference	1	┐
↪	True	-0.028688		
11	test_roc_auc	0.8480	0.7300	┐
↪	10	manual_tdc_admet_reference	1	┐
↪	True	-0.014626		
12	test_rmse	0.2160	0.4030	┐
↪	10	polaris	4	┐
↪	True	0.069363		
13	test_auprc	0.7660	0.5700	┐
↪	7	manual_tdc_admet_reference	5	┐
↪	True	0.082062		

14		test_spearman	0.9420	0.5820	␣
↪		16 manual_tdc_admet_reference; tdc		5	␣
↪	True	0.270673			
15		test_spearman	0.6330	0.4400	␣
↪		16 manual_tdc_admet_reference; tdc		3	␣
↪	True	0.109913			
16		test_roc_auc	0.7140	0.7140	␣
↪		1 manual_tdc_admet_reference		1	␣
↪	True	-0.077353			
17		test_rmse	0.4100	0.6300	␣
↪		2 literature_static		2	␣
↪	True	0.144603			
18		test_rmse	0.1130	0.2570	␣
↪		10 polaris		6	␣
↪	True	0.072158			
19		test_auprc	0.4500	0.3600	␣
↪		6 manual_tdc_admet_reference		3	␣
↪	True	0.022520			
20		test_roc_auc	0.9450	0.8520	␣
↪		6 manual_tdc_admet_reference		5	␣
↪	True	0.031957			
21		test_roc_auc	0.8800	0.8060	␣
↪		7 manual_tdc_admet_reference		3	␣
↪	True	0.019175			
22		test_mae	0.5570	0.7760	␣
↪		17 manual_tdc_admet_reference; tdc		3	␣
↪	True	0.170744			
23		test_roc_auc	0.9120	0.8340	␣
↪		7 manual_tdc_admet_reference		2	␣
↪	True	0.035983			
24		test_auprc	0.8770	0.7700	␣
↪		6 manual_tdc_admet_reference		3	␣
↪	True	0.066761			
25		test_rmse	0.5500	0.7300	␣
↪		2 literature_static		2	␣
↪	True	0.149850			
26		test_roc_auc	0.9410	0.9030	␣
↪		7 manual_tdc_admet_reference		2	␣
↪	True	0.008581			
27		test_mae	0.2920	0.6050	␣
↪		16 manual_tdc_admet_reference; tdc		5	␣
↪	True	0.288239			
28		test_mae	0.4060	0.5150	␣
↪		17 manual_tdc_admet_reference; tdc		9	␣
↪	True	0.086100			

29		test_mae	0.2560	0.2880	␣
↪		20 manual_tdc_admet_reference; tdc		3	␣
↪	True	0.016436			
30		test_roc_auc	0.9940	0.9760	␣
↪		8 manual_tdc_admet_reference		4	␣
↪	True	0.004082			
31		test_roc_auc	0.9940	0.9110	␣
↪		8 manual_tdc_admet_reference		5	␣
↪	True	0.069980			
32		test_roc_auc	0.6920	0.6510	␣
↪		7 manual_tdc_admet_reference		7	␣
↪	True	0.033207			
33		test_spearman	0.6520	0.5990	␣
↪		17 manual_tdc_admet_reference; tdc		10	␣
↪	True	0.046413			
34		test_roc_auc	0.8670	0.8330	␣
↪		6 manual_tdc_admet_reference		7	␣
↪	True	0.058761			
35		test_rmse	0.2220	0.3290	␣
↪		10 polaris		>10	␣
↪	False	0.121380			
36		test_spearman	0.6490	0.4850	␣
↪		16 manual_tdc_admet_reference; tdc		>10	␣
↪	False	0.257753			
gap_to_top10_cutoff					
0		-1.119161			
1		-0.616131			
2		-0.584984			
3		-0.390401			
4		-0.367312			
5		-0.275839			
6		-0.263009			
7		-0.216884			
8		-0.183502			
9		-0.139038			
10		-0.136688			
11		-0.132626			
12		-0.117637			
13		-0.113938			
14		-0.089327			
15		-0.083087			
16		-0.077353			
17		-0.075397			
18		-0.071842			
19		-0.067480			
20		-0.061043			

```

21         -0.054825
22         -0.048256
23         -0.042017
24         -0.040239
25         -0.030150
26         -0.029419
27         -0.024761
28         -0.022900
29         -0.015564
30         -0.013918
31         -0.013020
32         -0.007793
33         -0.006587
34         0.024761
35         0.014380
36         0.093753

```

```

[61]: # Run completeness and leaderboard-reference context

if RUN_STATUS is None or RUN_STATUS.empty:
    display_note("No per-dataset `run_status.json` files were found in this run_
↳directory.")
else:
    status_df = RUN_STATUS.copy()
    status_df["status"] = status_df["status"].astype(str)
    display_note("### Run completeness by dataset")
    display(status_df.sort_values(["status", "dataset"], ascending=[True,
↳True]).reset_index(drop=True))

    status_counts = status_df.groupby("status", dropna=False).size().
↳reset_index(name="datasets")
    fig = px.bar(status_counts, x="status", y="datasets", title="Run status_
↳counts")
    fig.update_layout(xaxis_title="Status", yaxis_title="Dataset count")
    fig.show()

ref = REFERENCE_LEADERBOARD_ROWS.copy() if "REFERENCE_LEADERBOARD_ROWS" in
↳globals() and REFERENCE_LEADERBOARD_ROWS is not None else pd.DataFrame()
if ref.empty:
    display_note("No combined leaderboard/reference rows were available.")
else:
    if "is_top10_entry" in ref.columns:
        include_mask = ref["is_top10_entry"].isna() | ref["is_top10_entry"].
↳apply(bool_like)
        ref = ref.loc[include_mask].copy()

    display_note("### Leaderboard/reference coverage")

```

```

coverage_cols = [
    "dataset",
    "benchmark_id",
    "benchmark_suite",
    "leaderboard_metric_name",
    "leaderboard_metric_direction",
    "leaderboard_dataset_split",
    "rank",
    "model",
    "metric_value",
    "reference_source",
    "paper_short",
    "year",
]
coverage_cols = [c for c in coverage_cols if c in ref.columns]
display(ref[coverage_cols].head(50))

ref["metric_value_numeric"] = pd.to_numeric(ref.get("metric_value_numeric",
↳pd.Series(dtype=float)), errors="coerce")
    if "leaderboard_metric_name" in ref.columns:
        ref["metric_kind"] = ref["leaderboard_metric_name"].astype(str).
↳apply(leaderboard_metric_kind)
    else:
        ref["metric_kind"] = ""
    if "leaderboard_metric_direction" in ref.columns:
        ref["direction"] = [leaderboard_direction(kind, explicit) for kind,
↳explicit in zip(ref["metric_kind"], ref["leaderboard_metric_direction"])]
    else:
        ref["direction"] = ref["metric_kind"].apply(leaderboard_direction)

coverage_rows = []
group_cols = [c for c in ["dataset", "leaderboard_metric_name",
↳"metric_kind", "direction"] if c in ref.columns]
for keys, group in ref.groupby(group_cols, dropna=False):
    if not isinstance(keys, tuple):
        keys = (keys,)
    payload = dict(zip(group_cols, keys))
    values = group["metric_value_numeric"].dropna().to_numpy(dtype=float)
    direction = str(payload.get("direction", "lower"))
    if len(values):
        sorted_values = np.sort(values)
        if direction == "higher":
            sorted_values = sorted_values[::-1]
        payload["reference_rows"] = int(len(group))
        payload["best_reference"] = float(sorted_values[0])
        payload["top10_or_worst_reference"] = float(sorted_values[min(9,
↳len(sorted_values) - 1)])

```



```

else:
    payload["reference_rows"] = int(len(group))
    payload["best_reference"] = np.nan
    payload["top10_or_worst_reference"] = np.nan
    if "reference_source" in group.columns:
        payload["reference_sources"] = "; ".
        join(sorted(group["reference_source"].dropna().astype(str).unique()))
        coverage_rows.append(payload)

coverage_summary = pd.DataFrame(coverage_rows)
if not coverage_summary.empty:
    display(coverage_summary.sort_values([c for c in ["dataset",
↳"leaderboard_metric_name"] if c in coverage_summary.columns])).
↳reset_index(drop=True))

```

1.0.14 Run completeness by dataset

	dataset	status	checkpoint_stage	
↳n_metrics_rows	started_at	completed_at	elapsed_seconds	
0	chemml_cep_homo	completed		23.
↳0	2026-04-28 20:51:34	7.936		
1	chemml_organic_density	completed		22.
↳0	2026-04-28 20:51:41	6.260		
2	chemml_xyz_polarizability	completed		32.
↳0	2026-04-28 20:46:02	2.145		
3	esol_delaney	completed		24.
↳0	2026-04-28 21:06:23	8.487		
4	freesolv_sampl	completed		22.
↳0	2026-04-28 20:55:34	4.596		
5	lipophilicity	completed		21.
↳0	2026-04-29 19:10:05	12598.991		
6	poduam_pod_nc_std	completed		23.
↳0	2026-04-29 03:37:53	5680.376		
7	poduam_pod_rd_std	completed		23.
↳0	2026-04-29 10:38:47	7060.314		
8	polaris_adme_fang_hppb_1	completed		23.
↳0	2026-04-29 02:03:13	3832.986		
9	polaris_adme_fang_perm_1	completed		21.
↳0	2026-04-29 11:40:59	3731.461		
10	polaris_adme_fang_rclint_1	completed		23.
↳0	2026-04-29 15:40:06	4756.863		
11	polaris_adme_fang_rppb_1	completed		24.
↳0	2026-04-28 21:05:47	12.247		
12	polaris_adme_fang_solu_1	completed		23.
↳0	2026-04-29 08:41:07	4669.024		
13	tdc_bbb_martins	completed		21.
↳0	2026-04-29 05:51:46	8032.827		

14	tdc_bioavailability_ma	completed		22.
↳0	2026-04-28 20:55:29		114.977	
15	tdc_caco2_wang	completed		24.
↳0	2026-04-28 21:06:00		13.241	
16	tdc_carcinogens_lagunin	completed		37.
↳0	2026-04-28 20:47:50		107.785	
17	tdc_clearance_hepatocyte_az	completed		23.
↳0	2026-04-28 21:22:12		704.395	
18	tdc_clearance_microsome_az	completed		22.
↳0	2026-04-28 21:06:15		14.654	
19	tdc_clintox	completed		21.
↳0	2026-04-29 00:04:07		7871.562	
20	tdc_cyp2c9_substrate_carbonmangels	completed		24.
↳0	2026-04-28 21:03:43		112.819	
21	tdc_cyp2d6_substrate_carbonmangels	completed		24.
↳0	2026-04-28 20:59:41		122.472	
22	tdc_cyp3a4_substrate_carbonmangels	completed		24.
↳0	2026-04-28 21:05:34		111.631	
23	tdc_dili	completed		25.
↳0	2026-04-28 20:51:26		112.809	
24	tdc_half_life_obach	completed		25.
↳0	2026-04-28 21:01:50		129.317	
25	tdc_herg	completed		24.
↳0	2026-04-28 20:57:38		119.695	
26	tdc_hia_hou	completed		22.
↳0	2026-04-28 20:53:34		113.478	
27	tdc_hydrationfreeenergy_freesolv	completed		22.
↳0	2026-04-28 20:55:38		4.706	
28	tdc_pampa_ncats	completed		19.
↳0	2026-04-29 07:23:18		5491.893	
29	tdc_pgp_broccatelli	completed		21.
↳0	2026-04-28 21:52:56		1843.711	
30	tdc_ppbr_az	completed		23.
↳0	2026-04-29 14:20:49		9590.413	
31	tdc_skin_reaction	completed		25.
↳0	2026-04-28 20:49:34		103.758	
32	tdc_toxcast	completed		19.
↳0	2026-04-29 00:59:20		3312.654	
33	tdc_vdss_lombardo	completed		25.
↳0	2026-04-28 21:10:27		244.339	
34	tdc_lipophilicity_astrazeneca	running		↳
↳NaN	2026-04-29 19:10:05		NaN	

1.0.15 Leaderboard/reference coverage

	dataset	benchmark_id	
benchmark_suite	leaderboard_metric_name	leaderboard_metric_direction	
leaderboard_dataset_split	rank	\	
0	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	1		
1	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	2		
2	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	3		
3	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	4		
4	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	5		
5	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	6		
6	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	7		
7	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	8		
8	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	9		
9	tdc_carcinogens_lagunin	carcinogens_lagunin	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	10		
10	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	1		
11	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	2		
12	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	3		
13	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	4		

14	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	5		
15	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	6		
16	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	7		
17	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	8		
18	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	9		
19	tdc_skin_reaction	skin_reaction	
↳tdc	AUROC	NaN	scaffold
↳(80/10/10)	10		
20	tdc_hydrationfreeenergy_freesolv	hydrationfreeenergy_freesolv	
↳tdc	RMSE	NaN	scaffold
↳(MoleculeNet proxy)	1		
21	tdc_hydrationfreeenergy_freesolv	hydrationfreeenergy_freesolv	
↳tdc	RMSE	NaN	scaffold
↳(MoleculeNet proxy)	2		
22	tdc_hydrationfreeenergy_freesolv	hydrationfreeenergy_freesolv	
↳tdc	RMSE	NaN	scaffold
↳(MoleculeNet proxy)	3		
23	tdc_hydrationfreeenergy_freesolv	hydrationfreeenergy_freesolv	
↳tdc	RMSE	NaN	scaffold
↳(MoleculeNet proxy)	4		
24	tdc_hydrationfreeenergy_freesolv	hydrationfreeenergy_freesolv	
↳tdc	RMSE	NaN	scaffold
↳(MoleculeNet proxy)	5		
25	tdc_hydrationfreeenergy_freesolv	hydrationfreeenergy_freesolv	
↳tdc	RMSE	NaN	scaffold
↳(MoleculeNet proxy)	6		
26	tdc_half_life_obach	half_life_obach	
↳tdc	Spearman	NaN	
↳Scaffold	1		
27	tdc_half_life_obach	half_life_obach	
↳tdc	Spearman	NaN	
↳Scaffold	2		
28	tdc_half_life_obach	half_life_obach	
↳tdc	Spearman	NaN	
↳Scaffold	3		

29	tdc_half_life_obach	half_life_obach		
↳tdc	Spearman	NaN		␣
↳Scaffold	4			␣
30	tdc_half_life_obach	half_life_obach		
↳tdc	Spearman	NaN		␣
↳Scaffold	5			
31	tdc_half_life_obach	half_life_obach		
↳tdc	Spearman	NaN		␣
↳Scaffold	6			␣
32	tdc_half_life_obach	half_life_obach		
↳tdc	Spearman	NaN		␣
↳Scaffold	7			
33	tdc_half_life_obach	half_life_obach		
↳tdc	Spearman	NaN		␣
↳Scaffold	8			␣
34	tdc_half_life_obach	half_life_obach		
↳tdc	Spearman	NaN		␣
↳Scaffold	9			
35	tdc_half_life_obach	half_life_obach		
↳tdc	Spearman	NaN		␣
↳Scaffold	10			␣
36	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	1			
37	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	2			
38	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	3			
39	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	4			
40	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	5			
41	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	6			
42	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	7			
43	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1		␣
↳polaris	mean_squared_error	NaN		␣
↳ predefined	8			

44	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1	□
↳polaris	mean_squared_error	NaN	□
↳predefined	9		
45	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1	□
↳polaris	mean_squared_error	NaN	□
↳predefined	10		
46	tdc_caco2_wang	caco2_wang	□
↳tdc	MAE	NaN	□
↳Scaffold	1		
47	tdc_caco2_wang	caco2_wang	□
↳tdc	MAE	NaN	□
↳Scaffold	2		
48	tdc_caco2_wang	caco2_wang	□
↳tdc	MAE	NaN	□
↳Scaffold	3		
49	tdc_caco2_wang	caco2_wang	□
↳tdc	MAE	NaN	□
↳Scaffold	4		

		model	metric_value	□
↳reference_source	paper_short	year		
0		FATE-Tox (MTL)	0.848 +/- 0.003	□
↳literature_static	NaN	NaN		
1		FATE-Tox (STL)	0.842 +/- 0.021	□
↳literature_static	NaN	NaN		
2		MAT	0.830 +/- 0.035	□
↳literature_static	NaN	NaN		
3		Uni-Mol	0.822 +/- 0.035	□
↳literature_static	NaN	NaN		
4		GraphMVP	0.795 +/- 0.047	□
↳literature_static	NaN	NaN		
5		SVM	0.783 +/- 0.001	□
↳literature_static	NaN	NaN		
6		MLP	0.783 +/- 0.002	□
↳literature_static	NaN	NaN		
7		RF	0.751 +/- 0.032	□
↳literature_static	NaN	NaN		
8		MolCLR	0.740 +/- 0.038	□
↳literature_static	NaN	NaN		
9		Molformer	0.730 +/- 0.002	□
↳literature_static	NaN	NaN		
10		FATE-Tox (MTL)	0.741 +/- 0.008	□
↳literature_static	NaN	NaN		
11		FATE-Tox (STL)	0.733 +/- 0.006	□
↳literature_static	NaN	NaN		

12				SVM	0.731 +/- 0.000	▮
↳ literature_static	NaN	NaN				
13				Uni-Mol	0.695 +/- 0.048	▮
↳ literature_static	NaN	NaN				
14				RF	0.677 +/- 0.021	▮
↳ literature_static	NaN	NaN				
15				MAT	0.659 +/- 0.011	▮
↳ literature_static	NaN	NaN				
16				Molformer	0.643 +/- 0.016	▮
↳ literature_static	NaN	NaN				
17				GraphMVP	0.613 +/- 0.035	▮
↳ literature_static	NaN	NaN				
18				MLP	0.499 +/- 0.138	▮
↳ literature_static	NaN	NaN				
19				MolCLR	0.451 +/- 0.062	▮
↳ literature_static	NaN	NaN				
20				PrismNet	0.654 +/- 0.046	▮
↳ literature_static	NaN	NaN				
21				MolGraph-xLSTM	1.024 +/- 0.076	▮
↳ literature_static	NaN	NaN				
22				HiGNN	1.068 +/- 0.092	▮
↳ literature_static	NaN	NaN				
23				FP-GNN	1.106 +/- 0.195	▮
↳ literature_static	NaN	NaN				
24				DMPNN	1.211 +/- 0.120	▮
↳ literature_static	NaN	NaN				
25				DeeperGCN	1.261 +/- 0.022	▮
↳ literature_static	NaN	NaN				
26				CFA	0.576 +/- 0.025	▮
↳ tdc	NaN	NaN				
27				MapLight	0.562 +/- 0.008	▮
↳ tdc	NaN	NaN				
28				MapLight + GNN	0.557 +/- 0.034	▮
↳ tdc	NaN	NaN				
29				Euclia ML model	0.547 +/- 0.032	▮
↳ tdc	NaN	NaN				
30				Voting Regressor (KNN, SVM)	0.544 +/- 0.034	▮
↳ tdc	NaN	NaN				
31				MiniMol	0.495 +/- 0.042	▮
↳ tdc	NaN	NaN				
32				DeepMol (AutoML)	0.485 +/- 0.039	▮
↳ tdc	NaN	NaN				
33				Basic ML	0.438 +/- 0.011	▮
↳ tdc	NaN	NaN				
34				SimGCN	0.392 +/- 0.065	▮
↳ tdc	NaN	NaN				

35			ADMETrix	0.372 +/- 0.026		┘
↳ tdc	NaN	NaN				
36			1B_MPNN_LargeMix-and-Phenomics	0.235		┘
↳ polaris	NaN	NaN				
37			1B_MPNN_MolGPS-ens_LargeMix	0.265		┘
↳ polaris	NaN	NaN				
38			nepare	0.23		┘
↳ polaris	NaN	NaN				
39			TabPFNv2-rdkit	0.316		┘
↳ polaris	NaN	NaN				
40			nepare_chemprop	0.361		┘
↳ polaris	NaN	NaN				
41			chemma-2b-sft	0.436		┘
↳ polaris	NaN	NaN				
42			adme-fang-RPPB-1_desc2D_RandomForestRegressor	0.457		┘
↳ polaris	NaN	NaN				
43			adme-fang-RPPB-1-GIRAFFE-wae	0.634		┘
↳ polaris	NaN	NaN				
44			CheMeleon	0.517		┘
↳ polaris	NaN	NaN				
45			chemlactica-1b-sft	0.555		┘
↳ polaris	NaN	NaN				
46			CaliciBoost	0.256 +/- 0.006		┘
↳ tdc	NaN	NaN				
47			XG Boost	0.274 +/- 0.004		┘
↳ tdc	NaN	NaN				
48			MapLight	0.276 +/- 0.005		┘
↳ tdc	NaN	NaN				
49			BaseBoosting	0.285 +/- 0.005		┘
↳ tdc	NaN	NaN				
			dataset leaderboard_metric_name metric_kind			┘
↳ direction	reference_rows	best_reference	top10_or_worst_reference			┘
↳		reference_sources				
0		esol_delaney	Test RMSE	rmse		┘
↳ lower	2	0.8851	1.7406			┘
↳		moleculenet				
1		lipophilicity	Test RMSE	rmse		┘
↳ lower	2	0.7806	0.9621			┘
↳		moleculenet				
2		poduam_pod_nc_std	RMSE	rmse		┘
↳ lower	2	0.5500	0.7300			┘
↳		literature_static				
3		poduam_pod_rd_std	RMSE	rmse		┘
↳ lower	2	0.4100	0.6300			┘
↳		literature_static				

4	polaris_adme_fang_hppb_1	mean_squared_error	mse	┐
↳lower	10	0.1430	0.3830	┐
↳	polaris			
5	polaris_adme_fang_perm_1	mean_squared_error	mse	┐
↳lower	10	0.1130	0.2570	┐
↳	polaris			
6	polaris_adme_fang_rclint_1	mean_squared_error	mse	┐
↳lower	10	0.2160	0.4030	┐
↳	polaris			
7	polaris_adme_fang_rppb_1	mean_squared_error	mse	┐
↳lower	10	0.2300	0.6340	┐
↳	polaris			
8	polaris_adme_fang_solu_1	mean_squared_error	mse	┐
↳lower	10	0.2220	0.3290	┐
↳	polaris			
9	tdc_ames	AUROC	roc_auc	┐
↳higher	7	0.9120	0.8340	┐
↳	manual_tdc_admet_reference			
10	tdc_bbb_martins	AUROC	roc_auc	┐
↳higher	7	0.9410	0.9030	┐
↳	manual_tdc_admet_reference			
11	tdc_bioavailability_ma	AUROC	roc_auc	┐
↳higher	8	0.7480	0.6400	┐
↳	manual_tdc_admet_reference			
12	tdc_caco2_wang	MAE	mae	┐
↳lower	20	0.2560	0.2880	┐
↳	manual_tdc_admet_reference; tdc			
13	tdc_carcinogens_lagunin	AUROC	roc_auc	┐
↳higher	20	0.8480	0.7950	┐
↳	literature_static; manual_tdc_admet_reference			
14	tdc_clearance_hepatocyte_az	Spearman	spearman	┐
↳higher	16	0.6330	0.4400	┐
↳	manual_tdc_admet_reference; tdc			
15	tdc_clearance_microsome_az	Spearman	spearman	┐
↳higher	17	0.6520	0.5990	┐
↳	manual_tdc_admet_reference; tdc			
16	tdc_clintox	AUROC	roc_auc	┐
↳higher	12	0.9960	0.8890	┐
↳	literature_static; manual_tdc_admet_reference			
17	tdc_cyp2c9_substrate_carbonmangels	AUPRC	auprc	┐
↳higher	6	0.4500	0.3600	┐
↳	manual_tdc_admet_reference			
18	tdc_cyp2c9_veith	AUPRC	auprc	┐
↳higher	6	0.8770	0.7700	┐
↳	manual_tdc_admet_reference			

19	tdc_cyp2d6_substrate_carbonmangels	AUPRC	auprc	␣
↪higher	7	0.7660	0.5700	␣
↪	manual_tdc_admet_reference			
20	tdc_cyp2d6_veith	AUPRC	auprc	␣
↪higher	6	0.8110	0.4640	␣
↪	manual_tdc_admet_reference			
21	tdc_cyp3a4_substrate_carbonmangels	AUROC	roc_auc	␣
↪higher	7	0.6920	0.6510	␣
↪	manual_tdc_admet_reference			
22	tdc_cyp3a4_veith	AUPRC	auprc	␣
↪higher	6	0.9230	0.7500	␣
↪	manual_tdc_admet_reference			
23	tdc_dili	AUROC	roc_auc	␣
↪higher	6	0.9450	0.8520	␣
↪	manual_tdc_admet_reference			
24	tdc_half_life_obach	Spearman	spearman	␣
↪higher	16	0.6490	0.4850	␣
↪	manual_tdc_admet_reference; tdc			
25	tdc_herg	AUROC	roc_auc	␣
↪higher	7	0.8800	0.8060	␣
↪	manual_tdc_admet_reference			
26	tdc_hia_hou	AUROC	roc_auc	␣
↪higher	8	0.9940	0.9760	␣
↪	manual_tdc_admet_reference			
27	tdc_hydrationfreeenergy_freesolv	RMSE	rmse	␣
↪lower	12	0.6540	1.2110	␣
↪	literature_static; manual_tdc_admet_reference			
28	tdc_ld50_zhu	MAE	mae	␣
↪lower	16	0.2920	0.6050	␣
↪	manual_tdc_admet_reference; tdc			
29	tdc_lipophilicity_astrazeneca	MAE	mae	␣
↪lower	17	0.4060	0.5150	␣
↪	manual_tdc_admet_reference; tdc			
30	tdc_pgp_broccatelli	AUROC	roc_auc	␣
↪higher	8	0.9940	0.9110	␣
↪	manual_tdc_admet_reference			
31	tdc_ppbr_az	MAE	mae	␣
↪lower	16	0.6790	7.9140	␣
↪	manual_tdc_admet_reference; tdc			
32	tdc_skin_reaction	AUROC	roc_auc	␣
↪higher	21	0.7410	0.6770	␣
↪	literature_static; manual_tdc_admet_reference			
33	tdc_solubility_aqsolddb	MAE	mae	␣
↪lower	17	0.5570	0.7760	␣
↪	manual_tdc_admet_reference; tdc			

34		tdc_tox21	AUROC	roc_auc	␣
	↪higher	12	0.8670	0.8400	␣
	↪literature_static; manual_tdc_admet_reference				
35		tdc_toxcast	AUROC	roc_auc	␣
	↪higher	2	0.7140	0.7140	␣
	↪literature_static; manual_tdc_admet_reference				
36		tdc_vdss_lombardo	Spearman	spearman	␣
	↪higher	16	0.9420	0.5820	␣
	↪ manual_tdc_admet_reference; tdc				

1.0.16 Cost vs value diagnostics across runs

This section compares all run folders under `benchmark_results/` and summarizes runtime cost vs performance value.

- Run-level value prefers leaderboard-relative metrics when available (`leaderboard_delta_top10_best`, then `leaderboard_delta_primary`) and falls back to held-out RMSE.
- Feature-selection scaling uses explicit selector-stage timings when available and otherwise uses a clearly labeled proxy (`selector timeout cap` or `time-to-first-model`).

```
[62]: # COST_VS_VALUE_DIAGNOSTICS_MARKER
# Cross-run cost/value diagnostics + feature-selection scaling diagnostics.

benchmark_root = RUN_DIR.parent if RUN_DIR.parent.name == "benchmark_results"␣
↪else Path("./benchmark_results").resolve()
run_dirs = sorted([p for p in benchmark_root.iterdir() if p.is_dir()],␣
↪key=lambda p: p.stat().st_mtime)

def load_run_metrics(run_dir):
    summary = standardize_metric_frame(read_optional_csv(run_dir /␣
↪"summary_metrics.csv"))
    if summary is not None and not summary.empty:
        return summary

    parts = []
    for metrics_path in sorted(run_dir.glob("*/metrics.csv")):
        dataset_name = metrics_path.parent.name
        frame = standardize_metric_frame(pd.read_csv(metrics_path))
        if frame is None or frame.empty:
            continue
        if "dataset" not in frame.columns:
            frame["dataset"] = dataset_name
        parts.append(frame)

    if not parts:
        return None
```

```

return pd.concat(parts, ignore_index=True)

def load_run_status(run_dir):
    rows = []
    for status_path in sorted(run_dir.glob("*/run_status.json")):
        payload = read_optional_json(status_path)
        if not isinstance(payload, dict):
            continue
        rows.append(
            {
                "dataset": status_path.parent.name,
                "status": payload.get("status", "unknown"),
                "checkpoint_stage": payload.get("checkpoint_stage", ""),
                "started_at": payload.get("started_at", ""),
                "completed_at": payload.get("completed_at", ""),
                "elapsed_seconds": payload.get("elapsed_seconds", np.nan),
            }
        )
    status_df = pd.DataFrame(rows)
    if not status_df.empty and "elapsed_seconds" in status_df.columns:
        status_df["elapsed_seconds"] = pd.
↳to_numeric(status_df["elapsed_seconds"], errors="coerce")
    return status_df

def bool_like_series(series):
    if series is None:
        return pd.Series(dtype=bool)
    return series.astype(str).str.strip().str.lower().isin({"true", "1", "yes",
↳"y", "t"})

def parse_stage_runtime_seconds(stage_frame):
    if stage_frame is None or stage_frame.empty:
        return np.nan
    label_col = pick_column(stage_frame.columns, ["stage_label", "stage",
↳"step", "name"])
    seconds_col = pick_column(stage_frame.columns, ["elapsed_seconds",
↳"duration_seconds", "seconds", "elapsed"])
    if label_col is None or seconds_col is None:
        return np.nan

    labels = stage_frame[label_col].astype(str).str.lower()
    mask = labels.str.contains("feature") | labels.str.contains("select")
    selector_rows = stage_frame.loc[mask].copy()
    if selector_rows.empty:

```

```

        return np.nan

    selector_rows[seconds_col] = pd.to_numeric(selector_rows[seconds_col],
↪errors="coerce")
    return float(selector_rows[seconds_col].sum()) if
↪selector_rows[seconds_col].notna().any() else np.nan

run_rows = []
dataset_cost_value_rows = []
selector_rows = []

for run_dir in run_dirs:
    run_name = run_dir.name
    run_metrics = load_run_metrics(run_dir)
    if run_metrics is None or run_metrics.empty:
        continue

    run_metrics = run_metrics.copy()
    rmse_col_run = metric_column(run_metrics, "AUTO", metric_hint="rmse")
    if rmse_col_run is None:
        continue

    run_metrics[rmse_col_run] = pd.to_numeric(run_metrics[rmse_col_run],
↪errors="coerce")

    numeric_candidates = [
        "elapsed_seconds",
        "n_molecules",
        "n_train",
        "n_test",
        "selected_feature_count",
        "original_feature_count",
        "leaderboard_delta_top10_best",
        "leaderboard_delta_primary",
        "leaderboard_rank",
        "leaderboard_estimated_rank_vs_top10",
        "leaderboard_gap_to_top10_cutoff",
        "selector_elasticnet_timeout_seconds",
    ]
    for col in numeric_candidates:
        if col in run_metrics.columns:
            run_metrics[col] = pd.to_numeric(run_metrics[col], errors="coerce")

    run_status = load_run_status(run_dir)

    best_by_dataset = (

```

```

        run_metrics.sort_values(rmse_col_run, ascending=True,
↪na_position="last")
        .groupby("dataset", as_index=False)
        .first()
    )

    dataset_elapsed_map = {}
    if not run_status.empty and "dataset" in run_status.columns and
↪"elapsed_seconds" in run_status.columns:
        dataset_elapsed_map = run_status.
↪set_index("dataset")["elapsed_seconds"].to_dict()

    status_completed_count = np.nan
    if not run_status.empty and "status" in run_status.columns:
        status_completed_count = int((run_status["status"].astype(str).str.
↪lower() == "completed").sum())

    total_elapsed_seconds = np.nan
    if not run_status.empty and "elapsed_seconds" in run_status.columns and
↪run_status["elapsed_seconds"].notna().any():
        total_elapsed_seconds = float(run_status["elapsed_seconds"].sum())

    if not np.isfinite(total_elapsed_seconds):
        if "dataset" in run_metrics.columns and "elapsed_seconds" in
↪run_metrics.columns:
            elapsed_proxy = run_metrics.groupby("dataset",
↪dropna=False)["elapsed_seconds"].max()
            if elapsed_proxy.notna().any():
                total_elapsed_seconds = float(elapsed_proxy.sum())

    delta_col = pick_column(best_by_dataset.columns,
↪["leaderboard_delta_top10_best", "leaderboard_delta_primary"])
    rank_col = pick_column(best_by_dataset.columns,
↪["leaderboard_estimated_rank_vs_top10", "leaderboard_rank"])
    top10_col = pick_column(best_by_dataset.columns,
↪["leaderboard_estimated_in_top10", "leaderboard_in_top10"])

    delta_values = pd.to_numeric(best_by_dataset[delta_col], errors="coerce")
↪if delta_col is not None else pd.Series(dtype=float)
    rank_values = pd.to_numeric(best_by_dataset[rank_col], errors="coerce") if
↪rank_col is not None else pd.Series(dtype=float)

    top10_rate = np.nan
    if top10_col is not None:
        bool_series = best_by_dataset[top10_col].astype(str).str.lower().
↪map({"true": True, "false": False})

```

```

        if bool_series.notna().any():
            top10_rate = float(bool_series.mean())

run_rows.append(
    {
        "run_name": run_name,
        "run_mtime": pd.Timestamp(run_dir.stat().st_mtime, unit="s"),
        "datasets": int(best_by_dataset["dataset"].nunique()),
        "completed_datasets": status_completed_count,
        "models": int(run_metrics["model"].nunique()) if "model" in
↪run_metrics.columns else np.nan,
        "total_elapsed_seconds": total_elapsed_seconds,
        "total_elapsed_hours": total_elapsed_seconds / 3600.0 if np.
↪isfinite(total_elapsed_seconds) else np.nan,
        "mean_best_rmse": float(best_by_dataset[rmse_col_run].
↪mean(skipna=True)),
        "median_best_rmse": float(best_by_dataset[rmse_col_run].
↪median(skipna=True)),
        "mean_leaderboard_delta": float(delta_values.mean(skipna=True)) if
↪len(delta_values) and delta_values.notna().any() else np.nan,
        "median_leaderboard_delta": float(delta_values.median(skipna=True))
↪if len(delta_values) and delta_values.notna().any() else np.nan,
        "mean_estimated_rank": float(rank_values.mean(skipna=True)) if
↪len(rank_values) and rank_values.notna().any() else np.nan,
        "top10_hit_rate": top10_rate,
        "value_metric_used": delta_col if delta_col is not None else
↪rmse_col_run,
    }
)

# Explicit selector stage timings (if emitted by future benchmark runs)
explicit_selector_seconds_by_dataset = {}
step_runtime_summary = read_optional_csv(run_dir / "step_runtime_summary.
↪csv")

if step_runtime_summary is not None and not step_runtime_summary.empty:
    ds_col = pick_column(step_runtime_summary.columns, ["dataset",
↪"dataset_name"])
    label_col = pick_column(step_runtime_summary.columns, ["stage_label",
↪"stage", "step", "name"])
    sec_col = pick_column(step_runtime_summary.columns, ["elapsed_seconds",
↪"duration_seconds", "seconds", "elapsed"])
    if ds_col is not None and label_col is not None and sec_col is not None:
        tmp = step_runtime_summary.copy()
        tmp[sec_col] = pd.to_numeric(tmp[sec_col], errors="coerce")
        tmp = tmp.loc[tmp[label_col].astype(str).str.lower().str.
↪contains("feature|select", regex=True)].copy()

```

```

        if not tmp.empty:
            grouped = tmp.groupby(ds_col, dropna=False)[sec_col].
↪sum(min_count=1)
            explicit_selector_seconds_by_dataset.update(grouped.to_dict())

    for step_path in run_dir.glob("*/step_runtime.csv"):
        dataset_name = step_path.parent.name
        if dataset_name in explicit_selector_seconds_by_dataset:
            continue
        stage_frame = read_optional_csv(step_path)
        seconds = parse_stage_runtime_seconds(stage_frame)
        if np.isfinite(seconds):
            explicit_selector_seconds_by_dataset[dataset_name] = float(seconds)

    for dataset_name, dataset_frame in run_metrics.groupby("dataset",
↪dropna=False):
        dataset_frame = dataset_frame.copy()
        dataset_frame = dataset_frame.sort_values(rmse_col_run, ascending=True,
↪na_position="last")
        best_row = dataset_frame.iloc[0]

        dataset_elapsed = dataset_elapsed_map.get(dataset_name, np.nan)
        if not np.isfinite(dataset_elapsed) and "elapsed_seconds" in
↪dataset_frame.columns:
            dataset_elapsed = float(dataset_frame["elapsed_seconds"]).
↪max(skipna=True))

        selector_seconds = np.nan
        selector_seconds_source = "unavailable"

        if dataset_name in explicit_selector_seconds_by_dataset:
            selector_seconds =
↪float(explicit_selector_seconds_by_dataset[dataset_name])
            selector_seconds_source = "stage_runtime"
        else:
            timed_out = False
            if "selector_timed_out" in dataset_frame.columns:
                timed_out =
↪bool(bool_like_series(dataset_frame["selector_timed_out"]).any())

            timeout_cap = np.nan
            if "selector_elasticnet_timeout_seconds" in dataset_frame.columns:
                timeout_candidates = pd.
↪to_numeric(dataset_frame["selector_elasticnet_timeout_seconds"],
↪errors="coerce")
                if timeout_candidates.notna().any():

```



```

        timeout_cap = float(timeout_candidates.dropna().iloc[0])

    if timed_out and np.isfinite(timeout_cap):
        selector_seconds = timeout_cap
        selector_seconds_source = "selector_timeout_cap"
    elif "elapsed_seconds" in dataset_frame.columns:
        first_elapsed = float(pd.
↳to_numeric(dataset_frame["elapsed_seconds"], errors="coerce").
↳min(skipna=True))
        if np.isfinite(first_elapsed):
            selector_seconds = first_elapsed
            selector_seconds_source = "first_model_elapsed_proxy"

    n_molecules_value = best_row.get("n_molecules", np.nan)
    n_train_value = best_row.get("n_train", np.nan)
    n_test_value = best_row.get("n_test", np.nan)

    selector_rows.append(
        {
            "run_name": run_name,
            "dataset": dataset_name,
            "n_molecules": pd.to_numeric(n_molecules_value,
↳errors="coerce"),
            "n_train": pd.to_numeric(n_train_value, errors="coerce"),
            "n_test": pd.to_numeric(n_test_value, errors="coerce"),
            "selector_seconds": pd.to_numeric(selector_seconds,
↳errors="coerce"),
            "selector_seconds_source": selector_seconds_source,
            "selector_timed_out": bool(bool_like_series(dataset_frame.
↳get("selector_timed_out", pd.Series([False]))).any()),
            "selector_method": best_row.get("selector_method", ""),
            "selected_feature_count": pd.to_numeric(best_row.
↳get("selected_feature_count", np.nan), errors="coerce"),
            "original_feature_count": pd.to_numeric(best_row.
↳get("original_feature_count", np.nan), errors="coerce"),
        }
    )

    dataset_cost_value_rows.append(
        {
            "run_name": run_name,
            "dataset": dataset_name,
            "dataset_elapsed_seconds": pd.to_numeric(dataset_elapsed,
↳errors="coerce"),
            "best_rmse": pd.to_numeric(best_row.get(rmse_col_run, np.nan),
↳errors="coerce"),

```

```

        "leaderboard_delta_top10_best": pd.to_numeric(best_row.
↳get("leaderboard_delta_top10_best", np.nan), errors="coerce"),
        "leaderboard_delta_primary": pd.to_numeric(best_row.
↳get("leaderboard_delta_primary", np.nan), errors="coerce"),
        "leaderboard_estimated_rank_vs_top10": pd.to_numeric(best_row.
↳get("leaderboard_estimated_rank_vs_top10", np.nan), errors="coerce"),
        "n_molecules": pd.to_numeric(best_row.get("n_molecules", np.
↳nan), errors="coerce"),
    }
)

RUN_COST_VALUE = pd.DataFrame(run_rows)
DATASET_COST_VALUE = pd.DataFrame(dataset_cost_value_rows)
SELECTOR_SCALING = pd.DataFrame(selector_rows)

if RUN_COST_VALUE.empty:
    display_note("No benchmark runs were detected under `benchmark_results/`_
↳with usable metrics.")
else:
    run_table = RUN_COST_VALUE.copy().sort_values("run_mtime", ascending=False)

    if not SELECTOR_SCALING.empty:
        selector_agg = (
            SELECTOR_SCALING.groupby("run_name", dropna=False)
            .agg(
                selector_timeout_rate=("selector_timed_out", "mean"),
                mean_selector_seconds=("selector_seconds", "mean"),
                median_selector_seconds=("selector_seconds", "median"),
                selector_rows=("dataset", "size"),
            )
            .reset_index()
        )
        run_table = run_table.merge(selector_agg, on="run_name", how="left")

    display_note("### Run-level cost vs value")
    display(
        run_table[
            [
                col
                for col in [
                    "run_name",
                    "run_mtime",
                    "datasets",
                    "completed_datasets",
                    "models",
                    "total_elapsed_hours",
                    "mean_best_rmse",

```

```

        "mean_leaderboard_delta",
        "mean_estimated_rank",
        "top10_hit_rate",
        "selector_timeout_rate",
        "mean_selector_seconds",
        "value_metric_used",
    ]
    if col in run_table.columns
]
].reset_index(drop=True)
)

run_plot_df = run_table.copy()
if run_plot_df["total_elapsed_hours"].notna().sum() >= 2:
    value_col = "mean_leaderboard_delta" if
↪run_plot_df["mean_leaderboard_delta"].notna().sum() >= 2 else
↪"mean_best_rmse"
    fig = px.scatter(
        run_plot_df,
        x="total_elapsed_hours",
        y=value_col,
        color="run_name",
        hover_data=[c for c in ["datasets", "completed_datasets", "models",
↪"mean_selector_seconds", "selector_timeout_rate"] if c in run_plot_df.
↪columns],
        title=f"Run-level cost vs value ({value_col})",
    )
    fig.update_layout(xaxis_title="Total runtime across datasets (hours)",
↪yaxis_title=value_col)
    fig.show()

if not DATASET_COST_VALUE.empty:
    dataset_plot = DATASET_COST_VALUE.copy()
    dataset_plot["dataset_elapsed_hours"] =
↪dataset_plot["dataset_elapsed_seconds"] / 3600.0
    value_col_dataset = (
        "leaderboard_delta_top10_best"
        if dataset_plot["leaderboard_delta_top10_best"].notna().sum() >= 3
        else "best_rmse"
    )
    fig = px.scatter(
        dataset_plot,
        x="dataset_elapsed_hours",
        y=value_col_dataset,
        color="run_name",
        hover_data=[c for c in ["dataset", "n_molecules",
↪"leaderboard_estimated_rank_vs_top10"] if c in dataset_plot.columns],

```

```

        title=f"Dataset-level runtime vs value ({value_col_dataset})",
    )
    fig.update_layout(xaxis_title="Dataset runtime (hours)",
        ↪yaxis_title=value_col_dataset)
    fig.show()

# Current-run model-level cost vs relative performance.
MODEL_COST_VALUE = pd.DataFrame()
model_cost_source = SUMMARY_ANALYSIS_METRICS.copy()
model_cost_error_col = pick_column(model_cost_source.columns, ["error",
    ↪"Error"])
if model_cost_error_col is not None:
    model_cost_source = model_cost_source.loc[
        model_cost_source[model_cost_error_col].isna()
        | (model_cost_source[model_cost_error_col].astype(str).str.strip() ==
    ↪"")
    ].copy()

model_cost_source = add_analysis_metric_columns(model_cost_source,
    ↪rmse_col=RMSE_COL, classification_col=BALANCED_ACCURACY_COL)

if "elapsed_seconds" in model_cost_source.columns and not model_cost_source.
    ↪empty:
    model_cost_source["elapsed_seconds"] = pd.
    ↪to_numeric(model_cost_source["elapsed_seconds"], errors="coerce")
    model_cost_source["analysis_metric_value"] = pd.
    ↪to_numeric(model_cost_source["analysis_metric_value"], errors="coerce")
    model_cost_source["analysis_delta_from_best"] = pd.
    ↪to_numeric(model_cost_source["analysis_delta_from_best"], errors="coerce")
    MODEL_COST_VALUE = model_cost_source.loc[
        model_cost_source["elapsed_seconds"].notna()
        & model_cost_source["analysis_metric_value"].notna()
        & model_cost_source["analysis_delta_from_best"].notna()
    ].copy()

if MODEL_COST_VALUE.empty:
    display_note("No per-model runtime rows with usable relative performance,
    ↪metrics were available.")
else:
    display_note("### Model cost vs relative performance")
    display(
        MODEL_COST_VALUE.sort_values(["task_kind", "analysis_delta_from_best",
    ↪"elapsed_seconds"])
        [[
            col

```

```

        for col in [
            "dataset",
            "task_kind",
            "model",
            "workflow",
            "elapsed_seconds",
            "analysis_metric",
            "analysis_metric_value",
            "analysis_delta_from_best",
        ]
        if col in MODEL_COST_VALUE.columns
    ]]
    .reset_index(drop=True)
)

cost_fig = px.scatter(
    MODEL_COST_VALUE,
    x="elapsed_seconds",
    y="analysis_delta_from_best",
    color="model",
    symbol="task_kind",
    facet_col="task_kind",
    facet_col_wrap=2,
    hover_data=[c for c in ["dataset", "workflow", "analysis_metric",
↪ "analysis_metric_value"] if c in MODEL_COST_VALUE.columns],
    title="Model runtime vs delta from dataset-best performance",
)
cost_fig.update_xaxes(type="log", title="Model runtime (seconds, log
↪ scale)")
cost_fig.update_yaxes(title="Delta from dataset-best metric (lower is
↪ better)")
cost_fig.show()

model_cost_summary = (
    MODEL_COST_VALUE.groupby(["model", "workflow", "task_kind"],
↪ dropna=False)
    .agg(
        datasets=("dataset", "nunique"),
        median_elapsed_seconds=("elapsed_seconds", "median"),
        mean_elapsed_seconds=("elapsed_seconds", "mean"),
        median_delta_from_best=("analysis_delta_from_best", "median"),
        mean_delta_from_best=("analysis_delta_from_best", "mean"),
    )
    .reset_index()
    .sort_values(["median_delta_from_best", "median_elapsed_seconds"],
↪ ascending=[True, True])
)

```

```

display_note("### Model cost/value summary")
display(model_cost_summary.reset_index(drop=True))

def publication_model_family(model_name, workflow_name=""):
    model_text = str(model_name or "").strip().lower()
    workflow_text = str(workflow_name or "").strip().lower()
    if model_text.startswith("cfa") or "cfa" in workflow_text:
        return "CFA"
    if model_text.startswith("ensemble") or "ensemble" in workflow_text:
        return "Ensemble"
    if "maplight + gnn" in workflow_text or "maplight + gnn" in model_text:
        return "MapLight+GNN"
    if "chemprop" in workflow_text or "chemprop" in model_text:
        return "Chemprop"
    if "tabpfnn" in workflow_text or "tabpfnn" in model_text:
        return "TabPFN"
    if "chemml" in workflow_text or "mlp" in model_text or "cnn" in_
↪model_text:
        return "Other neural baselines"
    return "Conventional ML"

def current_run_hardware_note(model_family):
    gpu_available = bool(RUN_CONFIG.get("gpu_available", False)) if_
↪isinstance(RUN_CONFIG, dict) else False
    if gpu_available:
        return "GPU available in current run; CPU/GPU use depends on_
↪backend"
    if model_family in {"Chemprop", "TabPFN", "MapLight+GNN", "Other neural_
↪baselines"}:
        return "CPU in current run; backend may support GPU"
    return "CPU in current run"

def trainable_parameter_note(model_family, exact_parameter_value=np.nan):
    if pd.isna(exact_parameter_value):
        return f"median recorded trainable parameters:_
↪{int(round(float(exact_parameter_value))),}"
    return {
        "Conventional ML": "Not applicable (non-neural estimators)",
        "CFA": "Not applicable (fusion over fitted model outputs)",
        "Ensemble": "Not applicable for averaging/linear meta-model; base_
↪model parameters counted separately",
        "MapLight+GNN": "Not recorded in benchmark artifacts; CatBoost head_
↪plus transferred graph features",
        "Chemprop": "Model-dependent; not recorded in benchmark artifacts",
        "TabPFN": "Pretrained model; no task-specific fine-tuning_
↪parameters recorded",
    }

```

```

        "Other neural baselines": "Model-dependent; not recorded in_
↳benchmark artifacts",
    }.get(model_family, "Not recorded")

inference_col = pick_column(
    MODEL_COST_VALUE.columns,
    [
        "test_prediction_seconds",
        "test_inference_seconds",
        "prediction_seconds",
        "predict_seconds",
        "inference_seconds",
        "inference_elapsed_seconds",
        "prediction_elapsed_seconds",
    ],
)
inference_count_col = pick_column(MODEL_COST_VALUE.columns,
↳["test_inference_molecules", "n_test", "test_rows", "inference_molecules",
↳"n_molecules"])
parameter_count_col = pick_column(MODEL_COST_VALUE.columns,
↳["model_trainable_parameters", "trainable_parameters", "parameter_count",
↳"n_parameters"])

publication_cost_source = MODEL_COST_VALUE.copy()
workflow_values = publication_cost_source["workflow"] if "workflow" in_
↳publication_cost_source.columns else pd.Series("",
↳index=publication_cost_source.index)
publication_cost_source["publication_model_family"] = [
    publication_model_family(model_name, workflow_name)
    for model_name, workflow_name in zip(publication_cost_source["model"],
↳workflow_values)
]
if inference_col is not None and inference_count_col is not None:
    publication_cost_source[inference_col] = pd.
↳to_numeric(publication_cost_source[inference_col], errors="coerce")
    publication_cost_source[inference_count_col] = pd.
↳to_numeric(publication_cost_source[inference_count_col], errors="coerce")
    publication_cost_source["inference_seconds_per_1000_molecules"] = (
        publication_cost_source[inference_col] /
↳publication_cost_source[inference_count_col].replace(0, np.nan) * 1000.0
    )
else:
    publication_cost_source["inference_seconds_per_1000_molecules"] = np.nan
if parameter_count_col is not None:
    publication_cost_source[parameter_count_col] = pd.
↳to_numeric(publication_cost_source[parameter_count_col], errors="coerce")

```

```

        publication_cost_source["recorded_trainable_parameters"] =
↪publication_cost_source[parameter_count_col]
        else:
            publication_cost_source["recorded_trainable_parameters"] = np.nan

    autoqsar_publication_cost = (
        publication_cost_source.groupby("publication_model_family",
↪dropna=False)
        .agg(
            source=("publication_model_family", lambda _: "AutoQSAR current
↪benchmark run"),
            datasets=("dataset", "nunique"),
            model_rows=("model", "size"),
            median_wall_clock_seconds=("elapsed_seconds", "median"),
            mean_wall_clock_seconds=("elapsed_seconds", "mean"),

            ↪
↪median_inference_seconds_per_1000_molecules=("inference_seconds_per_1000_molecules",
↪"median"),

            ↪
↪median_recorded_trainable_parameters=("recorded_trainable_parameters",
↪"median"),
        )
        .reset_index()
        .rename(columns={"publication_model_family": "model_family"})
    )
    autoqsar_publication_cost["hardware"] =
↪autoqsar_publication_cost["model_family"].map(current_run_hardware_note)
    autoqsar_publication_cost["trainable_parameters"] =
↪autoqsar_publication_cost.apply(
        lambda row: trainable_parameter_note(row["model_family"], row.
↪get("median_recorded_trainable_parameters", np.nan)),
        axis=1,
    )
    autoqsar_publication_cost["published_cost_comparison"] = "Measured in this
↪benchmark run; inference-only timing/parameter fields are populated when
↪emitted by benchmark artifacts"

    external_cost_rows = pd.DataFrame(
        [
            {
                "model_family": "MolGPS",
                "source": "Published comparator",
                "datasets": np.nan,
                "model_rows": np.nan,
                "median_wall_clock_seconds": np.nan,
                "mean_wall_clock_seconds": np.nan,
            }
        ]
    )

```



```

        "median_inference_seconds_per_1000_molecules": np.nan,
        "hardware": "GPU pretraining/inference reported in literature",
        "trainable_parameters": "~3B parameters",
        "published_cost_comparison": "Foundation-scale graph model;␣
↪include exact runtime from cited paper if used in manuscript table",
    },
    {
        "model_family": "MolE",
        "source": "Published comparator",
        "datasets": np.nan,
        "model_rows": np.nan,
        "median_wall_clock_seconds": np.nan,
        "mean_wall_clock_seconds": np.nan,
        "median_inference_seconds_per_1000_molecules": np.nan,
        "hardware": "GPU pretraining reported in literature",
        "trainable_parameters": "~100M parameters",
        "published_cost_comparison": "Pretrained on ~842M molecules;␣
↪include exact runtime from cited paper if used in manuscript table",
    },
    {
        "model_family": "ADMET-AI",
        "source": "Published comparator",
        "datasets": np.nan,
        "model_rows": np.nan,
        "median_wall_clock_seconds": np.nan,
        "mean_wall_clock_seconds": np.nan,
        "median_inference_seconds_per_1000_molecules": np.nan,
        "hardware": "GPU-capable Chemprop-RDKit deployment",
        "trainable_parameters": "Chemprop-RDKit; exact parameter count␣
↪not recorded here",
        "published_cost_comparison": "Published throughput reference:␣
↪~3.1 hours for 1M molecules",
    },
]
)

PUBLICATION_COST_TABLE = pd.concat([autoqsar_publication_cost,␣
↪external_cost_rows], ignore_index=True, sort=False)
display_note("### Publication-grade computational cost comparison")
display(
    PUBLICATION_COST_TABLE[
        [
            "model_family",
            "source",
            "datasets",
            "model_rows",
            "median_wall_clock_seconds",

```

```

        "mean_wall_clock_seconds",
        "hardware",
        "trainable_parameters",
        "median_recorded_trainable_parameters",
        "median_inference_seconds_per_1000_molecules",
        "published_cost_comparison",
    ]
    ].sort_values(["source", "model_family"]).reset_index(drop=True)
)
if inference_col is None:
    display_note(
        "Inference-only timing columns were not found in the current
↳ benchmark artifacts. "
        "The table keeps the inference-time field explicit so future runs
↳ can populate it without changing the manuscript table schema."
    )

# Paired statistical comparison of cost-performance effectiveness.
effectiveness_df = MODEL_COST_VALUE.copy()
effectiveness_df["elapsed_seconds"] = pd.
↳to_numeric(effectiveness_df["elapsed_seconds"], errors="coerce")
effectiveness_df["analysis_delta_from_best"] = pd.
↳to_numeric(effectiveness_df["analysis_delta_from_best"], errors="coerce")
effectiveness_df = effectiveness_df.loc[
    effectiveness_df["elapsed_seconds"].gt(0)
    & effectiveness_df["analysis_delta_from_best"].notna()
].copy()

if effectiveness_df.empty:
    display_note("No rows were available for cost-performance statistical
↳ testing.")
else:
    effectiveness_df["log_elapsed_seconds"] = np.
↳log10(effectiveness_df["elapsed_seconds"].clip(lower=1e-9))
    effectiveness_df["performance_percentile"] = effectiveness_df.
↳groupby("dataset")["analysis_delta_from_best"].rank(method="average",
↳pct=True, ascending=True)
    effectiveness_df["runtime_percentile"] = effectiveness_df.
↳groupby("dataset")["log_elapsed_seconds"].rank(method="average", pct=True,
↳ascending=True)
    effectiveness_df["cost_performance_score"] = 0.75 *
↳effectiveness_df["performance_percentile"] + 0.25 *
↳effectiveness_df["runtime_percentile"]
    effectiveness_df["cost_performance_ratio"] =
↳effectiveness_df["analysis_delta_from_best"] /
↳effectiveness_df["elapsed_seconds"].clip(lower=1e-9)

```

```

effectiveness_summary = (
    effectiveness_df.groupby(["model", "workflow"], dropna=False)
    .agg(
        datasets=("dataset", "nunique"),
        median_cost_performance_score=("cost_performance_score",
↪ "median"),
        mean_cost_performance_score=("cost_performance_score", "mean"),
        median_performance_percentile=("performance_percentile",
↪ "median"),
        median_runtime_percentile=("runtime_percentile", "median"),
        median_delta_from_best=("analysis_delta_from_best", "median"),
        median_elapsed_seconds=("elapsed_seconds", "median"),
    )
    .reset_index()
    .sort_values(["median_cost_performance_score",
↪ "median_delta_from_best", "median_elapsed_seconds"], ascending=[True, True,
↪ True])
    )
    display_note("### Cost-performance effectiveness summary")
    display(effectiveness_summary.reset_index(drop=True))

    min_datasets_for_test = max(5, int(math.ceil(0.50 *
↪ effectiveness_df["dataset"].nunique()))
    eligible_models = (
        effectiveness_df.groupby("model")["dataset"]
        .nunique()
        .loc[lambda s: s >= min_datasets_for_test]
        .index
        .tolist()
    )

    score_matrix = effectiveness_df.loc[effectiveness_df["model"]
↪ isin(eligible_models)].pivot_table(
        index="dataset",
        columns="model",
        values="cost_performance_score",
        aggfunc="mean",
    )
    complete_matrix = score_matrix.dropna(axis=0, how="any")

    stat_notes = []
    test_result_rows = []
    pairwise_rows = []
    best_effective_model = None

    try:

```

```

        from scipy import stats
    except Exception as exc:
        stats = None
        stat_notes.append(f"SciPy was not available, so formal Friedman/
↳Wilcoxon tests were skipped: {exc}")

    if stats is None:
        display_note("### Cost-performance statistical test")
        display_note(" ".join(stat_notes))
    elif complete_matrix.shape[0] < 5 or complete_matrix.shape[1] < 3:
        display_note("### Cost-performance statistical test")
        display_note(
            "Not enough paired complete model-by-dataset scores for a
↳Friedman test "
            f"(datasets={complete_matrix.shape[0]}, models={complete_matrix.
↳shape[1]})."
        )
    else:
        friedman_stat, friedman_p = stats.
↳friedmanchisquare(*[complete_matrix[col].to_numpy(dtype=float) for col in
↳complete_matrix.columns])
        test_result_rows.append(
            {
                "test": "Friedman repeated-measures test",
                "datasets": int(complete_matrix.shape[0]),
                "models": int(complete_matrix.shape[1]),
                "statistic": float(friedman_stat),
                "p_value": float(friedman_p),
                "score": "0.75*within-dataset performance percentile + 0.
↳25*within-dataset runtime percentile; lower is better",
            }
        )

        model_medians = complete_matrix.median(axis=0).
↳sort_values(ascending=True)
        best_effective_model = str(model_medians.index[0])
        best_scores = complete_matrix[best_effective_model]

        raw_pairwise = []
        for model_name in model_medians.index[1:]:
            candidate_scores = complete_matrix[model_name]
            diff = candidate_scores - best_scores
            if np.allclose(diff.to_numpy(dtype=float), 0.0,
↳equal_nan=False):
                stat_value, p_value = 0.0, 1.0
            else:

```

```

        stat_value, p_value = stats.wilcoxon(candidate_scores,
↪best_scores, alternative="greater", zero_method="wilcox")
        raw_pairwise.append(
            {
                "model": str(model_name),
                "comparison": f"{model_name} worse than",
↪{best_effective_model}",
                "datasets": int(len(diff)),
                "median_score": float(candidate_scores.median()),
                "median_score_delta_vs_best": float(diff.median()),
                "wilcoxon_statistic": float(stat_value),
                "raw_p_value": float(p_value),
            }
        )

    if raw_pairwise:
        order = sorted(range(len(raw_pairwise)), key=lambda i:
↪raw_pairwise[i]["raw_p_value"])
        adjusted = [np.nan] * len(raw_pairwise)
        running_max = 0.0
        m = len(raw_pairwise)
        for rank, row_idx in enumerate(order, start=1):
            holm_value = min(1.0, raw_pairwise[row_idx]["raw_p_value"]
↪* (m - rank + 1))
            running_max = max(running_max, holm_value)
            adjusted[row_idx] = running_max
        for row, adj in zip(raw_pairwise, adjusted):
            row["holm_adjusted_p_value"] = float(adj)
            row["significantly_worse_than_best_alpha_0_05"] = bool(adj
↪< 0.05)

        pairwise_rows = raw_pairwise

        display_note("### Cost-performance statistical test")
        display(pd.DataFrame(test_result_rows))
        if pairwise_rows:
            pairwise_df = pd.DataFrame(pairwise_rows).
↪sort_values(["holm_adjusted_p_value", "median_score_delta_vs_best"],
↪ascending=[True, False])
            display_note("Pairwise Wilcoxon tests vs the best median
↪cost-performance model")
            display(pairwise_df.reset_index(drop=True))

        recommendation_source = effectiveness_summary.copy()
        recommendation_source = recommendation_source.
↪loc[recommendation_source["model"].isin(complete_matrix.columns)].copy()

```

```

        if best_effective_model is not None and not recommendation_source.
↪empty:
            best_row = recommendation_source.
↪loc[recommendation_source["model"].eq(best_effective_model)].head(1)
            if not best_row.empty:
                row = best_row.iloc[0]
                display_note(
                    "Best single-model choice by paired cost-performance_
↪score: "
                    f"**{best_effective_model}** "
                    f"(median score={row['median_cost_performance_score']:.
↪3f}, "
                    f"median delta={row['median_delta_from_best']:.3g}, "
                    f"median runtime={row['median_elapsed_seconds']:.1f}s)._
↪"
                    "Lower score balances better relative performance and_
↪faster runtime within each dataset."
                )

if SELECTOR_SCALING.empty:
    display_note("No selector diagnostics could be inferred for these runs.")
else:
    selector_df = SELECTOR_SCALING.copy()
    selector_df["n_molecules"] = pd.to_numeric(selector_df["n_molecules"],_
↪errors="coerce")
    selector_df["selector_seconds"] = pd.
↪to_numeric(selector_df["selector_seconds"], errors="coerce")
    selector_df["selector_hours"] = selector_df["selector_seconds"] / 3600.0

    display_note("### Feature-selection time vs dataset size")
    display(
        selector_df.sort_values("selector_seconds", ascending=False)
        [[
            col
            for col in [
                "run_name",
                "dataset",
                "n_molecules",
                "selector_seconds",
                "selector_hours",
                "selector_seconds_source",
                "selector_timed_out",
                "selector_method",
                "selected_feature_count",
                "original_feature_count",
            ]
        ]
    )

```

```

        if col in selector_df.columns
    ]]
    .reset_index(drop=True)
)

fit_df = selector_df.loc[
    selector_df["n_molecules"].gt(0)
    & selector_df["selector_seconds"].gt(0)
].copy()

if len(fit_df) >= 3:
    log_x = np.log10(fit_df["n_molecules"].to_numpy(dtype=float))
    log_y = np.log10(fit_df["selector_seconds"].to_numpy(dtype=float))
    corr = float(np.corrcoef(log_x, log_y)[0, 1])
    slope, intercept = np.polyfit(log_x, log_y, 1)
    print(
        "Selector scaling fit (log10 seconds vs log10 n_molecules): "
        f"slope={slope:.3f}, intercept={intercept:.3f}, corr={corr:.3f}"
    )

    fig = px.scatter(
        fit_df,
        x="n_molecules",
        y="selector_seconds",
        color="run_name",
        symbol="selector_seconds_source",
        hover_data=[c for c in ["dataset", "selected_feature_count",
↪ "selector_timed_out"] if c in fit_df.columns],
        title="Feature-selection cost scaling across runs",
    )
    fig.update_xaxes(type="log", title="Dataset size (n_molecules, log_
↪ scale)")
    fig.update_yaxes(type="log", title="Selector runtime (seconds, log_
↪ scale)")
    fig.show()
else:
    display_note("Insufficient positive selector-time samples to fit a_
↪ size-scaling trend.")

source_counts = (
    selector_df.groupby("selector_seconds_source", dropna=False)
    .size()
    .reset_index(name="datasets")
    .sort_values("datasets", ascending=False)
)
display_note("Selector-time source diagnostics")
display(source_counts)

```

```

    if "first_model_elapsed_proxy" in source_counts["selector_seconds_source"].
↳astype(str).tolist():
        display_note(
            "`first_model_elapsed_proxy` indicates an estimated upper bound for
↳selector time when explicit stage timings were not saved "
            "for that run."
        )

```

C:\Users\scott\AppData\Local\Temp\ipykernel_7244\1714422776.py:114:
PerformanceWarning: DataFrame is highly fragmented. This is usually the result
of calling `frame.insert` many times, which has poor performance. Consider
joining all columns at once using `pd.concat(axis=1)` instead. To get a de-
fragmented frame, use `newframe = frame.copy()`
.first()

1.0.17 Run-level cost vs value

	run_name	run_mtime	datasets	
↳	completed_datasets	models	total_elapsed_hours	mean_best_rmse
↳	mean_leaderboard_delta	mean_estimated_rank	top10_hit_rate	\
0	benchmark_name_date	2026-04-30 02:10:05.760428905	34	
↳	34	26	22.401339	6.017494
↳		4.066667	0.866667	-0.053465
	selector_timeout_rate	mean_selector_seconds	value_metric_used	
0	0.0	771.086206	leaderboard_delta_top10_best	

1.0.18 Model cost vs relative performance

	dataset	task_kind	
↳	model	workflow	elapsed_seconds analysis_metric
↳	analysis_metric_value	\	
0	tdc_cyp3a4_substrate_carbonmangels	classification	
↳	LogisticRegression	conventional	35.506 test_roc_auc
↳			0.658793
1	tdc_carcinogens_lagunin	classification	
↳	AdaBoost	conventional	66.059 test_roc_auc
↳			0.862626
2	tdc_bioavailability_ma	classification	
↳	CatBoost	conventional	155.509 test_roc_auc
↳			0.776688
3	tdc_pgp_broccatelli	classification	
↳	HistGradientBoosting	conventional	206.544 test_roc_auc
↳			0.924020


```

4      tdc_cyp2c9_substrate_carbonmangels  classification      Chemprop v2
↳(AttentiveFP, ensemble=1)  Chemprop v2      220.111      test_roc_auc
↳      0.653283
..      ...
↳      ...
↳ ...
582      tdc_half_life_obach      regression  Chemprop v2
↳(AttentiveFP + Selected descriptor...  Chemprop v2      1097.251
↳test_rmse      33.535919
583      tdc_half_life_obach      regression
↳      ElasticNetCV  conventional      260.528      test_rmse
↳      36.479184
584      tdc_half_life_obach      regression
↳      Tabular CNN  conventional      599.380      test_rmse
↳      40.262086
585      tdc_half_life_obach      regression
↳      HistGradientBoosting  conventional      460.976      test_rmse
↳      41.924915
586      poduam_pod_nc_std      regression      Chemprop v2
↳(AttentiveFP, ensemble=1)  Chemprop v2      2310.658      test_rmse
↳      27.618390

      analysis_delta_from_best
0      0.000000
1      0.000000
2      0.000000
3      0.000000
4      0.000000
..      ...
582      16.313665
583      19.256930
584      23.039832
585      24.702661
586      26.917514

```

[587 rows x 8 columns]

1.0.19 Model cost/value summary

```

      model      workflow
↳ task_kind  datasets  median_elapsed_seconds  mean_elapsed_seconds
↳median_delta_from_best  mean_delta_from_best
0      Ensemble (Weighted average (inverse train RMSE))      ensemble
↳classification      13      5521.7190      4594.523308
↳      0.003909      0.011700

```

1			XGBoost	conventional	␣
	classification	22	611.5230	2212.995864	␣
		0.010247	0.022147		
2			Ensemble (OOF Stacking (RidgeCV, 5-fold))	ensemble	␣
	classification	22	1275.7955	3209.104000	␣
		0.013090	0.025729		
3			CatBoost	conventional	␣
	classification	22	624.1615	2234.524818	␣
		0.013681	0.022735		
4			Random forest	conventional	␣
	classification	22	531.6550	1816.220864	␣
		0.013911	0.028256		
5			Ensemble (Weighted average (inverse train RMSE))	ensemble	␣
	regression	9	1396.1950	3325.431778	␣
		0.014805	0.092756		
6			CFA (Combinatorial Fusion)	CFA fusion	␣
	classification	22	1274.7505	3208.052909	␣
		0.018284	0.026058		
7			MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN	␣
	regression	22	1688.4975	2893.231091	␣
		0.018306	0.297316		
8			Extra trees	conventional	␣
	classification	22	541.3935	1844.667500	␣
		0.020616	0.037516		
9			TabPFNRegressor	TabPFN deep learning	␣
	regression	11	661.7490	680.571182	␣
		0.021242	0.862981		
10			HistGradientBoosting	conventional	␣
	classification	22	566.9280	1887.914682	␣
		0.023085	0.037068		
11			Voting Classifier (KNN, SVM)	conventional	␣
	classification	22	572.5700	2023.493182	␣
		0.025171	0.044036		
12			Chemprop v2 (AttentiveFP + Selected descriptor...	Chemprop v2	␣
	classification	22	944.9605	2879.064182	␣
		0.025753	0.029977		
13			XGBoost	conventional	␣
	regression	22	910.8220	1959.541227	␣
		0.027483	0.604650		
14			Ensemble (OOF Stacking (RidgeCV, 5-fold))	ensemble	␣
	regression	22	1689.2595	2894.252318	␣
		0.029225	0.945077		
15			SVC	conventional	␣
	classification	22	523.4835	1777.967091	␣
		0.029344	0.053429		

16	Chemprop v2 (AttentiveFP, ensemble=1)		Chemprop v2	␣
↪classification	20	789.9945	2859.257400	␣
↪	0.030095	0.035912		
17	CFA (Combinatorial Fusion)		CFA fusion	␣
↪regression	22	1688.7280	2893.565455	␣
↪	0.032066	0.878631		
18	Tabular MLP		conventional	␣
↪classification	22	608.1575	2195.709636	␣
↪	0.034744	0.057928		
19	TabPFNClassifier	TabPFN deep learning	␣	
↪classification	10	184.2180	208.208500	␣
↪	0.036254	0.043494		
20	LogisticRegression		conventional	␣
↪classification	22	520.0355	1648.335864	␣
↪	0.039402	0.053219		
21	HistGradientBoosting		conventional	␣
↪regression	22	629.1320	1620.150773	␣
↪	0.042584	1.822758		
22	MapLight CatBoost (Strict Parity)		conventional	␣
↪regression	22	1073.0450	2130.613545	␣
↪	0.043543	0.276120		
23	CatBoost		conventional	␣
↪regression	22	932.6940	1978.720318	␣
↪	0.044646	0.405856		
24	AdaBoost		conventional	␣
↪classification	22	585.8310	2123.123455	␣
↪	0.051927	0.053453		
25	Random forest		conventional	␣
↪regression	22	458.0715	1295.241000	␣
↪	0.057760	0.660776		
26	ChemML MLP (PyTorch)	ChemML deep learning	␣	
↪classification	22	667.1180	2425.003591	␣
↪	0.059146	0.069860		
27	Extra trees		conventional	␣
↪regression	22	583.3945	1576.649409	␣
↪	0.074478	1.258322		
28	Chemprop v2 (AttentiveFP, ensemble=1)		Chemprop v2	␣
↪regression	20	1610.6525	2607.395200	␣
↪	0.082431	1.569825		
29	MapLight + GNN (CatBoost, Strict Parity)		MapLight + GNN	␣
↪classification	22	1274.2430	3207.383500	␣
↪	0.082586	0.101581		
30	ChemML MLP (PyTorch)	ChemML deep learning	␣	
↪regression	22	1154.4835	2246.492182	␣
↪	0.095805	1.310394		

31	Chemprop v2 (AttentiveFP + Selected descriptor...			Chemprop v2	□
	↪regression	22	1429.9395	2593.549227	□
	↪	0.098651	1.851287		
32			SVR	conventional	□
	↪regression	22	334.8435	1041.907818	□
	↪	0.101852	0.892226		
33			ElasticNetCV	conventional	□
	↪regression	22	333.3480	1032.391182	□
	↪	0.102517	2.801190		
34			Voting Regressor (KNN, SVM)	conventional	□
	↪regression	22	631.9855	1631.170045	□
	↪	0.142207	0.967670		
35			AdaBoost	conventional	□
	↪regression	22	675.4165	1716.619182	□
	↪	0.213480	1.201109		
36			Tabular MLP	conventional	□
	↪regression	22	732.6860	1783.404955	□
	↪	0.228423	1.654541		
37			Tabular CNN	conventional	□
	↪regression	22	899.0855	1946.835091	□
	↪	0.398852	2.234304		

1.0.20 Publication-grade computational cost comparison

	model_family		source	datasets	model_rows	□
	↪median_wall_clock_seconds	mean_wall_clock_seconds				□
	↪	hardware \				
0	CFA	AutoQSAR current benchmark run	44.0	44.0	□	
	↪	3781.0175	11549.174591	GPU available in current	□	
	↪run; CPU/GPU use depe...					
1	Chemprop	AutoQSAR current benchmark run	23.0	44.0	□	
	↪	1816.9595	2830.263841	GPU available in current	□	
	↪run; CPU/GPU use depe...					
2	Conventional ML	AutoQSAR current benchmark run	45.0	473.0	□	
	↪	975.4010	3717.537844	GPU available in current	□	
	↪run; CPU/GPU use depe...					
3	Ensemble	AutoQSAR current benchmark run	45.0	64.0	□	
	↪	197.9435	9162.682328	GPU available in current	□	
	↪run; CPU/GPU use depe...					
4	MapLight+GNN	AutoQSAR current benchmark run	7.0	7.0	□	
	↪	46514.5110	49997.540429	GPU available in current	□	
	↪run; CPU/GPU use depe...					
5	Other neural baselines	AutoQSAR current benchmark run	45.0	113.0	□	
	↪	1122.9100	3328.785611	GPU available in current	□	
	↪run; CPU/GPU use depe...					

6	TabPFN	AutoQSAR	current benchmark run	20.0	20.0	␣
	↪	385.4150	677.150200	GPU available in current		␣
	↪	run; CPU/GPU use depe...				
7	ADMET-AI		Published comparator	NaN	NaN	␣
	↪	NaN	NaN	GPU-capable		␣
	↪	Chemprop-RDKit deployment				
8	Mole		Published comparator	NaN	NaN	␣
	↪	NaN	NaN	GPU pretraining		␣
	↪	reported in literature				
9	MolGPS		Published comparator	NaN	NaN	␣
	↪	NaN	NaN	GPU pretraining/inference		␣
	↪	reported in literature				
			trainable_parameters			␣
	↪	median_recorded_trainable_parameters				␣
	↪	median_inference_seconds_per_1000_molecules				␣
	↪	published_cost_comparison				
0	Not applicable (fusion over fitted model outputs)					␣
	↪	NaN	NaN	Measured in this		␣
	↪	benchmark run; inference-only...				
1	median recorded trainable parameters: 395,618					␣
	↪	395618.0	6.269235	Measured in this		␣
	↪	benchmark run; inference-only...				
2	median recorded trainable parameters: 47,331,652					␣
	↪	47331652.0	0.128707	Measured in this		␣
	↪	benchmark run; inference-only...				
3	Not applicable for averaging/linear meta-model...					␣
	↪	NaN	NaN	Measured in this		␣
	↪	benchmark run; inference-only...				
4	Not recorded in benchmark artifacts; CatBoost ...					␣
	↪	NaN	NaN	Measured in this		␣
	↪	benchmark run; inference-only...				
5	median recorded trainable parameters: 341,761					␣
	↪	341761.0	0.017410	Measured in this		␣
	↪	benchmark run; inference-only...				
6	Pretrained model; no task-specific fine-tuning...					␣
	↪	NaN	NaN	Measured in this		␣
	↪	benchmark run; inference-only...				
7	Chemprop-RDKit; exact parameter count not reco...					␣
	↪	NaN	NaN	Published throughput		␣
	↪	reference: ~3.1 hours for...				
8			~100M parameters			␣
	↪	NaN	NaN	Pretrained on ~842M		␣
	↪	molecules; include exact r...				
9			~3B parameters			␣
	↪	NaN	NaN	Foundation-scale graph		␣
	↪	model; include exact ru...				

1.0.21 Cost-performance effectiveness summary

			model	workflow	
↳ datasets	median_cost_performance_score	mean_cost_performance_score			↳
↳ median_performance_percentile	\				
0			XGBoost	conventional	↳
↳ 44		0.337674		0.371318	↳
↳	0.263889				
1			HistGradientBoosting	conventional	↳
↳ 44		0.338235		0.394771	↳
↳	0.352941				
2			MapLight CatBoost (Strict Parity)	conventional	↳
↳ 22		0.356250		0.395165	↳
↳	0.263889				
3	Ensemble (Weighted average (inverse train RMSE))			ensemble	↳
↳ 22		0.362500		0.397666	↳
↳	0.150000				
4			CatBoost	conventional	↳
↳ 44		0.367647		0.379248	↳
↳	0.294118				
5			TabPFNRegressor	TabPFN deep learning	↳
↳ 11		0.388889		0.363690	↳
↳	0.277778				
6			Random forest	conventional	↳
↳ 44		0.408333		0.382284	↳
↳	0.486842				
7			TabPFNClassifier	TabPFN deep learning	↳
↳ 10		0.414624		0.428881	↳
↳	0.343137				
8			Extra trees	conventional	↳
↳ 44		0.455882		0.444794	↳
↳	0.529412				
9	Ensemble (OOF Stacking (RidgeCV, 5-fold))			ensemble	↳
↳ 44		0.457108		0.475948	↳
↳	0.285948				
10	Voting Classifier (KNN, SVM)			conventional	↳
↳ 22		0.463542		0.495872	↳
↳	0.500000				
11			SVC	conventional	↳
↳ 22		0.499020		0.499990	↳
↳	0.623529				
12			SVR	conventional	↳
↳ 22		0.513889		0.505762	↳
↳	0.649123				
13	CFA (Combinatorial Fusion)			CFA fusion	↳
↳ 44		0.521701		0.507847	↳
↳	0.381944				

14		ElasticNetCV	conventional	␣
↪	22	0.581506	0.534938	␣
↪		0.757310		
15		LogisticRegression	conventional	␣
↪	22	0.610294	0.529102	␣
↪		0.794118		
16		Chemprop v2 (AttentiveFP, ensemble=1)	Chemprop v2	␣
↪	40	0.661765	0.601819	␣
↪		0.647059		
17		Chemprop v2 (AttentiveFP + Selected descriptor...	Chemprop v2	␣
↪	44	0.664624	0.628799	␣
↪		0.629085		
18		Voting Regressor (KNN, SVM)	conventional	␣
↪	22	0.673026	0.635566	␣
↪		0.794737		
19		Tabular MLP	conventional	␣
↪	44	0.694444	0.677147	␣
↪		0.777778		
20		AdaBoost	conventional	␣
↪	44	0.722953	0.655397	␣
↪		0.837719		
21		MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN	␣
↪	44	0.729167	0.675408	␣
↪		0.694444		
22		ChemML MLP (PyTorch)	ChemML deep learning	␣
↪	44	0.743421	0.717108	␣
↪		0.750774		
23		Tabular CNN	conventional	␣
↪	22	0.862500	0.801156	␣
↪		1.000000		

	median_runtime_percentile	median_delta_from_best	median_elapsed_seconds
0	0.529412	0.017770	782.6275
1	0.277778	0.027607	629.1320
2	0.631579	0.043543	1073.0450
3	1.000000	0.008326	1925.7495
4	0.588235	0.021168	798.5825
5	0.684211	0.021242	661.7490
6	0.166667	0.031821	458.0715
7	0.647059	0.036254	184.2180
8	0.222222	0.042517	583.3945
9	0.975000	0.015394	1517.4560
10	0.352941	0.025171	572.5700
11	0.117647	0.029344	523.4835
12	0.105263	0.101852	334.8435
13	0.916667	0.026144	1516.9670
14	0.052632	0.102517	333.3480

15	0.058824	0.039402	520.0355
16	0.750000	0.036637	1189.4580
17	0.800000	0.039087	1249.1790
18	0.315789	0.142207	631.9855
19	0.444444	0.104158	732.6860
20	0.388889	0.080920	675.4165
21	0.858333	0.046841	1516.6945
22	0.700000	0.071605	955.6245
23	0.473684	0.398852	899.0855

1.0.22 Cost-performance statistical test

	test	datasets	models	statistic	p_value
			score		
0	Friedman repeated-measures test	17	18	143.379308	4.803259e-22
	↳0.75*within-dataset performance percentile + 0...				

Pairwise Wilcoxon tests vs the best median cost-performance model

			model	
	comparison	datasets	median_score	median_score_delta_vs_best
	↳wilcoxon_statistic	raw_p_value	\	
0			ChemML MLP (PyTorch)	ChemML MLP (PyTorch)
	↳worse than Ensemble (OOF ...	17	0.638889	0.347222
	↳149.0	0.000053		
1			Tabular CNN	Tabular CNN worse than
	↳Ensemble (OOF Stacking ...	17	0.875000	0.513158
	↳136.0	0.000218		
2			Tabular MLP	Tabular MLP worse than
	↳Ensemble (OOF Stacking ...	17	0.777778	0.444444
	↳151.0	0.000210		
3			AdaBoost	AdaBoost worse than
	↳Ensemble (OOF Stacking (Ri...	17	0.776316	0.
	↳447368	149.0	0.000299	
4			Chemprop v2 (AttentiveFP + Selected descriptor...	Chemprop v2 (AttentiveFP +
	↳Selected descriptor...	17	0.708333	0.325000
	↳149.0	0.000298		
5			Voting Regressor (KNN, SVM)	Voting Regressor (KNN,
	↳SVM) worse than Ensembl...	17	0.712500	0.394737
	↳146.0	0.000499		
6			Chemprop v2 (AttentiveFP, ensemble=1)	Chemprop v2 (AttentiveFP,
	↳ensemble=1) worse th...	17	0.644737	0.312500
	↳130.0	0.000673		
7			CFA (Combinatorial Fusion)	CFA (Combinatorial
	↳Fusion) worse than Ensemble...	17	0.527778	0.
	↳223684	143.0	0.000821	

8			ElasticNetCV	ElasticNetCV worse than
↳Ensemble (OOF Stacking...	17	0.472222	0.138889	↳
↳	137.0	0.002089		
9			SVR	SVR worse than Ensemble
↳(OOF Stacking (RidgeCV...	17	0.569444	0.194444	↳
↳	134.0	0.003238		
10			Extra trees	Extra trees worse than
↳Ensemble (OOF Stacking ...	17	0.513889	0.180556	↳
↳	124.0	0.011612		
11			Uni-Mol V1	Uni-Mol V1 worse than
↳Ensemble (OOF Stacking (...	17	0.462500	0.055556	↳
↳	104.0	0.103447		
12			Random forest	Random forest worse than
↳Ensemble (OOF Stackin...	17	0.421053	0.118421	↳
↳	88.0	0.150527		
13			HistGradientBoosting	HistGradientBoosting
↳worse than Ensemble (OOF ...	17	0.350000	0.069444	↳
↳	99.5	0.138059		
14			CatBoost	CatBoost worse than
↳Ensemble (OOF Stacking (Ri...	17	0.368421	0.	
↳065789	99.0	0.143380		
15			MapLight CatBoost (Strict Parity)	MapLight CatBoost (Strict
↳Parity) worse than E...	17	0.412500	0.041667	↳
↳	78.0	0.153312		
16			XGBoost	XGBoost worse than
↳Ensemble (OOF Stacking (Rid...	17	0.342105	0.	
↳013889	93.0	0.229286		

	holm_adjusted_p_value	significantly_worse_than_best_alpha_0_05
0	0.000908	True
1	0.003360	True
2	0.003360	True
3	0.004179	True
4	0.004179	True
5	0.005992	True
6	0.007404	True
7	0.008208	True
8	0.018804	True
9	0.025900	True
10	0.081284	False
11	0.620682	False
12	0.690295	False
13	0.690295	False
14	0.690295	False
15	0.690295	False
16	0.690295	False

Best single-model choice by paired cost-performance score: **Ensemble (OOF Stacking (RidgeCV, 5-fold))** (median score=0.375, median delta=0.0244, median runtime=120.5s). Lower score balances better relative performance and faster runtime within each dataset.

1.0.23 Feature-selection time vs dataset size

	run_name	dataset	n_molecules	
	↵ selector_seconds selector_hours selector_seconds_source selector_timed_out			
	↵ selector_method selected_feature_count \			
0	benchmark_name_date	lipophilicity	4200	
	↵ 6398.688 1.777413	stage_runtime	False	
	↵ elasticnet_cv	512		
1	benchmark_name_date	tdc_ppbr_az	2790	
	↵ 4995.537 1.387649	stage_runtime	False	
	↵ elasticnet_cv	512		
2	benchmark_name_date	poduam_pod_rd_std	2355	
	↵ 3367.248 0.935347	stage_runtime	False	
	↵ elasticnet_cv	512		
3	benchmark_name_date	tdc_bbb_martins	2030	
	↵ 2378.957 0.660821	stage_runtime	False	
	↵ elasticnet_cv	275		
4	benchmark_name_date	tdc_toxcast	1731	
	↵ 1962.854 0.545237	stage_runtime	False	
	↵ elasticnet_cv	68		
5	benchmark_name_date	poduam_pod_nc_std	1842	
	↵ 1684.384 0.467884	stage_runtime	False	
	↵ elasticnet_cv	342		
6	benchmark_name_date	tdc_pampa_ncats	2034	
	↵ 976.953 0.271376	stage_runtime	False	
	↵ elasticnet_cv	236		
7	benchmark_name_date	polaris_adme_fang_rclint_1	3054	
	↵ 897.377 0.249271	stage_runtime	False	
	↵ elasticnet_cv	512		
8	benchmark_name_date	polaris_adme_fang_hppb_1	1808	
	↵ 876.153 0.243376	stage_runtime	False	
	↵ elasticnet_cv	512		
9	benchmark_name_date	tdc_clintox	1478	
	↵ 689.068 0.191408	stage_runtime	False	
	↵ elasticnet_cv	126		
10	benchmark_name_date	polaris_adme_fang_perm_1	2642	
	↵ 578.371 0.160659	stage_runtime	False	
	↵ elasticnet_cv	512		
11	benchmark_name_date	polaris_adme_fang_solu_1	2173	
	↵ 476.823 0.132451	stage_runtime	False	
	↵ elasticnet_cv	224		

12	benchmark_name_date	tdc_pgp_broccatelli	1218		
	↪ 320.993 0.089165	stage_runtime	False	┐	
	↪elasticnet_cv	274			
13	benchmark_name_date	tdc_herg	655		
	↪ 62.760 0.017433	stage_runtime	False	┐	
	↪elasticnet_cv	131			
14	benchmark_name_date	tdc_cyp2d6_substrate_carbonmangels	667		
	↪ 61.215 0.017004	stage_runtime	False	┐	
	↪elasticnet_cv	142			
15	benchmark_name_date	tdc_dili	475		
	↪ 57.189 0.015886	stage_runtime	False	┐	
	↪elasticnet_cv	273			
16	benchmark_name_date	tdc_bioavailability_ma	640		
	↪ 57.178 0.015883	stage_runtime	False	┐	
	↪elasticnet_cv	12			
17	benchmark_name_date	tdc_hia_hou	578		
	↪ 56.718 0.015755	stage_runtime	False	┐	
	↪elasticnet_cv	86			
18	benchmark_name_date	tdc_cyp2c9_substrate_carbonmangels	669		
	↪ 56.070 0.015575	stage_runtime	False	┐	
	↪elasticnet_cv	8			
19	benchmark_name_date	tdc_cyp3a4_substrate_carbonmangels	670		
	↪ 55.932 0.015537	stage_runtime	False	┐	
	↪elasticnet_cv	176			
20	benchmark_name_date	tdc_skin_reaction	404		
	↪ 51.698 0.014361	stage_runtime	False	┐	
	↪elasticnet_cv	22			
21	benchmark_name_date	tdc_carcinogens_lagunin	280		
	↪ 51.066 0.014185	stage_runtime	False	┐	
	↪elasticnet_cv	83			
22	benchmark_name_date	tdc_clearance_hepatocyte_az	1213		
	↪ 15.864 0.004407	stage_runtime	False	┐	
	↪elasticnet_cv	512			
23	benchmark_name_date	tdc_vdss_lombardo	1130		
	↪ 15.620 0.004339	stage_runtime	False	┐	
	↪elasticnet_cv	512			
24	benchmark_name_date	tdc_clearance_microsome_az	1102		
	↪ 12.985 0.003607	stage_runtime	False	┐	
	↪elasticnet_cv	512			
25	benchmark_name_date	tdc_caco2_wang	910		
	↪ 11.746 0.003263	stage_runtime	False	┐	
	↪elasticnet_cv	355			
26	benchmark_name_date	polaris_adme_fang_rppb_1	885		
	↪ 10.698 0.002972	stage_runtime	False	┐	
	↪elasticnet_cv	221			

27	benchmark_name_date	tdc_half_life_obach	667	␣
↪	9.578	0.002661	stage_runtime	False ␣
↪	elasticnet_cv	512		
28	benchmark_name_date	esol_delaney	1128	␣
↪	7.150	0.001986	stage_runtime	False ␣
↪	elasticnet_cv	431		
29	benchmark_name_date	chemml_cep_homo	500	␣
↪	6.712	0.001864	stage_runtime	False ␣
↪	elasticnet_cv	339		
30	benchmark_name_date	chemml_organic_density	500	␣
↪	5.203	0.001445	stage_runtime	False ␣
↪	elasticnet_cv	241		
31	benchmark_name_date	tdc_hydrationfreeenergy_freesolv	642	␣
↪	3.639	0.001011	stage_runtime	False ␣
↪	elasticnet_cv	423		
32	benchmark_name_date	freesolv_sampl	642	␣
↪	3.472	0.000964	stage_runtime	False ␣
↪	elasticnet_cv	451		
33	benchmark_name_date	chemml_xyz_polarizability	38	␣
↪	1.032	0.000287	stage_runtime	False ␣
↪	elasticnet_cv	65		

	original_feature_count
0	10115
1	10115
2	10115
3	10115
4	10115
5	10115
6	10115
7	10115
8	10115
9	10115
10	10115
11	10115
12	10115
13	10115
14	10115
15	10115
16	10115
17	10115
18	10115
19	10115
20	10115
21	10115
22	10115
23	10115

```

24          10115
25          10115
26          10115
27          10115
28          10115
29          10115
30          10115
31          10115
32          10115
33          10115

```

Selector scaling fit (log10 seconds vs log10 n_molecules): slope=2.103,
intercept=-4.320, corr=0.763

Selector-time source diagnostics

```

    selector_seconds_source  datasets
0          stage_runtime          34

```

```

[63]: metric_plot_source = SUMMARY_ANALYSIS_METRICS.copy()
error_col = pick_column(metric_plot_source.columns, ["error", "Error"])
if error_col is not None:
    metric_plot_source = metric_plot_source.loc[
        metric_plot_source[error_col].isna() | (metric_plot_source[error_col].
        ↳astype(str).str.strip() == "")
    ].copy()
metric_plot_source = add_analysis_metric_columns(metric_plot_source,
        ↳rmse_col=RMSE_COL, classification_col=BALANCED_ACCURACY_COL)
metric_plot_source["analysis_metric_value"] = pd.
        ↳to_numeric(metric_plot_source["analysis_metric_value"], errors="coerce")
metric_plot_source["analysis_delta_from_best"] = pd.
        ↳to_numeric(metric_plot_source["analysis_delta_from_best"], errors="coerce")
metric_plot_source = metric_plot_source.
        ↳loc[metric_plot_source["analysis_metric_value"].notna()].copy()

if metric_plot_source.empty:
    display_note("No successful regression/classification metric rows were
        ↳available for model performance plots.")
else:
    for task_kind, task_df in metric_plot_source.groupby("task_kind",
        ↳sort=True):
        metric_names = sorted(task_df["analysis_metric"].dropna().astype(str).
        ↳unique().tolist())
        metric_label = metric_names[0] if len(metric_names) == 1 else "primary
        ↳metric"
        heatmap_df = task_df.pivot_table(index="dataset", columns="model",
        ↳values="analysis_metric_value", aggfunc="mean")

```

```

        color_scale = "Viridis_r" if task_df["analysis_metric_direction"].
↪astype(str).eq("lower").all() else "Viridis"
        fig = px.imshow(
            heatmap_df,
            aspect="auto",
            color_continuous_scale=color_scale,
            labels={"x": "Model", "y": "Dataset", "color": metric_label},
            text_auto='.3f',
            title=f"Held-out {metric_label} by dataset and model ({task_kind})",
        )
        fig.update_layout(height=max(450, 60 + 40 * len(heatmap_df.index)))
        fig.show()

        fig = px.box(
            task_df,
            x="model",
            y="analysis_metric_value",
            color="workflow" if "workflow" in task_df.columns else None,
            points="all",
            title=f"{metric_label} distribution across {task_kind} datasets",
        )
        fig.update_layout(xaxis_title="Model", yaxis_title=metric_label,
↪xaxis_tickangle=45)
        fig.show()

        delta_df = metric_plot_source.
↪loc[metric_plot_source["analysis_delta_from_best"].notna()].copy()

        if delta_df.empty:
            display_note("No valid rows available for delta-from-best model boxplot.
↪")
        else:
            model_order = (
                delta_df.groupby("model",
↪as_index=False)["analysis_delta_from_best"]
                .median()
                .sort_values("analysis_delta_from_best", ascending=True)["model"]
                .tolist()
            )

            positive_deltas = delta_df.loc[delta_df["analysis_delta_from_best"] >
↪0, "analysis_delta_from_best"]
            delta_floor = float(positive_deltas.min() * 0.5) if not positive_deltas.
↪empty else 1e-12
            delta_floor = max(delta_floor, 1e-12)

```

```

    delta_df["delta_from_best_log10"] = np.
↳log10(delta_df["analysis_delta_from_best"] + delta_floor)

    fig = px.box(
        delta_df,
        x="model",
        y="delta_from_best_log10",
        color="task_kind",
        points="all",
        category_orders={"model": model_order},
        title="Model delta from dataset-best model (RMSE or balanced_
↳accuracy, log10 scale)",
        hover_data=["dataset", "analysis_metric", "analysis_metric_value",
↳"analysis_delta_from_best"],
    )
    fig.update_layout(
        xaxis_title="Model",
        yaxis_title=f"log10(delta from best + {delta_floor:.2e}); lower is_
↳better",
        xaxis_tickangle=45,
    )
    fig.show()

```

```

[64]: if PREDICTIONS is not None and not PREDICTIONS.empty:
    pred_df = PREDICTIONS.copy()
    dataset_col = pick_column(pred_df.columns, ["dataset", "dataset_id",
↳"dataset_name"])
    model_col = pick_column(pred_df.columns, ["model", "Model"])
    observed_col = pick_column(pred_df.columns, ["observed", "y_true",
↳"actual"])
    predicted_col = pick_column(pred_df.columns, ["predicted", "y_pred",
↳"prediction"])
    split_col = pick_column(pred_df.columns, ["split", "Split"])

    if all(col is not None for col in [dataset_col, model_col, observed_col,
↳predicted_col]):
        pred_df = pred_df.rename(columns={
            dataset_col: "dataset",
            model_col: "model",
            observed_col: "observed",
            predicted_col: "predicted",
        })
        if split_col is not None:
            pred_df = pred_df.rename(columns={split_col: "split"})
            pred_df = pred_df.loc[pred_df["split"].astype(str).str.lower() ==
↳"test"].copy()

```

```

    pred_df["observed"] = pd.to_numeric(pred_df["observed"],
↳errors="coerce")
    pred_df["predicted"] = pd.to_numeric(pred_df["predicted"],
↳errors="coerce")
    pred_df = pred_df.loc[pred_df["observed"].notna() &
↳pred_df["predicted"].notna()].copy()

    metrics_best = SUMMARY_METRICS.copy()
    metrics_error_col = pick_column(metrics_best.columns, ["error",
↳"Error"])
    if metrics_error_col is not None:
        metrics_best = metrics_best.loc[
            metrics_best[metrics_error_col].isna() |
↳(metrics_best[metrics_error_col].astype(str).str.strip() == "")
        ].copy()
        metrics_best[RMSE_COL] = pd.to_numeric(metrics_best[RMSE_COL],
↳errors="coerce")
        metrics_best = metrics_best.loc[metrics_best[RMSE_COL].notna()].copy()

    if metrics_best.empty:
        display_note("No successful metric rows available to resolve best_
↳model per dataset for held-out scatter.")
    else:
        best_model_by_dataset = (
            metrics_best.sort_values(RMSE_COL, ascending=True)
                .groupby("dataset", as_index=False)
                .first()[["dataset", "model"]]
                .rename(columns={"model": "best_model"})
        )

        pred_df = pred_df.merge(best_model_by_dataset, on="dataset",
↳how="inner")
        pred_df = pred_df.loc[pred_df["model"].astype(str) ==
↳pred_df["best_model"].astype(str)].copy()

        if pred_df.empty:
            display_note("No held-out prediction rows matched each_
↳dataset's best model.")
        else:
            if len(pred_df) > int(prediction_sample_cap):
                dataset_count = max(1, pred_df["dataset"].nunique())
                per_dataset_cap = max(1, int(prediction_sample_cap //
↳dataset_count))

                pred_df = (
                    pred_df.groupby("dataset", group_keys=False)

```



```

        .apply(lambda frame: frame.sample(n=min(len(frame),
↳per_dataset_cap), random_state=42))
        .reset_index(drop=True)
    )

    fig = px.scatter(
        pred_df,
        x="observed",
        y="predicted",
        color="best_model",
        facet_col="dataset",
        facet_col_wrap=3,
        opacity=0.70,
        title="Observed vs predicted (held-out rows) for each
↳dataset's best model",
        hover_data=["dataset", "best_model"],
    )
    fig.update_traces(marker=dict(size=5))
    fig.update_layout(legend_title_text="Best model")
    fig.show()

    else:
        display_note("Predictions table exists, but the expected observed/
↳predicted columns were not found.")
else:
    display_note("No aggregate predictions table was found for this run.")

if GA_HISTORY is not None and not GA_HISTORY.empty:
    ga_df = GA_HISTORY.copy()
    dataset_col = pick_column(ga_df.columns, ["dataset", "dataset_id",
↳"dataset_name"])
    model_col = pick_column(ga_df.columns, ["model", "Model"])
    generation_col = pick_column(ga_df.columns, ["generation", "Generation"])
    best_col = pick_column(ga_df.columns, ["best_fitness", "best_rmse",
↳"best_score", "Best fitness"])
    if all(col is not None for col in [dataset_col, model_col, generation_col,
↳best_col]):
        ga_df = ga_df.rename(columns={dataset_col: "dataset", model_col:
↳"model", generation_col: "generation", best_col: "best_value"})
        fig = px.line(ga_df, x="generation", y="best_value", color="model",
↳facet_col="dataset", facet_col_wrap=3, title="GA convergence history")
        fig.update_layout(yaxis_title=best_col)
        fig.show()
    else:
        display_note("GA history exists, but the expected columns were not
↳found for plotting.")

```

No aggregate predictions table was found for this run.

```
[65]: # Ensemble diagnostics: method-aware value-add vs best base model

ENSEMBLE_EPS = 1e-12
METHOD_CFA = "CFA (Combinatorial Fusion)"
METHOD_OOF = "OOF stacking (RidgeCV)"
METHOD_INV_RMSE = "Model average (inverse train RMSE)"
EXPECTED_ENSEMBLE_METHODS = [METHOD_CFA, METHOD_OOF, METHOD_INV_RMSE]
BASELINE_LABEL = "Best base model"

def classify_ensemble_method(model_name, workflow_name=""):
    model_text = str(model_name or "").strip().lower()
    workflow_text = str(workflow_name or "").strip().lower()

    if model_text.startswith("cfa (") or "cfa" in workflow_text:
        return METHOD_CFA

    if model_text.startswith("ensemble (") or "ensemble" in workflow_text:
        if "oof stacking" in model_text:
            return METHOD_OOF
        if "weighted average" in model_text and "inverse" in model_text and
        ↪ "rmse" in model_text:
            return METHOD_INV_RMSE
    return None

perf_df = SUMMARY_METRICS.copy()
error_col = pick_column(perf_df.columns, ["error", "Error"])
if error_col is not None:
    perf_df = perf_df.loc[perf_df[error_col].isna() | (perf_df[error_col].
    ↪ astype(str).str.strip() == "")].copy()

perf_df[RMSE_COL] = pd.to_numeric(perf_df[RMSE_COL], errors="coerce")
perf_df = perf_df.loc[perf_df[RMSE_COL].notna()].copy()

if "workflow" not in perf_df.columns:
    perf_df["workflow"] = ""

perf_df["ensemble_method"] = [
    classify_ensemble_method(model_name, workflow_name)
    for model_name, workflow_name in zip(perf_df["model"], perf_df["workflow"])
]

ensemble_df = perf_df.loc[perf_df["ensemble_method"].notna()].copy()
base_df = perf_df.loc[perf_df["ensemble_method"].isna()].copy()

if perf_df.empty:
```

```

    display_note("No valid RMSE rows available for ensemble-vs-base diagnostics.
↳")
elif ensemble_df.empty or base_df.empty:
    display_note("Could not compare ensemble methods vs best base model because
↳one side is missing.")
else:
    method_availability = (
        ensemble_df.groupby("ensemble_method", dropna=False)
        .agg(rows=("dataset", "size"), datasets=("dataset", "nunique"))
        .reset_index()
    )
    method_availability = (
        method_availability.set_index("ensemble_method")
        .reindex(EXPECTED_ENSEMBLE_METHODS)
        .reset_index()
        .rename(columns={"index": "ensemble_method"})
    )
    method_availability[["rows", "datasets"]] = method_availability[["rows",
↳"datasets"]].fillna(0).astype(int)

    display_note("### Ensemble Method Coverage")
    display(method_availability)

    # Optional weight diagnostics when ensemble weight artifacts are present.
    ensemble_best_method_by_dataset = (
        ensemble_df.sort_values(RMSE_COL, ascending=True)
        .groupby("dataset", as_index=False)
        .first()[["dataset", "ensemble_method", "model"]]
        .rename(columns={"ensemble_method": "selected_ensemble_method", "model":
↳"selected_ensemble_model"})
    )
    selected_method_lookup =
↳dict(zip(ensemble_best_method_by_dataset["dataset"],
↳ensemble_best_method_by_dataset["selected_ensemble_method"]))

    ensemble_weight_rows = []
    for weights_path in sorted(RUN_DIR.glob("*/ensemble_weights.csv")):
        dataset_name = weights_path.parent.name
        try:
            wdf = pd.read_csv(weights_path)
        except Exception:
            continue
        if wdf.empty:
            continue
        model_col = pick_column(wdf.columns, ["model", "Model"])
        weight_col = pick_column(wdf.columns, ["weight", "Weight"])

```

```

abs_col = pick_column(wdf.columns, ["abs normalized contribution", "Abs_
↳normalized contribution", "abs_normalized_contribution"])
workflow_col = pick_column(wdf.columns, ["workflow", "Workflow"])
if model_col is None or weight_col is None:
    continue

local = pd.DataFrame(
    {
        "dataset": dataset_name,
        "model": wdf[model_col].astype(str),
        "weight": pd.to_numeric(wdf[weight_col], errors="coerce"),
        "abs_normalized_contribution": pd.to_numeric(wdf[abs_col],
↳errors="coerce") if abs_col is not None else np.nan,
        "workflow": wdf[workflow_col].astype(str) if workflow_col is
↳not None else "",
        "selected_ensemble_method": selected_method_lookup.
↳get(dataset_name, np.nan),
    }
)
local = local.loc[local["model"].str.strip().ne("") & local["weight"].
↳notna()].copy()
if not local.empty:
    ensemble_weight_rows.append(local)

if ensemble_weight_rows:
    ensemble_weights_df = pd.concat(ensemble_weight_rows, ignore_index=True)
    ensemble_weights_df["abs_weight"] = ensemble_weights_df["weight"].abs()

usage_summary = (
    ensemble_weights_df.groupby(["selected_ensemble_method", "model"],
↳dropna=False)
    .agg(
        datasets=("dataset", "nunique"),
        total_rows=("dataset", "size"),
        mean_weight=("weight", "mean"),
        median_weight=("weight", "median"),
        mean_abs_weight=("abs_weight", "mean"),
        median_abs_weight=("abs_weight", "median"),
        max_abs_weight=("abs_weight", "max"),
        mean_abs_contribution=("abs_normalized_contribution", "mean"),
        median_abs_contribution=("abs_normalized_contribution",
↳
↳"median"),
    )
    .reset_index()
    .sort_values(["selected_ensemble_method", "mean_abs_contribution",
↳
↳"mean_abs_weight", "datasets"], ascending=[True, False, False, False])

```

```

    )

    display_note("### Ensemble Weight Usage Statistics (when_
↳ `ensemble_weights.csv` is present)")
    display(usage_summary)

    weight_plot_df = ensemble_weights_df.copy()
    weight_plot_df["selected_ensemble_method"] =_
↳ weight_plot_df["selected_ensemble_method"].fillna("Unknown")
    fig = px.box(
        weight_plot_df,
        x="model",
        y="weight",
        color="selected_ensemble_method",
        points="all",
        title="Distribution of ensemble weights by selected ensemble_
↳ method",
    )
    fig.add_hline(y=0.0, line_dash="dash", line_color="black")
    fig.update_layout(xaxis_title="Model", yaxis_title="Weight",_
↳ xaxis_tickangle=45)
    fig.show()
    else:
        display_note("No `ensemble_weights.csv` files were found for this run;_
↳ skipping weight diagnostics.")

    method_best = (
        ensemble_df.groupby(["dataset", "ensemble_method"],_
↳ as_index=False)[RMSE_COL]
        .min()
        .rename(columns={RMSE_COL: "ensemble_rmse"})
    )
    base_best = (
        base_df.groupby("dataset", as_index=False)[RMSE_COL]
        .min()
        .rename(columns={RMSE_COL: "best_base_rmse"})
    )

    comparison = method_best.merge(base_best, on="dataset", how="inner")
    comparison["delta_rmse_ensemble_minus_best_base"] =_
↳ comparison["ensemble_rmse"] - comparison["best_base_rmse"]
    comparison["relative_delta_vs_best_base"] = np.where(
        comparison["best_base_rmse"].abs() > ENSEMBLE_EPS,
        comparison["delta_rmse_ensemble_minus_best_base"] /_
↳ comparison["best_base_rmse"],
        np.nan,

```

```

    )
    comparison["ensemble_beats_best_base"] =_
    ↪comparison["delta_rmse_ensemble_minus_best_base"] < -ENSEMBLE_EPS
    comparison["ensemble_ties_best_base"] =_
    ↪comparison["delta_rmse_ensemble_minus_best_base"].abs() <= ENSEMBLE_EPS

    if comparison.empty:
        display_note("No overlapping datasets with both ensemble-method and_
    ↪base-model RMSE values were found.")
    else:
        summary_rows = []
        for method_name in EXPECTED_ENSEMBLE_METHODS:
            local = comparison.loc[comparison["ensemble_method"] ==_
    ↪method_name].copy()
            if local.empty:
                summary_rows.append(
                    {
                        "ensemble_method": method_name,
                        "datasets_compared": 0,
                        "ensemble_beats_base_count": 0,
                        "ensemble_ties_base_count": 0,
                        "ensemble_loses_to_base_count": 0,
                        "ensemble_beats_base_fraction": np.nan,
                        "mean_delta_rmse": np.nan,
                        "median_delta_rmse": np.nan,
                        "std_delta_rmse": np.nan,
                        "min_delta_rmse": np.nan,
                        "max_delta_rmse": np.nan,
                        "mean_relative_delta": np.nan,
                        "median_relative_delta": np.nan,
                    }
                )
            else:
                beats = int((local["delta_rmse_ensemble_minus_best_base"] <_
    ↪-ENSEMBLE_EPS).sum())
                ties = int((local["delta_rmse_ensemble_minus_best_base"].abs() <=_
    ↪ENSEMBLE_EPS).sum())
                losses = int((local["delta_rmse_ensemble_minus_best_base"] >_
    ↪ENSEMBLE_EPS).sum())
                total = int(len(local))

                summary_rows.append(
                    {
                        "ensemble_method": method_name,
                        "datasets_compared": total,

```

```

        "ensemble_beats_base_count": beats,
        "ensemble_ties_base_count": ties,
        "ensemble_loses_to_base_count": losses,
        "ensemble_beats_base_fraction": beats / total if total else
↪ np.nan,
        "mean_delta_rmse":
↪ float(local["delta_rmse_ensemble_minus_best_base"].mean()),
        "median_delta_rmse":
↪ float(local["delta_rmse_ensemble_minus_best_base"].median()),
        "std_delta_rmse":
↪ float(local["delta_rmse_ensemble_minus_best_base"].std(ddof=1)) if total >=
↪ 2 else np.nan,
        "min_delta_rmse":
↪ float(local["delta_rmse_ensemble_minus_best_base"].min()),
        "max_delta_rmse":
↪ float(local["delta_rmse_ensemble_minus_best_base"].max()),
        "mean_relative_delta":
↪ float(local["relative_delta_vs_best_base"].mean(skipna=True)),
        "median_relative_delta":
↪ float(local["relative_delta_vs_best_base"].median(skipna=True)),
    }
)

method_vs_base_summary = pd.DataFrame(summary_rows).sort_values(
    ["ensemble_beats_base_fraction", "mean_delta_rmse"],
    ascending=[False, True],
).reset_index(drop=True)

display_note("### Ensemble Method Delta vs Best Base Model (across
↪ datasets)")
display(method_vs_base_summary)
display(comparison.sort_values(["ensemble_method",
↪ "delta_rmse_ensemble_minus_best_base"], ascending=[True, True]).
↪ reset_index(drop=True))

fig = px.box(
    comparison,
    x="ensemble_method",
    y="delta_rmse_ensemble_minus_best_base",
    color="ensemble_method",
    points="all",
    title="Ensemble RMSE delta vs best base model across datasets (by
↪ method)",
)
fig.add_hline(y=0.0, line_dash="dash", line_color="black")
fig.update_layout(

```

```

        xaxis_title="Ensemble method",
        yaxis_title="ensemble RMSE - best base RMSE (negative is better)",
        showlegend=False,
    )
    fig.show()

    # Pairwise superiority between ensemble methods on overlapping datasets.
    method_matrix = method_best.pivot(index="dataset",
    ↪columns="ensemble_method", values="ensemble_rmse")
    pairwise_rows = []
    for i, method_a in enumerate(EXPECTED_ENSEMBLE_METHODS):
        if method_a not in method_matrix.columns:
            continue
        for method_b in EXPECTED_ENSEMBLE_METHODS[i + 1 :]:
            if method_b not in method_matrix.columns:
                continue
            pair_df = method_matrix[[method_a, method_b]].dropna().copy()
            if pair_df.empty:
                continue
            delta = pair_df[method_a] - pair_df[method_b]
            wins_a = int((delta < -ENSEMBLE_EPS).sum())
            wins_b = int((delta > ENSEMBLE_EPS).sum())
            ties = int((delta.abs() <= ENSEMBLE_EPS).sum())
            compared = int(len(pair_df))
            pairwise_rows.append(
                {
                    "method_a": method_a,
                    "method_b": method_b,
                    "datasets_compared": compared,
                    "method_a_wins": wins_a,
                    "method_b_wins": wins_b,
                    "ties": ties,
                    "method_a_win_fraction": wins_a / compared if compared
    ↪else np.nan,
                    "mean_delta_rmse_a_minus_b": float(delta.mean()),
                    "median_delta_rmse_a_minus_b": float(delta.median()),
                }
            )

    if pairwise_rows:
        pairwise_df = pd.DataFrame(pairwise_rows).sort_values(
            ["datasets_compared", "method_a_win_fraction",
    ↪"mean_delta_rmse_a_minus_b"],
            ascending=[False, False, True],
        ).reset_index(drop=True)

```



```

        superiority_score = {m: {"wins": 0, "losses": 0, "ties": 0,
↪ "comparisons": 0} for m in EXPECTED_ENSEMBLE_METHODS}
        for _, row in pairwise_df.iterrows():
            a = row["method_a"]
            b = row["method_b"]
            aw = int(row["method_a_wins"])
            bw = int(row["method_b_wins"])
            tw = int(row["ties"])
            superiority_score[a]["wins"] += aw
            superiority_score[a]["losses"] += bw
            superiority_score[a]["ties"] += tw
            superiority_score[a]["comparisons"] +=
↪ int(row["datasets_compared"])
            superiority_score[b]["wins"] += bw
            superiority_score[b]["losses"] += aw
            superiority_score[b]["ties"] += tw
            superiority_score[b]["comparisons"] +=
↪ int(row["datasets_compared"])

        superiority_table = pd.DataFrame(
            [
                {
                    "ensemble_method": method_name,
                    "pairwise_wins": payload["wins"],
                    "pairwise_losses": payload["losses"],
                    "pairwise_ties": payload["ties"],
                    "pairwise_compared": payload["comparisons"],
                    "pairwise_win_fraction": payload["wins"] /
↪ payload["comparisons"] if payload["comparisons"] else np.nan,
                }
                for method_name, payload in superiority_score.items()
            ]
        ).sort_values(["pairwise_win_fraction", "pairwise_wins"],
↪ ascending=[False, False]).reset_index(drop=True)

        display_note("### Ensemble Method Pairwise Superiority
↪ (head-to-head)")
        display(pairwise_df)
        display(superiority_table)
    else:
        display_note("No overlapping datasets had RMSE values for pairwise
↪ method-vs-method ensemble comparisons.")

    # Dataset-level winner summary including the best base model.
    best_base_series = base_best.set_index("dataset")["best_base_rmse"]
    winner_matrix = method_matrix.copy()

```

```

winner_matrix[BASELINE_LABEL] = best_base_series
winner_matrix = winner_matrix.dropna(how="all")

if winner_matrix.empty:
    display_note("Could not compute cross-dataset winner summary (no
↳overlap between ensemble methods and base model).")
else:
    winner_weights = {name: 0.0 for name in [BASELINE_LABEL] +
↳EXPECTED_ENSEMBLE_METHODS}
    considered_datasets = 0
    for _, row in winner_matrix.iterrows():
        row_valid = row.dropna()
        if row_valid.empty:
            continue
        considered_datasets += 1
        best_val = float(row_valid.min())
        winners = [name for name, value in row_valid.items() if
↳abs(float(value) - best_val) <= ENSEMBLE_EPS]
        if not winners:
            continue
        share = 1.0 / len(winners)
        for winner in winners:
            winner_weights[winner] = winner_weights.get(winner, 0.0) +
↳share

    winner_summary = pd.DataFrame(
        [
            {
                "method_or_baseline": name,
                "dataset_best_weighted_wins": float(winner_weights.
↳get(name, 0.0)),
                "dataset_best_weighted_win_fraction": (
                    float(winner_weights.get(name, 0.0)) /
↳considered_datasets if considered_datasets else np.nan
                ),
                "datasets_considered": considered_datasets,
            }
            for name in [BASELINE_LABEL] + EXPECTED_ENSEMBLE_METHODS
        ]
    ).sort_values(["dataset_best_weighted_win_fraction",
↳"dataset_best_weighted_wins"], ascending=[False, False]).
    ↳reset_index(drop=True)

    display_note("### Cross-Dataset Winner Summary (best RMSE among
↳base + ensemble methods)")
    display(winner_summary)

```

1.0.24 Ensemble Method Coverage

	ensemble_method	rows	datasets
0	CFA (Combinatorial Fusion)	19	19
1	OOF stacking (RidgeCV)	20	20
2	Model average (inverse train RMSE)	8	8

No ensemble_weights.csv files were found for this run; skipping weight diagnostics.

1.0.25 Ensemble Method Delta vs Best Base Model (across datasets)

	ensemble_method	datasets_compared	ensemble_beats_base_count	ensemble_ties_base_count	ensemble_loses_to_base_count	ensemble_beats_base_fraction	mean_delta_rmse	\
0	Model average (inverse train RMSE)	8						
↪ 1		0				7		
↪ 0.125000		0.150031						
1	CFA (Combinatorial Fusion)	19						
↪ 2		0				17		
↪ 0.105263		0.836171						
2	OOF stacking (RidgeCV)	20						
↪ 1		0				19		
↪ 0.050000		0.788894						
	median_delta_rmse	std_delta_rmse	min_delta_rmse	max_delta_rmse				
↪ mean_relative_delta								
0	0.018589	0.377733	-0.007133	1.083601				
↪ 0.071694		0.048409						
1	0.045698	1.843293	-0.035164	5.949377				
↪ 0.162137		0.099778						
2	0.023337	1.838017	-0.002096	6.704866				
↪ 0.070736		0.041991						
	dataset	ensemble_method	ensemble_rmse	best_base_rmse	delta_rmse_ensemble_minus_best_base	relative_delta_vs_best_base	ensemble_beats_best_base	\
0	tdc_hydrationfreeenergy_freesolv	CFA (Combinatorial Fusion)	0.556425	0.591589	-0.035164			
↪ -0.059440		True						
1	poduam_pod_nc_std	CFA (Combinatorial Fusion)	0.700876	0.701017	-0.000141			
↪ -0.000201		True						
2	chemml_xyz_polarizability	CFA (Combinatorial Fusion)	0.008397	0.008000	0.000397			
↪ 0.049633		False						
3	chemml_organic_density	CFA (Combinatorial Fusion)	0.006350	0.005112	0.001238			
↪ 0.242224		False						

4	chemml_cep_homo	CFA (Combinatorial Fusion)	0.
↪102083	0.085692	0.016391	⌞
↪ 0.191283	False		
5	tdc_caco2_wang	CFA (Combinatorial Fusion)	0.
↪382279	0.349680	0.032599	⌞
↪ 0.093224	False		
6	polaris_adme_fang_rppb_1	CFA (Combinatorial Fusion)	0.
↪538896	0.501699	0.037197	⌞
↪ 0.074142	False		
7	polaris_adme_fang_rclint_1	CFA (Combinatorial Fusion)	0.
↪550578	0.512297	0.038281	⌞
↪ 0.074725	False		
8	polaris_adme_fang_solu_1	CFA (Combinatorial Fusion)	0.
↪619494	0.575097	0.044397	⌞
↪ 0.077199	False		
9	lipophilicity	CFA (Combinatorial Fusion)	0.
↪637404	0.591706	0.045698	⌞
↪ 0.077230	False		
10	polaris_adme_fang_hppb_1	CFA (Combinatorial Fusion)	0.
↪505969	0.456404	0.049565	⌞
↪ 0.108598	False		
11	polaris_adme_fang_perm_1	CFA (Combinatorial Fusion)	0.
↪466135	0.399061	0.067074	⌞
↪ 0.168081	False		
12	esol_delaney	CFA (Combinatorial Fusion)	0.
↪681754	0.612426	0.069327	⌞
↪ 0.113201	False		
13	poduam_pod_rd_std	CFA (Combinatorial Fusion)	0.
↪647592	0.550777	0.096815	⌞
↪ 0.175778	False		
14	freesolv_sampl	CFA (Combinatorial Fusion)	1.
↪210224	1.079859	0.130365	⌞
↪ 0.120724	False		
15	tdc_ppbr_az	CFA (Combinatorial Fusion)	12.
↪405624	11.711870	0.693754	⌞
↪ 0.059235	False		
16	tdc_clearance_microsome_az	CFA (Combinatorial Fusion)	39.
↪468904	35.888060	3.580845	⌞
↪ 0.099778	False		
17	tdc_vdss_lombardo	CFA (Combinatorial Fusion)	9.
↪807975	4.738737	5.069238	⌞
↪ 1.069744	False		
18	tdc_half_life_obach	CFA (Combinatorial Fusion)	23.
↪171631	17.222254	5.949377	⌞
↪ 0.345447	False		

19	lipophilicity	Model average (inverse train RMSE)	0.
↪584573	0.591706	-0.007133	⌞
↪-0.012055	True		
20	chemml_organic_density	Model average (inverse train RMSE)	0.
↪005233	0.005112	0.000121	⌞
↪0.023637	False		
21	chemml_xyz_polarizability	Model average (inverse train RMSE)	0.
↪009786	0.008000	0.001785	⌞
↪0.223166	False		
22	chemml_cep_homo	Model average (inverse train RMSE)	0.
↪096283	0.085692	0.010592	⌞
↪0.123601	False		
23	polaris_adme_fang_perm_1	Model average (inverse train RMSE)	0.
↪425648	0.399061	0.026587	⌞
↪0.066625	False		
24	freesolv_sampl	Model average (inverse train RMSE)	1.
↪112272	1.079859	0.032413	⌞
↪0.030016	False		
25	tdc_hydrationfreeenergy_freesolv	Model average (inverse train RMSE)	0.
↪643868	0.591589	0.052279	⌞
↪0.088370	False		
26	tdc_clearance_microsome_az	Model average (inverse train RMSE)	36.
↪971661	35.888060	1.083601	⌞
↪0.030194	False		
27	esol_delaney	OOF stacking (RidgeCV)	0.
↪610331	0.612426	-0.002096	⌞
↪-0.003422	True		
28	poduam_pod_nc_std	OOF stacking (RidgeCV)	0.
↪701051	0.701017	0.000034	⌞
↪0.000049	False		
29	chemml_organic_density	OOF stacking (RidgeCV)	0.
↪005253	0.005112	0.000141	⌞
↪0.027644	False		
30	chemml_cep_homo	OOF stacking (RidgeCV)	0.
↪085936	0.085692	0.000244	⌞
↪0.002846	False		
31	chemml_xyz_polarizability	OOF stacking (RidgeCV)	0.
↪008288	0.008000	0.000288	⌞
↪0.035982	False		
32	lipophilicity	OOF stacking (RidgeCV)	0.
↪597248	0.591706	0.005542	⌞
↪0.009366	False		
33	polaris_adme_fang_rclint_1	OOF stacking (RidgeCV)	0.
↪518468	0.512297	0.006171	⌞
↪0.012046	False		

34	tdc_hydrationfreeenergy_freesolv	00F stacking (RidgeCV)	0.
	↪602478 0.591589	0.010889	↪
	↪ 0.018407 False		
35	polaris_adme_fang_solu_1	00F stacking (RidgeCV)	0.
	↪595731 0.575097	0.020634	↪
	↪ 0.035880 False		
36	polaris_adme_fang_hppb_1	00F stacking (RidgeCV)	0.
	↪478996 0.456404	0.022592	↪
	↪ 0.049500 False		
37	polaris_adme_fang_rppb_1	00F stacking (RidgeCV)	0.
	↪525781 0.501699	0.024082	↪
	↪ 0.048000 False		
38	tdc_caco2_wang	00F stacking (RidgeCV)	0.
	↪379770 0.349680	0.030090	↪
	↪ 0.086049 False		
39	polaris_adme_fang_perm_1	00F stacking (RidgeCV)	0.
	↪429519 0.399061	0.030458	↪
	↪ 0.076324 False		
40	tdc_ppbr_az	00F stacking (RidgeCV)	11.
	↪772382 11.711870	0.060511	↪
	↪ 0.005167 False		
41	poduam_pod_rd_std	00F stacking (RidgeCV)	0.
	↪668464 0.550777	0.117687	↪
	↪ 0.213675 False		
42	freesolv_sampl	00F stacking (RidgeCV)	1.
	↪209470 1.079859	0.129611	↪
	↪ 0.120026 False		
43	tdc_vdss_lombardo	00F stacking (RidgeCV)	5.
	↪505192 4.738737	0.766454	↪
	↪ 0.161742 False		
44	tdc_clearance_hepatocyte_az	00F stacking (RidgeCV)	48.
	↪046579 44.472136	3.574442	↪
	↪ 0.080375 False		
45	tdc_half_life_obach	00F stacking (RidgeCV)	21.
	↪497500 17.222254	4.275246	↪
	↪ 0.248240 False		
46	tdc_clearance_microsome_az	00F stacking (RidgeCV)	42.
	↪592925 35.888060	6.704866	↪
	↪ 0.186827 False		
ensemble_ties_best_base			
0	False		
1	False		
2	False		
3	False		
4	False		
5	False		

6	False
7	False
8	False
9	False
10	False
11	False
12	False
13	False
14	False
15	False
16	False
17	False
18	False
19	False
20	False
21	False
22	False
23	False
24	False
25	False
26	False
27	False
28	False
29	False
30	False
31	False
32	False
33	False
34	False
35	False
36	False
37	False
38	False
39	False
40	False
41	False
42	False
43	False
44	False
45	False
46	False

1.0.26 Ensemble Method Pairwise Superiority (head-to-head)

	method_a	method_b	datasets_compared	method_a_wins	method_b_wins	ties	method_a_win_fraction	mean_delta_rmse_a_minus_b \
0	CFA (Combinatorial Fusion)	OOF stacking (RidgeCV)		19	4	15	0	0.210526
								0.193885
1	OOF stacking (RidgeCV)	Model average (inverse train RMSE)		8	3	5	0	0.375000
								0.710224
2	CFA (Combinatorial Fusion)	Model average (inverse train RMSE)		8	2	6	0	0.250000
								0.325825

	median_delta_rmse_a_minus_b
0	0.016148
1	0.001946
2	0.023144

	ensemble_method	pairwise_wins	pairwise_losses	pairwise_ties	pairwise_compared	pairwise_win_fraction
0	Model average (inverse train RMSE)	11	5	0	16	0.687500
1	OOF stacking (RidgeCV)	18	9	0	27	0.666667
2	CFA (Combinatorial Fusion)	6	21	0	27	0.222222

1.0.27 Cross-Dataset Winner Summary (best RMSE among base + ensemble methods)

	method_or_baseline	dataset_best_weighted_wins	dataset_best_weighted_win_fraction	datasets_considered
0	Best base model	16.0	0.80	20
1	CFA (Combinatorial Fusion)	2.0	0.10	20
2	OOF stacking (RidgeCV)	1.0	0.05	20
3	Model average (inverse train RMSE)	1.0	0.05	20

```
[66]: # NO_RERUN_PUBLICATION_HELPERS
# Shared helpers for no-rerun publication analyses.

import hashlib
import subprocess
```



```
NO_RERUN_EPS = 1e-12
```

```
def successful_primary_metric_rows(frame=None):
    work = (SUMMARY_ANALYSIS_METRICS if frame is None else frame).copy()
    if work is None or work.empty:
        return pd.DataFrame()
    error_col = pick_column(work.columns, ["error", "Error"])
    if error_col is not None:
        work = work.loc[work[error_col].isna() | (work[error_col].astype(str).
↪str.strip() == "")].copy()
    if "analysis_metric_value" not in work.columns:
        work = add_analysis_metric_columns(work, rmse_col=RMSE_COL,
↪classification_col=CLASSIFICATION_FALLBACK_COL)
    work["analysis_metric_value"] = pd.
↪to_numeric(work["analysis_metric_value"], errors="coerce")
    work = work.loc[work["analysis_metric_value"].notna()].copy()
    if work.empty:
        return work
    work["analysis_metric_direction"] = work["analysis_metric_direction"].
↪fillna("lower").astype(str)
    work["comparison_score"] = np.where(
        work["analysis_metric_direction"].str.lower().eq("higher"),
        -work["analysis_metric_value"],
        work["analysis_metric_value"],
    )
    work["rank_within_dataset"] = work.groupby("dataset")["comparison_score"].
↪rank(method="min", ascending=True)
    if "architecture_family" not in work.columns:
        if "classify_architecture_family" in globals():
            work["architecture_family"] = work.apply(
                lambda row: classify_architecture_family(row.get("workflow",
↪""), row.get("model", "")),
                axis=1,
            )
        else:
            work["architecture_family"] = work.get("workflow", pd.Series("",
↪index=work.index)).astype(str)
    return work

def classify_publication_ensemble_method(model_name, workflow_name=""):
    model_text = str(model_name or "").strip().lower()
    workflow_text = str(workflow_name or "").strip().lower()
    if model_text.startswith("cfa (") or "cfa" in workflow_text:
```

```

        return "CFA fusion"
    if model_text.startswith("ensemble (") or "ensemble" in workflow_text:
        if "oof stacking" in model_text:
            return "OOF stacking"
        if "weighted average" in model_text and "inverse" in model_text and
↪ "rmse" in model_text:
            return "Inverse-RMSE weighted average"
        return "Other ensemble"
    return None

def metric_relative_improvement(base_value, challenger_value, direction):
    if pd.isna(base_value) or pd.isna(challenger_value):
        return np.nan
    denom = max(abs(float(base_value)), NO_RERUN_EPS)
    if str(direction).lower() == "higher":
        return (float(challenger_value) - float(base_value)) / denom
    return (float(base_value) - float(challenger_value)) / denom

def write_publication_csv(frame, filename):
    path = RUN_DIR / filename
    if frame is None:
        frame = pd.DataFrame()
    frame.to_csv(path, index=False)
    return path

def sha256_file(path, block_size=1024 * 1024):
    path = Path(path)
    if not path.exists() or not path.is_file():
        return ""
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        while True:
            chunk = handle.read(block_size)
            if not chunk:
                break
            digest.update(chunk)
    return digest.hexdigest()

def git_value(args):
    try:
        return subprocess.check_output(["git", *args], cwd=Path(".").resolve(),
↪ text=True, stderr=subprocess.DEVNULL).strip()
    except Exception:

```

```

        return ""

def load_publication_predictions():
    if PREDICTIONS is not None and not PREDICTIONS.empty:
        return PREDICTIONS.copy()
    prediction_tables = []
    for predictions_path in sorted(RUN_DIR.glob("*/predictions.csv")):
        try:
            frame = pd.read_csv(predictions_path)
        except Exception:
            continue
        if frame.empty:
            continue
        if "dataset" not in frame.columns:
            frame["dataset"] = predictions_path.parent.name
        prediction_tables.append(frame)
    if not prediction_tables:
        return pd.DataFrame()
    return pd.concat(prediction_tables, ignore_index=True, sort=False)

```

```

[67]: # NO_RERUN_PUBLICATION_ENSEMBLE_VALUE
# Publication analysis 1: primary-metric-aware ensemble/CFA value-add.

ensemble_source = successful_primary_metric_rows()

if ensemble_source.empty:
    display_note("No successful metric rows were available for ensemble_
↪value-add analysis.")
else:
    ensemble_source["publication_ensemble_method"] = [
        classify_publication_ensemble_method(model_name, workflow_name)
        for model_name, workflow_name in zip(ensemble_source["model"],
↪ensemble_source.get("workflow", pd.Series("", index=ensemble_source.index)))
    ]
    ensemble_rows = ensemble_source.
↪loc[ensemble_source["publication_ensemble_method"].notna()].copy()
    base_rows = ensemble_source.
↪loc[ensemble_source["publication_ensemble_method"].isna()].copy()

    if ensemble_rows.empty or base_rows.empty:
        display_note("Could not compute ensemble value-add because either_
↪ensemble/CFA rows or base-model rows are missing.")
    else:
        base_best = (
            base_rows.sort_values(["dataset", "comparison_score", "model"],
↪ascending=[True, True, True])

```

```

        .groupby("dataset", as_index=False, dropna=False)
        .first()[[
            "dataset", "model", "workflow", "architecture_family",
↪ "analysis_metric", "analysis_metric_kind",
            "analysis_metric_direction", "analysis_metric_value",
↪ "comparison_score"
        ]]
        .rename(columns={
            "model": "best_base_model",
            "workflow": "best_base_workflow",
            "architecture_family": "best_base_family",
            "analysis_metric": "base_metric_column",
            "analysis_metric_kind": "base_metric_kind",
            "analysis_metric_direction": "metric_direction",
            "analysis_metric_value": "best_base_metric_value",
            "comparison_score": "best_base_comparison_score",
        })
    )

    competitive_rows = base_rows.merge(
        base_best[["dataset", "best_base_comparison_score"]],
        on="dataset",
        how="left",
    )
    competitive_rows["base_delta_from_best_base"] =
↪ competitive_rows["comparison_score"] -
↪ competitive_rows["best_base_comparison_score"]
    competitive_rows["competitive_threshold"] = np.maximum(0.05 *
↪ competitive_rows["best_base_comparison_score"].abs(), NO_RERUN_EPS)
    competitive_base_counts = (
        competitive_rows.loc[competitive_rows["base_delta_from_best_base"]
↪ <= competitive_rows["competitive_threshold"]]
        .groupby("dataset", as_index=False)["model"]
        .nunique()
        .rename(columns={"model": "competitive_base_model_count_5pct"})
    )

    ensemble_best = (
        ensemble_rows.sort_values(["dataset",
↪ "publication_ensemble_method", "comparison_score", "model"],
↪ ascending=[True, True, True, True])
        .groupby(["dataset", "publication_ensemble_method"],
↪ as_index=False, dropna=False)
        .first()[[
            "dataset", "publication_ensemble_method", "model", "workflow",
↪ "architecture_family", "task_kind",

```

```

        "analysis_metric", "analysis_metric_kind",
        ↪"analysis_metric_direction", "analysis_metric_value",
        "comparison_score", "rank_within_dataset"
    ]]
    .rename(columns={
        "model": "ensemble_model",
        "workflow": "ensemble_workflow",
        "architecture_family": "ensemble_family",
        "analysis_metric": "ensemble_metric_column",
        "analysis_metric_kind": "ensemble_metric_kind",
        "analysis_metric_direction": "ensemble_metric_direction",
        "analysis_metric_value": "ensemble_metric_value",
        "comparison_score": "ensemble_comparison_score",
        "rank_within_dataset": "ensemble_rank_within_dataset",
    })
)

ensemble_value_add = ensemble_best.merge(base_best, on="dataset",
↪how="inner")
ensemble_value_add = ensemble_value_add.merge(competitive_base_counts,
↪on="dataset", how="left")
ensemble_value_add["competitive_base_model_count_5pct"] =
↪ensemble_value_add["competitive_base_model_count_5pct"].fillna(1).astype(int)
ensemble_value_add["score_improvement_vs_best_base"] =
↪ensemble_value_add["best_base_comparison_score"] -
↪ensemble_value_add["ensemble_comparison_score"]
ensemble_value_add["relative_metric_improvement_vs_best_base"] = [
    metric_relative_improvement(base_value, ensemble_value, direction)
    for base_value, ensemble_value, direction in zip(
        ensemble_value_add["best_base_metric_value"],
        ensemble_value_add["ensemble_metric_value"],
        ensemble_value_add["metric_direction"],
    )
]
ensemble_value_add["ensemble_beats_best_base"] =
↪ensemble_value_add["score_improvement_vs_best_base"] > NO_RERUN_EPS
ensemble_value_add["ensemble_ties_best_base"] =
↪ensemble_value_add["score_improvement_vs_best_base"].abs() <= NO_RERUN_EPS
ensemble_value_add["ensemble_loses_to_best_base"] =
↪ensemble_value_add["score_improvement_vs_best_base"] < -NO_RERUN_EPS
ensemble_value_add["ensemble_wins_dataset"] = pd.
↪to_numeric(ensemble_value_add["ensemble_rank_within_dataset"],
↪errors="coerce") <= 1.0
ensemble_value_add["ensemble_top3_dataset"] = pd.
↪to_numeric(ensemble_value_add["ensemble_rank_within_dataset"],
↪errors="coerce") <= 3.0

```

```

ensemble_value_summary = (
    ensemble_value_add.groupby(["publication_ensemble_method",
    ↪ "task_kind"], dropna=False)
    .agg(
        datasets=("dataset", "nunique"),
        wins_dataset=("ensemble_wins_dataset", "sum"),
        top3_datasets=("ensemble_top3_dataset", "sum"),
        beats_best_base=("ensemble_beats_best_base", "sum"),
        ties_best_base=("ensemble_ties_best_base", "sum"),
        loses_to_best_base=("ensemble_loses_to_best_base", "sum"),
        median_rank=("ensemble_rank_within_dataset", "median"),
        mean_rank=("ensemble_rank_within_dataset", "mean"),
        median_score_improvement=("score_improvement_vs_best_base",
    ↪ "median"),
        mean_score_improvement=("score_improvement_vs_best_base",
    ↪ "mean"),
        ↵
    ↪ median_relative_metric_improvement=("relative_metric_improvement_vs_best_base",
    ↪ "median"),
        ↵
    ↪ median_competitive_base_models=("competitive_base_model_count_5pct",
    ↪ "median"),
    )
    .reset_index()
    .sort_values(["task_kind", "top3_datasets", "beats_best_base",
    ↪ "median_score_improvement"], ascending=[True, False, False, False])
    )
    ensemble_value_summary["top3_fraction"] =
    ↪ ensemble_value_summary["top3_datasets"] / ensemble_value_summary["datasets"].
    ↪ replace(0, np.nan)
    ensemble_value_summary["beats_best_base_fraction"] =
    ↪ ensemble_value_summary["beats_best_base"] /
    ↪ ensemble_value_summary["datasets"].replace(0, np.nan)

    display_note("### Publication Analysis: Ensemble/CFA Value-Add")
    display(ensemble_value_summary.reset_index(drop=True))
    display(
        ensemble_value_add.sort_values(["publication_ensemble_method",
    ↪ "task_kind", "score_improvement_vs_best_base"], ascending=[True, True,
    ↪ False])
        .reset_index(drop=True)
    )

    write_publication_csv(ensemble_value_summary,
    ↪ "publication_ensemble_value_add_summary.csv")

```

```

write_publication_csv(ensemble_value_add,
↳"publication_ensemble_value_add_by_dataset.csv")

fig = px.box(
    ensemble_value_add,
    x="publication_ensemble_method",
    y="score_improvement_vs_best_base",
    color="task_kind",
    points="all",
    hover_data=["dataset", "ensemble_model", "best_base_model",
↳"ensemble_rank_within_dataset", "relative_metric_improvement_vs_best_base"],
    title="Ensemble/CFA value-add vs best base model (primary metric;
↳positive is better)",
)
fig.add_hline(y=0.0, line_dash="dash", line_color="black")
fig.update_layout(xaxis_title="Fusion method",
↳yaxis_title="Primary-metric improvement vs best base")
fig.show()

```

1.0.28 Publication Analysis: Ensemble/CFA Value-Add

	publication_ensemble_method	task_kind	datasets	wins_dataset	↳
0	OOF stacking classification		22	7	↳
↳	12	9	0	13	3.0 4.
↳	772727	-0.003356			
1	Inverse-RMSE weighted average classification		13	4	↳
↳	7	6	0	7	3.0 4.
↳	000000	-0.002174			
2	CFA fusion classification		22	1	↳
↳	6	5	0	17	6.0 6.
↳	545455	-0.015364			
3	OOF stacking regression		22	0	↳
↳	8	2	0	20	5.5 6.
↳	000000	-0.029225			
4	Inverse-RMSE weighted average regression		9	1	↳
↳	7	3	0	6	3.0 2.
↳	666667	-0.000514			
5	CFA fusion regression		22	4	↳
↳	6	4	0	18	7.0 6.
↳	590909	-0.032066			

	mean_score_improvement	median_relative_metric_improvement	↳
0	-0.021946	-0.003474	↳
↳	8.5	0.545455	0.409091

1	-0.006150	-0.002397	
↪	10.0	0.538462	0.461538
2	-0.023982	-0.019396	
↪	8.5	0.272727	0.227273
3	-0.939874	-0.042270	
↪	3.0	0.363636	0.090909
4	-0.081111	-0.014325	
↪	3.0	0.777778	0.333333
5	-0.873428	-0.081835	
↪	3.0	0.272727	0.181818
	dataset	publication_ensemble_method	
↪	ensemble_model	ensemble_workflow	ensemble_family
↪	task_kind	ensemble_metric_column \	
0	tdc_skin_reaction	CFA fusion	CFA
↪	(Combinatorial Fusion)	CFA fusion	CFA combinatorial fusion
↪	classification	test_roc_auc	
1	tdc_cyp2c9_veith	CFA fusion	CFA
↪	(Combinatorial Fusion)	CFA fusion	CFA combinatorial fusion
↪	classification	test_auprc	
2	tdc_dili	CFA fusion	CFA
↪	(Combinatorial Fusion)	CFA fusion	CFA combinatorial fusion
↪	classification	test_roc_auc	
3	tdc_herg_karim	CFA fusion	CFA
↪	(Combinatorial Fusion)	CFA fusion	CFA combinatorial fusion
↪	classification	test_roc_auc	
4	tdc_hia_hou	CFA fusion	CFA
↪	(Combinatorial Fusion)	CFA fusion	CFA combinatorial fusion
↪	classification	test_roc_auc	
..	...	...	
↪	...	...	...
↪	...	...	...
105	tdc_ppbr_az	OOF stacking Ensemble (OOF	
↪	Stacking (RidgeCV, 5-fold))	ensemble	Ensemble meta-model
↪	regression	test_rmse	
106	tdc_vdss_lombardo	OOF stacking Ensemble (OOF	
↪	Stacking (RidgeCV, 5-fold))	ensemble	Ensemble meta-model
↪	regression	test_rmse	
107	tdc_half_life_obach	OOF stacking Ensemble (OOF	
↪	Stacking (RidgeCV, 5-fold))	ensemble	Ensemble meta-model
↪	regression	test_rmse	
108	tdc_clearance_microsome_az	OOF stacking Ensemble (OOF	
↪	Stacking (RidgeCV, 5-fold))	ensemble	Ensemble meta-model
↪	regression	test_rmse	
109	tdc_clearance_hepatocyte_az	OOF stacking Ensemble (OOF	
↪	Stacking (RidgeCV, 5-fold))	ensemble	Ensemble meta-model
↪	regression	test_rmse	

	ensemble_metric_kind	ensemble_metric_direction	ensemble_metric_value	
	ensemble_comparison_score	ensemble_rank_within_dataset		
	best_base_model	best_base_workflow	\	
0	roc_auc	higher	0.761935	
	-0.761935	2.0		
	TabPFNClassifier	TabPFN deep learning		
1	auprc	higher	0.796950	
	-0.796950	3.0		
	HistGradientBoosting	conventional		
2	roc_auc	higher	0.908913	
	-0.908913	2.0		
	Random forest	conventional		
3	roc_auc	higher	0.943185	
	-0.943185	2.0		
	Random forest	conventional		
4	roc_auc	higher	0.989918	
	-0.989918	1.0		
	CatBoost	conventional		
..	...	...	...	
	...	...	...	
	...	...	...	
105	rmse	lower	11.986943	
	11.986943	5.0	MapLight + GNN (CatBoost, Strict Parity)	
106	rmse	lower	5.541208	
	5.541208	7.0	MapLight + GNN (CatBoost, Strict Parity)	
107	rmse	lower	21.502196	
	21.502196	10.0	TabPFNRegressor	
108	rmse	lower	42.594895	
	42.594895	18.0	MapLight + GNN (CatBoost, Strict Parity)	
109	rmse	lower	52.132439	
	52.132439	14.0	MapLight + GNN (CatBoost, Strict Parity)	
	best_base_family	base_metric_column	base_metric_kind	
	metric_direction	best_base_metric_value	best_base_comparison_score	
	competitive_base_model_count_5pct	\		
0	TabPFN foundation model	test_roc_auc	roc_auc	
	higher	0.746774	-0.746774	
	9			
1	Conventional ML	test_auprc	auprc	
	higher	0.794078	-0.794078	
	9			

2		Conventional ML	test_roc_auc	roc_auc	␣
↪higher		0.906957	-0.906957		␣
↪	12				
3		Conventional ML	test_roc_auc	roc_auc	␣
↪higher		0.942149	-0.942149		␣
↪	10				
4		Conventional ML	test_roc_auc	roc_auc	␣
↪higher		0.988889	-0.988889		␣
↪	14				
..		...	...	...	␣
↪		...	...	...	␣
105	Graph transfer + tabular head		test_rmse	rmse	␣
↪lower		11.378376	11.378376		␣
↪	4				
106	Graph transfer + tabular head		test_rmse	rmse	␣
↪lower		4.684131	4.684131		␣
↪	3				
107	TabPFN foundation model		test_rmse	rmse	␣
↪lower		17.210966	17.210966		␣
↪	1				
108	Graph transfer + tabular head		test_rmse	rmse	␣
↪lower		36.331993	36.331993		␣
↪	5				
109	Graph transfer + tabular head		test_rmse	rmse	␣
↪lower		43.782188	43.782188		␣
↪	2				
score_improvement_vs_best_base relative_metric_improvement_vs_best_base ␣					
↪ensemble_beats_best_base	ensemble_ties_best_base	ensemble_loses_to_best_base	␣		
↪ensemble_wins_dataset	\				
0		0.015161		0.020302	␣
↪	True		False	False	␣
↪	False				
1		0.002872		0.003616	␣
↪	True		False	False	␣
↪	False				
2		0.001957		0.002157	␣
↪	True		False	False	␣
↪	False				
3		0.001036		0.001099	␣
↪	True		False	False	␣
↪	False				
4		0.001029		0.001040	␣
↪	True		False	False	␣
↪	True				

```

..          ...          ...          ...          ...
↪          ...          ...          ...          ...
↪          ...          ...          ...          ...
105          -0.608567          -0.053485
↪          False          False          True
↪          False
106          -0.857077          -0.182975
↪          False          False          True
↪          False
107          -4.291229          -0.249331
↪          False          False          True
↪          False
108          -6.262902          -0.172380
↪          False          False          True
↪          False
109          -8.350251          -0.190723
↪          False          False          True
↪          False

ensemble_top3_dataset
0          True
1          True
2          True
3          True
4          True
..          ...
105          False
106          False
107          False
108          False
109          False

```

[110 rows x 28 columns]

```

[68]: # NO_RERUN_PUBLICATION_COMPONENT_ABLATION
# Publication analysis 2: staged component ablation derived from existing
↪ metrics.

ablation_source = successful_primary_metric_rows()

def atomic_component_family(row):
    model_text = str(row.get("model", "")).strip().lower()
    workflow_text = str(row.get("workflow", "")).strip().lower()
    ensemble_method = classify_publication_ensemble_method(row.get("model",
↪ ""), row.get("workflow", ""))
    if ensemble_method == "CFA fusion":

```

```

        return "cfa"
    if ensemble_method is not None:
        return "ensemble"
    if "maplight + gnn" in workflow_text or "maplight + gnn" in model_text:
        return "deep_backend"
    if any(token in workflow_text or token in model_text for token in
↪["chemprop", "tabPFN", "chemML", "uni-mol", "unimol"]):
        return "deep_backend"
    if "maplight" in model_text or "maplight" in workflow_text:
        return "maplight_classic"
    return "conventional_ml"

if ablation_source.empty:
    display_note("No successful metric rows were available for_
↪component-ablation analysis.")
else:
    ablation_source["atomic_component_family"] = ablation_source.
↪apply(atomic_component_family, axis=1)
    stage_definitions = [
        (1, "Conventional ML only", {"conventional_ml"}),
        (2, "+ MapLight classic features", {"conventional_ml",
↪"maplight_classic"}),
        (3, "+ neural/deep backends", {"conventional_ml", "maplight_classic",
↪"deep_backend"}),
        (4, "+ CFA fusion", {"conventional_ml", "maplight_classic",
↪"deep_backend", "cfa"}),
        (5, "Full pipeline incl. ensembles", {"conventional_ml",
↪"maplight_classic", "deep_backend", "cfa", "ensemble"}),
    ]

    ablation_rows = []
    for stage_order, stage_name, allowed_components in stage_definitions:
        local = ablation_source.loc[ablation_source["atomic_component_family"].
↪isin(allowed_components)].copy()
        if local.empty:
            continue
        best_local = (
            local.sort_values(["dataset", "comparison_score", "model"],
↪ascending=[True, True, True])
            .groupby("dataset", as_index=False, dropna=False)
            .first()
        )
        for _, row in best_local.iterrows():
            ablation_rows.append(
                {

```

```

        "dataset": row.get("dataset", ""),
        "task_kind": row.get("task_kind", ""),
        "stage_order": stage_order,
        "stage": stage_name,
        "best_model_at_stage": row.get("model", ""),
        "best_workflow_at_stage": row.get("workflow", ""),
        "best_component_family_at_stage": row.
    ↪get("atomic_component_family", ""),
        "analysis_metric": row.get("analysis_metric", ""),
        "analysis_metric_kind": row.get("analysis_metric_kind", ""),
        "analysis_metric_direction": row.
    ↪get("analysis_metric_direction", ""),
        "analysis_metric_value": row.get("analysis_metric_value",
    ↪np.nan),
        "comparison_score": row.get("comparison_score", np.nan),
    }
)

component_ablation_by_dataset = pd.DataFrame(ablation_rows)
if component_ablation_by_dataset.empty:
    display_note("No staged ablation rows could be computed from the
    ↪existing metrics.")
else:
    component_ablation_by_dataset = component_ablation_by_dataset.
    ↪sort_values(["dataset", "stage_order"]).reset_index(drop=True)
    component_ablation_by_dataset["previous_stage_score"] =
    ↪component_ablation_by_dataset.groupby("dataset")["comparison_score"].shift(1)
    component_ablation_by_dataset["previous_stage_metric_value"] =
    ↪component_ablation_by_dataset.groupby("dataset")["analysis_metric_value"].
    ↪shift(1)
    component_ablation_by_dataset["marginal_score_improvement"] =
    ↪component_ablation_by_dataset["previous_stage_score"] -
    ↪component_ablation_by_dataset["comparison_score"]
    component_ablation_by_dataset.
    ↪loc[component_ablation_by_dataset["previous_stage_score"].isna(),
    ↪"marginal_score_improvement"] = np.nan
    component_ablation_by_dataset["marginal_relative_metric_improvement"] =
    ↪[
        metric_relative_improvement(prev_value, current_value, direction)
        for prev_value, current_value, direction in zip(
            component_ablation_by_dataset["previous_stage_metric_value"],
            component_ablation_by_dataset["analysis_metric_value"],
            component_ablation_by_dataset["analysis_metric_direction"],
        )
    ]
]

```

```

        component_ablation_by_dataset["improved_vs_previous_stage"] =
↪component_ablation_by_dataset["marginal_score_improvement"] > NO_RERUN_EPS
        component_ablation_by_dataset["stage_changed_winning_model"] = (
            component_ablation_by_dataset["best_model_at_stage"]
            != component_ablation_by_dataset.
↪groupby("dataset")["best_model_at_stage"].shift(1)
        )
        component_ablation_by_dataset.
↪loc[component_ablation_by_dataset["previous_stage_score"].isna(),
↪"stage_changed_winning_model"] = False

    component_ablation_summary = (
        component_ablation_by_dataset.groupby(["stage_order", "stage"],
↪dropna=False)
        .agg(
            datasets_evaluated=("dataset", "nunique"),
            datasets_improved_vs_previous=("improved_vs_previous_stage",
↪"sum"),
            datasets_changed_winner=("stage_changed_winning_model", "sum"),
            median_marginal_score_improvement=("marginal_score_improvement", "median"),
            mean_marginal_score_improvement=("marginal_score_improvement",
↪"mean"),
            median_relative_metric_improvement=("marginal_relative_metric_improvement",
↪"median"),
        )
        .reset_index()
        .sort_values("stage_order")
    )
    component_ablation_summary["improvement_fraction"] =
↪component_ablation_summary["datasets_improved_vs_previous"] /
↪component_ablation_summary["datasets_evaluated"].replace(0, np.nan)

    display_note("### Publication Analysis: Component Ablation from
↪Existing Metrics")
    display(component_ablation_summary.reset_index(drop=True))
    display(component_ablation_by_dataset.reset_index(drop=True))

    write_publication_csv(component_ablation_summary,
↪"publication_component_ablation_summary.csv")
    write_publication_csv(component_ablation_by_dataset,
↪"publication_component_ablation_by_dataset.csv")

    fig = px.bar(
        component_ablation_summary,

```

```

        x="stage",
        y="datasets_improved_vs_previous",
        hover_data=["datasets_evaluated", "improvement_fraction",
↪ "median_marginal_score_improvement"],
        title="Datasets improved by each added pipeline stage",
    )
    fig.update_layout(xaxis_title="Cumulative stage", yaxis_title="Datasets_
↪ improved vs previous stage")
    fig.show()

```

1.0.29 Publication Analysis: Component Ablation from Existing Metrics

	stage_order	stage	datasets_evaluated	↪
↪	datasets_improved_vs_previous	datasets_changed_winner	↪	
↪	median_marginal_score_improvement	mean_marginal_score_improvement	\	
0	1	Conventional ML only	44	↪
↪	0	0	NaN	↪
↪		NaN		
1	2	+ MapLight classic features	44	↪
↪	7	7	0.0	↪
↪		0.071573		
2	3	+ neural/deep backends	44	↪
↪	14	14	0.0	↪
↪		0.079094		
3	4	+ CFA fusion	44	↪
↪	9	9	0.0	↪
↪		0.002889		
4	5	Full pipeline incl. ensembles	44	↪
↪	12	12	0.0	↪
↪		0.001767		

	median_relative_metric_improvement	improvement_fraction
0	NaN	0.000000
1	0.0	0.159091
2	0.0	0.318182
3	0.0	0.204545
4	0.0	0.272727

	dataset	task_kind	stage_order	stage	↪
↪		best_model_at_stage	best_workflow_at_stage	↪	
↪	best_component_family_at_stage	analysis_metric	\		
0	chemml_cep_homo	regression	1	Conventional ML only	↪
↪		HistGradientBoosting		conventional	↪
↪	conventional_ml	test_rmse			
1	chemml_cep_homo	regression	2	+ MapLight classic features	↪
↪		HistGradientBoosting		conventional	↪
↪	conventional_ml	test_rmse			

2	chemml_cep_homo	regression	3	+ neural/deep backends	␣
↳		TabPFNRegressor	TabPFN deep learning		␣
↳	deep_backend	test_rmse			
3	chemml_cep_homo	regression	4	+ CFA fusion	␣
↳		TabPFNRegressor	TabPFN deep learning		␣
↳	deep_backend	test_rmse			
4	chemml_cep_homo	regression	5	Full pipeline incl. ensembles	␣
↳		TabPFNRegressor	TabPFN deep learning		␣
↳	deep_backend	test_rmse			
..	...	...	...	...	␣
↳		...	...		␣
↳...	...				
215	tdc_vdss_lombardo	regression	1	Conventional ML only	␣
↳		SVR	conventional		␣
↳	conventional_ml	test_rmse			
216	tdc_vdss_lombardo	regression	2	+ MapLight classic features	␣
↳		MapLight CatBoost (Strict Parity)	conventional		␣
↳	maplight_classic	test_rmse			
217	tdc_vdss_lombardo	regression	3	+ neural/deep backends	␣
↳		MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN		␣
↳	deep_backend	test_rmse			
218	tdc_vdss_lombardo	regression	4	+ CFA fusion	␣
↳		MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN		␣
↳	deep_backend	test_rmse			
219	tdc_vdss_lombardo	regression	5	Full pipeline incl. ensembles	␣
↳		MapLight + GNN (CatBoost, Strict Parity)	MapLight + GNN		␣
↳	deep_backend	test_rmse			
	analysis_metric_kind	analysis_metric_direction	analysis_metric_value		␣
↳	comparison_score	previous_stage_score	previous_stage_metric_value		␣
↳	marginal_score_improvement	\			
0		rmse	lower	0.103895	␣
↳	0.103895	NaN		NaN	␣
↳	NaN				
1		rmse	lower	0.103895	␣
↳	0.103895	0.103895		0.103895	␣
↳	0.000000				
2		rmse	lower	0.079691	␣
↳	0.079691	0.103895		0.103895	␣
↳	0.024204				
3		rmse	lower	0.079691	␣
↳	0.079691	0.079691		0.079691	␣
↳	0.000000				
4		rmse	lower	0.079691	␣
↳	0.079691	0.079691		0.079691	␣
↳	0.000000				

..	...	...	...	...	...
↪ ...	...	...	...	...	...
215	rmse	lower	4.897795		
↪ 4.897795	NaN		NaN		
↪ NaN					
216	rmse	lower	4.760954		
↪ 4.760954	4.897795		4.897795		
↪ 0.136841					
217	rmse	lower	4.684131		
↪ 4.684131	4.760954		4.760954		
↪ 0.076822					
218	rmse	lower	4.684131		
↪ 4.684131	4.684131		4.684131		
↪ 0.000000					
219	rmse	lower	4.684131		
↪ 4.684131	4.684131		4.684131		
↪ 0.000000					
marginal_relative_metric_improvement improved_vs_previous_stage					
↪ stage_changed_winning_model					
0	NaN	False			
↪ False					
1	0.000000	False			
↪ False					
2	0.232963	True			
↪ True					
3	0.000000	False			
↪ False					
4	0.000000	False			
↪ False					
..	...	...	...	...	...
↪ ...	...	...	...	...	...
215	NaN	False			
↪ False					
216	0.027939	True			
↪ True					
217	0.016136	True			
↪ True					
218	0.000000	False			
↪ False					
219	0.000000	False			
↪ False					

[220 rows x 18 columns]

```

[69]: # NO_RERUN_PUBLICATION_DATASET_DIFFICULTY
# Publication analysis 3: dataset difficulty stratification from existing
# artifacts.

difficulty_source = successful_primary_metric_rows()

if difficulty_source.empty:
    display_note("No successful metric rows were available for dataset
    difficulty stratification.")
else:
    best_difficulty_rows = (
        difficulty_source.sort_values(["dataset", "comparison_score", "model"],
        ascending=[True, True, True])
        .groupby("dataset", as_index=False, dropna=False)
        .first()
    )

    descriptor_cols = [
        col for col in [
            "dataset", "benchmark_suite", "benchmark_id", "n_molecules",
            "n_train", "n_test", "split_strategy",
            "target_transform", "selected_feature_count",
            "original_feature_count", "post_dedup_feature_count",
            "leaderboard_estimated_rank_vs_top10",
            "leaderboard_gap_to_top10_cutoff", "leaderboard_delta_top10_best",
            "leaderboard_delta_primary", "leaderboard_metric_normalized",
        ]
        if col in difficulty_source.columns
    ]
    dataset_descriptors = (
        difficulty_source.sort_values(["dataset", "comparison_score"])
        .groupby("dataset", as_index=False, dropna=False)
        .first()[descriptor_cols]
    )

    winner_cols = [
        "dataset", "task_kind", "model", "workflow", "architecture_family",
        "analysis_metric", "analysis_metric_kind",
        "analysis_metric_direction", "analysis_metric_value", "comparison_score"
    ]
    winner_cols = [col for col in winner_cols if col in best_difficulty_rows.
    columns]
    dataset_difficulty = dataset_descriptors.merge(
        best_difficulty_rows[winner_cols].rename(columns={
            "model": "winning_model",
            "workflow": "winning_workflow",

```

```

        "architecture_family": "winning_family",
        "analysis_metric_value": "best_primary_metric_value",
        "comparison_score": "best_comparison_score",
    }),
    on="dataset",
    how="left",
)

if "leaderboard_eval" in globals() and leaderboard_eval is not None and not_
↳leaderboard_eval.empty:
    rank_cols = [col for col in ["dataset", "estimated_rank_vs_top10",
↳"in_top10_estimate", "gap_to_top1", "gap_to_top10_cutoff"] if col in_
↳leaderboard_eval.columns]
    if rank_cols:
        rank_best = leaderboard_eval[rank_cols].copy()
        for col in [c for c in rank_cols if c not in {"dataset",
↳"in_top10_estimate"}]:
            rank_best[col] = pd.to_numeric(rank_best[col], errors="coerce")
            sort_col = "estimated_rank_vs_top10" if "estimated_rank_vs_top10"
↳in rank_best.columns else rank_cols[-1]
            rank_best = rank_best.sort_values(["dataset", sort_col]).
↳groupby("dataset", as_index=False).first()
            dataset_difficulty = dataset_difficulty.merge(rank_best,
↳on="dataset", how="left", suffixes=("", "_reference"))

prediction_descriptor_rows = []
scaffold_note = "RDKit unavailable; scaffold diversity skipped."
try:
    from rdkit import Chem
    from rdkit import RDLogger
    from rdkit.Chem.Scaffolds import MurckoScaffold
    RDLogger.DisableLog("rdApp.warning")
    rdkit_available = True
    scaffold_note = "Bemis-Murcko scaffold diversity computed from_
↳de-duplicated prediction SMILES."
except Exception as exc:
    Chem = None
    MurckoScaffold = None
    rdkit_available = False
    scaffold_note = f"RDKit unavailable; scaffold diversity skipped ({exc}).
↳"

publication_predictions = load_publication_predictions()
if publication_predictions is not None and not publication_predictions.
↳empty:
    pred = publication_predictions.copy()

```

```

dataset_col = pick_column(pred.columns, ["dataset", "dataset_id",
↪ "dataset_name"])
observed_col = pick_column(pred.columns, ["observed", "y_true",
↪ "actual"])
split_col = pick_column(pred.columns, ["split", "Split"])
smiles_col = pick_column(pred.columns, ["smiles", "SMILES",
↪ "canonical_smiles"])
model_col = pick_column(pred.columns, ["model", "Model"])
if dataset_col is not None and observed_col is not None:
    pred = pred.rename(columns={dataset_col: "dataset", observed_col:
↪ "observed"})
    pred["split"] = pred[split_col] if split_col is not None else "all"
    pred["smiles"] = pred[smiles_col] if smiles_col is not None else ""
    pred["model"] = pred[model_col] if model_col is not None else ""
    pred["observed"] = pd.to_numeric(pred["observed"], errors="coerce")
    dedupe_cols = ["dataset", "split", "smiles", "observed"] if
↪ smiles_col is not None else ["dataset", "split", "observed"]
    pred_unique = pred.loc[pred["observed"].notna()].
↪ drop_duplicates(dedupe_cols).copy()
    for dataset_name, group in pred_unique.groupby("dataset",
↪ sort=True):
        values = group["observed"].dropna()
        unique_values = sorted(values.unique().tolist())
        is_binary = len(unique_values) <= 2 and set(unique_values).
↪ issubset({0, 1, 0.0, 1.0})
        payload = {
            "dataset": dataset_name,
            "prediction_unique_rows": int(len(group)),
            "observed_unique_values": int(len(unique_values)),
            "class_positive_fraction": float(values.mean()) if
↪ is_binary and len(values) else np.nan,
            "class_minority_fraction": float(min(values.mean(), 1.0 -
↪ values.mean())) if is_binary and len(values) else np.nan,
        }
        if rdkit_available and smiles_col is not None:
            smiles_values = group["smiles"].dropna().astype(str).str.
↪ strip()
            smiles_values = smiles_values[smiles_values.ne("")]
            ↪ drop_duplicates()
            scaffolds = []
            for smiles in smiles_values:
                mol = Chem.MolFromSmiles(smiles)
                if mol is None:
                    continue
                scaffold = MurckoScaffold.MurckoScaffoldSmiles(mol=mol,
↪ includeChirality=False)

```

```

        scaffolds.append(scaffold or smiles)
        payload["scaffold_count"] = int(len(set(scaffolds))) if
↪scaffolds else np.nan
        payload["scaffold_diversity"] = float(len(set(scaffolds)) /
↪len(smiles_values)) if len(smiles_values) else np.nan
        prediction_descriptor_rows.append(payload)

    if prediction_descriptor_rows:
        dataset_difficulty = dataset_difficulty.merge(pd.
↪DataFrame(prediction_descriptor_rows), on="dataset", how="left")

    for col in ["n_molecules", "n_train", "n_test", "selected_feature_count",
↪"original_feature_count", "post_dedup_feature_count"]:
        if col in dataset_difficulty.columns:
            dataset_difficulty[col] = pd.to_numeric(dataset_difficulty[col],
↪errors="coerce")
        if "n_molecules" not in dataset_difficulty.columns or
↪dataset_difficulty["n_molecules"].isna().all():
            if {"n_train", "n_test"}.issubset(dataset_difficulty.columns):
                dataset_difficulty["n_molecules"] = pd.
↪to_numeric(dataset_difficulty["n_train"], errors="coerce") + pd.
↪to_numeric(dataset_difficulty["n_test"], errors="coerce")
            if "n_molecules" in dataset_difficulty.columns:
                try:
                    dataset_difficulty["dataset_size_quartile"] = pd.
↪qcut(dataset_difficulty["n_molecules"], q=4, duplicates="drop")
                except Exception:
                    dataset_difficulty["dataset_size_quartile"] = pd.NA

    numeric_candidates = [
        "n_molecules", "n_train", "n_test", "selected_feature_count",
↪"original_feature_count", "post_dedup_feature_count",
        "class_positive_fraction", "class_minority_fraction", "scaffold_count",
↪"scaffold_diversity",
    ]
    outcome_candidates = [
        "best_primary_metric_value", "estimated_rank_vs_top10",
↪"gap_to_top10_cutoff", "leaderboard_gap_to_top10_cutoff",
        "leaderboard_delta_primary", "leaderboard_delta_top10_best",
    ]
    corr_rows = []
    for x_col in [col for col in numeric_candidates if col in
↪dataset_difficulty.columns]:
        for y_col in [col for col in outcome_candidates if col in
↪dataset_difficulty.columns]:

```

```

        local = dataset_difficulty[[x_col, y_col]].apply(pd.to_numeric,
↳errors="coerce").dropna()
        if len(local) >= 4 and local[x_col].nunique() > 1 and local[y_col].
↳nunique() > 1:
            corr_rows.append({
                "descriptor": x_col,
                "outcome": y_col,
                "datasets": int(len(local)),
                "spearman_correlation": float(local[x_col].
↳corr(local[y_col], method="spearman")),
                "pearson_correlation": float(local[x_col].
↳corr(local[y_col], method="pearson")),
            })
        dataset_difficulty_correlations = pd.DataFrame(corr_rows).
↳sort_values(["outcome", "spearman_correlation"], ascending=[True, False]) if
↳corr_rows else pd.DataFrame()

        family_profile_cols = [col for col in ["winning_family", "task_kind",
↳"dataset_size_quartile"] if col in dataset_difficulty.columns]
        if family_profile_cols:
            family_difficulty_profile = (
                dataset_difficulty.groupby(family_profile_cols, dropna=False,
↳observed=False)
                .agg(
                    datasets=("dataset", "nunique"),
                    median_n_molecules=("n_molecules", "median") if "n_molecules"
↳in dataset_difficulty.columns else ("dataset", "size"),
                    median_best_metric=("best_primary_metric_value", "median"),
                    median_estimated_rank=("estimated_rank_vs_top10", "median") if
↳"estimated_rank_vs_top10" in dataset_difficulty.columns else ("dataset",
↳"size"),
                )
                .reset_index()
                .sort_values(["task_kind", "dataset_size_quartile", "datasets"],
↳ascending=[True, True, False])
            )
        else:
            family_difficulty_profile = pd.DataFrame()

        display_note("### Publication Analysis: Dataset Difficulty Stratification")
        display(dataset_difficulty.sort_values(["task_kind", "n_molecules",
↳"dataset"], ascending=[True, True, True]).reset_index(drop=True))
        if not dataset_difficulty_correlations.empty:
            display_note("Descriptor/outcome correlations")
            display(dataset_difficulty_correlations.reset_index(drop=True))
        if not family_difficulty_profile.empty:

```

```

display_note("Winning-family profile by dataset size and task type")
display(family_difficulty_profile.reset_index(drop=True))
display_note(scaffold_note)

write_publication_csv(dataset_difficulty,
↳"publication_dataset_difficulty_stratification.csv")
write_publication_csv(dataset_difficulty_correlations,
↳"publication_dataset_difficulty_correlations.csv")
write_publication_csv(family_difficulty_profile,
↳"publication_winning_family_difficulty_profile.csv")

if "n_molecules" in dataset_difficulty.columns:
    y_col = "estimated_rank_vs_top10" if "estimated_rank_vs_top10" in
↳dataset_difficulty.columns and dataset_difficulty["estimated_rank_vs_top10"].
↳notna().any() else "best_primary_metric_value"
    fig = px.scatter(
        dataset_difficulty,
        x="n_molecules",
        y=y_col,
        color="winning_family" if "winning_family" in dataset_difficulty.
↳columns else "task_kind",
        symbol="task_kind" if "task_kind" in dataset_difficulty.columns
↳else None,
        hover_data=[col for col in ["dataset", "winning_model",
↳"analysis_metric_kind", "class_minority_fraction", "scaffold_diversity"] if
↳col in dataset_difficulty.columns],
        title=f"Dataset size vs {y_col}, colored by winning family",
    )
    fig.update_xaxes(type="log", title="Dataset size (n molecules, log
↳scale)")
    fig.show()

```

1.0.30 Publication Analysis: Dataset Difficulty Stratification

	dataset	benchmark_suite	n_molecules	n_train	n_test	split_strategy	target_transform
0	tdc_carcinogens_lagunin	tdc					
↳	carcinogens_lagunin		280	224	56	random	
↳	raw		83				
1	tdc_skin_reaction	tdc					
↳	skin_reaction		404	323	81	random	raw
↳			22				
2	tdc_dili	tdc					
↳	dili		475	379	96	predefined	raw
↳			270				

3		tdc_hia_hou		tdc			
	↳ hia_hou	578	461 117	predefined		raw	↳
	↳	86					
4		tdc_bioavailability_ma		tdc			↳
	↳ bioavailability_ma		640 512	128 predefined			↳
	↳ raw		12				
5		tdc_herg		tdc			↳
	↳ herg	655	523 132	predefined		raw	↳
	↳	131					
6		tdc_cyp2d6_substrate_carbonmangels		tdc	↳		
	↳ cyp2d6_substrate_carbonmangels		667	532 135		predefined	↳
	↳ raw		142				
7		tdc_cyp2c9_substrate_carbonmangels		tdc	↳		
	↳ cyp2c9_substrate_carbonmangels		669	534 135		predefined	↳
	↳ raw		8				
8		tdc_cyp3a4_substrate_carbonmangels		tdc	↳		
	↳ cyp3a4_substrate_carbonmangels		670	535 135		predefined	↳
	↳ raw		176				
9		tdc_pgp_broccatelli		tdc			↳
	↳ pgp_broccatelli	1218	973 245	predefined			↳
	↳ raw	274					
10		tdc_clintox		tdc			↳
	↳ clintox	1478	1182 296	random		raw	↳
	↳	126					
11		tdc_toxcast		tdc			↳
	↳ toxcast	1731	1384 347	random		raw	↳
	↳	68					
12		tdc_bbb_martins		tdc			↳
	↳ bbb_martins	2030	1624 406	predefined		raw	↳
	↳	275					
13		tdc_pampa_ncats		tdc			↳
	↳ pampa_ncats	2034	1627 407	random		raw	↳
	↳	236					
14		tdc_tox21		tdc			↳
	↳ tox21	7258	5806 1452	random		raw	↳
	↳	512					
15		tdc_ames		tdc			↳
	↳ ames	7278	5821 1457	predefined		raw	↳
	↳	512					
16		tdc_cyp2c9_veith		tdc			↳
	↳ cyp2c9_veith	12092	9673 2419	predefined		raw	↳
	↳	512					
17		tdc_cyp3a4_veith		tdc			↳
	↳ cyp3a4_veith	12328	9861 2467	predefined		raw	↳
	↳	512					

18	tdc_cyp1a2_veith				tdc			
↳ cyp1a2_veith	12579	10063	2516		random		raw	↳
↳	512							
19	tdc_cyp2c19_veith				tdc			
↳ cyp2c19_veith	12665	10132	2533		random		raw	↳
↳	512							
20	tdc_cyp2d6_veith				tdc			
↳ cyp2d6_veith	13130	10504	2626		predefined		raw	↳
↳	512							
21	tdc_herg_karim				tdc			
↳ herg_karim	13445	10756	2689		random		raw	↳
↳	512							
22	chemml_cep_homo				None			
↳ None	500	400	100	target_quartiles			raw	↳
↳	339							
23	chemml_organic_density				None			
↳ None	500	400	100	target_quartiles			log10	↳
↳	241							
24	freesolv_sampl			moleculenet				
↳ freesolv_sampl	642	513	129		random			↳
↳ raw	451							
25	tdc_hydrationfreeenergy_freesolv				tdc	↳		
↳ hydrationfreeenergy_freesolv			642	513	129	target_quartiles		↳
↳	raw		423					
26	tdc_half_life_obach				tdc			
↳ half_life_obach	667	532	135		predefined			↳
↳ raw	512							
27	polaris_adme_fang_rppb_1			polaris		polaris/		
↳ adme-fang-rppb-1	885	708	177		predefined			↳
↳ raw	221							
28	tdc_caco2_wang				tdc			
↳ caco2_wang	910	728	182		predefined		raw	↳
↳	355							
29	tdc_clearance_microsome_az				tdc	↳		
↳ clearance_microsome_az		1102	881	221		predefined		↳
↳ raw	512							
30	esol_delaney			moleculenet				
↳ esol_delaney	1128	874	254		scaffold		raw	↳
↳	512							
31	tdc_vdss_lombardo				tdc			
↳ vdss_lombardo	1130	904	226		predefined		raw	↳
↳	512							
32	tdc_clearance_hepatocyte_az				tdc	↳		
↳ clearance_hepatocyte_az		1213	970	243		predefined		↳
↳ raw	512							

33	polaris_adme_fang_hppb_1		polaris	polaris/	
	↳adme-fang-hppb-1	1808 1446	362	predefined	␣
	↳raw	512			
34	poduam_pod_nc_std		literature		␣
	↳poduam_pod_nc_std	1842 1473	369	target_quartiles	␣
	↳raw	340			
35	polaris_adme_fang_solu_1		polaris	polaris/	
	↳adme-fang-solu-1	2173 1738	435	predefined	␣
	↳raw	224			
36	poduam_pod_rd_std		literature		␣
	↳poduam_pod_rd_std	2355 1884	471	target_quartiles	␣
	↳raw	512			
37	polaris_adme_fang_perm_1		polaris	polaris/	
	↳adme-fang-perm-1	2642 2113	529	predefined	␣
	↳raw	512			
38	tdc_ppbr_az		tdc		␣
	↳ppbr_az	2790 2231 559	predefined	raw	␣
	↳	512			
39	polaris_adme_fang_rclint_1		polaris	polaris/	
	↳adme-fang-rclint-1	3054 2443	611	predefined	␣
	↳ raw	512			
40	lipophilicity		moleculenet		␣
	↳lipophilicity	4200 3357	843	scaffold	raw␣
	↳	512			
41	tdc_lipophilicity_astrazeneca		tdc		␣
	↳lipophilicity_astrazeneca	4200	3360 840	predefined	␣
	↳ raw	512			
42	tdc_ld50_zhu		tdc		␣
	↳ld50_zhu	7385 5907 1478	predefined	raw	␣
	↳	512			
43	tdc_solubility_aqsolddb		tdc		␣
	↳solubility_aqsolddb	9980 7985	1995	predefined	␣
	↳ raw	512			
	original_feature_count	post_dedup_feature_count			␣
	↳leaderboard_estimated_rank_vs_top10	leaderboard_gap_to_top10_cutoff			␣
	↳leaderboard_delta_top10_best	leaderboard_delta_primary \			
0		10115	8428		␣
	↳ NaN		NaN	NaN	␣
	↳	NaN			
1		10115	7657		␣
	↳ NaN		NaN	NaN	␣
	↳	NaN			
2		10115	8800		␣
	↳ NaN		NaN	NaN	␣
	↳	NaN			

3		10115		8681		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
4		10115		8756		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
5		10115		8665		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
6		10115		8743		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
7		10115		8739		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
8		10115		8755		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
9		10115		8703		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
10		10115		9258		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
11		10115		8919		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
12		10115		8996		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
13		10115		8658		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
14		10115		9335		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
15		10115		9009		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
16		10115		9241		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
17		10115		9273		␣
↪	NaN		NaN		NaN	␣
↪		NaN				

18		10115		9275		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
19		10115		9276		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
20		10115		9254		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
21		10115		9178		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
22		10115		6550		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
23		10115		7231		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
24		10115		5880		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
25		10115		6013		␣
↪	NaN		NaN		NaN	␣
↪		NaN				
26		10115		8983		␣
↪	10.0		-0.019247		-0.184753	␣
↪		-0.184753				
27		10115		8687		␣
↪	3.0		-0.378624		0.037537	␣
↪		0.032537				
28		10115		8786		␣
↪	2.0		-0.053564		0.016436	␣
↪		0.016436				
29		10115		8550		␣
↪	11.0		0.000574		-0.058035	␣
↪		-0.058035				
30		10115		7936		␣
↪	1.0		-1.119161		-0.263661	␣
↪		-0.263661				
31		10115		9144		␣
↪	11.0		0.009788		-0.225788	␣
↪		-0.225788				
32		10115		8694		␣
↪	11.0		0.051021		-0.230324	␣
↪		-0.230324				

33		10115		8803		┐
↳	4.0		-0.165037		0.074963	┐
↳		0.074963				
34		10115		9028		┐
↳	1.0		-0.030150		-0.030150	┐
↳		-0.030150				
35		10115		8565		┐
↳	11.0		0.014380		0.121380	┐
↳		0.121380				
36		10115		9060		┐
↳	1.0		-0.075397		-0.075397	┐
↳		-0.075397				
37		10115		8685		┐
↳	6.0		-0.071842		0.072158	┐
↳		0.072158				
38		10115		8794		┐
↳	1.0		-1.350984		-0.110984	┐
↳		-0.110984				
39		10115		8700		┐
↳	4.0		-0.117637		0.069363	┐
↳		0.069363				
40		10115		8933		┐
↳	1.0		-0.367312		-0.176416	┐
↳		-0.176416				
41		10115		8897		┐
↳	5.0		-0.046900		0.036100	┐
↳		0.036100				
42		10115		9085		┐
↳	3.0		-0.049761		0.028239	┐
↳		0.028239				
43		10115		9180		┐
↳	1.0		-0.101256		-0.013256	┐
↳		-0.013256				

	leaderboard_metric_normalized	task_kind		winning_family	
↳	winning_model	winning_workflow			
↳	analysis_metric	analysis_metric_kind \			
0		None classification			┐
↳	AdaBoost	conventional		Conventional ML	┐
↳	test_roc_auc	roc_auc			
1		None classification	Ensemble (Weighted average		┐
↳	(inverse train RMSE))	ensemble		Ensemble meta-model	┐
↳	test_roc_auc	roc_auc			
2		None classification	Ensemble (OOF Stacking		┐
↳	(RidgeCV, 5-fold))	ensemble		Ensemble meta-model	┐
↳	test_roc_auc	roc_auc			

3		None	classification	CFA	
	↪(Combinatorial Fusion)		CFA fusion	CFA combinatorial fusion	
	↪ test_roc_auc		roc_auc		
4		None	classification		
	↪ CatBoost		conventional	Conventional ML	
	↪test_roc_auc		roc_auc		
5		None	classification		
	↪ AdaBoost		conventional	Conventional ML	
	↪test_roc_auc		roc_auc		
6		None	classification	Chemprop v2	
	↪(AttentiveFP, ensemble=1)		Chemprop v2	Graph neural networks	
	↪(Chemprop) test_auprc		auprc		
7		None	classification	Ensemble (OOF Stacking	
	↪(RidgeCV, 5-fold))		ensemble	Ensemble meta-model	
	↪ test_auprc		auprc		
8		None	classification		
	↪LogisticRegression		conventional	Conventional ML	
	↪ test_auprc		auprc		
9		None	classification		
	↪HistGradientBoosting		conventional	Conventional ML	
	↪ test_roc_auc		roc_auc		
10		None	classification		
	↪ CatBoost		conventional	Conventional ML	
	↪test_roc_auc		roc_auc		
11		None	classification		
	↪ CatBoost		conventional	Conventional ML	
	↪test_roc_auc		roc_auc		
12		None	classification		
	↪ Random forest		conventional	Conventional ML	
	↪test_roc_auc		roc_auc		
13		None	classification	Ensemble (Weighted average	
	↪(inverse train RMSE))		ensemble	Ensemble meta-model	
	↪ test_roc_auc		roc_auc		
14		None	classification		
	↪ Tabular MLP		conventional	Conventional ML	
	↪test_roc_auc		roc_auc		
15		None	classification	Ensemble (OOF Stacking	
	↪(RidgeCV, 5-fold))		ensemble	Ensemble meta-model	
	↪test_roc_auc		roc_auc		
16		None	classification	Ensemble (OOF Stacking	
	↪(RidgeCV, 5-fold))		ensemble	Ensemble meta-model	
	↪ test_auprc		auprc		
17		None	classification	Ensemble (OOF Stacking	
	↪(RidgeCV, 5-fold))		ensemble	Ensemble meta-model	
	↪ test_auprc		auprc		

18		None	classification	Ensemble (OOF Stacking	
	↪(RidgeCV, 5-fold))		ensemble	Ensemble meta-model	␣
	↪test_roc_auc		roc_auc		
19		None	classification	Ensemble (OOF Stacking	
	↪(RidgeCV, 5-fold))		ensemble	Ensemble meta-model	␣
	↪test_roc_auc		roc_auc		
20		None	classification	Ensemble (Weighted average	
	↪(inverse train RMSE))		ensemble	Ensemble meta-model	␣
	↪ test_aucprc		auprc		
21		None	classification	Ensemble (Weighted average	
	↪(inverse train RMSE))		ensemble	Ensemble meta-model	␣
	↪ test_roc_auc		roc_auc		
22		None	regression		␣
	↪ TabPFNRegressor	TabPFN	deep learning	TabPFN foundation model	␣
	↪ test_rmse		rmse		
23		None	regression		␣
	↪ TabPFNRegressor	TabPFN	deep learning	TabPFN foundation model	␣
	↪ test_rmse		rmse		
24		None	regression		␣
	↪ TabPFNRegressor	TabPFN	deep learning	TabPFN foundation model	␣
	↪ test_rmse		rmse		
25		None	regression	CFA	␣
	↪(Combinatorial Fusion)		CFA fusion	CFA combinatorial fusion	␣
	↪ test_rmse		rmse		
26		spearman	regression		␣
	↪ TabPFNRegressor	TabPFN	deep learning	TabPFN foundation model	␣
	↪ test_rmse		rmse		
27		mse	regression	MapLight + GNN	␣
	↪(CatBoost, Strict Parity)		MapLight + GNN	Graph transfer + tabular	␣
	↪head test_rmse		rmse		
28		mae	regression	MapLight	␣
	↪CatBoost (Strict Parity)		conventional	Conventional	␣
	↪ML test_rmse		rmse		
29		spearman	regression	MapLight + GNN	␣
	↪(CatBoost, Strict Parity)		MapLight + GNN	Graph transfer + tabular	␣
	↪head test_rmse		rmse		
30		rmse	regression		␣
	↪ TabPFNRegressor	TabPFN	deep learning	TabPFN foundation model	␣
	↪ test_rmse		rmse		
31		spearman	regression	MapLight + GNN	␣
	↪(CatBoost, Strict Parity)		MapLight + GNN	Graph transfer + tabular	␣
	↪head test_rmse		rmse		
32		spearman	regression	MapLight + GNN	␣
	↪(CatBoost, Strict Parity)		MapLight + GNN	Graph transfer + tabular	␣
	↪head test_rmse		rmse		

```

33      mse      regression      MapLight + GNN
↳(CatBoost, Strict Parity)      MapLight + GNN      Graph transfer + tabular
↳head      test_rmse      rmse
34      rmse      regression      CFA
↳(Combinatorial Fusion)      CFA fusion      CFA combinatorial fusion
↳      test_rmse      rmse
35      mse      regression
↳      XGBoost      conventional      Conventional ML
↳ test_rmse      rmse
36      rmse      regression
↳      XGBoost      conventional      Conventional ML
↳ test_rmse      rmse
37      mse      regression
↳      XGBoost      conventional      Conventional ML
↳ test_rmse      rmse
38      mae      regression      MapLight + GNN
↳(CatBoost, Strict Parity)      MapLight + GNN      Graph transfer + tabular
↳head      test_rmse      rmse
39      mse      regression
↳      CatBoost      conventional      Conventional ML
↳ test_rmse      rmse
40      rmse      regression      Ensemble (Weighted average
↳(inverse train RMSE))      ensemble      Ensemble meta-model
↳      test_rmse      rmse
41      mae      regression
↳      XGBoost      conventional      Conventional ML
↳ test_rmse      rmse
42      mae      regression      CFA
↳(Combinatorial Fusion)      CFA fusion      CFA combinatorial fusion
↳      test_rmse      rmse
43      mae      regression      CFA
↳(Combinatorial Fusion)      CFA fusion      CFA combinatorial fusion
↳      test_rmse      rmse

analysis_metric_direction  best_primary_metric_value  best_comparison_score
↳estimated_rank_vs_top10 in_top10_estimate  gap_to_top1  gap_to_top10_cutoff
↳prediction_unique_rows \
0      higher      0.862626      -0.862626
↳      1.0      True      -0.014626      -0.132626
↳      280
1      higher      0.774839      -0.774839
↳      1.0      True      -0.033839      -0.275839
↳      404
2      higher      0.913043      -0.913043
↳      5.0      True      0.031957      -0.061043
↳      475

```


3	higher		0.989918	-0.989918	␣
↩	4.0	True	0.004082	-0.013918	␣
↩	578				
4	higher		0.776688	-0.776688	␣
↩	1.0	True	-0.028688	-0.136688	␣
↩	640				
5	higher		0.860825	-0.860825	␣
↩	3.0	True	0.019175	-0.054825	␣
↩	651				
6	higher		0.683938	-0.683938	␣
↩	5.0	True	0.082062	-0.113938	␣
↩	665				
7	higher		0.427480	-0.427480	␣
↩	3.0	True	0.022520	-0.067480	␣
↩	666				
8	higher		0.716619	-0.716619	␣
↩	7.0	True	0.033207	-0.007793	␣
↩	667				
9	higher		0.924020	-0.924020	␣
↩	5.0	True	0.069980	-0.013020	␣
↩	1212				
10	higher		0.948884	-0.948884	␣
↩	2.0	True	0.047116	-0.216884	␣
↩	1478				
11	higher		0.791353	-0.791353	␣
↩	1.0	True	-0.077353	-0.077353	␣
↩	1731				
12	higher		0.932419	-0.932419	␣
↩	2.0	True	0.008581	-0.029419	␣
↩	1985				
13	higher		0.811231	-0.811231	␣
↩	NaN	NaN	NaN	NaN	␣
↩	2034				
14	higher		0.808239	-0.808239	␣
↩	7.0	True	0.058761	0.024761	␣
↩	7258				
15	higher		0.876017	-0.876017	␣
↩	2.0	True	0.035983	-0.042017	␣
↩	7255				
16	higher		0.810239	-0.810239	␣
↩	3.0	True	0.066761	-0.040239	␣
↩	12092				
17	higher		0.889038	-0.889038	␣
↩	2.0	True	0.033962	-0.139038	␣
↩	12328				

18	higher		0.951752	-0.951752	␣
↪	NaN	NaN	NaN	NaN	␣
↪	12579				
19	higher		0.904288	-0.904288	␣
↪	NaN	NaN	NaN	NaN	␣
↪	12665				
20	higher		0.727009	-0.727009	␣
↪	2.0	True	0.083991	-0.263009	␣
↪	13130				
21	higher		0.944478	-0.944478	␣
↪	NaN	NaN	NaN	NaN	␣
↪	13445				
22	lower		0.079691	0.079691	␣
↪	NaN	NaN	NaN	NaN	␣
↪	500				
23	lower		0.005035	0.005035	␣
↪	NaN	NaN	NaN	NaN	␣
↪	500				
24	lower		0.982177	0.982177	␣
↪	NaN	NaN	NaN	NaN	␣
↪	642				
25	lower		0.644869	0.644869	␣
↪	1.0	True	-0.009131	-0.616131	␣
↪	642				
26	lower		17.210966	17.210966	␣
↪	11.0	False	0.257753	0.093753	␣
↪	667				
27	lower		0.493558	0.493558	␣
↪	3.0	True	0.013599	-0.390401	␣
↪	885				
28	lower		0.349680	0.349680	␣
↪	3.0	True	0.016436	-0.015564	␣
↪	910				
29	lower		36.331993	36.331993	␣
↪	10.0	True	0.046413	-0.006587	␣
↪	1102				
30	lower		0.621439	0.621439	␣
↪	7.0	True	0.063439	-0.121561	␣
↪	1123				
31	lower		4.684131	4.684131	␣
↪	5.0	True	0.270673	-0.089327	␣
↪	1128				
32	lower		43.782188	43.782188	␣
↪	3.0	True	0.109913	-0.083087	␣
↪	1213				

33	lower		0.446652	0.446652	␣
↩	3.0	True	0.056498	-0.183502	␣
↩	1808				
34	lower		0.699850	0.699850	␣
↩	2.0	True	0.149850	-0.030150	␣
↩	1842				
35	lower		0.585986	0.585986	␣
↩	11.0	False	0.121380	0.014380	␣
↩	2173				
36	lower		0.554603	0.554603	␣
↩	2.0	True	0.144603	-0.075397	␣
↩	2355				
37	lower		0.430300	0.430300	␣
↩	6.0	True	0.072158	-0.071842	␣
↩	2642				
38	lower		11.378376	11.378376	␣
↩	3.0	True	6.650016	-0.584984	␣
↩	2790				
39	lower		0.534193	0.534193	␣
↩	4.0	True	0.069363	-0.117637	␣
↩	3054				
40	lower		0.594788	0.594788	␣
↩	6.0	True	0.045788	-0.037212	␣
↩	4200				
41	lower		0.644116	0.644116	␣
↩	9.0	True	0.086100	-0.022900	␣
↩	4200				
42	lower		0.829832	0.829832	␣
↩	5.0	True	0.288239	-0.024761	␣
↩	7376				
43	lower		1.001087	1.001087	␣
↩	3.0	True	0.170744	-0.048256	␣
↩	9980				

	observed_unique_values	class_positive_fraction	class_minority_fraction	␣
↩	scaffold_count	scaffold_diversity	dataset_size_quartile	
0	2	0.214286	0.214286	␣
↩	202	0.726619	(279.999, 667.0]	
1	2	0.678218	0.321782	␣
↩	248	0.613861	(279.999, 667.0]	
2	2	0.496842	0.496842	␣
↩	345	0.726316	(279.999, 667.0]	
3	2	0.865052	0.134948	␣
↩	403	0.697232	(279.999, 667.0]	
4	2	0.768750	0.231250	␣
↩	456	0.712500	(279.999, 667.0]	

5		2	0.686636	0.313364	␣
↪	410	0.632716	(279.999, 667.0]		
6		2	0.287218	0.287218	␣
↪	470	0.707831	(279.999, 667.0]		
7		2	0.211712	0.211712	␣
↪	471	0.707207	(667.0, 1604.5]		
8		2	0.530735	0.469265	␣
↪	473	0.709145	(667.0, 1604.5]		
9		2	0.533828	0.466172	␣
↪	700	0.577558	(667.0, 1604.5]		
10		2	0.075778	0.075778	␣
↪	910	0.623715	(667.0, 1604.5]		
11		2	0.255921	0.255921	␣
↪	1003	0.579434	(1604.5, 4200.0]		
12		2	0.760705	0.239295	␣
↪	1119	0.566582	(1604.5, 4200.0]		
13		2	0.854966	0.145034	␣
↪	1388	0.682399	(1604.5, 4200.0]		
14		2	0.042436	0.042436	␣
↪	3855	0.531138	(4200.0, 13445.0]		
15		2	0.544590	0.455410	␣
↪	2836	0.390903	(4200.0, 13445.0]		
16		2	0.334519	0.334519	␣
↪	7417	0.613381	(4200.0, 13445.0]		
17		2	0.414504	0.414504	␣
↪	7704	0.624919	(4200.0, 13445.0]		
18		2	0.463391	0.463391	␣
↪	7778	0.618332	(4200.0, 13445.0]		
19		2	0.459455	0.459455	␣
↪	7771	0.613581	(4200.0, 13445.0]		
20		2	0.191470	0.191470	␣
↪	8050	0.613100	(4200.0, 13445.0]		
21		2	0.499665	0.499665	␣
↪	6020	0.447750	(4200.0, 13445.0]		
22		214	NaN	NaN	␣
↪	500	1.000000	(279.999, 667.0]		
23		497	NaN	NaN	␣
↪	498	0.996000	(279.999, 667.0]		
24		486	NaN	NaN	␣
↪	382	0.595016	(279.999, 667.0]		
25		486	NaN	NaN	␣
↪	382	0.595016	(279.999, 667.0]		
26		193	NaN	NaN	␣
↪	459	0.690226	(279.999, 667.0]		
27		435	NaN	NaN	␣
↪	682	0.779429	(667.0, 1604.5]		

28		593	NaN	NaN	␣
↪	474	0.523179	(667.0, 1604.5]		
29		286	NaN	NaN	␣
↪	711	0.645191	(667.0, 1604.5]		
30		734	NaN	NaN	␣
↪	584	0.522829	(667.0, 1604.5]		
31		306	NaN	NaN	␣
↪	803	0.722772	(667.0, 1604.5]		
32		350	NaN	NaN	␣
↪	748	0.733333	(667.0, 1604.5]		
33		497	NaN	NaN	␣
↪	1207	0.672798	(1604.5, 4200.0]		
34		1834	NaN	NaN	␣
↪	1151	0.626906	(1604.5, 4200.0]		
35		1711	NaN	NaN	␣
↪	1888	0.869245	(1604.5, 4200.0]		
36		2347	NaN	NaN	␣
↪	1533	0.652896	(1604.5, 4200.0]		
37		2638	NaN	NaN	␣
↪	2326	0.880727	(1604.5, 4200.0]		
38		342	NaN	NaN	␣
↪	1168	0.649972	(1604.5, 4200.0]		
39		2694	NaN	NaN	␣
↪	2635	0.863085	(1604.5, 4200.0]		
40		553	NaN	NaN	␣
↪	2412	0.574286	(1604.5, 4200.0]		
41		553	NaN	NaN	␣
↪	2412	0.574286	(1604.5, 4200.0]		
42		3036	NaN	NaN	␣
↪	3606	0.491147	(4200.0, 13445.0]		
43		7871	NaN	NaN	␣
↪	4886	0.489579	(4200.0, 13445.0]		

Descriptor/outcome correlations

	descriptor		outcome	datasets	␣
	↪spearman_correlation	pearson_correlation			
0	post_dedup_feature_count	best_primary_metric_value	44		␣
↪	0.287104	0.040385			
1	class_positive_fraction	best_primary_metric_value	22		␣
↪	0.262564	0.306791			
2	selected_feature_count	best_primary_metric_value	44		␣
↪	0.194802	0.250419			
3	class_minority_fraction	best_primary_metric_value	22		␣
↪	0.189159	0.214335			
4	n_train	best_primary_metric_value	44		␣
↪	0.129753	-0.149356			

5	n_molecules	best_primary_metric_value	44	␣
↪	0.125247	-0.149422		
6	n_test	best_primary_metric_value	44	␣
↪	0.118020	-0.149683		
7	scaffold_count	best_primary_metric_value	44	␣
↪	0.040806	-0.155472		
8	scaffold_diversity	best_primary_metric_value	44	␣
↪	-0.233068	0.061572		
9	selected_feature_count	estimated_rank_vs_top10	37	␣
↪	0.364556	0.300972		
10	class_minority_fraction	estimated_rank_vs_top10	18	␣
↪	0.199094	0.151484		
11	scaffold_diversity	estimated_rank_vs_top10	37	␣
↪	0.122128	0.203070		
12	scaffold_count	estimated_rank_vs_top10	37	␣
↪	0.110474	-0.081042		
13	n_test	estimated_rank_vs_top10	37	␣
↪	0.099834	-0.105525		
14	n_train	estimated_rank_vs_top10	37	␣
↪	0.093653	-0.107086		
15	n_molecules	estimated_rank_vs_top10	37	␣
↪	0.092397	-0.106774		
16	post_dedup_feature_count	estimated_rank_vs_top10	37	␣
↪	-0.089383	0.104186		
17	class_positive_fraction	estimated_rank_vs_top10	18	␣
↪	-0.089540	-0.108122		
18	class_positive_fraction	gap_to_top10_cutoff	18	␣
↪	0.209494	0.181243		
19	class_minority_fraction	gap_to_top10_cutoff	18	␣
↪	0.176471	0.187941		
20	post_dedup_feature_count	gap_to_top10_cutoff	37	␣
↪	0.125415	0.547743		
21	n_train	gap_to_top10_cutoff	37	␣
↪	0.117361	0.055450		
22	scaffold_count	gap_to_top10_cutoff	37	␣
↪	0.112856	0.053997		
23	n_molecules	gap_to_top10_cutoff	37	␣
↪	0.111440	0.055475		
24	selected_feature_count	gap_to_top10_cutoff	37	␣
↪	0.110894	0.022300		
25	n_test	gap_to_top10_cutoff	37	␣
↪	0.102242	0.055571		
26	scaffold_diversity	gap_to_top10_cutoff	37	␣
↪	-0.196076	-0.064167		
27	scaffold_count	leaderboard_delta_primary	18	␣
↪	0.466701	0.401435		

28	n_train	leaderboard_delta_primary	18	␣
↪	0.312693	0.258462		
29	scaffold_diversity	leaderboard_delta_primary	18	␣
↪	0.295302	0.288994		
30	n_molecules	leaderboard_delta_primary	18	␣
↪	0.289107	0.257008		
31	n_test	leaderboard_delta_primary	18	␣
↪	0.273478	0.251146		
32	post_dedup_feature_count	leaderboard_delta_primary	18	␣
↪	-0.155831	0.118356		
33	selected_feature_count	leaderboard_delta_primary	18	␣
↪	-0.352822	-0.421422		
34	scaffold_count	leaderboard_delta_top10_best	18	␣
↪	0.445018	0.398802		
35	scaffold_diversity	leaderboard_delta_top10_best	18	␣
↪	0.314920	0.290713		
36	n_train	leaderboard_delta_top10_best	18	␣
↪	0.283798	0.256170		
37	n_molecules	leaderboard_delta_top10_best	18	␣
↪	0.261229	0.254715		
38	n_test	leaderboard_delta_top10_best	18	␣
↪	0.246646	0.248849		
39	post_dedup_feature_count	leaderboard_delta_top10_best	18	␣
↪	-0.168215	0.117156		
40	selected_feature_count	leaderboard_delta_top10_best	18	␣
↪	-0.382578	-0.426439		
41	scaffold_diversity	leaderboard_gap_to_top10_cutoff	18	␣
↪	0.234383	0.208528		
42	post_dedup_feature_count	leaderboard_gap_to_top10_cutoff	18	␣
↪	0.075335	0.476596		
43	selected_feature_count	leaderboard_gap_to_top10_cutoff	18	␣
↪	-0.103438	-0.111861		
44	scaffold_count	leaderboard_gap_to_top10_cutoff	18	␣
↪	-0.133196	0.178222		
45	n_train	leaderboard_gap_to_top10_cutoff	18	␣
↪	-0.174407	0.070615		
46	n_molecules	leaderboard_gap_to_top10_cutoff	18	␣
↪	-0.209603	0.068634		
47	n_test	leaderboard_gap_to_top10_cutoff	18	␣
↪	-0.279670	0.060679		

Winning-family profile by dataset size and task type

	winning_family	task_kind	dataset_size_quartile	␣
↪ datasets	median_n_molecules	median_best_metric	median_estimated_rank	
0	Conventional ML	classification	(279.999, 667.0]	␣
↪ 3	640.0	0.860825	1.0	

1		Ensemble meta-model	classification	(279.999, 667.0]	␣
↪ 2		439.5	0.843941	3.0	
2		CFA combinatorial fusion	classification	(279.999, 667.0]	␣
↪ 1		578.0	0.989918	4.0	
3		Graph neural networks (Chemprop)	classification	(279.999, 667.0]	␣
↪ 1		667.0	0.683938	5.0	
4		Graph transfer + tabular head	classification	(279.999, 667.0]	␣
↪ 0		NaN	NaN	NaN	
5		TabPFN foundation model	classification	(279.999, 667.0]	␣
↪ 0		NaN	NaN	NaN	
6		Conventional ML	classification	(667.0, 1604.5]	␣
↪ 3		1218.0	0.924020	5.0	
7		Ensemble meta-model	classification	(667.0, 1604.5]	␣
↪ 1		669.0	0.427480	3.0	
8		CFA combinatorial fusion	classification	(667.0, 1604.5]	␣
↪ 0		NaN	NaN	NaN	
9		Graph neural networks (Chemprop)	classification	(667.0, 1604.5]	␣
↪ 0		NaN	NaN	NaN	
10		Graph transfer + tabular head	classification	(667.0, 1604.5]	␣
↪ 0		NaN	NaN	NaN	
11		TabPFN foundation model	classification	(667.0, 1604.5]	␣
↪ 0		NaN	NaN	NaN	
12		Conventional ML	classification	(1604.5, 4200.0]	␣
↪ 2		1880.5	0.861886	1.5	
13		Ensemble meta-model	classification	(1604.5, 4200.0]	␣
↪ 1		2034.0	0.811231	NaN	
14		CFA combinatorial fusion	classification	(1604.5, 4200.0]	␣
↪ 0		NaN	NaN	NaN	
15		Graph neural networks (Chemprop)	classification	(1604.5, 4200.0]	␣
↪ 0		NaN	NaN	NaN	
16		Graph transfer + tabular head	classification	(1604.5, 4200.0]	␣
↪ 0		NaN	NaN	NaN	
17		TabPFN foundation model	classification	(1604.5, 4200.0]	␣
↪ 0		NaN	NaN	NaN	
18		Ensemble meta-model	classification	(4200.0, 13445.0]	␣
↪ 7		12579.0	0.889038	2.0	
19		Conventional ML	classification	(4200.0, 13445.0]	␣
↪ 1		7258.0	0.808239	7.0	
20		CFA combinatorial fusion	classification	(4200.0, 13445.0]	␣
↪ 0		NaN	NaN	NaN	
21		Graph neural networks (Chemprop)	classification	(4200.0, 13445.0]	␣
↪ 0		NaN	NaN	NaN	
22		Graph transfer + tabular head	classification	(4200.0, 13445.0]	␣
↪ 0		NaN	NaN	NaN	
23		TabPFN foundation model	classification	(4200.0, 13445.0]	␣
↪ 0		NaN	NaN	NaN	

24		TabPFN foundation model	regression	(279.999, 667.0]	␣
↪	4	571.0	0.530934	11.0	
25		CFA combinatorial fusion	regression	(279.999, 667.0]	␣
↪	1	642.0	0.644869	1.0	
26		Conventional ML	regression	(279.999, 667.0]	␣
↪	0	NaN	NaN	NaN	
27		Ensemble meta-model	regression	(279.999, 667.0]	␣
↪	0	NaN	NaN	NaN	
28		Graph neural networks (Chemprop)	regression	(279.999, 667.0]	␣
↪	0	NaN	NaN	NaN	
29		Graph transfer + tabular head	regression	(279.999, 667.0]	␣
↪	0	NaN	NaN	NaN	
30		Graph transfer + tabular head	regression	(667.0, 1604.5]	␣
↪	4	1116.0	20.508062	4.0	
31		Conventional ML	regression	(667.0, 1604.5]	␣
↪	1	910.0	0.349680	3.0	
32		TabPFN foundation model	regression	(667.0, 1604.5]	␣
↪	1	1128.0	0.621439	7.0	
33		CFA combinatorial fusion	regression	(667.0, 1604.5]	␣
↪	0	NaN	NaN	NaN	
34		Ensemble meta-model	regression	(667.0, 1604.5]	␣
↪	0	NaN	NaN	NaN	
35		Graph neural networks (Chemprop)	regression	(667.0, 1604.5]	␣
↪	0	NaN	NaN	NaN	
36		Conventional ML	regression	(1604.5, 4200.0]	␣
↪	5	2642.0	0.554603	6.0	
37		Graph transfer + tabular head	regression	(1604.5, 4200.0]	␣
↪	2	2299.0	5.912514	3.0	
38		CFA combinatorial fusion	regression	(1604.5, 4200.0]	␣
↪	1	1842.0	0.699850	2.0	
39		Ensemble meta-model	regression	(1604.5, 4200.0]	␣
↪	1	4200.0	0.594788	6.0	
40		Graph neural networks (Chemprop)	regression	(1604.5, 4200.0]	␣
↪	0	NaN	NaN	NaN	
41		TabPFN foundation model	regression	(1604.5, 4200.0]	␣
↪	0	NaN	NaN	NaN	
42		CFA combinatorial fusion	regression	(4200.0, 13445.0]	␣
↪	2	8682.5	0.915460	4.0	
43		Conventional ML	regression	(4200.0, 13445.0]	␣
↪	0	NaN	NaN	NaN	
44		Ensemble meta-model	regression	(4200.0, 13445.0]	␣
↪	0	NaN	NaN	NaN	
45		Graph neural networks (Chemprop)	regression	(4200.0, 13445.0]	␣
↪	0	NaN	NaN	NaN	
46		Graph transfer + tabular head	regression	(4200.0, 13445.0]	␣
↪	0	NaN	NaN	NaN	

47	TabPFN foundation model	regression	(4200.0, 13445.0]	□
↪ 0	NaN	NaN	NaN	

Bemis-Murcko scaffold diversity computed from de-duplicated prediction SMILES.

```
[70]: # NO_RERUN_PUBLICATION_REPRODUCIBILITY
# Publication analysis 4: reproducibility audit and archive manifest from
↪existing artifacts.

required_dataset_files = [
    "metrics.csv",
    "predictions.csv",
    "selected_features.csv",
    "selector_coefficients.csv",
    "run_status.json",
    "step_runtime.csv",
]

manifest_rows = []
for dataset_dir in sorted([path for path in RUN_DIR.iterdir() if path.
↪is_dir()]):
    metrics_path = dataset_dir / "metrics.csv"
    first_metric = {}
    if metrics_path.exists():
        try:
            metric_head = pd.read_csv(metrics_path, nrows=1)
            if not metric_head.empty:
                first_metric = metric_head.iloc[0].to_dict()
        except Exception:
            first_metric = {}
    row = {
        "dataset": dataset_dir.name,
        "dataset_dir": str(dataset_dir).replace("\\", "/"),
        "git_commit": git_value(["rev-parse", "HEAD"]),
        "git_is_dirty": bool(git_value(["status", "--porcelain"])),
        "configured_random_seed": (RUN_CONFIG or {}).get("random_seed", "") if
↪isinstance(RUN_CONFIG, dict) else "",
        "configured_chemprop_seed": (RUN_CONFIG or {}).
↪get("chemprop_random_seed", "") if isinstance(RUN_CONFIG, dict) else "",
        "configured_maplight_parity_seeds": (RUN_CONFIG or {}).
↪get("maplight_parity_seeds", "") if isinstance(RUN_CONFIG, dict) else "",
        "split_strategy": first_metric.get("split_strategy", ""),
        "split_train_hash": first_metric.get("split_train_hash", ""),
        "split_test_hash": first_metric.get("split_test_hash", ""),
        "n_train": first_metric.get("n_train", np.nan),
        "n_test": first_metric.get("n_test", np.nan),
    }
    present_count = 0
```

```

for filename in required_dataset_files:
    path = dataset_dir / filename
    present = path.exists()
    present_count += int(present)
    stem = filename.replace(".", "_")
    row[f"has_{stem}"] = bool(present)
    row[f"sha256_{stem}"] = sha256_file(path) if present else ""
    row[f"bytes_{stem}"] = int(path.stat().st_size) if present else 0
row["required_dataset_artifacts_present"] = present_count
row["required_dataset_artifacts_total"] = len(required_dataset_files)
row["required_dataset_artifact_fraction"] = present_count /
↳len(required_dataset_files)
manifest_rows.append(row)

reproducibility_manifest = pd.DataFrame(manifest_rows)

repo_level_files = [
    "README.md",
    "publication_recommendations.md",
    "portable_colab_qsar_bundle/run_autoqsar_ga_benchmarks.py",
    "portable_colab_qsar_bundle/benchmark_results_summary.ipynb",
    "data/benchmark_dataset_catalog.csv",
    "data/benchmark_leaderboards/TDC_ADMET_Benchmark_Performance_by_Model.csv",
    "data/benchmark_leaderboards/ESOL_Lipophilicity_Current_Benchmarks_csv.csv",
]
repo_manifest_rows = []
for relative_path in repo_level_files:
    path = resolve_repo_file(relative_path)
    repo_manifest_rows.append({
        "relative_path": relative_path,
        "exists": bool(path.exists()),
        "bytes": int(path.stat().st_size) if path.exists() else 0,
        "sha256": sha256_file(path) if path.exists() else "",
    })
repo_reproducibility_manifest = pd.DataFrame(repo_manifest_rows)

repro_summary = pd.DataFrame([
    {
        "datasets_audited": int(len(reproducibility_manifest)),
        "datasets_with_metrics": int(reproducibility_manifest.
↳get("has_metrics_csv", pd.Series(dtype=bool)).sum()) if not
↳reproducibility_manifest.empty else 0,
        "datasets_with_predictions": int(reproducibility_manifest.
↳get("has_predictions_csv", pd.Series(dtype=bool)).sum()) if not
↳reproducibility_manifest.empty else 0,
    }
])

```

```

        "datasets_with_selected_features": int(reproducibility_manifest.
↪get("has_selected_features_csv", pd.Series(dtype=bool)).sum()) if not
↪reproducibility_manifest.empty else 0,
        "datasets_with_split_hashes": int((reproducibility_manifest.
↪get("split_train_hash", pd.Series(dtype=object)).astype(str).str.len() > 0).
↪sum()) if not reproducibility_manifest.empty else 0,
        "median_required_artifact_fraction":
↪float(reproducibility_manifest["required_dataset_artifact_fraction"].
↪median()) if not reproducibility_manifest.empty else np.nan,
        "git_commit": git_value(["rev-parse", "HEAD"]),
        "git_is_dirty": bool(git_value(["status", "--porcelain"])),
        "run_config_present": bool((RUN_DIR / "run_config.json").exists()),
    }
])

write_publication_csv(reproducibility_manifest,
↪"publication_reproducibility_manifest.csv")
write_publication_csv(repo_reproducibility_manifest,
↪"publication_reproducibility_repo_files.csv")
write_publication_csv(repro_summary, "publication_reproducibility_summary.csv")
archive_file_list = reproducibility_manifest[["dataset", "dataset_dir"] +
↪[f"has_{name.replace('.', '_')}" for name in required_dataset_files if
↪f"has_{name.replace('.', '_')}" in reproducibility_manifest.columns]].copy()
↪if not reproducibility_manifest.empty else pd.DataFrame()
write_publication_csv(archive_file_list,
↪"publication_zenodo_archive_dataset_file_list.csv")

availability_statement = (
    "Benchmark artifacts include per-dataset metrics, predictions, selected
↪features, selector coefficients, "
    "run configuration, split hashes, and checksum manifests. The generated
↪publication_reproducibility_manifest.csv "
    "and publication_reproducibility_repo_files.csv files are intended as the
↪Zenodo archive checklist."
)
(RUN_DIR / "publication_data_availability_statement.txt").
↪write_text(availability_statement, encoding="utf-8")

if reproducibility_manifest.empty:
    display_note("No dataset directories were available for reproducibility
↪auditing.")
else:
    display_note("### Publication Analysis: Reproducibility Audit and Archive
↪Manifest")
    display(repro_summary)

```

```

display(reproducibility_manifest.
↳sort_values(["required_dataset_artifact_fraction", "dataset"],
↳ascending=[True, True]).reset_index(drop=True))
display_note("Repo-level files proposed for the archive")
display(repo_reproducibility_manifest)

```

1.0.31 Publication Analysis: Reproducibility Audit and Archive Manifest

```

datasets_audited  datasets_with_metrics  datasets_with_predictions  ↵
↳datasets_with_selected_features  datasets_with_split_hashes  ↵
↳median_required_artifact_fraction  \
0          45          44          44          ↵
↳          44          44          ↵
↳  1.0

```



```

git_commit  git_is_dirty  run_config_present
0  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  True

```



```

dataset  ↵
↳dataset_dir  git_commit  git_is_dirty  ↵
↳configured_random_seed  configured_chemprop_seed  \
0          tdc_herg_central  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42
1          chemml_cep_homo  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42
2          chemml_organic_density  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42
3          esol_delaney  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42
4          freesolv_sampl  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42
5          lipophilicity  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42
6          poduam_pod_nc_std  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42
7          poduam_pod_rd_std  C:/Users/Scott.Coffin/OneDrive - ↵
↳California OE...  b06baf5ef2ebdd2a4c6bc76824103dfbd35be861  True  ↵
↳          13          42

```

8	polaris_adme_fang_hppb_1	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
9	polaris_adme_fang_perm_1	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
10	polaris_adme_fang_rclint_1	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
11	polaris_adme_fang_rppb_1	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
12	polaris_adme_fang_solu_1	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
13	tdc_ames	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
14	tdc_bbb_martins	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
15	tdc_bioavailability_ma	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
16	tdc_caco2_wang	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
17	tdc_carcinogens_lagunin	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
18	tdc_clearance_hepatocyte_az	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
19	tdc_clearance_microsome_az	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
20	tdc_clintox	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
21	tdc_cyp1a2_veith	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		
22	tdc_cyp2c19_veith	C:/Users/Scott.Coffin/OneDrive	-	u
	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
	13	42		

23	tdc_cyp2c9_substrate_carbonmangels	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
24	tdc_cyp2c9_veith	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
25	tdc_cyp2d6_substrate_carbonmangels	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
26	tdc_cyp2d6_veith	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
27	tdc_cyp3a4_substrate_carbonmangels	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
28	tdc_cyp3a4_veith	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
29	tdc_dili	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
30	tdc_half_life_obach	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
31	tdc_herg	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
32	tdc_herg_karim	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
33	tdc_hia_hou	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
34	tdc_hydrationfreeenergy_freesolv	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
35	tdc_ld50_zhu	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
36	tdc_lipophilicity_astrazeneca	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	
37	tdc_pampa_ncats	C:/Users/Scott.Coffin/OneDrive -	California OE...	b06baf5ef2ebdd2a4c6bc76824103dfbd35be861	True	
			↳	13	42	

```

38          tdc_pgp_broccatelli  C:/Users/Scott.Coffin/OneDrive -
↳California OEm... b06baf5ef2ebdd2a4c6bc76824103dfbd35be861      True      13      42      42
39          tdc_ppbr_az  C:/Users/Scott.Coffin/OneDrive -
↳California OEm... b06baf5ef2ebdd2a4c6bc76824103dfbd35be861      True      13      42      42
40          tdc_skin_reaction  C:/Users/Scott.Coffin/OneDrive -
↳California OEm... b06baf5ef2ebdd2a4c6bc76824103dfbd35be861      True      13      42      42
41          tdc_solubility_aqsolddb  C:/Users/Scott.Coffin/OneDrive -
↳California OEm... b06baf5ef2ebdd2a4c6bc76824103dfbd35be861      True      13      42      42
42          tdc_tox21  C:/Users/Scott.Coffin/OneDrive -
↳California OEm... b06baf5ef2ebdd2a4c6bc76824103dfbd35be861      True      13      42      42
43          tdc_toxcast  C:/Users/Scott.Coffin/OneDrive -
↳California OEm... b06baf5ef2ebdd2a4c6bc76824103dfbd35be861      True      13      42      42
44          tdc_vdss_lombardo  C:/Users/Scott.Coffin/OneDrive -
↳California OEm... b06baf5ef2ebdd2a4c6bc76824103dfbd35be861      True      13      42      42

    configured_maplight_parity_seeds      split_strategy      split_test_hash
↳      split_train_hash
↳n_train  n_test  has_metrics_csv  \
0          1,2,3,4,5
↳
↳ NaN      NaN      False
1          1,2,3,4,5  target_quartiles
↳7bca3443e9455134a9e500ae47d65ef8a9258d17fc844f...
↳2cf9245678766eb17714604f0f4651d32bd496fde0b205...      400.0      100.0
↳True
2          1,2,3,4,5  target_quartiles
↳3a2203cf8e9934f0de0a2d6aca47be30df6b0507780a18...
↳e6718ea91d5848923d4910c905793d1e3e7d3ae27315e6...      400.0      100.0
↳True
3          1,2,3,4,5      scaffold
↳6f01da23b222c7d7cd5476ad9b5cb52d01f6d1626756e4...
↳9ee48128aa2c8388f7a9825139781be3d4fde955fb02d6...      874.0      254.0
↳True
4          1,2,3,4,5      random
↳1a4d3541a62887d29772b7142a7571b65d0c668794fde3...
↳751e782db79be4730236c0e1232d599506645e8018bd2d...      513.0      129.0
↳True

```


5	1,2,3,4,5	scaffold	↳1c9600e79ad39c5f75209604a7171b2bd6a9c7fc733939...	↳26aaf8bb369bd22baa01dd6ed101857bdecaacd32b2d1b...	3357.0	843.0	↳True	↳
6	1,2,3,4,5	target_quartiles	↳9d583a348e68e5eb6f8758c9ee58e1fba3f422444d608a...	↳0e14d87142390e9c66b8902d81dc53edbe48383f564d1c...	1473.0	369.0	↳True	↳
7	1,2,3,4,5	target_quartiles	↳ab369ddebf9ec5b0caf99728d370fe5b63c08ca47dd9c8...	↳d7bdd743de243e744443452ffc332473e683901c242abf...	1884.0	471.0	↳True	↳
8	1,2,3,4,5	predefined	↳40484d6673195f8d2fda68d20af4f3531c3597678494cf...	↳66dc633f35cdc8f4287a582e07b4fab1e1979e18c548df...	1446.0	362.0	↳True	↳
9	1,2,3,4,5	predefined	↳f23c02546e879de2b690e822ca8e7d9784b0e69103c2da...	↳3ba0ff6ed8e4af670d029a7f058511621c8f85055a78c1...	2113.0	529.0	↳True	↳
10	1,2,3,4,5	predefined	↳c1f606248434d901ce9795dcc1d798999d72a2f5087468...	↳a5d43bd96e3b17263757ea33b18ec349714c42ed3cc771...	2443.0	611.0	↳True	↳
11	1,2,3,4,5	predefined	↳ae4bc1e9cbf4bcb42b90d3918f46fc2204cce1c855bdab...	↳5b6e0686a4ba6e5084f43967bb4264892c24fc294700fa...	708.0	177.0	↳True	↳
12	1,2,3,4,5	predefined	↳65f12df6b9e2a7abe204db5cff958f2c01aedc49ad9158...	↳5d717152f002212ed3406ed747d093a08f52e348ad0efd...	1738.0	435.0	↳True	↳
13	1,2,3,4,5	predefined	↳c1e188ce665dcf4a30bb7a4559236c9d4e987cc57029df...	↳c7e6d660eaddec638ff06bf1964ad194d2d84171c1b3cf...	5821.0	1457.0	↳True	↳
14	1,2,3,4,5	predefined	↳a6dccd56b1ec87c5d47ec856e472f8a0ad4ada513e8cff...	↳72058152bdf05cccd6688d989dca2779c615cfbf70ffec...	1624.0	406.0	↳True	↳
15	1,2,3,4,5	predefined	↳037d79f4c2a5c2cc45214f6cf911003856ccb2898fac60...	↳fc65a1d4a0e74f3e44e8cc5e23e7d0819dea458fd07af4...	512.0	128.0	↳True	↳

16	1,2,3,4,5	predefined	┐			
	↳13d5e39f6646aa71e41f87ffe270075e70b8a52f61135b...	┐				
	↳f99907fd252c7e33efd6a6bd6e733ed93b18b9f7f215c6...			728.0	182.0	┐
	↳True					
17	1,2,3,4,5	random	┐			
	↳1337c1898cc4bf67c33e02adcbc529871eddbbaefb81b3...	┐				
	↳f414f70ef733f2b850b272e8be910f7a3b0c37a1900db8...			224.0	56.0	┐
	↳True					
18	1,2,3,4,5	predefined	┐			
	↳a46c9f3a6c94a805fe648eb11c1df1ff43877cfc351de0...	┐				
	↳d0e0deddb5a46b1f69771818604af0cbc5f1b0650450de...			970.0	243.0	┐
	↳True					
19	1,2,3,4,5	predefined	┐			
	↳8690f4b589419add143c92f467fed5d2e0e93865d7a8ce...	┐				
	↳e6f80dfa2e43d14511bd426e9803b2dc209eb886e94ec8...			881.0	221.0	┐
	↳True					
20	1,2,3,4,5	random	┐			
	↳e5e14762d4103ddc1d8dcdf09946434ff28b8cbf683713...	┐				
	↳dd389882ff3161b285bb5e4188ca97f3821de461c6e852...			1182.0	296.0	┐
	↳True					
21	1,2,3,4,5	random	┐			
	↳087e7c82ee97b98eab92ea0fc6a54c22ff7e479089465d...	┐				
	↳6c06095d482b89c62fcc5e4483d67502f106ad0d7127db...			10063.0	2516.0	┐
	↳True					
22	1,2,3,4,5	random	┐			
	↳52669581a0562314195e2c243ac9a99c5d7d1566264f11...	┐				
	↳2601c72e638d3a95d6153eb24f869f06d6d124250eaded...			10132.0	2533.0	┐
	↳True					
23	1,2,3,4,5	predefined	┐			
	↳57f35ddc7eb10b8be0aec4b9c51a913508fbdfad4625a6...	┐				
	↳1eeea5eb9e728890b1600cd9d94766119ef0e001fae063...			534.0	135.0	┐
	↳True					
24	1,2,3,4,5	predefined	┐			
	↳e2125e288f4f697bb0db68036cc6e783ea1d70cf7badec...	┐				
	↳f62c2f668ec14fb2f8b59f10fce0887d959d5295d26b22...			9673.0	2419.0	┐
	↳True					
25	1,2,3,4,5	predefined	┐			
	↳c18495eb2efc7a330ac8ca3084478a3c913778b1b51249...	┐				
	↳4a6356582ba17f809e42c5dd38e58474251b1eb75cd105...			532.0	135.0	┐
	↳True					
26	1,2,3,4,5	predefined	┐			
	↳fb682d3286f211f487cd8ef11cc87dfdfc7c51b279fb95...	┐				
	↳cb779209dbc7aedc1f979fc119cae0d99fa9a3f75c951a...			10504.0	2626.0	┐
	↳True					

```

27          1,2,3,4,5          predefined  ␣
  ↪a0afa0b1ff4ebd42751a56d4ac75fdc5486468a3f86154... ␣
  ↪6969c0ddabbc518e5ae53ff95c44559c6e1cab23e373b4... 535.0 135.0 ␣
  ↪True

28          1,2,3,4,5          predefined  ␣
  ↪a1ab095f0c322114596b10a8320f111c8e89c1663ecef4... ␣
  ↪876c64a9605af73779282733ee229aeaf989f33defd93f... 9861.0 2467.0 ␣
  ↪True

29          1,2,3,4,5          predefined  ␣
  ↪f2a7e7ea20931ec51770fa53937aeb46667b093259fb17... ␣
  ↪884555c3aeea80e8c967b851b0d53fb258541a7556a1c2... 379.0 96.0 ␣
  ↪True

30          1,2,3,4,5          predefined  ␣
  ↪9ff78871525cf39b693ad6c13d7fb061dfcbb16aefa93d... ␣
  ↪7a672eea1805369566039d2ddada0aad2788b40fcc83c2... 532.0 135.0 ␣
  ↪True

31          1,2,3,4,5          predefined  ␣
  ↪b87a3f409c7ed81e12a689659fa459f46c1d7c55577627... ␣
  ↪75c42c053c342f6b55693d4118ce7106803c73bc7e4134... 523.0 132.0 ␣
  ↪True

32          1,2,3,4,5          random ␣
  ↪4d0193670289862cb3d527783257aee805bdb705484720... ␣
  ↪8fed49b82cb54f60590aad27f0939d2169af9624cc7e75... 10756.0 2689.0 ␣
  ↪True

33          1,2,3,4,5          predefined  ␣
  ↪914fef277489c120c49df649a2490602576a68e2a53328... ␣
  ↪90f0a4c6a2d8c250c8b3b314301993ec3924a13ab062c0... 461.0 117.0 ␣
  ↪True

34          1,2,3,4,5  target_quartiles ␣
  ↪e99e95d86b39d0c1fef7984f923de08a194f8df032ab3e... ␣
  ↪a7d3716155ad13c774cbf0d46d4807f3f90616db84e81f... 513.0 129.0 ␣
  ↪True

35          1,2,3,4,5          predefined  ␣
  ↪6256f884a78ae46696836eded007c1cf018bb56304934a... ␣
  ↪e152a1f0ee254a8a1feed858602665e37dd2bb3f2e9e7f... 5907.0 1478.0 ␣
  ↪True

36          1,2,3,4,5          predefined  ␣
  ↪e9e48833907af3b489e2d3008c23bf6bfb6bcaa84fa80d... ␣
  ↪2759247fc6c7db07c9b24e2b1effd0da5cb1ce427fb855... 3360.0 840.0 ␣
  ↪True

37          1,2,3,4,5          random ␣
  ↪1048ae3a05c4b54b6e3ed275c4983f5ef94b1f2fad1b9e... ␣
  ↪6f4328da3e8c84c6c57bb789b864af6fda491b4d35981a... 1627.0 407.0 ␣
  ↪True

```

```

38          1,2,3,4,5          predefined  ☐
↳120d8959389d7ad774ede3b365a34cd05f02c37d38763a... ☐
↳8578a46994ebb1793c605e32b31005192abf044deca5d5... 973.0 245.0 ☐
↳True

39          1,2,3,4,5          predefined  ☐
↳d630d01d567dbbc8d507e7ba380f353a013b3144e39f21... ☐
↳7eff1d13f182b487cc433086f79c51f708253bf7c31305... 2231.0 559.0 ☐
↳True

40          1,2,3,4,5          random ☐
↳b80b7ee7dca55253349eeb1cff6cca5bad5ebcdc6a6694... ☐
↳37395861f4023d5cba7474c0fc4457e6e0fdc9fb69587f... 323.0 81.0 ☐
↳True

41          1,2,3,4,5          predefined  ☐
↳228f17b48ac14a4b14e267564d566d2fc603217b87afd7... ☐
↳bbe67cdaca56f4a02331e8809da8e6d33c6864d01eaa7c... 7985.0 1995.0 ☐
↳True

42          1,2,3,4,5          random ☐
↳a7157c6c2569e1a4ac8ffa5b67765f00e987cb131ba912... ☐
↳94e4a93eb05d17ff2c0df161ebdcdefa8c3c75c223ebfe... 5806.0 1452.0 ☐
↳True

43          1,2,3,4,5          random ☐
↳60c66705e1ff672ca2c7c57bcc087df571d79187e27cad... ☐
↳1965f617270211d436d82df13cbaef09e3016bb39caa5b... 1384.0 347.0 ☐
↳True

44          1,2,3,4,5          predefined  ☐
↳d6fec356993676713dad15a80f329f23642670ee9f5fc5... ☐
↳0b147b50b8c9ffffe271ce20d7095c73177486f61ab5e4... 904.0 226.0 ☐
↳True

          sha256_metrics_csv  bytes_metrics_csv ☐
↳has_predictions_csv          sha256_predictions_csv ☐
↳bytes_predictions_csv  has_selected_features_csv  \

0          0          0 ☐
↳          False ☐
↳          0          False

1  e04a6232c08c75528283bf7a88ed76e07ef94908dea820... 36106 ☐
↳          True  a90dcc0367102ef25a58ef62f5baa3c7476db3bfdd07ae... ☐
↳1328935          True

2  c728980d4e62e0749c518cf766b9c787d3efeb22b4e2af... 36614 ☐
↳          True  14b44b4d726142ae4e151c6acb200e5df9e839cd18a1c9... ☐
↳1391952          True

3  4976bd39c50cc39eab183a8ba54c739812804fce8adabe... 47053 ☐
↳          True  574a3066063aa5904ba4ed03049cb1f9ee02b5329b5611... ☐
↳2087442          True

4  2a0f93dfd48d7bb3eeae49e033b7eac1cf206c2a4c907... 36964 ☐
↳          True  275057d272c3f9bf8231c30f26ee99a5427c64bfb330cb... ☐
↳1263870          True

```

5	89fb2824aaa7f8e29603de9e71a232d7a59c5ab968b218...	42759		
	True 6644890de41a431b0d0203dca1a7125937a884b331d972...			
	10306645 True			
6	e48014a2cd0a7d7b41347e0a9975bbf046d712f12f090c...	47933		
	True 53ac36590165e356529ebc66bd97f3c25590d695b1f3d8...			
	4037886 True			
7	305eeb1f6bccb939bceb71c8445c6dfb777c9dda9c27d7...	48785		
	True c6ecf1acb45c82c6290337733d15b59a9a51c5b67d0fd0...			
	5220894 True			
8	5ee64344cefed4a9d75ee8e8e55d4140b0c8ec4a43d731...	70279		
	True a505fe0fb83afa5b1f847b11056a023f752726c80da054...			
	4656443 True			
9	ba47749057c66f4abea09b8c96c68b76daed987cb6888c...	65467		
	True 37f4c0e7fc9041e56b74d7857242d2c37e66f39751810a...			
	6892278 True			
10	338e1d3c56e5c27d88eadfae0344b2acc4a5504e3ea2b4...	71237		
	True 10629261e721f6b0aab7afbc9db6b7de8e30e41f3cf2e9...			
	6998798 True			
11	60884d7cae0d2906218d11a65bd275a768bad7cc68a7d2...	70261		
	True 2cdc60700f14d9c4b4bacdf02d2db44851c8d3df7ab250...			
	2406317 True			
12	d0a2deeeb1f635cbbaa33bb9da6504462bcc266626447f...	68477		
	True 554c380e1d609c0539898741f5f696286796dd96824701...			
	4919406 True			
13	53a1113a852ef99e61bde33cb01f10eac6c6b2fc3875bb...	34053		
	True 71a3261d5f89502f21dd1b31ad1ab4e9b999388ab75011...			
	11097502 True			
14	f17cc1b914c2a49d269df4c05d4f9cc5c0ce6a5e6d489c...	35310		
	True 1e5c18d3fd31ea50293140a7b7df368611389e3eea5cfb...			
	3565556 True			
15	d11aa2940403466d08ab6027a10046b7a71a3640ba4cd4...	31627		
	True de7eb395ad138f92d18a24a200e6ee281181535549a0eb...			
	1580718 True			
16	07d71296b7325634bb559c6e8741ef0d7774033bcecf9...	67436		
	True ee8f262e8acb30bb05f067046fe3f41de38948d6c87211...			
	2419403 True			
17	ff0404bc81516e826f04ed4bcd85baeaf8e3d7cf78dfcd...	35055		
	True 051bc2692dd0461673ed29c314489d9ea2fd9af4ab7534...			
	644667 True			
18	f29434fd25f44bc2f736e0f697ff7cec35e4d297a8075d...	67981		
	True 7c18f71da17dc8b1b72e693a43e50db3af96a050575218...			
	3160349 True			
19	aef695d1b610a5722f75fa09a404f41ec8804b99787639...	63501		
	True a088d3ff796090732766c2c31358ab4df0b5e624b599c9...			
	3176837 True			

20	41e18a12e49b76015415d4ae7c6f39eb30d6c1ce73c4e2...	34009		
	True a8d2c18904a9261a6c7d27012dbb43d25bee834ece2c1f...			
	2752082 True			
21	3bd9dc65feec0e51bd171ffbeb2c5c54d4a8fe5a3a00fb...	31715		
	True c78cecad48f5cdf0ee9279a18cba57ed420b51bde36d24...			
	27349733 True			
22	1859d4e56e3117d9050deebbb4b51866432ece4a779569...	31458		
	True c6e8aee8e8ef92b05497a3f9863eb2f9a2296bc9d6bea3...			
	27864195 True			
23	79660224e678e6ca93e21225a23b6683bfe9f58986181e...	35928		
	True 935032dcc3b32bbfc14e5e1f1cf900ae77848217d3425a...			
	1587178 True			
24	6446c35eaa8cc1a77e4391b3f6f602dcfa11cd20431f7b...	32234		
	True 6bef8257fb263c05cf411767b5c8d9f11bcb7571da373c...			
	26518935 True			
25	780ecd56eaf85c3e7ea9a4d81b0c9453f160576a491526...	35874		
	True 2de416bc81e81ccd8979c986f2ac434a1a594d3264d1fa...			
	1568987 True			
26	578793fbb067f3e78841a02214f145d698fe7dc6d029c3...	32110		
	True 6792c20f769f6cfbcd2225882f7ae02d4e9f9fd8436513...			
	28797708 True			
27	674a35c7f3aab09e27891bba59c6572da64b69a7f7c449...	35666		
	True bece233ed436aef2542943519b8db537602b0d3b9efb0f...			
	1576756 True			
28	d2361cc2e7389533cbac44a74fb8442c138c0b00d596e9...	32161		
	True 71fd583e062ad33308131b73e7759650bc282163443ce4...			
	26872100 True			
29	4eb4b5986e516e1a6337f9eb0ee95ae1d8b775f78efb8a...	31973		
	True 5862365b306295f9e8bfc30eeb22ca375c70cc89cef2e5...			
	989765 True			
30	3a772c111b687b48d0f18fd32956d8f50a20e4983ab634...	67761		
	True bb35fa3feb2530852a6c7e97a9e965af480eeea4b4a3b4...			
	1618893 True			
31	2b4415dcd1a7a7a659733923716dfcf50ed3f6ae9c5283...	34361		
	True 0004cdb474bbbb8864bfd8ea991becb95aa22aef2c7396...			
	1291976 True			
32	18a80faa16dec5a43104ba7fad22a54ca6244b029fee8b...	31454		
	True 41926897ed352c1d18f46372f42296d79854f94c20db66...			
	31064728 True			
33	76fd1d9abe85f8e9238b0b757f77b83f529b82386fe29d...	31960		
	True f6701d45857aa22b1523121b64c1adbb995202422bd5ef...			
	1292532 True			
34	6b0bd9af659bde9fe457f50b7c6065d3a2675d90852fa6...	37221		
	True 256e80428a2c6387209532f48a76c82f6baf0e66b1a4b7...			
	1495154 True			

```

35 5c768a4ef841f9a2f86e8e43597e98f51c7da3a695372a... 66391
    True a916d2e6e94edd98aaef3171d52fe901ab1d0991c4e120...
    ↳13519757 True
36 62b6a9aa69222e78c9861bffc32ce81f73165a65ab0... 62688
    True 5bd4c61c0aa2e40764ea31daacdc845ed4946f2d8c5546...
    ↳11583775 True
37 333c16999ffe450070119d45f0711a40125608e8c6541b... 31161
    True e9709f29696619ec34aaa07e7e31fe18a5911a7cb1770a...
    ↳4472344 True
38 d93cffdb3cf68b5888ce922cdae4e158a7f705b7425261... 35375
    True baed41db7165b49723894a8114eab69400d520fe3af16e...
    ↳2580700 True
39 c7ed778beaef0b9546e912d06e6fc7965492169d1de7e7... 65932
    True 0409deabf5448b5032029116a4e37a79978f5b82db0722...
    ↳6156812 True
40 3d86681ff9787029fac9003d382f069b1771b3a73514d0... 31278
    True 85c395dff81d19289db95ce1d0a10174ce16d61540508d...
    ↳790472 True
41 3cade4868513fd41ff4c3ce23fa9c258bc1a8700dbf9e4... 61911
    True acbadb727db8c4310d1500958d1d5f184079463a2f1119...
    ↳24370571 True
42 89c128160bdd56a497fde9f00f1d0971e3a217385629ce... 31067
    True 8550020429f0f97451cc6a328ecede6566a1dc068f60b7...
    ↳13797021 True
43 fb6ae49a6752ca0f7a3eb1ad9e54e3d369b3420d9cd3b9... 31290
    True 47c5db5abff7ce2f8d09d8d1e9235a096878c533f7812a...
    ↳3219510 True
44 196a3cc0e05c172ecc717a88af16670671c2db86281f37... 67495
    True fa96825e6c14facdbaceeb7b72db743195f9a295cfebbe...
    ↳2685551 True

    sha256_selected_features_csv
    ↳bytes_selected_features_csv has_selector_coefficients_csv
    ↳sha256_selector_coefficients_csv bytes_selector_coefficients_csv \
0
    ↳0 False
    ↳ 0
1 1a2325b2f5664359ec908c8c32f95626b2c7a07cc04e0f... 6025
    True
    ↳0525eed288abdf77ff06ca78e5568958361d717ae6461e...
    ↳186011
2 90db7e9c311d72cc8f573bf54bc50695582e88d36e3105... 4467
    True
    ↳da2c49f3482ad3e417d56aaa5028ccee714d9c183d7be5...
    ↳203256

```

3	80670272430de20b26e31745207f17217486da85521d60...	9643
	↪ True	
	↪ 8469a3dd59e8144b6da780009da7bd370d560f920af556...	
	↪ 236693	
4	a7c6c15f16e8fe3446bc6d585f9f9c3d84a4124ecd507d...	8411
	↪ True	
	↪ f076f8a726cc0b455099eb2d355d19cc95f90c58c75566...	
	↪ 175529	
5	48687a44952790d70b15226a27f2cfcf19d9b456986426...	9358
	↪ True	
	↪ 9863fbe67a6bcf695602d9f2434a33069535867f6e4887...	
	↪ 270603	
6	07a0240348586a08ae3078a5d03182f835e04045868162...	6464
	↪ True	
	↪ 1fe8e71dd9c70e54d253b73fe74c2ff3323df3d67f0a5d...	
	↪ 257882	
7	abaae731ec5e5b1c563db7b66e8104f75c661446b7fbbc...	9604
	↪ True	
	↪ 1044b3ee7985751acddd97e90061034be898bf66a93dbd...	
	↪ 268808	
8	80d855a5a18588c0759acb3c02a470019635b8edad8c90...	9460
	↪ True	
	↪ 3a7147ac21e7004a0d97b98452d49167444efe58807497...	
	↪ 256377	
9	49856390d4de6c097f97aee8e89f34c9e0e83019cbd912...	9415
	↪ True	
	↪ 36af2a1cfd98de7414a362657e7fa289da767f562adeb8...	
	↪ 255395	
10	13194c677ecb8b9bcda69f1fdd69ffcb82afa120fa613c...	9649
	↪ True	
	↪ 4cd7a03c2b19560a8ca23e00936c2c325888e447b64cf0...	
	↪ 258037	
11	4aff0fff13ebf3622bc9d230424bdcc75f2d29d04543ca...	4189
	↪ True	
	↪ ccb03ee701d5d4796c456b613e389bdac436c26a6b8f44...	
	↪ 242748	
12	eaaa61642e2f0f47d0f706bfb59d068c0d68763cd3a0d0...	4141
	↪ True	
	↪ 5f11afb477e30c508b538ec3576aff9693198c9eb384e5...	
	↪ 238962	
13	73d84b712e1e9d06cf63916754c1d2d97b456754abbb7c...	9055
	↪ True	
	↪ 5e8b7eddfb6a62f451c5f1ddde4bdacd6bc02f71cb6389...	
	↪ 365479	

14	bcc54763d8ce5188747ffece9b2970a68d9be3c0fc6cb8...	5145
	True	
	30614a7c3a2ae849f467c460dcace9bb13b38af54327ca...	
	253709	
15	cecd9cfacbb85c31870d2f7b0b6c1d29de998999d0808d...	254
	True	
	8c0b49f2d028654aacac486d9a02d3afb0bad0360eebdc...	
	238051	
16	30aba757fc4e4dea52466ff8dee698ed2f0c8b402313ef...	6433
	True	
	9939b65431c53c2183594fef280aa5338cca525fa04d02...	
	250490	
17	1f9c69dc4fdbbe46dec5acfe9c03649fb80e7796ecddae9...	1565
	True	
	4583711165fc053582a886f4587e0821efccf6abe7bb6d...	
	231976	
18	f7b0a0e425c97d0cd207921ba18c20dd814cc82dd9bf9e...	9173
	True	
	49164a1f6c06222ef1b07c5b87eca937cc55d9f7cd2dad...	
	347900	
19	e9cb5e49c2cbdb7a0a3437261a1df25ffc6218f313d84a...	9287
	True	
	d0c4d8e5c8879efc10a26f14e1080280be484739353d73...	
	346299	
20	38582daa2910aa012366c942b0b6a5afbb54ed6cf5a295...	2369
	True	
	d654a73cf59ae73ae533c51bb8db7644b1fa742628bf...	
	256891	
21	02786267f72f5d2c035d83efce47c78b1e34eea203c3a9...	8951
	True	
	76da8fdd8bf86f66ee9ce32778547ff7984f59b43280a9...	
	378975	
22	ac438d511c146a4ed01d781a4912b598d4bcfafba925e4...	8923
	True	
	996495c1fc9284ddeac1619eed1801a6a4d07d8c836266...	
	379320	
23	8e9346148add55696c82faaa58ed86c89014ccb9bf6501...	152
	True	
	a14e37cb046445b0b126c2d1098b11d70c316eff729669...	
	237813	
24	4ced6177e9b92fb151d7fd21fcb45fba6961f0998d1e8f...	8905
	True	
	f0054570ce9ccb71136ed2a369d94bf3598cd5bc666c06...	
	377912	

25	21e0d3c10800a4f6bae8da336259a5f538d0d5af5a360e...	2557
	↪ True	
	↪0589f68e6bbec76fd9d620ca1dca74bfd4454a5e435092...	
	↪242630	
26	4936421034d9cad5cf292439a5a07875d74e8451222d94...	9047
	↪ True	
	↪bee41c5ee4aa0bb9f94f04bfee39e0e8b1b5c4e57a5b3a...	
	↪378221	
27	3a7b38721089225253064465ec72657c29ae473a5a5512...	3208
	↪ True	
	↪8f3ec37997b1b460f95620c29da5fb50e20066eca3d42d...	
	↪243007	
28	ac642276bf8ef73db4a7edd0f754fb5ecdd1ce271b9ca8...	8915
	↪ True	
	↪5cf7bd1674c2c3652ef88aab8acd6fecbc19be7020cdbf...	
	↪378685	
29	f4b8a27ac42afe0689f0ac59fa01a8b7315b8757858b53...	5049
	↪ True	
	↪c792646a56dd49503a55ccb03defeb312f6fa42618e17b...	
	↪247821	
30	37d53568116abafec45ab68d9a659e6f3ac33a2ae635f7...	9400
	↪ True	
	↪4672d6a6e6fc2882bf57ec0892ae79bbaab8154055bff3...	
	↪345470	
31	035752036f16aa4adbac43f898fb79edbc8af01d9cf100...	2347
	↪ True	
	↪b13125771146c9ac8ffc8cc35421efcc8a9177f98eca35...	
	↪238458	
32	bc3e66cee81e1002a420afaa24d3f2fe52342955396751...	8562
	↪ True	
	↪043bc778424bb44733527f606d4e800538689744bc68a5...	
	↪375987	
33	3b6fba4771d172faeaa7ba1d1125293681516f7a2d69a2...	1585
	↪ True	
	↪651ff9e38e5ca9762027a081c0e2ac3cf736193038974b...	
	↪237257	
34	b6c1b16e19dc243dd82c44c2324269c359eccec7dcc85d...	7925
	↪ True	
	↪27c35d0b05517178436188087eaa2d8512a0582f91b514...	
	↪178400	
35	28bbf65419435ad8ba86c9da62b80d6acc6cf59aaa35b2...	9260
	↪ True	
	↪f7cbaadddf3bbf03560d1b4419985d9752b7ef844c82695...	
	↪372464	

```

36 7d83b17cfc10a1417b549f284f781f8654fc0aed35e886... 9276
    True
    ↪930e8cf9b9313c25b7f31cddbc464a50edf6aa0b63b84d...
    ↪266367
37 785ba669685e22a2e44cc06dfe543054b6f6fa64bfdca1... 4168
    True
    ↪1f2ac5933a0ed595114d5645a5f89514522ad211aadd07...
    ↪241898
38 83f260d63c861653486c5e55735d865e271a4fcdc0433a... 5055
    True
    ↪e3ab169e6546f0f5e00a5d54ddaa837df2bdea13785753...
    ↪245346
39 427474c1cff5aed9b3bb90726cab66694932965be9c7c4... 9478
    True
    ↪c897b5da8eee7a935a2dc4f407e63f37d86a27651adbd6...
    ↪322428
40 f18d1a9e68dbbea2e6edbf0f3a6b156af81c034dc5076c... 398
    True
    ↪0fda22095ed3c7c610a98b1be29385cb60c47adf823e69...
    ↪208690
41 5ca012ee05fa7d5510a192d23b8c3d354324d00d698daf... 8925
    True
    ↪542c478d66ba059d6fb30fd902c2526f5388e9f9d49491...
    ↪377466
42 8092a2c8478fca4c4f427cd8a504f64a3959314177642e... 9184
    True
    ↪ae2ac477f4282dee3bb5027f371342c803618caf7c492f...
    ↪333922
43 f9d35a4af1e0196a02b3ccd05b9a37404ba685fba78cb4... 1285
    True
    ↪adca9d68e67a1856f81c5fa7d75ab2f8ab4a3da85ea2fd...
    ↪244411
44 f59a6644b7ca23ac70943a25fc132c1820d99da241d1a8... 9475
    True
    ↪7466112e517d00f86146a60668312b4ab4990196feb97d...
    ↪327936

    has_run_status_json sha256_run_status_json
    ↪bytes_run_status_json has_step_runtime_csv
    ↪sha256_step_runtime_csv bytes_step_runtime_csv \
0 True 180607fda121b7a6ff290e5f6f2bd257c1bb31e0ba081a...
    ↪171 True
    ↪e669ea9361dc36cfab2f12639f6f1accf22e21c8a4e17c... 524
1 True 64f2460642243538ba752c567ab89fe53d542d00b28cf1...
    ↪235 True
    ↪5bd48194623a1679af1a9793d23aa3d926ff8f6b926379... 4452

```

2	True	825fe13239e0027ed53d796980c57579125daf55939656...	4626
	250	True	
		e076bb7b9be7aedab92803f2ee6838de1c876ac349c6ee...	
3	True	d7c949f224c1573311a209460cdb4b6c6c29faf742e8d5...	4411
	246	True	
		c0e561fca906d8c612b91b7487e3bd0af8f7ab09e2bb47...	
4	True	828262081e423d0acb72f181a8aede5ba056accb8e9c08...	4447
	250	True	
		66092efd0a10ffef6320f7e9128d6e3f4fa399a0a36cf0...	
5	True	41f75cb69a194b0ef59f6b1542b47c86b8b9ca1d3823db...	4246
	247	True	
		38ae558631b0153bf87f5f03b4d8d69f41e740e35057cc...	
6	True	49dee3068f40561c1e7840339fef619e8cf7c7ad0654b...	4382
	301	True	
		2c3d5e9fc9fc6ec19f96174b6413e7a176789f1fba65b7...	
7	True	f0ebe7d9892c466d993d6e6b8d166451903bb95e24b820...	4394
	317	True	
		6deb6f29f41e1588d2b977cafb9ae614b183939d5c5e4...	
8	True	323b0dd8ee60b9cf6d400dc482667c1c724edfdccc7c1d...	4478
	260	True	
		edfa0c3e803e5bbc7675b16f5bc6453b8bf280b7351be3...	
9	True	6b3a1ce04e26cf0cdb967c044c34b4c6f8ce2a6a54f9b5...	4473
	260	True	
		111dc762ea99c5a3f445bf1d436d1a28824de0d97fca3c...	
10	True	791b26571f7754054a3d4bf637a88b385aac42dcff66f1...	4526
	264	True	
		16379e9cf78c6fd1668aac51af1e79562ef8b27d547318...	
11	True	26d1457955d1ee80e16b10b1bb84e92f9fe8d3bf507b6a...	4670
	259	True	
		88eea53905b86dae916dbe93194496d144e1ac7f6211f...	
12	True	db2081a913c33d2ee16093aeb036a92a529f5709573b99...	4475
	260	True	
		839b674cf43123818a4bd860639889ff40f0850a8cde46...	
13	True	675d5258eae9bafb3248eed9c8ad677d91e1226dd83f3b...	3774
	260	True	
		200f2f29e9e469f77db3d135dc35d72117ff24e109cd16...	
14	True	d3b9357acdedc742347e33597d86561a5b4f1a92ad5e1c...	3899
	275	True	
		a0afcf943e0b98991f3c73f7dbbb9bc3a5ac08fbb7d04f...	
15	True	d340714140d6b8984e54ea2ea5027d9485de81fcb5ff3f...	4246
	288	True	
		aeb2cfa8b3635b962bde337382672d24c4017b6a8b3efd...	
16	True	3fcf93a58ec960abe38790a5d375eed52e473af308ef6c...	4480
	273	True	
		c35a36dfd7672a1e0133d3ea3bace2bb00fa9bb70c047b...	

17	True	7338565c24ae90cd9effa2dc9e24e9dbd5efee1826fc96...	↳
	251	True	↳
		e811eda59d6825ce2fb9e600134cfa3ba094285c0d8d2f...	4226
18	True	336286ba40d31b2fb3d69965a94027bcc55dffa7240096...	↳
	299	True	↳
		32e9c30119b956cd5aa9a1ddba4560bca1f6d5fb663e27...	4773
19	True	1fa11d1c3143c6426eeaf1b2048d4c594e181ff488d5d6...	↳
	297	True	↳
		6e8bcb75a1a0ac80755f8477f661c5fb011ce696d08107...	4750
20	True	22d0adb92c48da3c633f250a8379c52081be3064099541...	↳
	227	True	↳
		94cfe5297d69ec937623e865934b97ceebc6724465f4ce...	3780
21	True	9ecccc5d3b6de17cd9ebd5a39d2565f149b10f5c393028...	↳
	239	True	↳
		08690cbeb0724a001e10f128e1e9f9dc6d650b2d21f77a...	3893
22	True	5f57e86f5b297fa38e6b941cae8ce55ad260fd7dbf20a5...	↳
	241	True	↳
		d6b5b39ee102e14b5ce0dd8e3d2e12f22f54f5f4e07863...	3909
23	True	c856b0a454d2825994d4288aca5a32a23fee9f24402b36...	↳
	312	True	↳
		1e97176b82182f371a59cb5b6755e69df2b6d09476a901...	4502
24	True	79ed85590aab821bac2350e16f89e664994d8cf129237f...	↳
	277	True	↳
		8813f3ce3498954476a25970dc06c4baa4b8e5c03dd992...	3934
25	True	5f3f693a18f57b4d61ff99078395252882536faf70d370...	↳
	312	True	↳
		251d80d71c1deef527e34493804053609188599243bfe5...	4491
26	True	9aecdad22ac55219bd2a796b984d6844470e506217210c...	↳
	277	True	↳
		5f76bf1f165aca9c12566d8d42efb1072149771ad1b7f3...	3938
27	True	bd1c4d002d6e0be74af734b783fb5f2a991d35948c2a04...	↳
	312	True	↳
		322e34cf8c39f4f56052d9ff4a20df3deff8d95cf0d746...	4496
28	True	d4648d925c1387e94d9472e8346cf86573676815d5aefb...	↳
	277	True	↳
		cd1a74f9e02037fe0c6b4eb918bce887f52b4fe0177a6e...	3936
29	True	5830b3961a4244076a8288709c62ec135a3c11d24db1bd...	↳
	259	True	↳
		84fd1dcfd674635429fbd58e87155526d197b67d829cce...	3944
30	True	c1bb7e6f7f8ac4eb1b3570bb26b97c88aff732e43ef2da...	↳
	283	True	↳
		8ed93d55552a3a7039f508a6f7869a446134fd2d374cb3...	4593
31	True	7060eebf1fbc0bc156fc86c62db010b7bb7c891c93f62c...	↳
	259	True	↳
		edfa9a8f037f6899169af30de77dd3b3c72f500c41495e...	3946

32	True	c1b17a779a2f7b630f743baa92349fbb5bb6388394e3a1...	3854	␣
↪	234	True		
↪		2a821d3bc2862fdbb80dc8020380987a3a73a49842b3fa...		
33	True	76b05ba94bf7b34541da2eb6731691af3184ce2e11dc5d...		␣
↪	267	True		
↪		3fe5c1c8b5bf4e12696b3aac2cb6d79f96612ff69f7138...	4010	
34	True	812e0e3e9323c4233abb76851f1c0d68fbb32659b03fe8...		␣
↪	270	True		
↪		3856eb5f2d97cc6105cf94fdb508f1dbe8cbfd4e4cb...	4853	
35	True	34c5d323edd115e51709fed9c2b796b0052d12f3b2562d...		␣
↪	268	True		
↪		357b53edf76eba9984b163e521ddd85edb4309b1e6eda3...	4244	
36	True	95599ac6c08bc966526d65d77c6d9d3dd3eb87564bf34a...		␣
↪	303	True		
↪		7c0868d2655ab5c9b1a40047307415d8bc292143fa2a20...	4622	
37	True	d6ae3db1acd39c6151d390f764b54d898db605a33c319f...		␣
↪	237	True		
↪		06dc0c3e7a9992ddc70bf87d4b27192afef59c3b279ecd...	3855	
38	True	30e01ff44b6c2ac7c63b644c0c85a8d2faea512000c7bf...		␣
↪	282	True		
↪		867ac7d92bae698fa635627dba6548c55f27cf4c973889...	4176	
39	True	5ebf564526debc1fb580460895f83b3f5baf8034b4727...		␣
↪	267	True		
↪		2b9e06080567f94ed00e6d30e90b93f0bbbc85afb2e626...	4224	
40	True	3470fe1da52b1982f0923c094856ff32646879669e53d1...		␣
↪	238	True		
↪		64c6ec657f19eb65848f788802e9b2e284b9bcb6d13717...	4102	
41	True	c216b6edf6da687b04e45849de135f93600e02b155b439...		␣
↪	289	True		
↪		485596355763c590e66624e52af75352369a6847bec665...	4470	
42	True	f3d8ede42e261f795027a05d27ef8b47195cf9fbe70526...		␣
↪	238	True		
↪		22ed223a9166c53da839b4d3f402e91f0399e55da4af90...	3757	
43	True	f42e5000364f328b093c5e879a20c662622825557fad7...		␣
↪	260	True		
↪		703335da4fcda5cd6eea19495b7b473b177f0dcf2e9294...	3815	
44	True	de2f5aba1a2d48f703fc5b52de3f303624a7229152a804...		␣
↪	279	True		
↪		8399d53e9ea04e0d16892ef96d9d313e220d5a3363bbca...	4545	
	required_dataset_artifacts_present	required_dataset_artifacts_total		␣
↪	required_dataset_artifact_fraction			
0		2	6	␣
↪	0.333333			
1		6	6	␣
↪	1.000000			

2		6	6	␣
↪	1.000000			
3		6	6	␣
↪	1.000000			
4		6	6	␣
↪	1.000000			
5		6	6	␣
↪	1.000000			
6		6	6	␣
↪	1.000000			
7		6	6	␣
↪	1.000000			
8		6	6	␣
↪	1.000000			
9		6	6	␣
↪	1.000000			
10		6	6	␣
↪	1.000000			
11		6	6	␣
↪	1.000000			
12		6	6	␣
↪	1.000000			
13		6	6	␣
↪	1.000000			
14		6	6	␣
↪	1.000000			
15		6	6	␣
↪	1.000000			
16		6	6	␣
↪	1.000000			
17		6	6	␣
↪	1.000000			
18		6	6	␣
↪	1.000000			
19		6	6	␣
↪	1.000000			
20		6	6	␣
↪	1.000000			
21		6	6	␣
↪	1.000000			
22		6	6	␣
↪	1.000000			
23		6	6	␣
↪	1.000000			
24		6	6	␣
↪	1.000000			

25		6	6	␣
↪	1.000000			
26		6	6	␣
↪	1.000000			
27		6	6	␣
↪	1.000000			
28		6	6	␣
↪	1.000000			
29		6	6	␣
↪	1.000000			
30		6	6	␣
↪	1.000000			
31		6	6	␣
↪	1.000000			
32		6	6	␣
↪	1.000000			
33		6	6	␣
↪	1.000000			
34		6	6	␣
↪	1.000000			
35		6	6	␣
↪	1.000000			
36		6	6	␣
↪	1.000000			
37		6	6	␣
↪	1.000000			
38		6	6	␣
↪	1.000000			
39		6	6	␣
↪	1.000000			
40		6	6	␣
↪	1.000000			
41		6	6	␣
↪	1.000000			
42		6	6	␣
↪	1.000000			
43		6	6	␣
↪	1.000000			
44		6	6	␣
↪	1.000000			

Repo-level files proposed for the archive

	relative_path	exists	bytes	
↪	sha256			
0	README.md	True	23482	␣
↪	6ef5e2f81b788c937ee84905faf151b18619e7b2229305...			

1	publication_recommendations.md	True	23963	└
	↳df0e7fd54208f05c6ff563b332a6cd8c59bd9f6f6de534...			
2	portable_colab_qsar_bundle/run_autoqsar_ga_ben...	True	430846	└
	↳9f904ca663e23b22781ca09afc98b0bcd793029c7d70c5...			
3	portable_colab_qsar_bundle/benchmark_results_s...	True	1879987	└
	↳54b44fbc5ec2affadc89e29725a0de544795f193471f1...			
4	data/benchmark_dataset_catalog.csv	True	150667	└
	↳927ea321997d61382adfaef5c15b5ba032b73c00313f0a...			
5	data/benchmark_leaderboards/TDC_ADMET_Benchmar...	True	28282	└
	↳ac9a0ae65240e25434779711e2caca6be3d8aa8034b980...			
6	data/benchmark_leaderboards/ESOL_Lipophilicity...	True	7320	└
	↳332f21a3db5628aa50e4dd27e38324cb18ba781f710cf8...			

```
[71]: # NO_RERUN_PUBLICATION_LEADERBOARD_REFERENCES
# Publication analysis 5: updated leaderboard-reference audit and regenerated
↳comparison tables.

def load_current_physchem_reference(path="data/benchmark_leaderboards/
↳ESOL_Lipophilicity_Current_Benchmarks_csv.csv"):
    path = resolve_repo_file(path)
    frame = read_optional_csv(path)
    if frame is None or frame.empty:
        return pd.DataFrame()
    required = {"Dataset", "Metric", "Direction", "Model", "Score_Mean"}
    if not required.issubset(set(frame.columns)):
        return pd.DataFrame()
    work = frame.copy()
    work["Score_Mean"] = pd.to_numeric(work["Score_Mean"], errors="coerce")
    work["Score_StdDev"] = pd.to_numeric(work.get("Score_StdDev", np.nan),
↳errors="coerce")
    captured_at = pd.Timestamp(path.stat().st_mtime, unit="s").
↳strftime("%Y-%m-%d") if path.exists() else ""
    rows = []
    for _, item in work.iterrows():
        score_mean = item.get("Score_Mean", np.nan)
        if pd.isna(score_mean):
            continue
        dataset_name = normalize_reference_dataset_name(item.get("Dataset"),
↳"moleculenet")
        rank_value = pd.to_numeric(item.get("Rank", np.nan), errors="coerce")
        rows.append({
            "dataset": dataset_name,
            "dataset_source": f"Current MoleculeNet literature reference: {item.
↳get('Dataset', '')}",
            "benchmark_suite": "moleculenet",
            "benchmark_id": clean_reference_text(item.get("Dataset")),
```

```

        "leaderboard_url": clean_reference_text(item.get("DOI_or_URL")),
        "leaderboard_dataset_split": clean_reference_text(item.
↳get("Split")),
        "leaderboard_metric_name": clean_reference_text(item.get("Metric")),
        "leaderboard_metric_direction": clean_reference_text(item.
↳get("Direction")),
        "rank": str(int(rank_value)) if pd.notna(rank_value) else "",
        "rank_numeric": float(rank_value) if pd.notna(rank_value) else np.
↳nan,
        "model": clean_reference_text(item.get("Model")),
        "metric_value": format_reference_metric_value(score_mean, item.
↳get("Score_StdDev", "")) if pd.notna(item.get("Score_StdDev", np.nan)) else_
↳"",
        "metric_value_numeric": float(score_mean),
        "is_top10_entry": True,
        "reference_source": "current_moleculenet_literature_reference",
        "captured_at": captured_at,
        "paper_short": clean_reference_text(item.get("Paper_Short")),
        "year": clean_reference_text(item.get("Year")),
        "reference": clean_reference_text(item.get("Reference")),
        "doi_or_url": clean_reference_text(item.get("DOI_or_URL")),
        "notes": clean_reference_text(item.get("Notes")),
        "manual_reference_source_file": str(path).replace("\\", "/"),
    })
    return pd.DataFrame(rows)

CURRENT_PHYSCHEM_REFERENCE = load_current_physchem_reference()
UPDATED_REFERENCE_ROWS = combine_leaderboard_reference_frames(
    LEADERBOARD_TOP10,
    MANUAL_LEADERBOARD_REFERENCE,
    CURRENT_PHYSCHEM_REFERENCE,
)

if UPDATED_REFERENCE_ROWS.empty:
    display_note("No reference rows were available for updated leaderboard_
↳comparison.")
else:
    UPDATED_REFERENCE_ROWS = UPDATED_REFERENCE_ROWS.copy()
    if "leaderboard_metric_name" in UPDATED_REFERENCE_ROWS.columns:
        UPDATED_REFERENCE_ROWS["metric_kind"] =_
↳UPDATED_REFERENCE_ROWS["leaderboard_metric_name"].astype(str).
↳apply(leaderboard_metric_kind)
        UPDATED_REFERENCE_ROWS["metric_value_numeric"] = pd.
↳to_numeric(UPDATED_REFERENCE_ROWS.get("metric_value_numeric", pd.
↳Series(dtype=float)), errors="coerce")

```

```

UPDATED_LEADERBOARD_COMPARISON =
↳ comparison_rows_from_reference(UPDATED_REFERENCE_ROWS)

tdc_admet_group_ids = {
    "caco2_wang", "hia_hou", "pgp_broccatelli", "bioavailability_ma",
↳ "lipophilicity_astazeneca",
    "solubility_aqsoldb", "bbb_martins", "ppbr_az", "vdss_lombardo",
↳ "cyp2d6_veith",
    "cyp3a4_veith", "cyp2c9_veith", "cyp2d6_substrate_carbonmangels",
    "cyp3a4_substrate_carbonmangels", "cyp2c9_substrate_carbonmangels",
↳ "half_life_obach",
    "clearance_microsome_az", "clearance_hepatocyte_az", "herg", "ames",
↳ "dili", "ld50_zhu",
}
ref_dataset_key = UPDATED_REFERENCE_ROWS.get("benchmark_id", pd.Series("",
↳ index=UPDATED_REFERENCE_ROWS.index)).astype(str).str.lower()
ref_dataset_name = UPDATED_REFERENCE_ROWS.get("dataset", pd.Series("",
↳ index=UPDATED_REFERENCE_ROWS.index)).astype(str).str.lower().str.
↳ replace("^tdc_", "", regex=True)
is_tdc22 = ref_dataset_key.isin(tdc_admet_group_ids) | ref_dataset_name.
↳ isin(tdc_admet_group_ids)
has_maxqsaring = UPDATED_REFERENCE_ROWS.get("model", pd.Series("",
↳ index=UPDATED_REFERENCE_ROWS.index)).astype(str).str.contains("maxqsaring",
↳ case=False, na=False)
maxq_datasets = sorted(set(ref_dataset_key.loc[is_tdc22 & has_maxqsaring].
↳ replace("", np.nan).dropna().tolist()) | set(ref_dataset_name.loc[is_tdc22 &
↳ has_maxqsaring].replace("", np.nan).dropna().tolist()))

physchem_current_mask = UPDATED_REFERENCE_ROWS.get("dataset", pd.Series("",
↳ index=UPDATED_REFERENCE_ROWS.index)).astype(str).str.lower().
↳ isin({"esol_delaney", "lipophilicity"})
physchem_current_rows = UPDATED_REFERENCE_ROWS.loc[physchem_current_mask].
↳ copy()

reference_audit = pd.DataFrame([
    {
        "audit_item": "TDC-22 datasets with MaxQsaring reference rows",
        "count": len(maxq_datasets),
        "expected": len(tdc_admet_group_ids),
        "status": "complete" if len(maxq_datasets) >=
↳ len(tdc_admet_group_ids) else "partial",
        "details": "; ".join(maxq_datasets),
    },
    {

```

```

        "audit_item": "Current ESOL/Lipophilicity literature reference_
↪rows",
        "count": int(len(physchem_current_rows)),
        "expected": 2,
        "status": "complete" if physchem_current_rows["dataset"].nunique()_
↪>= 2 else "partial",
        "details": "; ".join(sorted(physchem_current_rows.get("model", pd.
↪Series(dtype=object)).dropna().astype(str).unique().tolist())[:20]),
    },
    {
        "audit_item": "Updated reference rows total",
        "count": int(len(UPDATED_REFERENCE_ROWS)),
        "expected": np.nan,
        "status": "available",
        "details": f"datasets={UPDATED_REFERENCE_ROWS['dataset'].nunique()_
↪if 'dataset' in UPDATED_REFERENCE_ROWS.columns else 0}",
    },
    {
        "audit_item": "Updated comparison rows generated",
        "count": int(len(UPDATED_LEADERBOARD_COMPARISON)) if_
↪UPDATED_LEADERBOARD_COMPARISON is not None else 0,
        "expected": np.nan,
        "status": "available" if UPDATED_LEADERBOARD_COMPARISON is not None_
↪and not UPDATED_LEADERBOARD_COMPARISON.empty else "missing",
        "details": f"datasets={UPDATED_LEADERBOARD_COMPARISON['dataset'].
↪nunique() if UPDATED_LEADERBOARD_COMPARISON is not None and not_
↪UPDATED_LEADERBOARD_COMPARISON.empty and 'dataset' in_
↪UPDATED_LEADERBOARD_COMPARISON.columns else 0}",
    },
])

write_publication_csv(UPDATED_REFERENCE_ROWS,_
↪"publication_leaderboard_top10_reference.csv")
write_publication_csv(UPDATED_LEADERBOARD_COMPARISON,_
↪"publication_leaderboard_comparison_by_dataset.csv")
write_publication_csv(reference_audit,_
↪"publication_leaderboard_reference_audit.csv")

display_note("### Publication Analysis: Updated Leaderboard References")
display(reference_audit)
if not UPDATED_LEADERBOARD_COMPARISON.empty:
    display(UPDATED_LEADERBOARD_COMPARISON.sort_values(["dataset",_
↪"estimated_rank_vs_top10"]).reset_index(drop=True))

rank_counts = (

```

```

UPDATED_LEADERBOARD_COMPARISON.
↪assign(in_top10_estimate=UPDATED_LEADERBOARD_COMPARISON.
↪get("in_top10_estimate", False).astype(bool))
    .groupby("in_top10_estimate", dropna=False)["dataset"]
    .nunique()
    .reset_index(name="datasets")
)
fig = px.bar(
    rank_counts,
    x="in_top10_estimate",
    y="datasets",
    title="AutoQSAR top-10 status using updated local reference tables",
)
fig.update_layout(xaxis_title="Estimated in top 10",
↪yaxis_title="Datasets")
fig.show()

```

1.0.32 Publication Analysis: Updated Leaderboard References

	audit_item	count	expected	status
	details			
0	TDC-22 datasets with MaxQsaring reference rows	22	22.0	complete
	↪ ames; bbb_martins; bioavailability_ma; caco2_w...			
1	Current ESOL/Lipophilicity literature referenc...	73	2.0	complete
	↪ 3D INFOMAX; 3D PGT; AutoML; AutoML (xLSTM pape...			
2	Updated reference rows total	407	NaN	available
	↪ datasets=37			
3	Updated comparison rows generated	37	NaN	available
	↪ datasets=37			
	dataset	benchmark_id		
↪	leaderboard_metric_name	metric_kind	direction	
↪	best_model	best_workflow	\	
0	esol_delaney	esol_delaney		
	↪ RMSE	rmse	lower	
	↪ TabPFNRegressor	TabPFN	deep learning	
1	lipophilicity	lipophilicity		
	↪ RMSE	rmse	lower	Ensemble (Weighted average (inverse train
	↪ RMSE))	ensemble		
2	poduam_pod_nc_std	poduam_pod_nc_std		
	↪ RMSE	rmse	lower	CFA (Combinatorial
	↪ Fusion)	CFA fusion		
3	poduam_pod_rd_std	poduam_pod_rd_std		
	↪ RMSE	rmse	lower	
	↪ XGBoost	conventional		

4	polaris_adme_fang_hppb_1	polaris/adme-fang-hppb-1	□
↪	mean_squared_error mse lower	MapLight + GNN (CatBoost,	
↪	Strict Parity)		
5	polaris_adme_fang_perm_1	polaris/adme-fang-perm-1	□
↪	mean_squared_error mse lower		□
↪	XGBoost conventional		
6	polaris_adme_fang_rclint_1	polaris/adme-fang-rclint-1	□
↪	mean_squared_error mse lower		□
↪	CatBoost conventional		
7	polaris_adme_fang_rppb_1	polaris/adme-fang-rppb-1	□
↪	mean_squared_error mse lower	MapLight + GNN (CatBoost,	
↪	Strict Parity)		
8	polaris_adme_fang_solu_1	polaris/adme-fang-solu-1	□
↪	mean_squared_error mse lower		□
↪	XGBoost conventional		
9	tdc_ames	ames	□
↪	AUROC roc_auc higher	Ensemble (OOF Stacking (RidgeCV,	
↪	5-fold))	ensemble	
10	tdc_bbb_martins	bbb_martins	□
↪	AUROC roc_auc higher	Random	□
↪	forest conventional		
11	tdc_bioavailability_ma	bioavailability_ma	□
↪	AUROC roc_auc higher		□
↪	CatBoost conventional		
12	tdc_caco2_wang	caco2_wang	□
↪	MAE mae lower	MapLight CatBoost (Strict	
↪	Parity)	conventional	
13	tdc_carcinogens_lagunin	carcinogens_lagunin	□
↪	AUROC roc_auc higher		□
↪	AdaBoost conventional		
14	tdc_clearance_hepatocyte_az	clearance_hepatocyte_az	□
↪	Spearman spearman higher	MapLight + GNN (CatBoost, Strict	
↪	Parity)		
15	tdc_clearance_microsome_az	clearance_microsome_az	□
↪	Spearman spearman higher	MapLight + GNN (CatBoost, Strict	
↪	Parity)		
16	tdc_clintox	clintox	□
↪	AUROC roc_auc higher		□
↪	CatBoost conventional		
17	tdc_cyp2c9_substrate_carbonmangels	cyp2c9_substrate_carbonmangels	□
↪	AUPRC auprc higher	Ensemble (OOF Stacking (RidgeCV,	
↪	5-fold))	ensemble	
18	tdc_cyp2c9_veith	cyp2c9_veith	□
↪	AUPRC auprc higher	Ensemble (OOF Stacking (RidgeCV,	
↪	5-fold))	ensemble	

19	tdc_cyp2d6_substrate_carbonmangels	cyp2d6_substrate_carbonmangels		
↪	AUPRC	auprc	higher	Chemprop v2 (AttentiveFP, ␣
↪	ensemble=1)	Chemprop v2		
20		tdc_cyp2d6_veith		cyp2d6_veith ␣
↪	AUPRC	auprc	higher	Ensemble (Weighted average (inverse train, ␣
↪	RMSE))	ensemble		
21	tdc_cyp3a4_substrate_carbonmangels	cyp3a4_substrate_carbonmangels		␣
↪	AUROC	roc_auc	higher	␣
↪	LogisticRegression	conventional		
22		tdc_cyp3a4_veith		cyp3a4_veith ␣
↪	AUPRC	auprc	higher	Ensemble (OOF Stacking (RidgeCV, ␣
↪	5-fold))	ensemble		
23		tdc_dili		dili ␣
↪	AUROC	roc_auc	higher	Ensemble (OOF Stacking (RidgeCV, ␣
↪	5-fold))	ensemble		
24		tdc_half_life_obach		half_life_obach ␣
↪	Spearman	spearman	higher	␣
↪	TabPFNRegressor	TabPFN deep learning		
25		tdc_herg		herg ␣
↪	AUROC	roc_auc	higher	␣
↪	AdaBoost	conventional		
26		tdc_hia_hou		hia_hou ␣
↪	AUROC	roc_auc	higher	CFA (Combinatorial, ␣
↪	Fusion)	CFA fusion		
27	tdc_hydrationfreeenergy_freesolv	hydrationfreeenergy_freesolv		␣
↪	RMSE	rmse	lower	CFA (Combinatorial, ␣
↪	Fusion)	CFA fusion		
28		tdc_ld50_zhu		ld50_zhu ␣
↪	MAE	mae	lower	CFA (Combinatorial, ␣
↪	Fusion)	CFA fusion		
29	tdc_lipophilicity_astrazeneca	lipophilicity_astrazeneca		␣
↪	MAE	mae	lower	␣
↪	XGBoost	conventional		
30		tdc_pgp_broccatelli		pgp_broccatelli ␣
↪	AUROC	roc_auc	higher	␣
↪	HistGradientBoosting	conventional		
31		tdc_ppbr_az		ppbr_az ␣
↪	MAE	mae	lower	MapLight + GNN (CatBoost, Strict, ␣
↪	Parity)	MapLight + GNN		
32		tdc_skin_reaction		skin_reaction ␣
↪	AUROC	roc_auc	higher	Ensemble (Weighted average (inverse train, ␣
↪	RMSE))	ensemble		
33	tdc_solubility_aqsoldb	solubility_aqsoldb		␣
↪	MAE	mae	lower	CFA (Combinatorial, ␣
↪	Fusion)	CFA fusion		

34		tdc_tox21	tox21	
↪	AUROC	roc_auc higher		
↪	Tabular MLP	conventional		
35		tdc_toxcast	toxcast	
↪	AUROC	roc_auc higher		
↪	CatBoost	conventional		
36		tdc_vdss_lombardo	vdss_lombardo	
↪	Spearman	spearman higher	MapLight + GNN (CatBoost, Strict	
↪	Parity)	MapLight + GNN		

	best_value	best_value_raw_model_metric	best_value_metric_column	
↪	leaderboard_top1	leaderboard_top10_cutoff	known_reference_count	
↪	top10_reference_count	\		

0	0.621439	0.621439	test_rmse	
↪	0.558	0.615	38	10
1	0.594788	0.594788	test_rmse	
↪	0.549	0.587	35	10
2	0.699850	0.699850	test_rmse	
↪	0.550	0.730	2	2
3	0.554603	0.554603	test_rmse	
↪	0.410	0.630	2	2
4	0.199498	0.446652	test_rmse	
↪	0.143	0.383	10	10
5	0.185158	0.430300	test_rmse	
↪	0.113	0.257	10	10
6	0.285363	0.534193	test_rmse	
↪	0.216	0.403	10	10
7	0.243599	0.493558	test_rmse	
↪	0.230	0.634	10	10
8	0.343380	0.585986	test_rmse	
↪	0.222	0.329	10	10
9	0.876017	0.876017	test_roc_auc	
↪	0.912	0.834	7	7
10	0.932419	0.932419	test_roc_auc	
↪	0.941	0.903	7	7
11	0.776688	0.776688	test_roc_auc	
↪	0.748	0.640	8	8
12	0.272436	0.272436	test_mae	
↪	0.256	0.288	20	10
13	0.862626	0.862626	test_roc_auc	
↪	0.848	0.730	10	10
14	0.523087	0.523087	test_spearman	
↪	0.633	0.440	16	10
15	0.605587	0.605587	test_spearman	
↪	0.652	0.599	17	10

16	0.948884	0.948884	test_roc_auc	6
	↪0.996	0.732		
17	0.427480	0.427480	testauprc	6
	↪0.450	0.360		
18	0.810239	0.810239	testauprc	6
	↪0.877	0.770		
19	0.683938	0.683938	testauprc	7
	↪0.766	0.570		
20	0.727009	0.727009	testauprc	6
	↪0.811	0.464		
21	0.658793	0.658793	test_roc_auc	7
	↪0.692	0.651		
22	0.889038	0.889038	testauprc	6
	↪0.923	0.750		
23	0.913043	0.913043	test_roc_auc	6
	↪0.945	0.852		
24	0.391247	0.391247	test_spearman	16
	↪0.649	0.485		
25	0.860825	0.860825	test_roc_auc	7
	↪0.880	0.806		
26	0.989918	0.989918	test_roc_auc	8
	↪0.994	0.976		
27	0.644869	0.644869	test_rmse	6
	↪0.654	1.261		
28	0.580239	0.580239	test_mae	16
	↪0.292	0.605		
29	0.492100	0.492100	test_mae	17
	↪0.406	0.515		
30	0.924020	0.924020	test_roc_auc	8
	↪0.994	0.911		
31	7.329016	7.329016	test_mae	16
	↪0.679	7.914		
32	0.774839	0.774839	test_roc_auc	11
	↪0.741	0.499		
33	0.727744	0.727744	test_mae	17
	↪0.557	0.776		
34	0.808239	0.808239	test_roc_auc	6
	↪0.867	0.833		
35	0.791353	0.791353	test_roc_auc	1
	↪0.714	0.714		
36	0.671327	0.671327	test_spearman	16
	↪0.942	0.582		
		reference_sources		
	↪	reference_models	estimated_rank_vs_top10	in_top10_estimate
	↪	gap_to_top1	gap_to_top10_cutoff	

```

0  current_moleculenet_literature_reference; lite... PrismNet; HiGNN; DMPNN;
  ↳MGPhyche; DeeperGCN; A... 11 False 0.
  ↳063439 0.006439
1  current_moleculenet_literature_reference; lite... PrismNet; GRAPHMSL; MV-Mol;
  ↳InstructMol; GEM +... 11 False 0.045788
  ↳ 0.007788
2  literature_static PODUAM BNN
  ↳(PODnc); PODUAM BNN (PODnc) 2 True 0.
  ↳149850 -0.030150
3  literature_static PODUAM BNN
  ↳(PODrd); PODUAM BNN (PODrd) 2 True 0.
  ↳144603 -0.075397
4  polaris
  ↳1B_MPNN_LargeMix-and-Phenomics; 1B_MPNN_MolGPS... 3
  ↳ True 0.056498 -0.183502
5  polaris
  ↳1B_MPNN_MolGPS-ens_LargeMix; 1B_MPNN_LargeMix-... 6
  ↳ True 0.072158 -0.071842
6  polaris
  ↳1B_MPNN_MolGPS-ens_LargeMix; 1B_MPNN_LargeMix-... 4
  ↳ True 0.069363 -0.117637
7  polaris
  ↳1B_MPNN_LargeMix-and-Phenomics; 1B_MPNN_MolGPS... 3
  ↳ True 0.013599 -0.390401
8  polaris
  ↳1B_MPNN_MolGPS-ens_LargeMix; 1B_MPNN_LargeMix-... 11
  ↳ False 0.121380 0.014380
9  manual_tdc_admet_reference QW-MTL; Meta-model (NIST);
  ↳ ADMETboost (XGBoost... 2 True 0.035983
  ↳ -0.042017
10 manual_tdc_admet_reference MolGPS (3B); MaxQsaring;
  ↳QW-MTL; Auto-ADMET (M... 2 True 0.
  ↳008581 -0.029419
11 manual_tdc_admet_reference MaxQsaring; Auto-ADMET
  ↳(MTE); Meta-model (NIST... 1 True -0.
  ↳028688 -0.136688
12 manual_tdc_admet_reference; tdc CaliciBoost; XG Boost;
  ↳MapLight; BaseBoosting;... 3 True 0.
  ↳016436 -0.015564
13 manual_tdc_admet_reference FATE-Tox (MTL); FATE-Tox
  ↳(STL); MAT; Uni-Mol; ... 1 True -0.
  ↳014626 -0.132626
14 manual_tdc_admet_reference; tdc CFA; MapLight + GNN;
  ↳MapLight; ADMETrix; MiniM... 3 True 0.
  ↳109913 -0.083087

```

15	manual_tdc_admet_reference; tdc	MapLight + GNN; MiniMol; ┐
	└MapLight; CFA; RFStac... 10	True 0.
	└046413 -0.006587	
16	manual_tdc_admet_reference	PrismNet; MolGraph-xLSTM; ┐
	└DMPNN; DeeperGCN; Hi... 2	True 0.047116 ┐
	└ -0.216884	
17	manual_tdc_admet_reference	MaxQsaring; QW-MTL; MolE; ┐
	└Auto-ADMET (MTE); AD... 3	True 0.022520 ┐
	└ -0.067480	
18	manual_tdc_admet_reference	MaxQsaring; MolGPS (3B); ┐
	└ADMETboost (XGBoost);... 3	True 0.
	└066761 -0.040239	
19	manual_tdc_admet_reference	MaxQsaring; QW-MTL; ┐
	└Auto-ADMET (MTE); MolE; AD... 5	True 0.
	└082062 -0.113938	
20	manual_tdc_admet_reference	MaxQsaring; ADMETboost ┐
	└(XGBoost); Meta-model (... 2	True 0.
	└083991 -0.263009	
21	manual_tdc_admet_reference	MolE; MaxQsaring; ┐
	└Meta-model (NIST); ADMETboos... 7	True ┐
	└0.033207 -0.007793	
22	manual_tdc_admet_reference	MaxQsaring; ADMETboost ┐
	└(XGBoost); QW-MTL; Meta... 2	True 0.
	└033962 -0.139038	
23	manual_tdc_admet_reference	Meta-model (NIST); ┐
	└MiniMol (GINE); ADMETboost ... 5	True 0.
	└031957 -0.061043	
24	manual_tdc_admet_reference; tdc	CFA; MapLight; MapLight + ┐
	└GNN; Euclia ML model... 11	False 0.257753 ┐
	└ 0.093753	
25	manual_tdc_admet_reference	MaxQsaring; MolGPS (3B); ┐
	└MiniMol (GINE); QW-MT... 3	True 0.
	└019175 -0.054825	
26	manual_tdc_admet_reference	MiniMol (GINE); ┐
	└MaxQsaring; QW-MTL; ADMETboost... 4	True ┐
	└ 0.004082 -0.013918	
27	manual_tdc_admet_reference	PrismNet; MolGraph-xLSTM; ┐
	└HiGNN; FP-GNN; DMPNN... 1	True -0.009131 ┐
	└ -0.616131	
28	manual_tdc_admet_reference; tdc	BaseBoosting KyQVZ6b2; ┐
	└ADMETrix; MiniMol; MACC... 5	True 0.
	└288239 -0.024761	
29	manual_tdc_admet_reference; tdc	MiniMol; Chemprop-RDKit; ┐
	└Chemprop; BaseBoostin... 9	True 0.
	└086100 -0.022900	

```

30          manual_tdc_admet_reference  MiniMol (GINE); MolGPS
↳(3B); MaxQsaring; MolE;...          5          True    0.
↳069980          -0.013020
31          manual_tdc_admet_reference; tdc  Gradient Boost; MapLight
↳+ GNN; MapLight; Mini...          3          True    6.
↳650016          -0.584984
32          manual_tdc_admet_reference  FATE-Tox (MTL); FATE-Tox
↳(STL); SVM; ET (singl...          1          True    -0.
↳033839          -0.275839
33          manual_tdc_admet_reference; tdc  MiniMol; Chemprop-RDKit;
↳DeepMol (AutoML); Att...          3          True    0.
↳170744          -0.048256
34          manual_tdc_admet_reference  PrismNet; MolGraph-xLSTM;
↳DMPNN; HiGNN; Deeper...          7          True    0.058761
↳          0.024761
35          manual_tdc_admet_reference
↳          PrismNet          1          True    -0.
↳077353          -0.077353
36          manual_tdc_admet_reference; tdc  MapLight + GNN; MapLight;
↳CFA; Basic ML; Eucli...          5          True    0.270673
↳          -0.089327

```